FOUNDATION OF EMBODIED AI

English Course Reader

Foundation of Embodied AI

14 chapters

Chapters follow the repository manifest

zengxiong111 / Foundation_of_Embodied_AI

Contents

Select a chapter to jump to the start.

01	Git Basics	3
02	ROS 2 Basics	23
03	Camera Calibration	47
04	Visual Perception and State Estimation	66
05	Robot Dynamics	79
06	Isaac / MuJoCo Simulation	99
07	Embodied RL	120
08	Motion Control RL	135
09	PPO	161
10	SAC	181
11	Data Collection and Imitation Learning	200
12	Flow Matching	212
13	GR00T & SONIC	233
14	Advanced Robot Agents	248

Original chapter math and links are preserved; local page numbers may differ.

Git Tutorial: 30-Minute Essentials and Research Reference

Course home · 简体中文 · PDF

Item	Details
Applicability	Git 2.x; Ubuntu terminal; GitHub workflow examples
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- Part I: The 30-Minute Essentials
  - 1. Git Basics: Four Areas and Essential Commands
  - 2. Install Git and Configure Your Identity
  - 3. The One End-to-End Workflow: Clone to Pull Request
  - 4. Inspect Changes with Status, Diff, and Log
  - 5. Syncing Remote Code and Pre-Commit Safety
- Part II: Research-Oriented Reference
  - 6. New Repositories, SSH, and GitHub Authentication
  - 7. Branches, Team Collaboration, and Conflicts
  - 8. Undoing, Correcting, and Recovering History
  - 9. Ignoring Files, Git LFS, and DVC
  - 10. Stash, Rebase, Show, and Blame
  - 11. Version Tracking and Reproducible Research
  - 12. Common Errors and Safe Troubleshooting
  - 13. Git in VS Code
  - 14. Advanced Command Reference and Official Docs

Part I: The 30-Minute Essentials

Goal: In 30 minutes, understand Git's model and safely edit, commit, and push a branch in an existing research repository. Study Part I in order; consult Part II only when needed.

Time	Topic
0–5 minutes	The four areas and key commands
5–10 minutes	Installation and identity
10–22 minutes	One complete GitHub workflow
22–26 minutes	Diffs and history
26–30 minutes	Sync and safety

1. Git Basics: Four Areas and Essential Commands

Git is a distributed version control system (VCS). It records the history of changes to files so that you can compare versions, restore older code, experiment in independent branches, and integrate changes from collaborators.

1.1 What problems does Git solve?

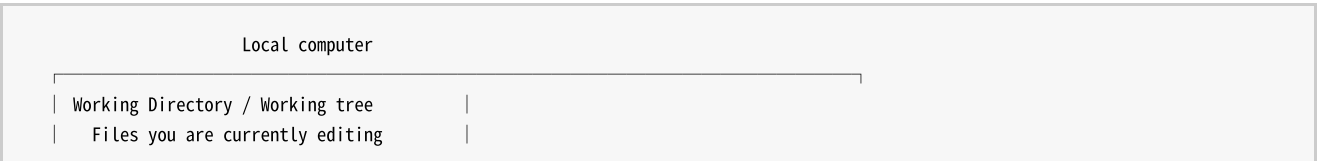
- **Version control:** Each commit identifies a traceable state of your code.
- **Safe experimentation:** Try a different reward function or training algorithm without disturbing a stable version.
- **Recovery:** Retrieve accidentally changed code or undo a buggy commit.
- **Collaboration:** Work independently on PPO, SAC, perception, and sim-to-real modules, then integrate them.
- **Cross-machine synchronization:** Use GitHub or GitLab to synchronize source code between a workstation and GPU servers.
- **Reproducibility:** Record the Commit ID used for a training run together with its configuration, seed, and dataset version.

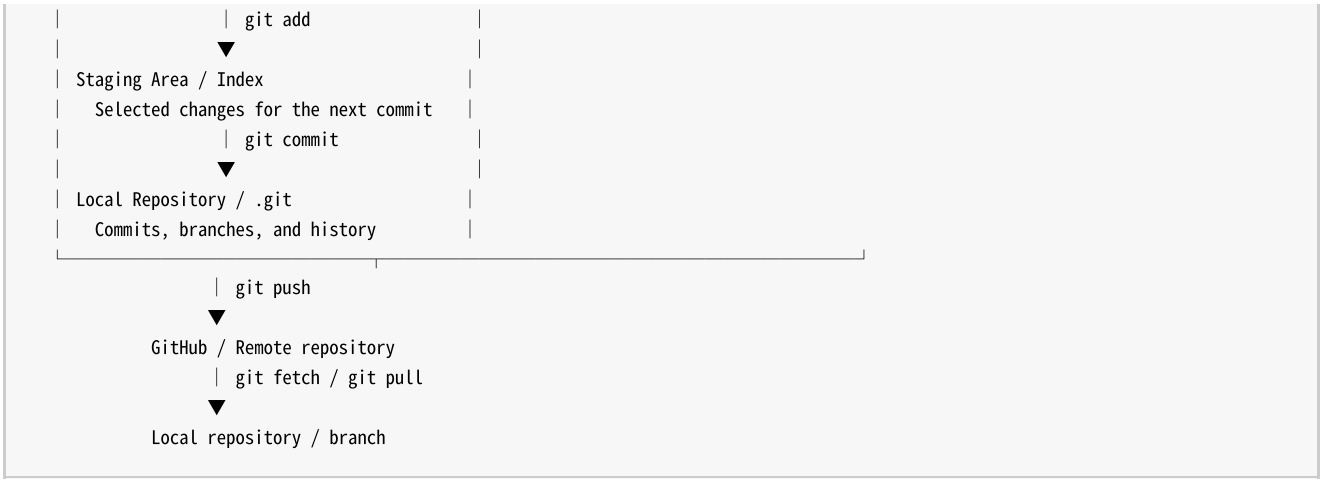
1.2 Git versus GitHub, GitLab, and VS Code

Tool	Purpose
Git	Local version control, commits, branches, and merging; most operations work offline.
GitHub	Online Git hosting with pull requests, code review, and automation.
GitLab	Another platform for hosting Git repositories and collaborating.
VS Code	A code editor with built-in Source Control support for Git.

Git is not GitHub. You can run `git init`, `git add`, `git commit`, and `git branch` without any GitHub account or network connection.

1.3 Working tree, staging area, local repository, remote repository





The following commands perform **three different operations**:

```
git add .
git commit -m "Implement PPO training"
git push
```

- `git add` places **selected file changes** in the staging area.
- `git commit` records the **staged content** in a local commit.
- `git push` sends **local commits** to a remote repository.

A commit is effectively a snapshot of the project at a particular moment, identified by a hash. Uncommitted working-tree edits do **not** get uploaded just because you run `git push`.

1.4 Ten essential commands

Command	Purpose
`git init`	Initialize a local repository
`git clone URL`	Clone a remote repository
`git status`	Inspect the current state
`git add PATH`	Stage file changes
`git commit -m "..."`	Create a local commit
`git push`	Publish commits
`git pull`	Fetch and integrate remote changes
`git log --oneline`	View compact commit history
`git switch -c NAME`	Create and switch to a branch
`git merge NAME`	Merge a specified branch

2. Install Git and Configure Your Identity

2.1 Install Git

Open an Ubuntu terminal and run:

```
sudo apt update
sudo apt install git
git --version
```

You should see a version such as `git version 2.x.y`; the exact version depends on your Ubuntu release and repositories.

2.2 Configure the commit author identity

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

These values are written into the author identity of new commits. Use an email address associated with GitHub if you want GitHub to attribute the contributions correctly. Inspect the global configuration:

```
git config --global --list
```

You can also configure a different identity for **one repository** by omitting `--global`:

```
git config user.name "Lab Account"
git config user.email "lab@example.com"
```

2.3 Set the initial branch name

```
git config --global init.defaultBranch main
```

This affects repositories initialized with `git init` in the future. It does not rename existing branches.

3. The One End-to-End Workflow: Clone to Pull Request

This is the tutorial's **only fully sequenced day-to-day workflow**. It replaces overlapping local-repository, collaboration, research-workflow, and beginner-workflow examples in the previous edition.

3.1 Before you begin

Assume you have push access to an existing GitHub repository named `robot-policy`, and its default branch is `main`. If you do not have a remote repository yet, see Chapter 6 in Part II for `git init` and remote setup. HTTPS cloning can be anonymous for public repos; pushing requires GitHub authentication (Credential Manager, a suitable token, or SSH).

3.2 Clone, change, inspect, commit, and open a PR

```
# Clone once
git clone https://github.com/YOUR_USERNAME/robot-policy.git
cd robot-policy

# Start one isolated task from the latest main
git switch main
git pull --ff-only
git switch -c feature/pose-estimation

# Edit an existing source file, then inspect it
git status
git diff
git add train.py
git diff --staged
git commit -m "Improve visual pose estimation"

# First push of this feature branch
git push -u origin feature/pose-estimation
```

On GitHub, open a Pull Request from `feature/pose-estimation` into `main` for checks and review. After it is merged, run `git switch main` and `git pull --ff-only` locally. **Do not practice pushing to upstream open-source repositories where you lack write access**; fork first when necessary.

3.3 Local versus remote changes

`git add` modifies the index, `git commit` records local history, and only `git push` publishes local commits. A GitHub PR is a review/merge request, not a commit. Uncommitted working-tree changes are never sent by `git push`.

4. Inspect Changes with Status, Diff, and Log

When debugging a training run, knowing what the code **actually changed** is often more valuable than remembering what you *intended* to change.

4.1 Inspect the working tree

```
git status
```

The output identifies your current branch, staged edits, unstaged edits, and untracked files.

4.2 Compare code

```
git diff                    # Working tree vs staging area
git diff --staged           # Staging area vs latest commit
git diff -- train.py        # Changes in one file
```

A diff might look like:

```
- learning_rate = 3e-4
+ learning_rate = 1e-4
```

The - line is removed; the + line is added.

4.3 Visualize the commit graph

```
git log --oneline
git log --oneline --graph --all
```

For example:

```
* a34bd21 (HEAD -> main) Merge SAC implementation
|\
| * d72f8a1 Implement SAC algorithm
| * f61b2c9 Add replay buffer
|/
* b83fc92 Implement PPO
* 28ad613 Initial commit
```

Make a habit of combining `git status`, `git diff`, and `git log --oneline --graph --all`. Before each commit, confirm that you have not staged temporary artifacts, model weights, or secrets.

5. Syncing Remote Code and Pre-Commit Safety

5.1 Update local code: Fetch versus Pull

```
cd robot-policy
git fetch origin
```

`fetch` retrieves remote objects and updates remote-tracking references. It **does not automatically merge them into the current local branch**.

If your local `main` tracks `origin/main`, you may then run:

```
git merge origin/main
```

A common shorthand is:

```
git pull
```

`pull` performs `fetch` followed by integration, using merge or rebase according to the command options and Git configuration. To avoid an unexpected merge commit when updating a stable branch, beginners may prefer:

```
git pull --ff-only
```

This succeeds only when the update can be fast-forwarded; if the histories have diverged, Git stops and lets you decide how to integrate them.

> **Note:** You normally cannot push directly to another team's open-source repository unless you have permission. You can Fork the repository first and work in your own Fork, or contribute

> through the project's pull request process.

5.2 Three checks before committing

1. `git status`: confirm the active branch, staged files, and untracked files.
2. `git diff --staged`: review exactly what the next commit will contain.
3. Never commit passwords, private keys, model checkpoints, large datasets, or training outputs. See Chapter 9 for `.gitignore` and Git LFS/DVC.

5.3 Minimal safe undo

If you staged a file accidentally, `git restore --staged FILE` only unstages it and preserves working-tree edits. Before discarding work or undoing a shared commit, consult the risk table in Chapter 8.

Part II: Research-Oriented Reference

Use this part as a reference when a real need arises; beginners do not need to memorize it.

6. New Repositories, SSH, and GitHub Authentication

6.1 Initialize a new research project

If no remote repository exists yet, create a local project:

```
mkdir robot-policy && cd robot-policy
git init
echo '# Robot Policy' > README.md
git add README.md
git commit -m "Initial commit"
```

Create an **empty repository without starter files** on GitHub, then configure `origin` and push using Section 6.3. Use `git clone` for existing repositories; do not run `git init` again.

6.2 Configure an SSH key

SSH authentication is convenient for long-term GitHub development on Ubuntu.

First check for existing keys:

```
ls -al ~/.ssh
```

If you do not already have a suitable Ed25519 key, generate one:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Follow the prompts to choose the key location and passphrase. If a key file already exists, **do not overwrite it without checking**.

Start an SSH agent and load the private key:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Display and copy the **public key**:

```
cat ~/.ssh/id_ed25519.pub
```

In GitHub, navigate to **Settings** → **SSH and GPG keys** → **New SSH key**, paste the public key, and save. The file `id_ed25519` is your **private key**: never publish or upload it. The `.pub` file is the one to share.

Test connectivity:

```
ssh -T git@github.com
```

On the first connection, verify GitHub's host fingerprint before accepting it. A successful result generally confirms authentication while noting that shell access is not available.

Reference: GitHub's official SSH documentation.

6.3 Push a local project to GitHub

Create an **empty** repository named `robot-policy` on GitHub. If your local repository already has an initial commit, avoid adding a README, `.gitignore`, or License on the website during repository creation; doing so would create a separate initial history.

Then, locally:

```
cd robot-policy
git remote add origin git@github.com:YOUR_USERNAME/robot-policy.git
git remote -v
git branch -M main
git push -u origin main
```

Interpretation:

- `origin` is the conventional **name** of a remote; it is not a special server.
- `git remote -v` shows the fetch and push URLs.
- `git branch -M main` names the current branch `main`.
- `-u` establishes upstream tracking so later pushes usually need only `git push`.

6.4 Clone an open-source project

For example:

```
git clone https://github.com/ajwagen/dsrl.git
cd robot-policy
```

With SSH configured and appropriate access, you can instead use the repository's SSH URL:

```
git clone git@github.com:ajwagen/dsrl.git
```

`clone` downloads the repository history, creates the project directory, and normally configures an `origin` remote. Do **not** run `git init` again inside a cloned repository.

HTTPS	SSH
Public repositories can typically be cloned anonymously	Typically requires SSH authentication
Uses an `https://...` URL	Uses a `git@github.com:...` URL
Pushing needs appropriate account credentials	Key-based authentication makes pushing convenient

7. Branches, Team Collaboration, and Conflicts

Branches let you maintain a stable implementation and experimental features side by side. For example, `main` can keep stable PPO code while `feature/sac` contains a new SAC implementation:

```
A --- B ---------------- M       main
       \                /
        C --- D -------/        feature/sac
```

Here `M` is a possible merge commit. If the merge can be fast-forwarded, Git may not create a separate merge commit.

7.1 List, create, and switch branches

```
git branch                      # List local branches
git branch feature/sac          # Create only
git switch -c feature/sac       # Create and switch (alternative)
git switch main                 # Switch to main
git switch feature/sac          # Return to the feature branch
```

If you already created the branch using `git branch feature/sac`, switch with `git switch feature/sac`; do not run `switch -c` with the same name again.

Switching branches updates files in your working tree. Git may block the switch if an uncommitted modification would be overwritten. Commit the work or use `git stash` first.

7.2 Merge into the main branch

```
git switch main
git merge feature/sac
```

If there are no conflicts, Git completes the merge. Otherwise, see Section 7.5.

7.3 Delete a finished branch

```
git branch -d feature/sac
```

The `-d` option checks that the branch has been safely integrated. To delete the corresponding remote branch:

```
git push origin --delete feature/sac
```

For shared repositories, it is generally better to merge feature branches via a reviewed GitHub Pull Request into a protected `main` branch.

A research group might organize work as follows:

```
main
 ├── feature/ppo
 ├── feature/sac
 ├── feature/pose-estimation
 └── feature/sim2real
```

A recommended workflow is **Feature Branch → Push → Pull Request → Review → Merge**.

7.4 Push the branch and open a PR

```
git push -u origin feature/pose-estimation
```

Create a Pull Request on GitHub, typically from `feature/pose-estimation` into `main`. Other team members can inspect changes, review code, discuss implementation details, run CI checks, and approve the integration.

A useful team rule: **do not make large, untested algorithm changes directly on a shared stable branch**. Validate them in isolation first.

A merge conflict may arise when two branches modify overlapping lines and Git cannot choose an automatic resolution.

Suppose two branches changed `train.py`:

```
<<<<<<< HEAD
learning_rate = 0.0001
=======
learning_rate = 0.0003
>>>>>>> feature/sac
```

- Between `<<<<<<< HEAD` and `=======` is the current branch's version.
- Between `=======` and `>>>>>>> feature/sac` is the incoming branch's version.

7.5 Resolve a conflict

1. Run `git status` to identify conflicting files.
2. Open the file and understand the intent of both changes.
3. Keep one side, or write a result that correctly combines both behaviors.
4. **Remove all conflict markers** (`<<<<<<<`, `=======`, `>>>>>>>`).
5. Run tests and validate the behavior.

6. Stage the resolved file and complete the merge commit.

For example, choose:

```
learning_rate = 0.0001
```

Then run:

```
git add train.py
git commit
```

To cancel an in-progress merge:

```
git merge --abort
```

For conflicts during a rebase, use `git rebase --continue` or `git rebase --abort` instead. `git merge --abort` is for a merge operation.

7.6 Resolve conflicts using VS Code

VS Code's Merge Editor compares the current version, the incoming version, and the merged result. Regardless of whether you use the editor or terminal, run tests afterward—especially for reward functions, action scaling, robot reference frames, and other changes whose semantics are not obvious from a text diff.

8. Undoing, Correcting, and Recovering History

Recovery depends on whether changes are **unstaged, staged, committed, or already shared**. The following commands are not interchangeable.

8.1 Unstaged changes: Discard one file's working-tree edits

```
git restore train.py
```

This restores `train.py` in the working tree from the staging area. **Unbacked working-tree edits may be lost.**

8.2 Staged changes but no commit: Unstage

```
git restore --staged train.py
```

The edits remain in the working tree, but they will not be included in the next commit unless staged again.

8.3 A local, unshared commit: Undo the commit but keep edits

Assume the latest commits are:

```
a82bc31 Bad reward modification
b41d2ab Working PPO version
```

Undo the last commit while retaining its staged changes:

```
git reset --soft HEAD~1
```

Undo the last commit and unstage the changes, but retain the working-tree edits:

```
git reset HEAD~1
```

Undo the last commit and discard the corresponding working-tree changes:

```
git reset --hard HEAD~1
```

`reset --hard` is destructive. Verify that you do not need the affected code, experiment results, or uncommitted work before using it.

8.4 A pushed/shared commit: Use Revert

```
git revert a82bc31
```

`revert` creates a new commit that attempts to cancel the changes introduced by an earlier commit, preserving an auditable shared history. Conflicts are possible if later commits have modified the same code.

Operation	Effect	Typical use
`git restore FILE`	Discard unstaged file edits	Abandon temporary local changes
`git restore --staged FILE`	Unstage, keep edits	Accidental staging
`git reset --soft HEAD~1`	Move branch pointer, keep staging	Refine unshared commits
`git reset HEAD~1`	Move pointer, keep working edits	Restage selected changes
`git reset --hard HEAD~1`	Move pointer and discard edits	Only when no recovery is needed
`git revert HASH`	Add a reverse commit	Undo an already shared commit

General principle: **use `reset` cautiously for private history; prefer `revert` for shared history.**

9. Ignoring Files, Git LFS, and DVC

Machine learning projects routinely produce gigabytes of datasets, checkpoints, videos, and logs. Such files are usually **not suitable for an ordinary source-code Git repository**.

9.1 A useful template

Create `.gitignore` in the project root:

```
touch .gitignore
nano .gitignore
```

Example:

```
# Python cache
__pycache__/
*.pyc

# Local environments
.venv/
venv/

# Secrets
.env
*.pem

# Model artifacts
*.pt
*.pth
*.ckpt
checkpoints/

# Training outputs
outputs/
logs/
wandb/

# Large datasets
data/
datasets/

# ROS recordings
*.bag
*.db3

# Optional IDE state
.vscode/
```

Adapt these rules to your project. Some projects intentionally version tiny test weights or share certain `.vscode/` settings, so blanket exclusions may not be appropriate.

9.2 What `.gitignore` does—and does not—do

Ignore rules mostly affect **untracked** files. If `checkpoint.pt` is already tracked, adding `*.pt` to `.gitignore` does not automatically stop tracking it.

To remove a file from Git tracking while keeping the local copy:

```
git rm --cached checkpoint.pt
git commit -m "Stop tracking checkpoint"
```

This does not remove old copies from existing commits. If you accidentally published an API key, password, or other secret, revoke/rotate the affected credentials promptly and evaluate whether public history must be cleaned.

9.3 Separate source code, data, and models

Recommended division:

- Git: source code, configurations, scripts, documentation, and small test assets.
- Object storage or experiment tracking: checkpoints, videos, logs, and large datasets.
- Experiment metadata: Commit ID, dataset version, weight checksum, and training parameters.

For versioned large artifacts, separately consider Git LFS, DVC, or a specialized model/data management solution.

9.4 Choosing Git LFS, DVC, or Object Storage

Option	Best suited for	Important caveat
Plain Git + `.gitignore`	Code, config, docs; ignore transient outputs	Ignore rules do not remove previous history
Git LFS	Larger binary assets that need Git-associated versions	Git stores pointers; the binary lives in LFS storage with provider support and quotas
DVC + remote storage	Large datasets and versioned pipeline metadata	Git stores `.dvc` metadata; the actual data needs configured remote storage
Object storage / experiment platform	Very large videos, checkpoints, and logs	Record versions, paths, and checksums rather than a mutable URL alone

Minimal Git LFS example (install Git LFS first and ensure your host supports it):

```
git lfs install
git lfs track "*.onnx"
git add .gitattributes models/policy.onnx
git commit -m "Track policy model via LFS"
```

With DVC, use `dvc add datasets/trajectories` on a data directory and commit the generated `.dvc` metadata and ignore rules to Git. You must also configure and synchronize a DVC remote so other machines can obtain the actual data. **Neither LFS nor DVC automatically provides unlimited remote storage.**

10. Stash, Rebase, Show, and Blame

10.1 `git stash`: Temporarily shelve uncommitted work

Suppose you are implementing SAC but must switch to `main` to fix an urgent issue:

```
git stash
```

By default, this mainly saves changes to tracked files. Include untracked files by running:

```
git stash push -u -m "WIP SAC training"
```

Then:

```
git switch main
# Fix the issue and commit it
git switch feature/sac
git stash pop
```

Command	Purpose
`git stash list`	List saved stashes
`git stash show -p`	Inspect the most recent stash's patch
`git stash pop`	Restore a stash and remove it if successful
`git stash apply`	Restore a stash but keep the stash record

Stash application can also conflict; it is not a substitute for reliable long-term backups.

10.2 `git rebase`: Move experimental work onto a newer base

Original history:

```
main:    A -- B -- C
feature: A -- B -- D -- E
```

Run:

```
git switch feature
git rebase main
```

Conceptually, the result is:

```
main:    A -- B -- C
feature: A -- B -- C -- D' -- E'
```

`rebase` replays changes and creates new commit IDs. It is useful for organizing **your own private branches**, but do not casually rewrite shared history that other developers are using.

10.3 Inspect a commit or trace a changed line

```
git show <commit-id>      # Full details of one commit
git blame train.py        # Last modifying commit for each line
```

`git blame` can help answer “when was reward scaling changed?” or “which commit changed the control frequency?” It is a historical diagnostic tool, not a judgment about individual responsibility.

10.4 Inspect local and remote branches

```
git branch -a
```

Example:

```
* main
  feature/sac
  remotes/origin/main
  remotes/origin/feature/ppo
```

Remote-tracking references (`remotes/origin/...`) represent Git's locally recorded state of remote branches. Use `git fetch` to refresh them.

11. Version Tracking and Reproducible Research

11.1 Name branches by the research task

Research scenario	Suggested branch
Implement PPO or SAC	`feature/ppo` / `feature/sac`
Explore a different reward	`experiment/reward-v2`
Adapt an online residual velocity field	`experiment/residual-velocity`
Update pose estimation or camera calibration	`feature/pose-estimation`
Fix deployed robot control latency	`fix/control-latency`
Reproduce a paper's experiment	Pin the code using a Commit ID or a Tag

11.2 Record the Commit ID with every training run

```
git rev-parse HEAD
```

Save it alongside the experiment configuration, for example:

```
experiment:
  name: residual_velocity_v3
  git_commit: a82bc31
  random_seed: 42
training:
  learning_rate: 0.0003
  batch_size: 256
```

The short hash above is illustrative; for production experiment records, store the **full hash** returned by the command.

> Important: A Commit ID identifies committed code only. If you launch an experiment with uncommitted modifications, the hash alone cannot fully reproduce that run. Check `git status` before official runs, and record a patch or create a dedicated experiment commit if needed.

11.3 Separate code, configuration, and large files

- **Git repository:** Source code, configs, entry points, evaluation scripts, README, environment documentation.

- **Experiment output storage:** TensorBoard/W&B logs, videos, weights, and large caches—usually ignored by Git.
- **Reproducibility metadata:** Commit ID, environment/dependency versions, dataset version, seed, simulator version, and GPU/driver information.
- **Robot deployment:** Tag a verified policy release (e.g., `v1.0-deploy`) and store compatible controller settings and hardware versions.

Example: tag a verified deployment version:

```
git tag -a v1.0-deploy -m "Validated robot deployment"
git push origin v1.0-deploy
```

11.4 What metadata must accompany a research run?

Bind every simulation or robot run to the **full Commit ID, clean/dirty working-tree state, config files, random seeds, dependency and simulator versions, dataset versions/checksums, and checkpoint hash**. For real robots also record consequential URDF, firmware, camera extrinsic, and controller-rate settings.

A Commit ID identifies only committed code. If `git status` shows uncommitted edits, retain a patch or make a traceable commit. The Commit ID does not replace data and environment versioning.

12. Common Errors and Safe Troubleshooting

12.1 `fatal: not a git repository`

Git cannot find a valid repository in the current directory or its parents. Check:

```
pwd
ls -a
```

Make sure you are in the project directory. Run `git init` only if you actually intend to create a **new** repository.

12.2 `Permission denied (publickey)`

SSH authentication failed. Check:

```
ssh -T git@github.com
```

Verify that the SSH agent has your correct private key loaded and that its corresponding public key is attached to the intended GitHub account.

12.3 `Updates were rejected`

This usually means that the remote has commits missing from your local history, so the push cannot fast-forward. Inspect first:

```
git fetch origin
git status
```

For your own feature branch with a correctly configured upstream, you can choose this workflow if it matches your team's conventions:

```
git pull --rebase
# Resolve potential conflicts and run tests
git push
```

Do not blindly use `git push --force` just to make the error disappear; it can overwrite collaborators' commits.

12.4 `Your local changes would be overwritten`

Uncommitted local changes might be overwritten by a switch or update. Choose one of the following before retrying:

```
# Option A: Commit the current work
git add .
git commit -m "Save work in progress"
```

```
# Option B: Temporarily stash it, including untracked files
git stash push -u
```

12.5 A general troubleshooting rule

When Git reports an unfamiliar error, start with `git status`, read its suggested next steps, and determine whether you are in a merge, rebase, or ordinary working-tree state. Avoid chains of `reset --hard`, `clean -fd`, or force-push commands when you do not understand the current state.

13. Git in VS Code

Open the repository with:

```
code .
```

Open the **Source Control** pane, commonly with `Ctrl + Shift + G`.

Action	VS Code workflow
View changed files	Source Control file list
Compare versions	Click a modified file in the list
Stage edits	+ / Stage Changes
Commit	Enter a message and choose Commit
Push	Push or Sync Changes

Action	VS Code workflow
Switch branches	Status bar branch label or Command Palette
Resolve conflicts	Merge Editor

For example, change:

```
reward = success_reward
```

to:

```
reward = success_reward + progress_reward
```

VS Code's Diff Editor clearly shows the removed and added lines. For RL experiments with many configuration files, reviewing the complete diff before each commit often reveals accidental hyperparameter changes.

Recommended combination: Use the terminal for reproducible commands and precise control, and use VS Code to review large diffs and resolve conflicts.

14. Advanced Command Reference and Official Docs

Category	Command	Purpose
Setup	`git init`	Create a repository
Clone	`git clone URL`	Download a repository
Status	`git status`	Inspect current state
Diff	`git diff`	Inspect unstaged changes
History	`git log --oneline`	Browse commit history
Stage	`git add .`	Stage changes in the current directory
Commit	`git commit -m "msg"`	Create a commit
Publish	`git push`	Send local commits to a remote
Fetch	`git fetch`	Retrieve remote state without merging
Sync	`git pull --ff-only`	Fast-forward-only update
Branch	`git branch`	List local branches
Branch	`git switch -c name`	Create and switch branch
Branch	`git switch name`	Switch to an existing branch
Merge	`git merge name`	Merge a named branch
Undo	`git restore file`	Discard unstaged file edits
Undo	`git restore --staged file`	Unstage a file
Undo	`git revert HASH`	Add a reversing commit

Category	Command	Purpose
Shelf	`git stash`	Temporarily save edits
Shelf	`git stash pop`	Restore a stash
Inspect	`git show HASH`	Examine a commit
Inspect	`git blame file`	Inspect per-line commit attribution

14.1 Official references

- Pro Git (Chinese)
- Pro Git (English)
- Git Command Reference
- GitHub: Connect with SSH
- GitHub: Pull Request Documentation

Conclusion: Git's main contribution to research is not simply “uploading code to GitHub.” It is connecting **source-code versions, experimental changes, and reproducible results**. Start with small commits, separate experiment branches, review diffs, and keep large artifacts out of the main source repository. Add team collaboration practices as your projects grow.

ROS 2 for Robot Learning: A Complete Practical Tutorial

Course home · 简体中文 · PDF

Item	Details
Applicability	Ubuntu 24.04; ROS 2 Jazzy; Python examples
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

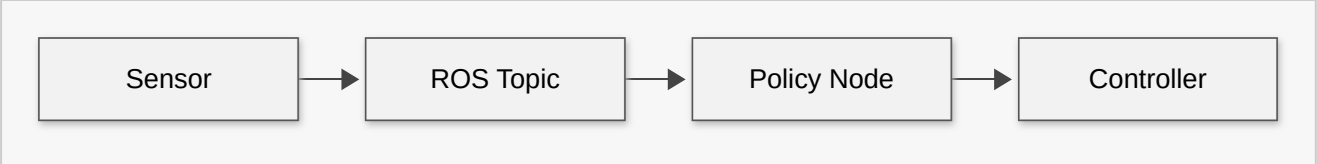
Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. What Is ROS, and Why Use It in Robot Learning?
  - 1.1 Definition of ROS
  - 1.2 A typical robot learning system
  - 1.3 Is ROS always required for robot learning?
- 2. ROS 1 vs. ROS 2 and Choosing a Distribution
  - 2.1 Main architectural differences
  - 2.2 Which ROS distribution should you use in 2026?
- 3. Core ROS 2 Concepts
  - 3.1 Node
  - 3.2 Topic: publish / subscribe
  - 3.3 Message: data schema
  - 3.4 Service: request / response
  - 3.5 Action: longer operations with progress feedback
  - 3.6 Parameter: node configuration
  - 3.7 TF2: coordinate-frame management
  - 3.8 QoS: Quality of Service
- 4. Installing ROS 2 Jazzy on Ubuntu 24.04
  - 4.1 Check your system and configure repositories
  - 4.2 Install ROS 2 and developer tools
  - 4.3 Source the environment and verify
  - 4.4 First communication test
  - 4.5 Essential command-line tools

- 5. Hands-on: Writing a Robot Learning Policy Node in Python
  - 5.1 Create a workspace
  - 5.2 Create a Python package
  - 5.3 Implement the policy node
  - 5.4 Declare dependencies and entry point
  - 5.5 Build and run
  - 5.6 Publish simulated state and inspect output
  - 5.7 Replace the toy mapping with a PPO / SAC policy
- 6. Recording Robot Learning Data with ROS 2
  - 6.1 Teleoperation data collection architecture
  - 6.2 What is rosbag2?
  - 6.3 Record and replay a session
  - 6.4 What should a robot learning dataset contain?
  - 6.5 Time alignment and dataset conversion
- 7. URDF, TF2, RViz, and ros2_control
  - 7.1 URDF: robot model description
  - 7.2 Relationship between URDF and TF2
  - 7.3 RViz 2: visualization
  - 7.4 ros2_control: connecting commands to hardware
- 8. Connecting ROS to Isaac Lab / MuJoCo RL
  - 8.1 Simulated training usually needs no ROS
  - 8.2 Real-world deployment needs integration
  - 8.3 Consistent semantics matter more than transport
  - 8.4 Frequency and latency
- 9. A Tron2 + Revo3 Dexterous Manipulation Architecture
  - 9.1 Suggested ROS node architecture
  - 9.2 Suggested workspace layout
  - 9.3 Splitting arm and hand actions
  - 9.4 Integrating FoundationPose
- 10. Common ROS Issues in Robot Learning
  - 10.1 ROS 2 and Conda environments
  - 10.2 Safety boundary for real hardware
- 11. A Recommended Learning Path
- 12. Summary and Practical Recommendations
- 13. Official Documentation and References

Chapter concept map



1. What Is ROS, and Why Use It in Robot Learning?

1.1 Definition of ROS

Despite its name, the Robot Operating System (ROS) is not an operating system in the sense of Ubuntu or Windows. It is a robotics software framework and ecosystem. Its central role is to allow separate robot software components to communicate, share information, and work together.

A robot may have cameras, a manipulator, a dexterous hand, an object pose estimator, a neural policy, and low-level controllers. The components can be written in different languages, run in different processes, or even live on different computers. ROS supplies a common messaging, communication, and tooling layer.

1.2 A typical robot learning system

Consider a Tron2 arm with a Revo3 hand performing a screwdriver pick-up and rotation task:

```
                Real Robot
                    |
        +-----------+-----------+
        |                       |
        v                       v
  RGB-D / RGB Cameras     Arm + Hand
        |                       |
        v                       v
     Images                Joint States
        +-----------+-----------+
                    v
                 ROS 2
                    v
          Observation Builder
                    v
                RL Policy
              (PPO / SAC)
                    v
                 Action
                    v
                 ROS 2
                    v
          Low-level Control / SDK
                    v
               Arm + Hand
                    +----> Next Observation
```

The surrounding software systems have different responsibilities:

Component	Primary role in robot learning
ROS 2	Inter-module communication, system integration, transforms, recording, debugging

Component	Primary role in robot learning
PyTorch	Neural-network training, loading, and inference
PPO / SAC	Reinforcement learning algorithms
Isaac Lab / MuJoCo	Physics simulation and RL training environments
ros2_control	Hardware abstraction, controller management, and command interfaces
MoveIt 2	Motion planning, kinematics, and collision checking
RViz 2	Visualization of robot state, sensors, and transforms
rosbag2	Recording and replaying ROS messages

The essential distinction is that ROS handles integration and communication, learning frameworks handle training and inference, and controllers execute the resulting commands. They complement rather than replace one another.

1.3 Is ROS always required for robot learning?

No. If a task only involves Python and a physics simulator, ROS may be unnecessary.

Scenario	ROS required?	Reason
Parallel PPO training in Isaac Lab	No	Direct simulator APIs reduce overhead
SAC training in MuJoCo	No	Python environment and algorithm can interact directly
Offline Diffusion Policy / VLA training	No	Dataset loader plus PyTorch is sufficient
Controlling a single arm through its SDK	Not necessarily	Direct vendor SDK calls may be sufficient
Multi-camera + arm + hand integration	Recommended	Simplifies modularity, monitoring, and synchronization
Teleoperation data collection	Recommended	Simplifies multimodal recording and replay
Real-robot deployment of RL policies	Recommended, not mandatory	Standardizes observation/action and device boundaries
Distributed or multi-robot systems	Recommended	Facilitates inter-process and inter-machine communication

> **Practical rule:** Keep the high-throughput simulated training loop outside ROS when possible. ROS is particularly useful around real-robot sensing, state estimation, data collection, device control, and deployment.

---

2. ROS 1 vs. ROS 2 and Choosing a Distribution

2.1 Main architectural differences

ROS 1 is the earlier ecosystem. ROS 2 was redesigned for contemporary robotics and offers better foundations for distributed discovery, QoS, lifecycle management, cross-platform support, and integration with real-time control systems.

Feature	ROS 1	ROS 2
Discovery	Typically depends on a ROS Master	Common DDS implementations support distributed discovery
Python API	`rospy`	`rclpy`
C++ API	`roscpp`	`rclcpp`
Bag recording	`rosbag`	`rosbag2`
Parameters	ROS Parameter Server	Parameters managed by individual nodes
QoS	More limited	Reliability, history, depth, deadlines, and more
Control framework	`ros_control`	`ros2_control`
Recommended for new projects	Generally no	Yes

Importantly, ROS 2 does **not** automatically make an application hard real-time. Determinism still depends on the OS, scheduling, hardware drivers, and system design.

2.2 Which ROS distribution should you use in 2026?

As of **October 2026**, ROS 2 **Lyrical Luth** has been released (May 22, 2026). The correct distribution nevertheless depends on the installed Ubuntu release and vendor-driver compatibility.

Ubuntu version	Common matching ROS 2 release	Support note
Ubuntu 22.04	Humble Hawksbill	LTS through May 2027
Ubuntu 24.04	**Jazzy Jalisco**	LTS through May 2029
Ubuntu 26.04	Lyrical Luth	LTS through May 2031

This tutorial targets Ubuntu 24.04 + ROS 2 Jazzy. If a hardware vendor only provides drivers for a specific ROS release, confirm driver compatibility before choosing a distribution.

---

3. Core ROS 2 Concepts

3.1 Node

A Node is a basic ROS computational unit: a software component with a clear responsibility. A node is not necessarily a separate OS process; multiple nodes can be composed into one process.

```
ROS 2 System
|-- camera_node          # Acquire images
|-- robot_state_node     # Read joint states
|-- object_pose_node     # Estimate object 6D pose
|-- rl_policy_node       # Run policy inference
`-- controller_node      # Execute commands
```

The separation allows a perception component such as FoundationPose to be updated without rewriting the arm driver.

3.2 Topic: publish / subscribe

Topics are used for continuous, asynchronous data streams:

```
camera_node --Publish--> /camera/image_raw --Subscribe--> perception_node
```

The publisher does not need to know its subscribers, and multiple subscribers may receive the same stream. Common robot learning topics include:

Example topic	Data
`/camera/color/image_raw`	RGB image
`/camera/depth/image_rect_raw`	Depth image
`/joint_states`	Robot joint state
`/object_pose`	Object pose
`/policy/action`	Action generated by the policy
`/tf`, `/tf_static`	Dynamic and static transforms

The main role of topics in robot learning is to carry data among perception, observation construction, the policy, and the controller. Actual topic names depend on the driver and namespace.

3.3 Message: data schema

A Message specifies the fields and types transmitted by a topic. Common messages include:

Message type	Purpose
`sensor_msgs/msg/Image`	Image
`sensor_msgs/msg/CameraInfo`	Camera calibration information and intrinsics
`sensor_msgs/msg/JointState`	Joint names, positions, velocities, and effort
`sensor_msgs/msg/Imu`	IMU measurements
`geometry_msgs/msg/Pose`	Position and orientation
`geometry_msgs/msg/PoseStamped`	Timestamped pose in a named frame
`geometry_msgs/msg/Twist`	Linear and angular velocity

Message type	Purpose
`trajectory_msgs/msg/JointTrajectory`	Joint trajectory
`std_msgs/msg/Float64MultiArray`	A basic float array

For example, `sensor_msgs/msg/JointState` includes:

```
header       # Timestamp and metadata
name         # Joint names, e.g. ['joint_1', 'joint_2']
position     # Joint positions
velocity     # Joint velocities
effort       # Joint torques or forces
```

Whether Revo3 exposes this standard message and the correct joint ordering depends on the actual driver or adapter. **Never assume that array indices have the same joint meaning across devices.**

3.4 Service: request / response

A Service is useful for one-off operations that should return a response:

```
policy_node --Reset Request--> reset_server
policy_node <--Reset Result----- reset_server
```

Typical uses include resetting a robot, querying device information, switching modes, or starting calibration. A service resembles a remote function call, but can still fail or time out.

3.5 Action: longer operations with progress feedback

An Action supports the workflow "submit goal -> receive progress -> get result". Examples include executing a manipulator trajectory, navigating to a goal, and running a multi-second grasp skill. Goals can be canceled.

Communication method	Best fit	Robot learning example
Topic	Continuous, asynchronous streaming	Images, joint state, policy actions
Service	One-off request and reply	Reset, query, mode switch
Action	Long operation with feedback and result	Execute a grasp or trajectory

For periodic closed-loop commands at, say, 50 Hz, one normally uses a topic or a direct in-process interface rather than creating a new Action every control tick.

3.6 Parameter: node configuration

Parameters configure nodes without changing their main code:

```
policy_node:
  ros__parameters:
    control_frequency: 50.0
    use_camera: true
```

```
    observation_dim: 32
    policy_path: "/home/robot/policy.pt"
```

Common settings include control frequency, camera names, model paths, reference frames, action scaling, and joint limits.

3.7 TF2: coordinate-frame management

A typical robot has many frames:

```
world
 `-- base_link
     |-- arm_base
     |    `-- ee_link
     |         `-- hand_base
     `-- head_link
          `-- camera_link
               `-- camera_optical_frame
```

Suppose FoundationPose estimates the pose of an object in the camera frame, `T(C,O)`. The manipulation policy needs the pose in the robot base frame:

Transform composition: `T(B,O) = T(B,C) x T(C,O)`

Here `B` means robot base, `C` means camera, and `O` means object. TF2 stores and queries transforms between frames, including time-dependent transforms.

TF2 does not calibrate a camera for you. The extrinsic transform `T(B,C)` must first come from URDF geometry, hand-eye calibration, or another valid calibration procedure. This is especially relevant when working with Tron2 head/wrist cameras and the offsets among yaw/pitch joints.

3.8 QoS: Quality of Service

QoS controls message delivery behavior:

Policy	Meaning
Reliability	`Reliable` vs. `Best Effort`
History / Depth	How messages are buffered
Durability	Whether retained data can be delivered to new subscribers
Deadline	Expected update interval and deadline monitoring

Camera streams often use `Best Effort` to avoid retransmitting stale frames. Some important state or command streams may use `Reliable`, but queues of outdated commands must still be prevented. Incompatible publisher/subscriber QoS can make a topic visible while delivering no data.

---

4. Installing ROS 2 Jazzy on Ubuntu 24.04

The commands below follow the official **Jazzy Ubuntu Debian package instructions**. Installation requires network access and administrator privileges. First check that another ROS installation will not conflict.

4.1 Check your system and configure repositories

```
lsb_release -a                # Verify Ubuntu 24.04
locale                        # Verify a UTF-8 locale

sudo apt update
sudo apt install software-properties-common curl -y
sudo add-apt-repository universe

export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F 'tag_name' | awk -F\" '{print $4}')

curl -L -o /tmp/ros2-apt-source.deb \
  "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-
source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"

sudo dpkg -i /tmp/ros2-apt-source.deb
```

If the system does not have a UTF-8 locale, see the official locale setup procedure, for example:

```
sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
```

4.2 Install ROS 2 and developer tools

```
sudo apt update
sudo apt install ros-jazzy-desktop ros-dev-tools
```

`ros-jazzy-desktop` includes RViz, demos, and common tools. For a non-GUI installation, `ros-jazzy-ros-base` is a smaller alternative.

4.3 Source the environment and verify

```
source /opt/ros/jazzy/setup.bash
echo $ROS_DISTRO              # Expected: jazzy
```

To source the environment automatically in new Bash sessions:

```
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
```

Avoid unintentionally sourcing multiple ROS distributions in the same shell.

4.4 First communication test

Terminal A:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

Terminal B:

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

Expected output includes messages similar to `Publishing: Hello World` and `I heard: Hello World`, verifying Python/C++ interoperability.

4.5 Essential command-line tools

```
ros2 node list
ros2 node info /camera_node
ros2 topic list
ros2 topic echo /joint_states
ros2 topic type /joint_states
ros2 topic hz /joint_states
ros2 topic bw /camera/color/image_raw
ros2 topic info -v /joint_states
ros2 interface show sensor_msgs/msg/JointState
ros2 service list
ros2 action list
ros2 run tf2_ros tf2_echo base_link camera_link
```

The node/topic/frame names are examples. During real-robot debugging, `ros2 topic list`, `echo`, `hz`, and `info -v` are especially valuable for verifying message content, update rates, and QoS compatibility.

---

5. Hands-on: Writing a Robot Learning Policy Node in Python

We will build a minimal reproducible pipeline that **does not command any real motors**:

```
/demo/joint_states
       | (JointState)
       v
    Policy Node
       | (Float64MultiArray)
       v
/demo/policy_target
```

5.1 Create a workspace

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

Typical structure:

```
ros2_ws/
|-- src/      # Source code
|-- build/    # Build artifacts (generated)
|-- install/  # Installed packages (generated)
`-- log/      # Build logs (generated)
```

5.2 Create a Python package

```
cd ~/ros2_ws/src
ros2 pkg create \
    --build-type ament_python \
    --license Apache-2.0 \
    --node-name demo_policy \
    robot_learning_demo
```

The generated files resemble:

```
robot_learning_demo/
|-- package.xml
|-- setup.py
|-- setup.cfg
|-- resource/
`-- robot_learning_demo/
    |-- __init__.py
    `-- demo_policy.py
```

`package.xml` declares dependencies; `setup.py` defines Python installation and executable entry points; `demo_policy.py` contains the policy node.

5.3 Implement the policy node

Edit `~/ros2_ws/src/robot_learning_demo/robot_learning_demo/demo_policy.py`:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from std_msgs.msg import Float64MultiArray


class PolicyNode(Node):
    def __init__(self):
        super().__init__("policy_node")

        # Subscribe: virtual robot observation
        self.subscription = self.create_subscription(
            JointState,
            "/demo/joint_states",
            self.state_callback,
            10,
        )

        # Publish: demonstration action (not for real hardware)
        self.publisher = self.create_publisher(
            Float64MultiArray,
            "/demo/policy_target",
            10,
        )
```

```
    def state_callback(self, msg):
        q = list(msg.position)
        if not q:
            return

        # An illustrative mapping, not a trained RL policy
        action = [0.5 * x for x in q]

        command = Float64MultiArray()
        command.data = action
        self.publisher.publish(command)


def main(args=None):
    rclpy.init(args=args)
    node = PolicyNode()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == "__main__":
    main()
```

The demonstration mapping is `a_t = 0.5 q_t`. It illustrates how observations become actions; it is **not** a trained or safe robot control policy. `Float64MultiArray` is chosen for simplicity; actual systems must define joint ordering, units, command semantics, limits, and timeout handling.

5.4 Declare dependencies and entry point

Add these dependencies to `package.xml` (do not duplicate existing entries):

```
<exec_depend>rclpy</exec_depend>
<exec_depend>sensor_msgs</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

Check that `setup.py` includes the following `console_scripts` entry:

```
'demo_policy = robot_learning_demo.demo_policy:main'
```

5.5 Build and run

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 run robot_learning_demo demo_policy
```

The node now waits for state messages. Ensure Jazzy is sourced before building with `colcon`.

5.6 Publish simulated state and inspect output

In a new terminal, source ROS:

```
source /opt/ros/jazzy/setup.bash
```

Publish a virtual two-joint state at 5 Hz:

```
ros2 topic pub -r 5 \
  /demo/joint_states \
  sensor_msgs/msg/JointState \
  "{name: ['j1', 'j2'], position: [0.4, -0.2]}"
```

In another terminal:

```
source /opt/ros/jazzy/setup.bash
ros2 topic echo /demo/policy_target
```

Expected output:

```
data:
- 0.2
- -0.1
```

You have completed the minimal pipeline: publish state -> subscribe -> compute an action -> publish the action.

5.7 Replace the toy mapping with a PPO / SAC policy

Once a trained neural policy is available, replace:

```
action = [0.5 * x for x in q]
```

with PyTorch inference (illustrative only):

```
with torch.inference_mode():
    action_tensor = policy(observation_tensor)
    action = action_tensor.cpu().numpy().tolist()
```

A real deployment must load model weights, construct the correct observation, normalize inputs, map joint order, scale actions, manage devices/dtypes, enforce limits, and handle failures.

It is often better to separate sensor callbacks from the fixed-frequency inference loop:

```
Sensor Callback -> Observation Buffer
                          |
                          v
                Policy Timer (e.g. 50 Hz)
                          |
                          v
                   Model Inference
                          |
                          v
                Safety Filter -> Publisher
```

50 Hz is illustrative. The proper rate depends on inference cost, driver capabilities, and the task. Running inference unconditionally inside every subscription callback can lead to irregular execution, queue buildup, and latency.

---

6. Recording Robot Learning Data with ROS 2

6.1 Teleoperation data collection architecture

For example, collecting demonstrations by teleoperating Tron2 + Revo3 with PICO:

```
Human Operator / PICO / Glove
          |
          v
      Teleoperation
          |
          v
   Robot Controller / SDK
          |
          v
       Real Robot
          |
      +---+-------------+
      v                 v
   RGB-D / RGB       Joint States
      |                 |
      +--------+--------+
               v
             ROS 2
               v
            rosbag2
               v
         Offline Dataset
               v
    BC / Diffusion Policy / VLA
```

6.2 What is rosbag2?

rosbag2 is the ROS 2 tool for recording and replaying topic messages and related metadata. Common storage backends include SQLite3 and MCAP. It is useful for reproducing data collection sessions and diagnosing sensing or algorithm behavior.

6.3 Record and replay a session

Assume that these topics exist:

```
/joint_states
/camera/color/image_raw
/camera/depth/image_rect_raw
/tf
/tf_static
```

Record a selected set of topics:

```
ros2 bag record -o demo_001 \
  /joint_states \
  /camera/color/image_raw \
  /camera/depth/image_rect_raw \
  /tf \
  /tf_static
```

Stop with `Ctrl+C`, then inspect or replay:

```
ros2 bag info demo_001
ros2 bag play demo_001
```

Safety warning: Replaying a bag can republish control topics if those topics were recorded. Isolate the real hardware control interface or disconnect actuation before replaying such commands.

6.4 What should a robot learning dataset contain?

An imitation learning dataset can be written as:

$D = \{ (o_t, a_t) \}_{t=0,\ldots,T}$

Recommended fields include:

Category	Examples
Images	Head RGB, wrist RGB, head depth
Robot state	Arm/hand joint positions and velocities, end-effector pose
Action	Issued control commands and target joint positions/velocities
Time	Original timestamps from each data source
Calibration	Camera intrinsics/extrinsics, TF, URDF version
Episode metadata	Task, start/end time, success/failure
Optional	Tactile, torques, motor current, object pose

A crucial distinction is:

- **Commanded action:** A target generated by teleoperation or a policy.
- **Executed state:** What onboard sensors indicate the robot actually did.

They may differ due to delays, saturation, controller limits, or contact and must not be treated as interchangeable.

6.5 Time alignment and dataset conversion

A typical pipeline:

```
ROS Bag
   |
   v
Read Topics
   |
```

```
   v
Timestamp Alignment
   |
   v
Episode Segmentation
   |
   v
Observation / Action Pairing
   |
   v
HDF5 / Parquet / LeRobot-compatible Dataset
   |
   v
Policy Training
```

Example rates (not specifications for Tron2 or Revo3):

Modality	Illustrative rate
RGB / depth cameras	30 Hz
Robot joint state	100 Hz
Tactile sensing	100 Hz
Policy action	50 Hz

Choose a reference timeline, interpolate states, match image frames, or construct observation histories as appropriate. **Do not synchronize by the order in which ROS messages arrive; use meaningful timestamps and account for clock offsets.**

---

7. URDF, TF2, RViz, and ros2_control

7.1 URDF: robot model description

URDF (Unified Robot Description Format) describes links, joints, geometry, and inertial properties. For example:

```
Tron2
 |-- Shoulder Joint
 |-- Elbow Joint
 |-- Wrist Joint
 `-- Revo3 Hand
     |-- Finger / Joint 1
     |-- Finger / Joint 2
     `-- ...
```

Common XML elements include:

```
<link>      <!-- Rigid-body link -->
<joint>     <!-- Joint and connectivity -->
<visual>    <!-- Display geometry -->
<collision> <!-- Collision geometry -->
<inertial>  <!-- Mass and inertia -->
```

URDF provides structure for visualization, kinematics, and conversion into simulation models, although simulation imports may need extra parameters and format conversion.

7.2 Relationship between URDF and TF2

URDF describes structure; TF2 manages coordinate transforms over time.

```
           URDF
             |
             v
robot_state_publisher <- /joint_states
             |
             v
          TF Tree
             |
             v
            RViz
```

For Tron2, this helps inspect the end effector, Revo3 hand base, head camera, and wrist camera. Incorrect URDF extrinsics will not be corrected by TF2 automatically.

7.3 RViz 2: visualization

Start RViz:

```
rviz2
```

RViz can display the robot model, TF tree, camera images, point clouds, joint state, pose markers, and trajectories. With FoundationPose, it is useful for checking where the estimated object pose lies relative to the robot base.

7.4 ros2_control: connecting commands to hardware

```
High-Level Policy
        |
        v
Controller (ros2_control)
        |
        v
Hardware Interface
        |
        v
Vendor SDK / Driver
        |
        v
Motor / Robot
```

A Hardware Interface maps low-level device read/write operations to the control framework. A Controller implements trajectory, position, velocity, or other command semantics.

Controller / concept	Purpose
Joint Trajectory Controller	Execute joint trajectories

Controller / concept	Purpose
Forward Command Controller	Forward command values to selected interfaces
Joint position/velocity command interfaces	Send commands as supported by configuration
Diff Drive Controller	Control a differential-drive mobile base

Install the packages:

```
sudo apt install \
  ros-jazzy-ros2-control \
  ros-jazzy-ros2-controllers
```

Once a compatible `controller_manager` is running:

```
ros2 control list_controllers
```

Installing ros2_control alone does not provide a Tron2/Revo3 driver. You still need a compatible Hardware Interface or your own vendor-SDK adapter. A standalone ROS driver node without ros2_control is also possible.

---

8. Connecting ROS to Isaac Lab / MuJoCo RL

8.1 Simulated training usually needs no ROS

PPO/SAC training in Isaac Lab or MuJoCo often calls the environment API directly:

```
observations = env.reset()
for step in range(num_steps):
    actions = policy(observations)
    observations, rewards, dones, info = env.step(actions)
```

This is conceptual pseudocode, not an exact API signature for every Gymnasium or Isaac Lab version. GPU-parallel environments require high throughput; using a ROS topic at every simulation step usually adds unnecessary serialization and scheduling costs.

8.2 Real-world deployment needs integration

```
Isaac Lab / MuJoCo
        |
        v
  PPO / SAC Training
        |
        v
   Exported Policy
        |
        v
  ROS 2 Policy Node
        |
        v
```

```
  Safety / Controller
        |
        v
    Real Robot
```

Typical execution architecture:

```
Camera --------+
Joint States --+--> Observation Builder --> RL Policy
Object Pose ---+                                |
                                                v
                                          Safety Filter
                                                |
                                                v
                                          Robot Controller
                                                |
                                                v
                                            Real Robot
```

8.3 Consistent semantics matter more than transport

Item	Training	Real-robot requirement
Joint order	Simulator model order	Explicit mapping to driver order
Joint units	Often radians	Check radians, degrees, or vendor-specific units
Observation	Normalized state	Identical normalization and concatenation order
Action	Position, delta, velocity, etc.	Match the hardware control mode
Frequency	Simulation timestep / decimation	Define policy and control periods
Frames	Simulation world/base	Define corresponding real-world frames

ROS standardizes communication but does not automatically fix joint ordering, camera calibration, normalization, dynamics mismatch, or latency.

8.4 Frequency and latency

Distinguish three clocks:

- `f_camera`: sensor acquisition rate.
- `f_policy`: neural-policy inference/update rate.
- `f_control`: low-level controller update rate.

They need not be equal. For example, a 30 Hz camera, 50 Hz policy, and faster servo loop is a conceivable design, not a hardware requirement.

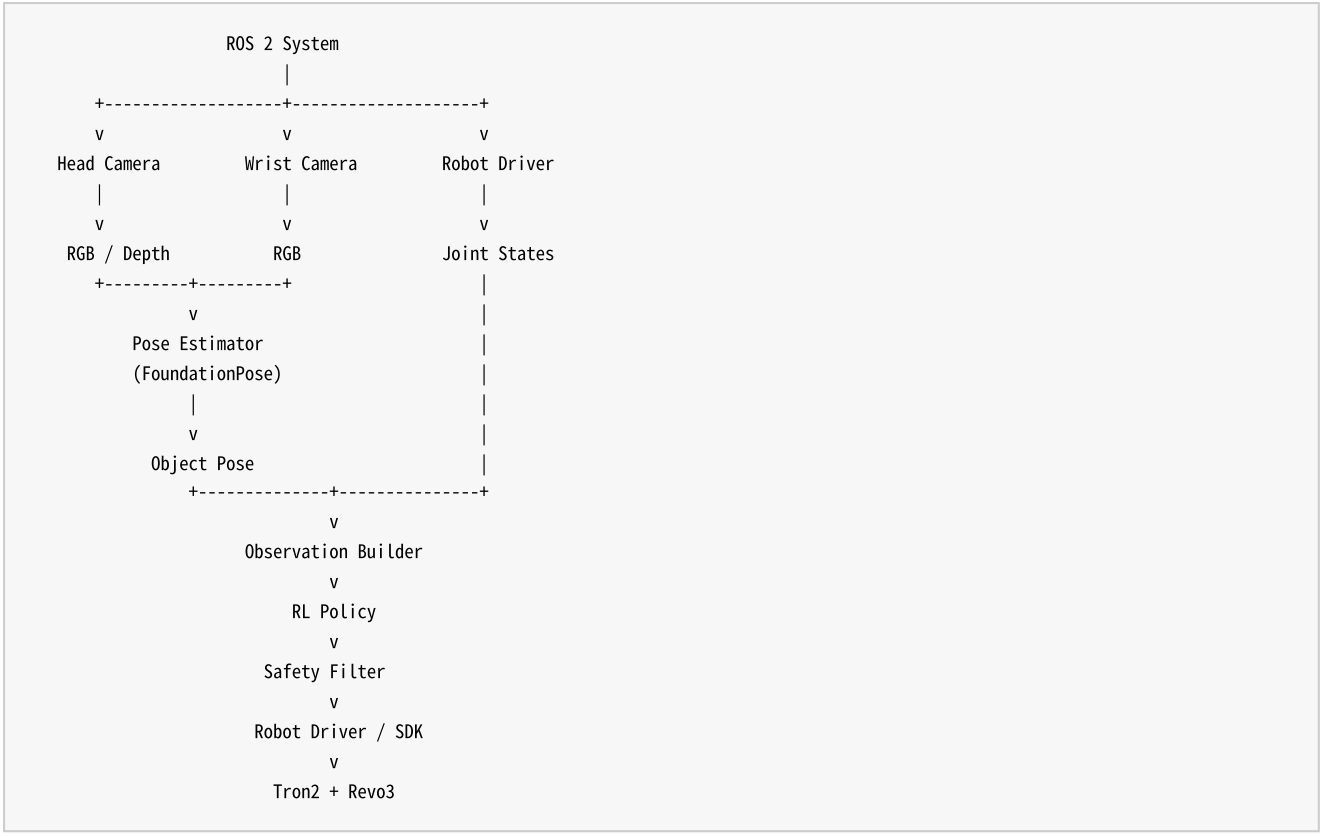
Python ROS 2 nodes do not automatically provide hard real-time execution. For robust control, separate GPU inference from servo control, place timing-critical tasks in appropriate C++/real-time threads, avoid blocking in the control path, and implement command timeouts, safety limits, and an emergency stop.

---

9. A Tron2 + Revo3 Dexterous Manipulation Architecture

Target task: **pick a screwdriver up from a table, carry it to a specified location, and rotate it.** The proposed system contains Tron2, BrainCo Revo3, a head RGB-D camera, a wrist RGB camera, FoundationPose, and a PPO/SAC policy.

9.1 Suggested ROS node architecture



9.2 Suggested workspace layout

```
robot_learning_ws/
`-- src/
    |-- tron2_driver/
    |-- revo3_driver/
    |-- camera_driver/
    |-- robot_description/
    |-- pose_estimation/
    |-- observation_builder/
    |-- rl_policy/
    |-- safety_controller/
    |-- teleoperation/
    `-- data_recorder/
```

This is a proposed engineering layout, **not** a claim that the vendors distribute packages with these names. If a vendor only provides a Python/C++ SDK, wrap it in a ROS driver or adapter.

9.3 Splitting arm and hand actions

A joint policy may produce:

```
a_t = [ a_t^arm, a_t^hand ]
```

Here the arm action dimension is `n_a`, and the hand action dimension is `n_h`. Actions can then be routed as:

```
          RL Policy
              |
              v
            Action
           +--+--+
           v     v
        Arm Cmd Hand Cmd
           v     v
       Tron2 SDK Revo3 SDK
```

The design could use separate topics for the two subsystems or a unified Robot Action message with a dispatcher. Either approach must specify time alignment, joint ordering, command semantics, and failure handling across the arm and hand.

9.4 Integrating FoundationPose

```
Head RGB-D / Wrist RGB
          |
          v
     FoundationPose
          |
          v
      /object_pose
          |
          v
     TF Transform
          |
          v
Object Pose in Robot Base
          |
          v
       RL Policy
```

The pose estimation node can be developed independently of the controller, provided its `PoseStamped` timestamps and `frame_id` are accurate. Real implementations should also handle occlusion, tracking loss/reinitialization, and temporal filtering.

---

10. Common ROS Issues in Robot Learning

Symptom	Possible cause	What to check
`ros2 topic list` cannot see hardware	Driver not running, discovery or domain issue	Nodes, DDS, `ROS_DOMAIN_ID`
Topic exists but no messages arrive	Incompatible QoS or message types	`ros2 topic info -v`
Camera and joint states are misaligned	Different rates or clock domains	Timestamps, hardware sync, buffering

Symptom	Possible cause	What to check
Object pose is biased	Calibration, TF chain, or frame mismatch	Extrinsics, optical frame convention
RL commands behave incorrectly	Joint order, normalization, or command mode mismatch	Compare training/deployment observation and action
End-to-end latency is high	Sensor, model, transport, SDK, controller delay	Measure each segment and queue occupancy
ROS conflicts with Conda	System Python vs. Conda ABI/dependency mismatch	Compatible environment or separate processes
Bag recordings are huge	Multiple high-bandwidth RGB-D streams	Compression, MCAP/storage settings, splitting
RViz robot model is wrong	URDF, JointState, or TF inconsistency	Inspect Robot Model and TF step by step

10.1 ROS 2 and Conda environments

Robot learning workflows commonly use Conda for PyTorch, CUDA, Isaac Lab, and RL libraries, while the apt-installed `rclpy` is tied to the system Python environment. Running `import rclpy` inside an arbitrary Conda environment may trigger ABI or shared-library problems.

A robust architecture is:

```
  ROS 2 Driver Process
          |
      Observation
          v
PyTorch Inference Process
      (Conda / CUDA)
          |
        Action
          v
  ROS 2 Control Process
```

Processes can communicate through ROS topics, dedicated IPC, or shared memory, rather than requiring every dependency in the same Python environment.

10.2 Safety boundary for real hardware

A learned policy output is not a trusted low-level motor command. Real deployments need joint limits, velocity/acceleration bounds, command timeouts, stopping and emergency-stop mechanisms, disconnection fallback behavior, and initial testing at low speed without payload or contact. ROS is not a replacement for a hardware safety system.

---

11. A Recommended Learning Path

Stage	Topics	Demonstrable milestone
1	Nodes, topics, messages	Inspect and explain the robot's topics

Stage	Topics	Demonstrable milestone
2	`rclpy`, publishers, subscribers	Run a custom Python policy node
3	rosbag2	Record and replay a complete episode
4	URDF, TF2, RViz	Verify robot and camera coordinate frames
5	Robot SDK, ros2_control	Safely read state and send test targets
6	PyTorch + ROS	Deploy PPO/SAC inference on a robot
7	QoS, synchronization, latency	Quantify sensor-to-action end-to-end latency

For researchers already comfortable with Python, control, and robot learning, the most efficient path is to learn what is needed for **data collection** -> **policy deployment** -> **reproducible experiments**, rather than covering every ROS subsystem in isolation.

---

12. Summary and Practical Recommendations

The value of ROS 2 in robot learning can be grouped into three areas:

Area	ROS responsibilities	Typical application
Data collection	Gather and record camera, state, and teleoperation streams	BC, Diffusion Policy, VLA
Policy deployment	Connect observations, policy inference, and controllers	PPO, SAC, Sim2Real
System integration	Frame transforms, device communication, visualization, diagnostics	Dexterous manipulation, visual feedback, real-robot experiments

For a **Tron2** + **Revo3** research platform, prioritize five skills:

1. **Topics** + **rclpy:** Connect state and camera observations to the RL policy.
2. **rosbag2:** Record teleoperation demonstrations for offline datasets.
3. **TF2** + **URDF:** Maintain the head-camera, wrist-camera, end-effector, and object frames.
4. **Robot SDK / Hardware Interface:** Control Tron2 and Revo3 reliably.
5. **Sim2Real deployment:** Transfer policies trained in Isaac Lab / MuJoCo to a physical closed-loop system.

The goal is not to put more ROS into training. It is to build an observable, reproducible, maintainable sensor -> policy -> controller -> robot pipeline.

---

13. Official Documentation and References

1. ROS 2 Jazzy Tutorials
2. ROS 2 Jazzy Ubuntu Debian Package Installation

3. Python Publisher / Subscriber Tutorial
4. ROS 2 TF2 Tutorials
5. ros2_control Documentation
6. MoveIt 2 Documentation
7. rosbag2 GitHub Repository
8. ROS 2 Distribution Support Information
9. ROS 2 Lyrical Release Records

Document note: This tutorial is educational and intended for system design. The sample action topic is for simulated testing only. All real-robot integrations must be checked against vendor documentation and safety requirements.

Camera Calibration for Robot Learning

Course home · 简体中文 · PDF

Item	Details
Applicability	OpenCV 4.x concepts; ROS 2 Jazzy; device SDK version must be selected separately
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

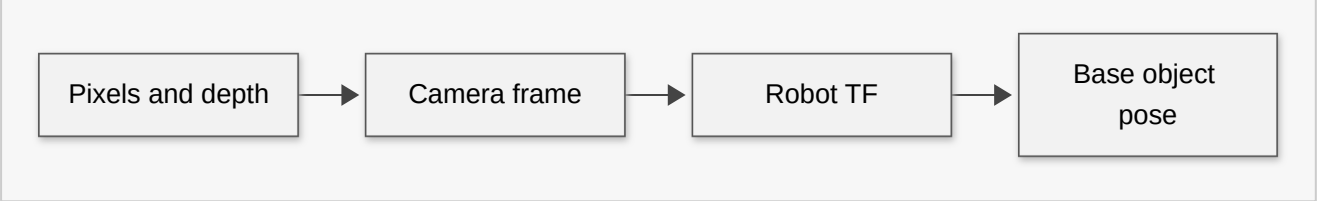
Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Types of Camera Calibration and Their Uses
- 2. Essential Mathematics and Coordinate Frames
  - 2.1 Frame definitions
  - 2.2 Robot frames versus optical camera frames
- 3. Intrinsic Calibration
  - 3.1 Intrinsic matrix and distortion
  - 3.2 Why intrinsics matter
  - 3.3 Choosing a calibration target
  - 3.4 Practical intrinsic-calibration procedure
- 4. Extrinsic Calibration
  - 4.1 What an extrinsic transform means
  - 4.2 Known target pose: PnP followed by transform composition
- 5. Hand-Eye Calibration
  - 5.1 The two standard configurations
  - 5.2 Eye-in-hand: calibrating the wrist camera
  - 5.3 Eye-to-hand: fixed camera with a board on the robot
- 6. Specific Considerations for Tron2 Head and Wrist Cameras
  - 6.1 Camera mounted on a moving head
  - 6.2 Wrist-mounted camera
  - 6.3 Relative transform between head and wrist cameras
- 7. RGB-D and Depth-Camera Calibration
  - 7.1 Internal camera frames
  - 7.2 From an RGB pixel to a point in the robot base frame
  - 7.3 Should factory depth calibration be redone?

- 8. Complete OpenCV Hand-Eye Example
  - 8.1 Data conventions
  - 8.2 Code supporting both configurations
  - 8.3 Validating the output
- 9. Camera Calibration in ROS 2
  - 9.1 Useful tools
  - 9.2 Intrinsic calibration in ROS 2
  - 9.3 Hand-eye calibration in ROS 2
- 10. Using Calibration Results in ROS TF2 and URDF
  - 10.1 Recommended TF structure
  - 10.2 Static extrinsic publishers
  - 10.3 Dynamic transform checks
- 11. How Calibration Is Used in Robot Learning
  - 11.1 6D pose estimation and visually guided grasping
  - 11.2 Imitation learning and VLAs
  - 11.3 Reinforcement learning and sim-to-real
  - 11.4 Visual feedback and synchronization
- 12. Accuracy Metrics and Calibration Validation
  - 12.1 Reprojection error
  - 12.2 3D localization error
  - 12.3 Grasp-level validation
- 13. Common Failure Modes and Troubleshooting
- 14. Recommended Open-Source Tools and References
- 15. Recommended Tron2 + Revo3 Implementation Plan
  - Phase 1: Intrinsics and depth verification
  - Phase 2: Head camera calibration
  - Phase 3: Wrist camera and tool center point
  - Phase 4: Visual control and learning deployment
- 16. Summary and Deployment Checklist

Chapter concept map



1. Types of Camera Calibration and Their Uses

In robot learning, *camera calibration* is broader than estimating intrinsics. It also involves the geometric and temporal relationships among cameras, robots, and the workspace.

Type	Main purpose	Typical applications
Intrinsic calibration	Estimate imaging geometry and lens distortion	Projection, PnP, 6D pose
Extrinsic calibration	Estimate a camera pose relative to a reference frame	Grasping, spatial localization
Hand-eye calibration	Estimate a rigid transform between a camera and robot end effector or base	Wrist and fixed cameras
Multi-camera calibration	Estimate relative transforms between cameras	Multi-view perception, fusion
RGB-D/depth calibration	Verify color-depth alignment and depth scale	Point clouds, 3D localization
Temporal calibration	Align image timestamps with robot/sensor measurements	Tracking, feedback control

Closely related topics are **robot kinematic calibration** and **camera-IMU calibration**.

Important distinction: A RealSense device usually comes with factory intrinsics, but installing it on Tron2 still requires determining its pose relative to the robot. If the camera is mounted on a moving head, its transform relative to the base varies with head-joint angles.

2. Essential Mathematics and Coordinate Frames

2.1 Frame definitions

Symbol	Frame	Meaning
W	World	Global or simulation world
B	Robot base	Base frame of the robot
G	Gripper/end effector	Robot terminal link frame
C	Camera optical	Optical coordinate frame
O	Object	Object-centered frame
T	Calibration target	Board or fiducial frame

We use an explicit transform convention throughout:

$$ {}^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}t_{B} \\ 0 & 1 \end{bmatrix}. $$

The transform ${}^{A}T_{B}$ maps the coordinates of a point **expressed in frame B into frame A**. Here $R \in SO(3)$, $t \in \mathbb{R}^3$, and $T \in SE(3)$.

In homogeneous coordinates,

$$ \bar{p}_A = {}^{A}T_{B}\bar{p}_B. $$

Its inverse and composition obey

$$ {}^{B}T_{A} = ({}^{A}T_{B})^{-1}, \qquad {}^{A}T_{C} = {}^{A}T_{B}\,{}^{B}T_{C}. $$

For example, if FoundationPose estimates an object pose ${}^{C}T_{O}$ in the camera frame, a base-frame controller needs

$$ \boxed{{}^{B}T_{O} = {}^{B}T_{C}\,{}^{C}T_{O}.} $$

One of the principal objectives of calibration is therefore to obtain an accurate ${}^{B}T_{C}$.

2.2 Robot frames versus optical camera frames

The typical ROS REP 103 conventions are:

Axis	Robot body frame	Camera optical frame
X	Forward	Right in the image
Y	Left	Down in the image
Z	Up	Forward along the optical axis

Consequently, `camera_link` and `camera_optical_frame` may be different frames connected by a fixed rotation. Do not interpret optical coordinates returned by OpenCV as robot body-frame coordinates without checking the transform. See ROS REP 103.

3. Intrinsic Calibration

3.1 Intrinsic matrix and distortion

For an ideal pinhole camera,

$$ K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, $$

where f_x, f_y are focal lengths measured in pixels and c_x, c_y define the principal point. For an undistorted camera-frame point (X, Y, Z),

$$ u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y. $$

Real lenses introduce distortion. One common OpenCV model uses

$$ D = [k_1, k_2, p_1, p_2, k_3] $$

for radial and tangential distortion; other models are appropriate for fisheye and specialized lenses.

3.2 Why intrinsics matter

Use 1: Projecting 3D points into images. Given a point $p_C = (X, Y, Z)$, camera intrinsics determine its image pixel (u, v). This is needed for visualization, reprojection, visual servoing, and synthetic-real image alignment.

Use 2: RGB-D back-projection. If an undistorted image pixel (u, v) has a known **optical-axis depth** Z, then

$$\boxed{p_C = \begin{bmatrix} (u - c_x)Z/f_x \\ (v - c_y)Z/f_y \\ Z \end{bmatrix}.}$$

For distorted pixels, undistort first or use a correct camera-model deprojection function. After aligning depth to color, use the camera model appropriate to the aligned representation.

Use 3: 6D object-pose estimation. PnP, FoundationPose, MegaPose, and CAD-model tracking typically use intrinsics. An ordinary RGB pixel by itself determines a ray, not the absolute distance of a point along that ray.

3.3 Choosing a calibration target

Target	Advantages	Limitations	Typical use
Checkerboard	Accurate corners; inexpensive	Corners must be visible	Basic intrinsics
ArUco / AprilTag	Easy identification; markers have IDs	Single-marker corner accuracy varies	PnP and target tracking
ChArUco	Accurate chessboard-like corners plus marker IDs; partial occlusion support	Requires accurate printing and corner indexing	Precise intrinsics and hand-eye calibration

For ordinary robot labs, **ChArUco** is a good starting point, although a checkerboard is also effective. See the OpenCV ChArUco detection tutorial.

3.4 Practical intrinsic-calibration procedure

1. **Prepare a rigid board.** For example, a 7-by-5-square ChArUco board with 30 mm squares and 20 mm tags. These are illustrative dimensions, not a requirement. Print at true scale on a flat backing.
2. **Capture diverse views.** Start with roughly 20–40 sharp, useful images. Move the board across the center, corners, distances, tilt angles, and apparent scales. Keep important camera settings controlled.
3. **Detect corresponding corners.** Use `cv2.findChessboardCornersSB()` or a ChArUco detector. Verify exact 2D–3D point correspondence.
4. **Optimize intrinsics and distortion.** Minimize reprojection residuals across images:

$$\min_{K, D, \{R_j, t_j\}} \sum_{j,i} \|p_{ij} - \pi(K, D, R_j P_i + t_j)\|_2^2 .$$

5. **Validate with held-out views.** Inspect RMS reprojection error, residuals across the image, and straight lines after undistortion. An RMS around 0.3–0.5 pixels can be a useful reference for high-quality captures; it is **not** a universal acceptance threshold, nor a measure of millimeter-scale robot accuracy.

4. Extrinsic Calibration

4.1 What an extrinsic transform means

Intrinsics answer “how does a 3D point project into the image?” Extrinsics answer “where is the camera with respect to a reference frame?” For a head-mounted Tron2 camera, a common objective is

$$
{}^{B}T_{C} = \begin{bmatrix} {}^{B}R_{C} & {}^{B}t_{C} \\ 0 & 1 \end{bmatrix}.
$$

4.2 Known target pose: PnP followed by transform composition

Suppose a calibration board has a precisely known base-frame pose ${}^{B}T_{T}$. PnP estimates the **target-to-camera** transform ${}^{C}T_{T}$ using intrinsics and image corners. Then

$$
{}^{B}T_{T} = {}^{B}T_{C}\,{}^{C}T_{T},
$$

which gives

$$
\boxed{{}^{B}T_{C} = {}^{B}T_{T}({}^{C}T_{T})^{-1}.}
$$

Procedure: Fix the board; measure or touch-probe ${}^{B}T_{T}$; acquire a clear image; detect corresponding corners; run PnP; compute ${}^{B}T_{C}$; validate at other known target positions.

```
import cv2
import numpy as np

# N-by-3 board corners in meters
object_points = np.asarray(..., dtype=np.float64)
# Corresponding N-by-2 image corners in pixels
image_points = np.asarray(..., dtype=np.float64)
K = np.asarray(..., dtype=np.float64)
dist = np.asarray(..., dtype=np.float64)

ok, rvec, tvec = cv2.solvePnP(
    object_points, image_points, K, dist
)
if not ok:
    raise RuntimeError("PnP failed")

R, _ = cv2.Rodrigues(rvec)
T_C_T = np.eye(4)
T_C_T[:3, :3] = R
T_C_T[:3, 3] = tvec.reshape(3)

# Must come from an independent measurement
T_B_T = np.asarray(..., dtype=np.float64)
T_B_C = T_B_T @ np.linalg.inv(T_C_T)
print(T_B_C)
```

Direction warning: `solvePnP` returns ${}^{C}T_{T}$, not its inverse.

Limitation: If the board is placed arbitrarily and its base-frame pose is unknown, one image normally cannot determine ${}^{B}T_{C}$. This motivates hand-eye calibration.

5. Hand-Eye Calibration

5.1 The two standard configurations

Property	Eye-in-hand	Eye-to-hand
Camera	Rigidly mounted on the moving end effector	Fixed to the base or workspace
Target board	Usually fixed in the environment	Usually fixed to the end effector
Typical unknown	${}^{G}T_{C}$	${}^{B}T_{C}$
Example	Wrist camera	Static overhead camera

A Tron2 head camera that moves with yaw/pitch is **not a truly static eye-to-hand camera**. It can only be modeled as static while the head joints remain fixed.

5.2 Eye-in-hand: calibrating the wrist camera

The goal is to estimate the constant ${}^{G}T_{C}$. Keep the board stationary while the robot visits several poses. At each pose record:

- End-effector pose from forward kinematics or TF: ${}^{B}T_{G,i}$.
- Calibration-board pose estimated by PnP: ${}^{C}T_{T,i}$.

Because the board does not move,

$$ {}^{B}T_{G,i}\,{}^{G}T_{C}\,{}^{C}T_{T,i} = {}^{B}T_{T}. $$

Pairs of observations yield the classical $AX = XB$ problem, with $X = {}^{G}T_{C}$. Robot motion must contain **diverse, nonparallel rotation axes**. Pure translations or repeated rotation about a single axis are generally inadequate.

Practical dataset: Begin with about 20–30 valid poses with varied orientations, distances, and image locations. Keep the board visible, ensure image-state alignment, and reject blurred or badly detected frames. Motion diversity is usually more important than simply increasing sample count.

OpenCV implements several solvers, including Tsai, Park, Horaud, Andreff, and Daniilidis. See `calibrateHandEye`.

5.3 Eye-to-hand: fixed camera with a board on the robot

Here the camera is static, and the board is rigidly mounted on the end effector. Each observation satisfies

$$ {}^{B}T_{G,i}\,{}^{G}T_{T} = {}^{B}T_{C}\,{}^{C}T_{T,i}. $$

Both the base-to-camera transform ${}^{B}T_{C}$ and the gripper-to-target transform ${}^{G}T_{T}$ are fixed unknowns. Multiple robot poses can determine ${}^{B}T_{C}$ without knowing an external board location.

Critical convention: When using OpenCV `cv2.calibrateHandEye()` for this layout, provide ${}^{G}T_{B} = ({}^{B}T_{G})^{-1}$ as the *robot* pose input, while continuing to supply ${}^{C}T_{T}$ as the target pose. The output is then

interpreted as ${}^{B}T_{C}$. Reusing the eye-in-hand inputs unchanged can give a plausible-looking but incorrect transform.

6. Specific Considerations for Tron2 Head and Wrist Cameras

6.1 Camera mounted on a moving head

Let H denote the terminal link frame of the head kinematic chain, C_h the head optical frame, and q_h the head-joint vector. The camera pose is

$$\boxed{{}^{B}T_{C_h}(q_h) = {}^{B}T_{H}(q_h)\,{}^{H}T_{C_h}.}$$

- ${}^{B}T_{H}(q_h)$ is a **dynamic** transform from head forward kinematics.
- ${}^{H}T_{C_h}$ is a **fixed** camera-mount transform to be calibrated or measured.

Suggested procedure: Fix a ChArUco board in the workspace. Capture images across different yaw/pitch poses and record their joint values. Compute ${}^{B}T_{H}$ from FK and ${}^{C_h}T_{T}$ from PnP. Solve for ${}^{H}T_{C_h}$ using the eye-in-hand formulation, then test on held-out head poses.

What if the yaw and pitch axes do not intersect?

A naive expression $R_z(q_y)R_y(q_p)$ omits the offset between rotation axes. Use the full kinematic chain:

$${}^{B}T_{C_h} = {}^{B}T_{Y}\,T_{Y}(q_y)\,{}^{Y}T_{P}\,T_{P}(q_p)\,{}^{P}T_{C_h}.$$

The fixed transform ${}^{Y}T_{P}$ contains both rotation and translation between yaw and pitch frames. First inspect the `origin` and `axis` entries in the URDF. If the actual axis geometry differs from the kinematic model, correcting only the camera-mount extrinsic will **not necessarily eliminate** posture-dependent errors.

For an initial screwdriver-grasping experiment with a fixed head viewpoint, calibrating a single base-to-camera pose is a reasonable starting point. Add full dynamic head modeling when active vision is needed.

6.2 Wrist-mounted camera

Let C_w be the wrist camera and q_a the arm configuration:

$$\boxed{{}^{B}T_{C_w}(q_a) = {}^{B}T_{G}(q_a)\,{}^{G}T_{C_w}.}$$

The first term is obtained from FK; the second is estimated through eye-in-hand calibration. If the camera is mounted on `wrist_link` rather than the precise terminal frame used for FK, consistently compose the corresponding link transforms.

What changes when installing the Revo3 hand?

Keep the following transforms distinct:

$${}^{B}T_{G}, \quad {}^{G}T_{C_w}, \quad {}^{G}T_{\mathrm{TCP}}.$$

They describe the end-effector pose, wrist-camera mount, and tool-center-point (TCP) mount, respectively. If changing the hand does **not** move the camera relative to the same gripper frame, the prior camera extrinsic can remain valid. However, the TCP and fingertip geometry may change. Recheck calibration if the camera bracket moved, the flange/gripper frame was redefined, or the hardware became loose.

6.3 Relative transform between head and wrist cameras

Once each camera's base-frame pose is known,

$$\boxed{{}^{C_h}T_{C_w} = ({}^{B}T_{C_h})^{-1}\,{}^{B}T_{C_w}.}$$

A separate two-camera calibration is not always necessary, although observing the same target from both cameras is an excellent consistency check. Since the head and arm move, the relative camera-to-camera transform is generally **time-varying**.

7. RGB-D and Depth-Camera Calibration

7.1 Internal camera frames

Devices such as RealSense D435 and D455 often expose color, infrared, and depth camera frames. Their optical centers usually differ, requiring a known rigid transform such as

$${}^{C_{\text{rgb}}}T_{C_{\text{depth}}}.$$

The RealSense SDK can normally provide factory intrinsics, extrinsics, and depth scale. Begin by **reading and validating** these values before attempting new device-level calibration.

7.2 From an RGB pixel to a point in the robot base frame

The key pipeline is:

```
RGB image
   -> Detect pixel (u, v)
   -> Align depth to RGB
   -> Read depth Z
   -> Deproject using intrinsics
   -> Point p_C in optical camera frame
   -> Transform using T_B_C
   -> Point p_B in robot base frame
```

A minimal RealSense example is:

```
import pyrealsense2 as rs
import numpy as np

pipeline = rs.pipeline()
config = rs.config()
config.enable_stream(rs.stream.depth)
config.enable_stream(rs.stream.color)
pipeline.start(config)

align = rs.align(rs.stream.color)
```

```
try:
    frames = pipeline.wait_for_frames()
    aligned = align.process(frames)

    depth = aligned.get_depth_frame()
    color = aligned.get_color_frame()
    if not depth or not color:
        raise RuntimeError("Missing RGB-D frame")

    u, v = 320, 240  # Example: check image bounds
    z = depth.get_distance(u, v)  # Meters
    if z <= 0:
        raise RuntimeError("Invalid depth")

    intr = depth.profile.as_video_stream_profile().get_intrinsics()
    point_C = rs.rs2_deproject_pixel_to_point(
        intr, [float(u), float(v)], z
    )
    print("Point in camera optical frame:", point_C)
finally:
    pipeline.stop()
```

Given a separately calibrated camera-to-base transform,

```
point_C_h = np.r_[np.asarray(point_C), 1.0]
point_B_h = T_B_C @ point_C_h
print("Point in base frame:", point_B_h[:3])
```

This implements "RGB pixel → 3D camera point → 3D robot-base point."

7.3 Should factory depth calibration be redone?

Not automatically. First verify device calibration at the intended operating range:

Item	Purpose
RGB intrinsics	Accurate color-image projection
Depth intrinsics	Accurate depth deprojection
RGB-depth extrinsics	Correct spatial alignment
Depth scale	Correct metric units
Depth accuracy near the target	Reliable working-distance estimation
Camera-to-robot extrinsics	Correct conversion into the robot base

Thin and reflective objects, including screwdrivers, can cause missing depth, incorrect boundaries, and reflection-related failures. Such errors are not always fixed by re-estimating RGB intrinsics. See the RealSense SDK repository.

8. Complete OpenCV Hand-Eye Example

8.1 Data conventions

For each of N sampled configurations, store two time-matched 4-by-4 transforms:

```
base_T_gripper.npy   shape = (N, 4, 4)
camera_T_target.npy  shape = (N, 4, 4)
```

These are ${}^{B}T_{G,i}$ and ${}^{C}T_{T,i}$. Use meters for translations. Rotation blocks must be orthonormal with determinants near one, and each last row must be `[0, 0, 0, 1]`.

8.2 Code supporting both configurations

```
import cv2
import numpy as np

# ^B T_G : gripper -> robot base
T_B_G = np.load("base_T_gripper.npy")
# ^C T_T : calibration target -> camera optical frame
T_C_T = np.load("camera_T_target.npy")

assert T_B_G.shape == T_C_T.shape
assert T_B_G.ndim == 3 and T_B_G.shape[1:] == (4, 4)
assert len(T_B_G) >= 3

mode = "eye_in_hand"  # or "eye_to_hand"

if mode == "eye_in_hand":
    robot_poses = T_B_G
elif mode == "eye_to_hand":
    robot_poses = np.linalg.inv(T_B_G)
else:
    raise ValueError("Unknown calibration mode")

R_robot = [T[:3, :3] for T in robot_poses]
t_robot = [T[:3, 3] for T in robot_poses]
R_target = [T[:3, :3] for T in T_C_T]
t_target = [T[:3, 3] for T in T_C_T]

R, t = cv2.calibrateHandEye(
    R_robot, t_robot, R_target, t_target,
    method=cv2.CALIB_HAND_EYE_PARK
)

X = np.eye(4)
X[:3, :3] = R
X[:3, 3] = t.reshape(3)

if mode == "eye_in_hand":
    print("^G T_C (gripper <- camera):")
else:
    print("^B T_C (base <- camera):")
print(X)
np.save("handeye_result.npy", X)
```

The example illustrates correct API conventions. For real deployment, reject outliers, ensure sufficiently rich rotational motion, and validate on held-out poses. Confirm that `camera_T_target` refers to the **optical** camera frame, not an ambiguous `camera_link`.

8.3 Validating the output

For eye-in-hand, compute for each observation

$$\hat{T}_{BT,i} = {}^{B}T_{G,i}\,{}^{G}T_{C}\,{}^{C}T_{T,i}.$$

The estimated board pose must remain nearly constant across robot motions.

For eye-to-hand, compute

$$\hat{T}_{GT,i} = ({}^{B}T_{G,i})^{-1}\,{}^{B}T_{C}\,{}^{C}T_{T,i}.$$

This estimated board-to-gripper transform should likewise be constant. Report translational and angular residuals separately, and test with robot poses excluded from calibration.

9. Camera Calibration in ROS 2

9.1 Useful tools

Tool	Main use
`camera_calibration`	Checkerboard-based intrinsic calibration
`image_pipeline`	Image undistortion and rectification
`tf2`	Coordinate-transform management
`robot_state_publisher`	Publish robot frames from URDF and joint states
`easy_handeye2`	ROS 2 hand-eye workflow
MoveIt Calibration	GUI hand-eye workflow
Kalibr	Multi-camera and camera-IMU spatiotemporal calibration

9.2 Intrinsic calibration in ROS 2

On a compatible ROS 2 distribution:

```
sudo apt install \
  ros-${ROS_DISTRO}-camera-calibration \
  ros-${ROS_DISTRO}-image-pipeline

ros2 topic list
```

If images and camera information are published as `/camera/image_raw` and `/camera/camera_info`, a checkerboard workflow can be launched with:

```
ros2 run camera_calibration cameracalibrator \
  --size 8x6 \
  --square 0.025 \
  image:=/camera/image_raw \
  camera:=/camera
```

`8x6` is the number of **internal corners**; `0.025` is the physical square width in meters. Both must match the board. Actual topics depend on the camera driver. See ROS image_pipeline.

9.3 Hand-eye calibration in ROS 2

Useful projects include easy_handeye2 and MoveIt Calibration. A typical workflow is:

1. Publish end-effector poses in TF.
2. Detect the board and publish its observed pose.
3. Collect observations across diverse robot poses.
4. Select eye-in-hand or eye-to-hand.
5. Compute, validate, and save the estimated transform.
6. Integrate the result into URDF or the TF tree.

A short Python/OpenCV script often suffices for a one-off experiment. ROS tooling becomes valuable when the team needs a repeatable, standardized calibration procedure.

10. Using Calibration Results in ROS TF2 and URDF

10.1 Recommended TF structure

The following is an illustration; replace link names with the actual Tron2 URDF definitions:

```
base_link
├── arm_link_1
│   └── ...
│       └── wrist_link
│           └── wrist_camera_link
│               └── wrist_camera_optical_frame
└── head_yaw_link
    └── head_pitch_link
        └── head_camera_link
            └── head_camera_optical_frame
```

Separate fixed camera-mount transforms from dynamic robot kinematics:

```
URDF + joint_states + robot_state_publisher
               +
      calibrated camera mount
               |
               v
              TF2
               |
               v
      Base-to-camera transform
```

10.2 Static extrinsic publishers

The following translation and rotation values are **syntax examples**, not actual Tron2 calibration parameters:

```
ros2 run tf2_ros static_transform_publisher \
  --x 0.03 --y 0.00 --z 0.08 \
  --qx 0 --qy 0 --qz 0 --qw 1 \
  --frame-id wrist_link \
  --child-frame-id wrist_camera_optical_frame
```

Warning: The rotation into a real optical frame is often **not** the identity quaternion. If the same parent-child transform is already published through URDF and `robot_state_publisher`, do not publish it again.

10.3 Dynamic transform checks

```
ros2 run tf2_ros tf2_echo \
  base_link wrist_camera_optical_frame

ros2 run tf2_ros tf2_echo \
  base_link head_camera_optical_frame
```

As arm and head joints move, these base-to-camera transforms should update. For recorded images, query TF at the **image acquisition timestamp**, not blindly at the current robot-state timestamp.

11. How Calibration Is Used in Robot Learning

11.1 6D pose estimation and visually guided grasping

Suppose FoundationPose estimates a screwdriver pose ${}^{C_h}T_O$ from the head RGB-D camera. Convert it to the base frame using

$$ {}^{B}T_O = {}^{B}T_{C_h}(q_h)\,{}^{C_h}T_O. $$

A common pipeline is:

```
Head RGB-D
  -> Object pose estimation
  -> ^C T_O
  -> Calibrated ^B T_C
  -> ^B T_O
  -> Grasp / task planning
  -> IK / motion control
  -> Real robot + feedback
```

Distinguish **pose-estimation error** from **camera-extrinsic error**. Even a very accurate 6D pose estimator produces systematically wrong robot-frame positions if ${}^{B}T_C$ is incorrect.

11.2 Imitation learning and VLAs

This section only establishes **when explicit extrinsics are needed**. BC/ACT and demonstrations belong in Data and Imitation Learning; SAM/Foundation models, temporal estimation and reacquisition belong in Visual State Estimation.

Actions may be represented as TCP increments $a_t = \Delta T_{\text{TCP}}$ or future joint sequences $a_t = q_{t:t+H}$. Whether explicit extrinsics are required depends on the architecture and task:

Task	Explicit extrinsics?
End-to-end behavior cloning from RGB to joint commands	Not necessarily
End-to-end RGB + proprioception policy	Not necessarily
Recovering object 3D positions from depth and pixels	Yes
Retargeting video trajectories into robot SE(3) motion	Usually
Multi-view 3D sensor fusion	Yes
Matching camera geometry across simulation and hardware	Yes
FoundationPose coupled with IK	Yes

End-to-end VLA models can learn an implicit visual-to-action mapping. Nevertheless, explicit, consistent camera geometry is often valuable for transfer across cameras, robots, and workspaces.

11.3 Reinforcement learning and sim-to-real

Consider an image-based control policy

$$a_t = \pi(I_t, q_t).$$

If camera poses, viewing angles, focal lengths, crop windows, or resizing differ between simulation and hardware, the observation distributions shift. Match at least

$$K_{\text{sim}} \approx K_{\text{real}}, \qquad {}^{B}T_{C,\text{sim}} \approx {}^{B}T_{C,\text{real}}.$$

Then randomize residual camera-calibration uncertainty in training, for instance

$$\delta t \sim \mathcal{N}(0, \Sigma_t), \qquad \delta\theta \sim \mathcal{N}(0, \Sigma_R).$$

Camera domain randomization improves robustness but should not replace accurate initial calibration.

11.4 Visual feedback and synchronization

For object target pose T_O^* and current pose $T_{O,t}$, an $SE(3)$-valued tracking error may be constructed as

$$e_t = \operatorname{Log}\left((T_O^*)^{-1} T_{O,t}\right).$$

Accurate visual control requires temporal alignment as well as spatial calibration. If the image was acquired at t_{image}, use the robot joint state and

$${}^{B}T_C(t_{\text{image}})$$

at that time, rather than pairing an older image with the latest arm pose. This matters especially for moving wrist and head cameras.

12. Accuracy Metrics and Calibration Validation

Evaluate the system at **image, geometry, and task** levels:

Level	Metric	Purpose
Image	RMS reprojection error and spatial residuals	Validate intrinsics and corner detection
Geometry	Translation and rotation consistency	Validate extrinsic and hand-eye constraints
Task	3D localization error and grasp success	Validate full robot-system performance

12.1 Reprojection error

For observed image corner p_i and predicted corner $\hat{p}_i$,

$$e_i = \|p_i - \hat{p}_i\|_2$$

with RMS

$$e_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}.$$

An image-space error measured in pixels is not directly a millimeter-level robot-endpoint error.

12.2 3D localization error

Given a target point with known base-frame position p_B^*, compare it to the visually estimated $\hat{p}_B$:

$$\boxed{e_p = \|\hat{p}_B - p_B^*\|_2.}$$

Test the workspace center and boundaries, several heights and viewing angles, and multiple yaw/pitch configurations. If head motion causes a substantial error increase, examine head kinematics, axis offsets, timestamps, and camera-mount calibration.

12.3 Grasp-level validation

One useful metric is TCP position error,

$$e_{\text{grasp}} = \|p_{\text{TCP,actual}} - p_{\text{TCP,target}}\|_2.$$

Also evaluate screwdriver grasp success, repeatability, orientation error, and disturbance recovery. These depend on FK, IK, actuation, TCP geometry, and object perception as well as camera calibration. A grasp failure does **not** prove the camera was calibrated incorrectly.

13. Common Failure Modes and Troubleshooting

Symptom	Likely cause	Check
Consistent spatial offset	Extrinsic translation bias	Inspect ${}^{B}T_{C}$

Symptom	Likely cause	Check
Error grows with viewpoint changes	Rotation calibration or kinematic error	Test many joint poses
Localization drifts after head motion	Incorrect yaw/pitch axes or offsets	Compare URDF to hardware
Unstable wrist hand-eye estimates	Weak rotational excitation / outliers	Use multi-axis poses, reject bad frames
Incorrect object distance	Depth scale, reflections, depth noise	Measure known distances
RGB-depth pixel mismatch	Alignment or camera-model mismatch	Check alignment and intrinsics
Vision axes disagree with control axes	Incorrect optical-frame mapping	Inspect TF frame conventions
Static tests succeed but moving tests fail	Image/joint timestamp mismatch	Query TF at exposure time
Grasp offset after hand replacement	TCP or flange definition changed	Inspect ${}^{G}T_{\mathrm{TCP}}$
Simulation succeeds; robot fails	Camera, dynamics, or control sim-to-real gap	Diagnose perception and actuation separately

A conservative data-collection sequence is: **move into pose** → **wait until stable** → **acquire image and corresponding robot state** → **save sample**. This reduces motion blur and time-alignment error.

14. Recommended Open-Source Tools and References

Tool	Main function	Recommended use
OpenCV	Intrinsics, PnP, hand-eye	Learn first
ROS image_pipeline	Intrinsics and image processing in ROS	ROS deployments
easy_handeye2	ROS 2 hand-eye workflow	Repeatable calibration
MoveIt Calibration	GUI calibration tools	Interactive setup
Kalibr	Multi-camera, camera-IMU, temporal calibration	Complex sensor suites
RealSense SDK	Device parameters, alignment, deprojection	D435/D455 and related devices
ROS REP 103	ROS frame conventions	Optical-frame verification

For this two-camera system, prioritize TF, PnP, and hand-eye calibration.

15. Recommended Tron2 + Revo3 Implementation Plan

A four-phase rollout helps isolate different error sources instead of combining them from the start.

Phase 1: Intrinsics and depth verification

- Read RealSense RGB/depth intrinsics and distortion coefficients.

- Verify or recalibrate wrist RGB intrinsics when necessary.
- Inspect RGB-depth alignment, depth scale, and error at typical grasping distances.
- Ensure actual image resolution, crop, and resizing match the camera model used by the perception pipeline.

This phase generally does not require robot movement.

Phase 2: Head camera calibration

Goal: obtain ${}^{B}T_{C_h}(q_h)$.

1. Inspect the Tron2 yaw/pitch axes, fixed offsets, and URDF definitions.
2. Fix a ChArUco board and capture multiple head configurations.
3. Record $q_{h,i}$ and use FK for ${}^{B}T_{H}(q_{h,i})$.
4. Run PnP to get ${}^{C_h}T_{T,i}$.
5. Calibrate the fixed mount ${}^{H}T_{C_h}$.
6. Verify 3D localization on held-out yaw/pitch poses.

If experiments currently use only one head orientation, start by locking the head and calibrating the corresponding base-camera transform.

Phase 3: Wrist camera and tool center point

Goals: obtain ${}^{G}T_{C_w}$ and ${}^{G}T_{\mathrm{TCP}}$.

1. Fix the target board and capture 20–30 diverse robot poses.
2. Save the matched end-effector FK and target pose observations.
3. Run `calibrateHandEye()`.
4. Validate board-pose constancy on unseen robot poses.
5. Recheck tool and fingertip geometry after mounting the Revo3 hand.

Phase 4: Visual control and learning deployment

Suggested unified data flow:

```
Head RGB-D / Wrist RGB
  -> Object perception
  -> Camera-frame object pose
  -> Calibrated camera transform
  -> Robot-base object pose
  -> Policy / grasp planner
  -> IK / control
  -> Tron2 + Revo3
  -> Visual feedback / replanning
```

Prioritize three transforms:

$$\boxed{{}^{B}T_{C_h}(q_h)}, \qquad \boxed{{}^{G}T_{C_w}}, \qquad \boxed{{}^{G}T_{\mathrm{TCP}}}.$$

Combined with FK, TF2, and synchronized timestamps, these connect head perception, wrist vision, and grasp control in one consistent coordinate system.

16. Summary and Deployment Checklist

Core question	Main technique	Role in robot learning
How does an image encode 3D geometry?	Intrinsics, distortion, PnP, RGB-D	Back-projection and pose estimation
Where is the camera relative to the robot?	Extrinsics and hand-eye	Robot-frame grasp planning
Do transforms stay valid during motion?	FK, URDF, TF2, synchronization	Active vision and closed-loop control

Use the following checklist before deployment:

- [ ] Image resolution and intrinsics match, and the image's distortion/rectification status is known.
- [ ] Each transform has a clearly documented direction (for example, ${}^{B}T_{C}$).
- [ ] `camera_link` and `camera_optical_frame` are not confused.
- [ ] FK, PnP, and hand-eye measurements use consistent units and link definitions.
- [ ] Head yaw/pitch axes, fixed offsets, and wrist mount extrinsics are verified.
- [ ] RGB-D data are aligned, and depth definitions/scales are known.
- [ ] Extrinsics are evaluated with independent poses, not just the fit data.
- [ ] Camera exposure times are aligned with joint-state and TF timestamps.
- [ ] TCP and fingertip frames have been checked after any hand replacement.
- [ ] Final quality is tested using real 3D points and actual grasps.

For a Tron2 manipulation stack, OpenCV PnP, hand-eye calibration, robot forward kinematics, and ROS TF2 are the four most valuable foundations. Together, they enable head/wrist vision and dexterous closed-loop manipulation without requiring an external Vicon system.

Note: Board dimensions, sample counts, paths, frame names, and some command arguments in this tutorial are examples or engineering recommendations, not verified Tron2 manufacturer specifications or measured calibration results. Real deployments must be validated against the actual device, URDF, logs, and physical tests.

Visual Perception and State Estimation

Course home · 简体中文 · PDF

Item	Details
Applicability	RGB-D, 6D pose, temporal tracking and recovery concepts
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Conceptual methods and experiment design; no robot or accuracy validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

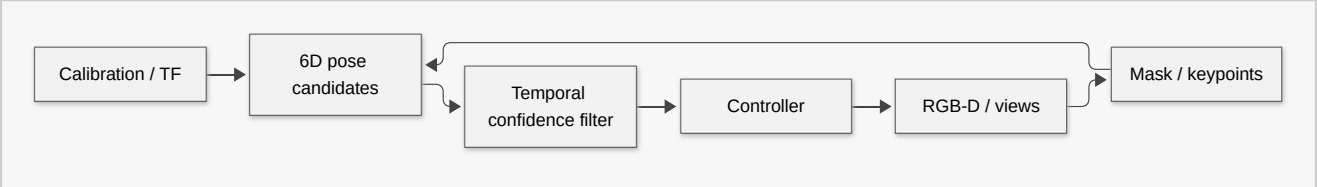
- 1. Why Robots Need State Estimation
  - 1.1 Observation, Estimate, and True State
  - 1.2 When 6D Pose Is Needed
- 2. From Pixels to Robot Base Frames
  - 2.1 Head and Wrist Cameras Have Different Extrinsics
  - 2.2 Timestamp Delay Becomes Spatial Error
- 3. A Modular Visual Estimation Pipeline
  - 3.1 Separate Initialization from Tracking
  - 3.2 Symmetry Can Make Pose Ambiguous
- 4. Temporal Filtering, Confidence and Recovery
  - 4.1 Confidence-Aware Control
  - 4.2 Lost-Track and Reacquisition Are Core Capabilities
- 5. Training with Estimator Error
  - 5.1 Evaluate Error Against the Control Objective
- 6. A Minimal Closed-Loop Panda Experiment
  - 6.1 Acceptance Checklist
- 7. From Pixels to 3D State With Uncertainty
  - 7.1 What Can One RGB Image Actually Measure?
  - 7.2 Stereo Depth and Error Propagation
  - 7.3 Object Point Clouds From Masks and Depth
- 8. Visual Foundation Models: Roles and Limitations
  - 8.1 Segmentation, Stereo, Pose, and Reconstruction Are Different Tasks
  - 8.2 SAM 2: Promptable Temporal Instance Masks
  - 8.3 SAM 3 and SAM 3.1: Open-Vocabulary Language Grounding

    - 8.4 FoundationPose: 6D Pose With CAD or Reference Views
    - 8.5 FoundationStereo and Fast-FoundationStereo as Depth Components
    - 8.6 SAM 3D Objects and Body: Shape Priors, Not Precision Calibration
- 9. Building a Modular Perception Pipeline
    - 9.1 Two Complementary Perception Routes
    - 9.2 Asynchronous Scheduling Rather Than Every Model Every Frame
    - 9.3 A Consistent State Interface
- 10. Temporal Estimation, Latency, and Error Propagation
    - 10.1 Prediction and EKF Measurement Update
    - 10.2 Delay Is Spatial Error, Not Just Slower FPS
    - 10.3 Occlusion, Observability, and Active Reacquisition
- 11. Selecting Models for Actual Robot Cameras
    - 11.1 Head RGB-D and Wrist RGB
    - 11.2 Model Selection by Task
    - 11.3 Isolate Perception Error From Control Error
- 12. A Minimal Reproduction and Acceptance Protocol
    - 12.1 Report Both Perception and Robot Metrics
    - 12.2 Shared Panda Tabletop Ablations
- 13. Connections and References
- 8. Measurement Models and State Estimation
    - 8.1 RGB-D and Stereo Depth
    - 8.2 Recursive Filtering and Confidence
- 9. General-Purpose Vision Models
    - 9.1 Capability Comparison
    - 9.2 Composable Perception Stack
    - 9.3 End-to-End Latency
- 10. Shared Experiments and References

Learning outcomes: Understand pose chains; handle latency and tracking loss; connect estimates to closed-loop control.

1. Why Robots Need State Estimation

Visual measurements are partial, delayed, and noisy. Grasping policies need object pose, end-effector pose, and contact state; cameras provide pixels and depth. State estimation combines images, proprioception, calibrated geometry, and temporal history into control-ready state and uncertainty.



1.1 Observation, Estimate, and True State

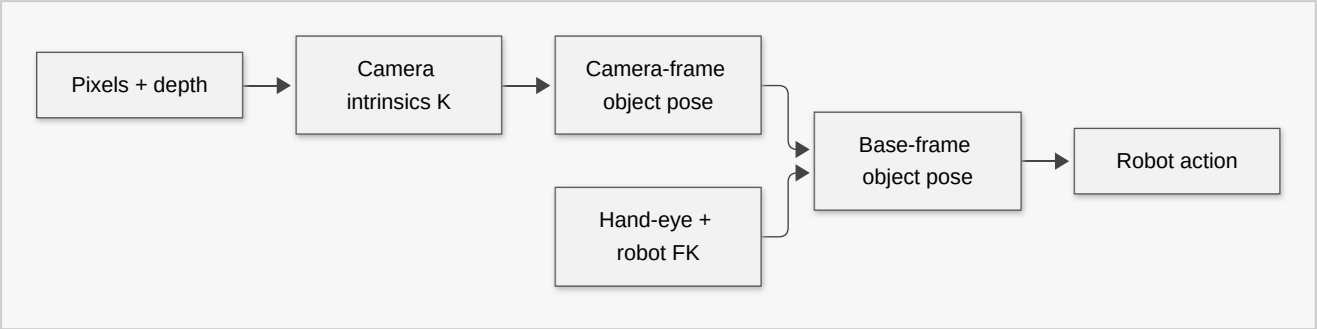
The true environment state s_t is generally not fully observable. Camera observation o_t changes with occlusion, exposure, and viewpoint. An estimator supplies $\hat{s}_t$ and a confidence signal. In simulation, keep ground truth, rendered observations, and estimated state separate to prevent privileged-information leakage.

1.2 When 6D Pose Is Needed

Insertion, tool rotation, and grasping can depend on the object's 3D position and orientation, so an SE(3) pose may be useful. For symmetric, occluded, or textureless objects, task keypoints, relative pose, contours, or direct visual policies may be better. **Not every VLA requires an explicit 6D pose.**

2. From Pixels to Robot Base Frames

Calibrate intrinsics, distortion, depth scale, and extrinsics first. If the camera estimates T_C^O, then $T_B^O = T_B^C T_C^O$. For a wrist camera, $T_B^C(t)$ changes with robot configuration and requires timestamp-aligned forward kinematics and hand-eye calibration.



2.1 Head and Wrist Cameras Have Different Extrinsics

A fixed head camera may have constant extrinsics; a pan/tilt head still has time-varying $T_B^C(t)$. A wrist camera commonly uses $T_B^E(q_t)T_E^C$. The existence of a URDF does not eliminate real-world camera-to-axis errors: extrinsics must be validated.

2.2 Timestamp Delay Becomes Spatial Error

When the robot moves, transforming an old wrist-camera frame with the newest joint state creates systematic pose error. Log exposure/arrival, robot-state, inference-output and command-effective times; interpolate historical transforms. Measure end-to-end latency before tuning aggressive control.

3. A Modular Visual Estimation Pipeline

Common modules are detection/segmentation, correspondences and pose hypotheses, pose refinement, temporal tracking, failure detection, and uncertainty. A segmentation mask is not a 6D pose; PnP needs reliable 2D-3D correspondences. FoundationPose supports novel-object estimation/tracking under the input assumptions described in the paper, such as CAD or reference views.

3.1 Separate Initialization from Tracking

Initialization can propose multiple hypotheses from masks, depth, and geometry; tracking refines near the prior pose, often faster but prone to drift. Occlusion, out-of-view loss, or false detections should trigger global reinitialization rather than indefinite extrapolation.

3.2 Symmetry Can Make Pose Ambiguous

For symmetric cylinders or textureless tools, some rotations are not visually identifiable. Evaluate with symmetry-aware metrics or predict task-relevant contact keypoints. A bigger network cannot recover information absent from measurements.

4. Temporal Filtering, Confidence and Recovery

Maintain a predict → measurement update → validity check → recovery loop. The prediction uses previous state plus motion priors; the update uses new images. EKF, particle filtering, learned recurrent estimators, or simpler smoothing are possible. Evaluate error, latency, and reinitialization counts rather than only displaying smooth trajectories.

4.1 Confidence-Aware Control

Attach uncertainty or a validated quality score. At sustained low confidence, slow down, hold a safe pose, reacquire an observation, or request intervention; **do not continue high-force contact with an untrusted pose**. Calibrate confidence on held-out scenes: a raw network score is not a guaranteed error bound.

4.2 Lost-Track and Reacquisition Are Core Capabilities

Use an explicit state machine: TRACKING, UNCERTAIN, LOST, RELOCALIZING. Test transitions under occlusion, active camera movement, drops, and viewpoint jumps. Measure recovery success/time and maintain object identity to avoid silently switching to a similar object.

5. Training with Estimator Error

A simulator-privileged teacher is not a deployed visual policy. Visual distillation or pose-noise domain randomization may help, but include bias, temporal correlation, delays, missed detections, and outliers where justified. Compare oracle pose, synthetic noisy pose, and estimated pose to isolate perception versus control limitations.

5.1 Evaluate Error Against the Control Objective

Track translation/orientation error, track-loss rate, confidence calibration, inference latency, occlusion recovery time, and grasp success. Better numerical pose error does not always imply better manipulation performance, so include task outcomes.

6. A Minimal Closed-Loop Panda Experiment

In the shared Panda + cube + tabletop setup, first establish an oracle-state baseline, then a fixed RGB-D estimator, and finally a wrist-camera case with time-varying extrinsics. Hold the controller fixed while changing only the perception pipeline. Test nominal lighting, occlusion, and randomized start poses; active camera recovery is optional.

6.1 Acceptance Checklist

Check	Acceptance criterion
Frame chain	Known object maps to the base frame
Timestamps	Images and FK use aligned sample times
Nominal error	Measured quantitatively on held-out scenes
Recovery	Reacquires after loss, no permanently stale pose
Safety	Can slow/stop when confidence is poor
Task metric	Logs success, latency, and recovery

7. From Pixels to 3D State With Uncertainty

7.1 What Can One RGB Image Actually Measure?

For a camera-frame point $P_C = (X, Y, Z)$, the ideal pinhole camera maps it to $u = f_x X/Z + c_x,\ v = f_y Y/Z + c_y$. A pixel (u, v) alone does not uniquely specify a 3D point. Depth, calibrated stereo, geometric correspondences or other priors are required. For the derivation of intrinsics, extrinsics, hand-eye calibration and TF, consult Camera Calibration; this chapter focuses on **estimating object state from calibrated observations** rather than repeating calibration theory.

7.2 Stereo Depth and Error Propagation

For synchronized, rectified stereo images, disparity $d = u_L - u_R$ yields the simplified metric depth and sensitivity

$$Z = \frac{f_x b}{d}, \qquad \sigma_Z \approx \frac{f_x b}{d^2}\sigma_d,$$

where b is the measured baseline. Small disparity at large distances amplifies depth error, while incorrect matching and reflections create invalid depth. **FoundationStereo** performs zero-shot dense disparity estimation from calibrated stereo; **Fast-FoundationStereo** (CVPR 2026) improves speed using distillation, architecture search and pruning. Neither is a single-image 6D object pose estimator.

7.3 Object Point Clouds From Masks and Depth

Given mask $M(u, v)$ and valid depth $Z(u, v)$, back-project foreground pixels to

$$\mathcal{P}_O = \left\{ Z(u,v) K^{-1} [u, v, 1]^\top : M(u,v) = 1,\ Z(u,v) > 0 \right\}.$$

Evaluate mask boundaries and background leakage separately from invalid/reflective depth, self-occlusion and segmentation identity failures. A point cloud in the camera frame is not automatically an SE(3) object pose in the robot base frame; it still requires geometry and timestamp-aligned transforms.

8. Visual Foundation Models: Roles and Limitations

8.1 Segmentation, Stereo, Pose, and Reconstruction Are Different Tasks

Model	Typical input	Output	Reasonable robot role	Does NOT guarantee
SAM 2 / 2.1	Image/video and click/box/mask prompt	Instance masks propagated through video	Track an initialized target	Depth and 6D pose
SAM 3 / 3.1	Image/video, text concepts or exemplars	Open-vocabulary detection, masks and tracking	Language grounding and target identity	Contact forces or grasp precision
Foundation Stereo	Synchronized calibrated stereo	Disparity and derived depth	Stereo metric 3D measurements	Monocular depth or object orientation
Fast-Foundation Stereo	Same	Lower-latency disparity family	Real-time stereo candidate	Realtime speed on all hardware
Foundation Pose	RGB(-D), target mask, CAD or reference images	6D pose and tracking	Rigid-object localization	Pose with zero object-reference assumptions
SAM 3D Objects	Single object image	Hypothesized 3D geometry, texture, layout	Candidate shape priors without CAD	Verified millimetre-scale collision geometry
SAM 3D Body	Human image	3D human body shape and pose	Human demonstration understanding	Robot hand contact forces

“Foundation” suggests generalization across categories and scenes; it does not remove the need for calibration, synchronization, device benchmarking or safety checks. Audit checkpoint license, memory footprint, camera requirements and operating domain before use.

8.2 SAM 2: Promptable Temporal Instance Masks

SAM 2 initializes targets from point, box or mask prompts, and propagates masks using video memory. It is a strong baseline for tracking a known screwdriver or gripper in video. Severe occlusion, out-of-view targets, reflections and similar-looking instances can break identity continuity. Log track IDs and allow re-prompting. Smooth video masks do not imply correct physical pose or grasp contact.

8.3 SAM 3 and SAM 3.1: Open-Vocabulary Language Grounding

SAM 3 accepts short text concepts, visual exemplars and prompts to find and track matching instances. “The red-handled screwdriver on the table” can yield masks for a downstream 3D

estimator. SAM 3.1 further improves multi-object video throughput. Yet open-vocabulary localization returns **image regions**, not a control-ready grasp trajectory; assess identity switches, false matches, latency and occlusion recovery.

8.4 FoundationPose: 6D Pose With CAD or Reference Views

FoundationPose (CVPR 2024) supports model-based pose estimation (CAD provided) and its model-free setup (a few reference views are captured). It can handle novel test objects without task-specific fine-tuning under these assumptions. A practical pipeline is segmentation/depth input → pose hypotheses and scoring → pose refinement → temporal tracking, with reinitialization after loss. **Model-free does not mean no reference information.**

With 3D model points P_i and trusted image correspondences u_i, the geometric idea of reprojection consistency can be written

$$(\hat{R}, \hat{t}) = \arg\min_{R,t} \sum_i \rho\left(\|u_i - \pi(K(RP_i + t))\|_2^2\right).$$

This is a generic explanatory pose objective, **not a claim that FoundationPose exactly optimizes this loss**. Symmetry, texturelessness and occlusion can make rotations unobservable; evaluate using symmetry-aware pose errors or task-relevant keypoints.

8.5 FoundationStereo and Fast-FoundationStereo as Depth Components

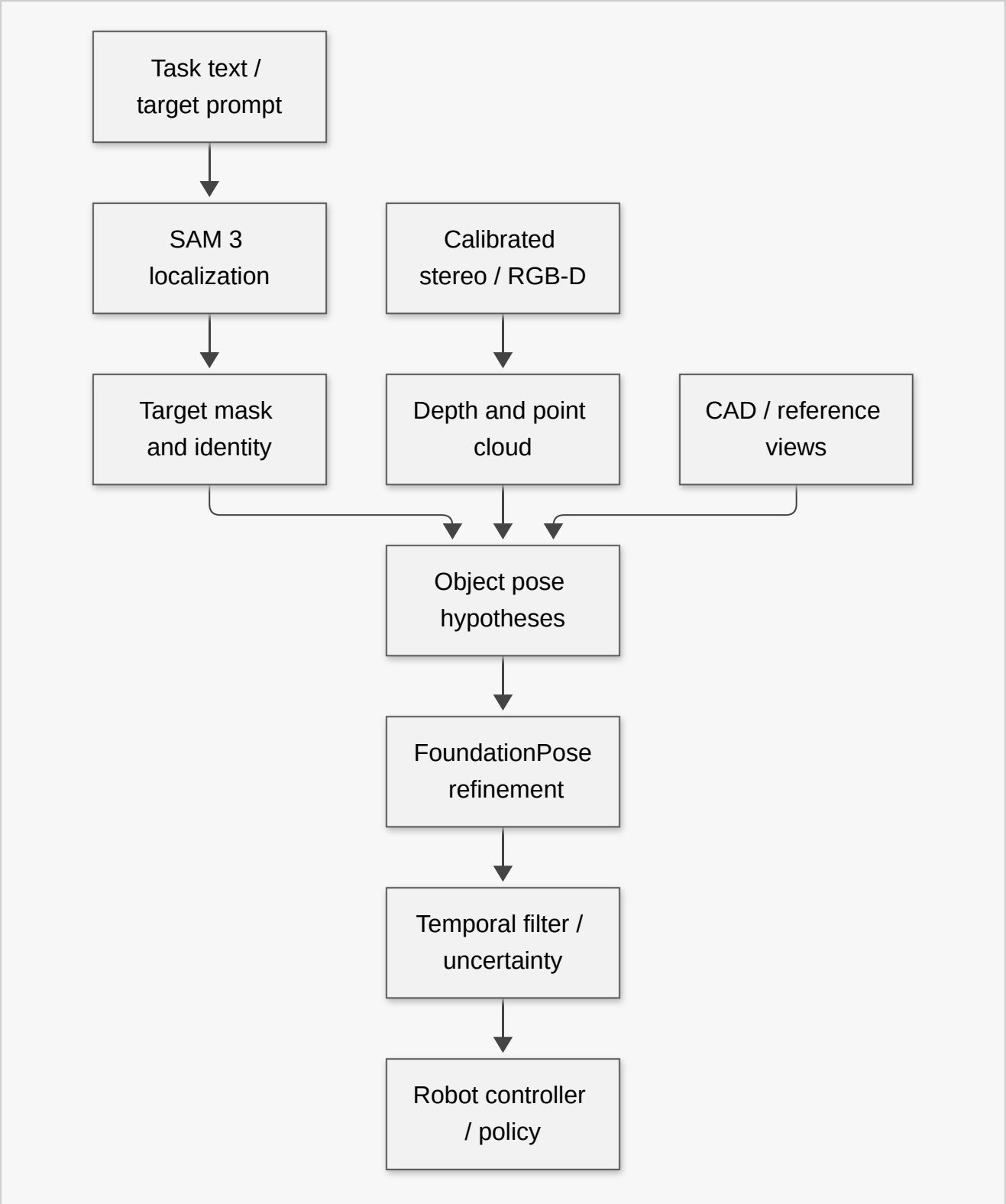
FoundationStereo is zero-shot stereo matching: calibrated/synchronized $[I_L, I_R]$ → disparity → depth using $Z = f_x b/d$. Fast-FoundationStereo is an accelerated model family; runtime depends on checkpoint, iterations, input resolution and GPU. **High-quality depth is not equivalent to high-quality 6D pose.** Where a depth camera already provides reliable aligned RGB-D, its SDK should be the first baseline. Adding a stereo foundation model must justify extra cost by measured accuracy, latency or robustness gains.

8.6 SAM 3D Objects and Body: Shape Priors, Not Precision Calibration

SAM 3D Objects infers shape, texture and layout from a single object image; SAM 3D Body estimates human 3D form. A single image leaves hidden surfaces and metric scale ambiguous. Its 3D output can serve as a **candidate geometry prior** but must be scale-checked, aligned and validated before rigid registration, collision simulation, or high-precision insertion. A plausible 3D mesh does not guarantee measured mass, inertia, contact coefficients or millimetre accuracy.

9. Building a Modular Perception Pipeline

9.1 Two Complementary Perception Routes



Explicit geometry: SAM 3 for target selection, stereo/RGB-D for metric depth, FoundationPose for 6D pose, and a temporal estimator to provide stable control feedback. **Implicit policy:** send images and task goals to a visual action model without necessarily estimating a 6D pose, while independently monitoring target visibility, progress and safety. Deformable, transparent, and deeply occluded objects may not have a unique estimable rigid pose.

9.2 Asynchronous Scheduling Rather Than Every Model Every Frame

At task start, run SAM 3 to identify the object. Track subsequent video with a lighter tracker or the SAM video mode; run more expensive pose refinement less often; rerun global detection when identity or confidence is lost. Invoke stereo models only when synchronized true binocular input and an accuracy or dataset need justify them. Every measurement must preserve its **image acquisition timestamp**, not only inference completion time.

9.3 A Consistent State Interface

A target tracking message should include frame_id, timestamp, track_id, mask/detection region, $\hat{T}_B^O$, translation/rotation uncertainty, estimator checkpoint and validity state: TRACKING / UNCERTAIN / LOST / RELOCALIZING. Record invalid-depth fraction and pose hypothesis ambiguity. Policy and high-level agents should only consume audited fields. A raw classifier confidence value is not a certified bound on physical error.

10. Temporal Estimation, Latency, and Error Propagation

10.1 Prediction and EKF Measurement Update

For a local Euclidean state, let $x_t = f(x_{t-1}, u_t) + w_t$ and $z_t = h(x_t) + v_t$ with covariances Q_t, R_t. Linearization gives the extended Kalman gain/update

$$K_t = P_t^- H_t^\top (H_t P_t^- H_t^\top + R_t)^{-1}, \qquad \hat{x}_t = \hat{x}_t^- + K_t(z_t - h(\hat{x}_t^-)).$$

SE(3) poses are not ordinary vectors under large rotation changes. An error-state update can use $\hat{T}_t = \hat{T}_t^- \operatorname{Exp}(\delta\xi_t^\wedge)$. EKF will not fix a mistaken instance identity; robust gating, lost-track detection and reacquisition are separate design requirements.

10.2 Delay Is Spatial Error, Not Just Slower FPS

At approximately constant object speed $\|v\|$ and lag δ, stale observations can induce position error on the order of $\|v\|\delta$. A wrist-camera frame captured at $t - \delta$ must be transformed using robot FK/TF for the **same historical time**. Combining a stale image with the newest FK introduces systematic error. Log exposure, receipt, estimator finish, policy finish and command-effective times; evaluate end-to-end p95 latency.

10.3 Occlusion, Observability, and Active Reacquisition

Turn confidence into explicit controller modes. High confidence permits bounded feedback, low confidence slows or holds, and lost tracking triggers safe observation and relocalization. Moving the wrist or head for an informative view may help, provided collision and reachability are verified first. Symmetric objects can have fundamentally unobservable rotations; larger models alone do not resolve missing information.

11. Selecting Models for Actual Robot Cameras

11.1 Head RGB-D and Wrist RGB

A fixed head camera may provide a stable global view; a pan/tilt camera has time-varying extrinsics. A wrist RGB camera sees close interactions but moves with the end-effector and suffers occlusion. A head RGB-D plus a wrist RGB **does not automatically form a synchronized, rectified stereo pair** suitable for FoundationStereo. Build the SDK-depth + SAM 2/3 + FoundationPose baseline first.

11.2 Model Selection by Task

Requirement	First baseline	Optional component to test
Track a prompted object mask	SAM 2	SAM 3 for language selection
Ground an unknown linguistic target	SAM 3	Stereo/depth and pose estimator
Calibrated stereo	Device SDK / FoundationStereo	Fast variant for low latency
Rigid object with CAD/reference views	FoundationPose	External SAM + temporal tracker
No CAD and incomplete shape	SAM 3D Objects candidate	Metric-scale verification
Precise insertion/contact	Pose + proprioception + contact signals	Mask never substitutes force feedback

11.3 Isolate Perception Error From Control Error

Keep controller parameters fixed. Compare oracle pose, matched synthetic noisy pose, and estimated visual pose. Measure mask quality, track loss, relocalization, 6D error and downstream grasping success. Otherwise one cannot tell whether a failed pick resulted from an estimator failure or an inadequate controller.

12. A Minimal Reproduction and Acceptance Protocol

12.1 Report Both Perception and Robot Metrics

Measure mask IoU, identity switches, stereo depth MAE and invalid fraction, translation error, symmetry-aware rotation error, tracking dropout, reacquisition success, p50/p95 inference latency, robot success and unsafe contact rate. Ground truth must come from an independent measurement or an explicitly identified simulator source. Bind every result to GPU, model/checkpoint, image resolution, threshold and camera calibration.

12.2 Shared Panda Tabletop Ablations

Hold the Panda + cube + table task and controller fixed. Test A) oracle state, B) SDK RGB-D + simple mask, C) SAM 2 + FoundationPose, D) SAM 3 + FoundationPose, E) FoundationStereo or Fast on an actual calibrated stereo rig. Vary occlusion, camera motion, similar-looking objects, slips, and full track loss. Always evaluate safety-stop and reacquisition behavior.

Chapter ownership: Calibration mathematics are in Camera Calibration; demonstrations and policy learning are in Data and Imitation Learning; high-level planning, tool use and success checking are in Robot Agent Workflows.

13. Connections and References

This chapter follows camera calibration and supplies an observation interface for imitation learning, motion RL, and visual VLA deployment.

- FoundationPose, CVPR 2024
- FoundationStereo, CVPR 2025
- Fast-FoundationStereo, CVPR 2026
- SAM 3 and SAM 3.1
- SAM 3D Objects
- SAM 2
- OpenCV calib3d
- ROS 2 tf2

Verification boundary: this is a conceptual methods and experiment-design chapter. Real-time performance on any particular camera, GPU, or robot has not been measured.

8. Measurement Models and State Estimation

8.1 RGB-D and Stereo Depth

Pinhole projection obeys $u = f_x X/Z + c_x$, $v = f_y Y/Z + c_y$. With valid depth, $P_C = ZK^{-1}[u, v, 1]^\top$. Rectified stereo gives $Z = fb/d$ with baseline b and disparity d; approximate depth uncertainty is $\sigma_Z \approx (fb/d^2)\sigma_d$. Depth, segmentation and extrinsic calibration are all noisy. For intrinsic and hand–eye calibration methods, use the dedicated Camera Calibration chapter.

8.2 Recursive Filtering and Confidence

Use $x_t = f(x_{t-1}, u_{t-1}) + w_t$, $y_t = h(x_t) + v_t$. An EKF computes

$$\hat{x}_t^- = f(\hat{x}_{t-1}, u_{t-1}), \quad P_t^- = F_t P_{t-1} F_t^\top + Q_t,$$

$$K_t = P_t^- H_t^\top (H_t P_t^- H_t^\top + R_t)^{-1}, \quad \hat{x}_t = \hat{x}_t^- + K_t(y_t - h(\hat{x}_t^-)).$$

SE(3) orientation updates should use a local tangent-space residual; evaluate symmetric objects with symmetry-aware metrics. During occlusion, uncertainty must grow, and lost tracks must reinitialize rather than silently reuse a stale pose.

9. General-Purpose Vision Models

9.1 Capability Comparison

Model	Primary output	Required or typical input	Robotics application	Not a guarantee of
FoundationPose	object 6D pose estimation/tracking	target region, RGB(-D), CAD or reference views as applicable	manipulation pose feedback	universal unconditional object recognition
FoundationStereo	dense stereo disparity and depth	calibrated rectified stereo images	metric depth and point cloud	object semantics or 6D pose
SAM 2	image masks and video mask tracking	images/video plus click, box or mask prompts	maintain target identity in video	metric depth or physical contact
SAM 3	text/exemplar-prompted concept detection, masks and tracking	images/video and language or examples	referential object discovery	physical pose or task success
SAM 3D Objects	estimated single-image 3D geometry, texture and layout	image/object region	candidate shape prior	metrology-grade CAD or real-time tracking

SAM 3 is not SAM 3D, and neither is FoundationPose. Monocular reconstruction is ambiguous in geometry and scale. Do not use unverified SAM 3D output as precision collision geometry. FoundationStereo is optional if RGB-D depth is already available.

9.2 Composable Perception Stack

SAM 3 locates a task referent from language; SAM 2/3 tracks object masks; RGB-D or FoundationStereo supplies depth; FoundationPose or another appropriate estimator generates camera-frame pose T_C^O; a temporal filter produces base-frame $\hat{T}_B^O$ and uncertainty for the policy. SAM 3D may generate a provisional object shape when CAD is missing, but scale and contact surfaces require validation. Some tasks need only segmentation and depth keypoints, not full object orientation.

9.3 End-to-End Latency

$$T_{\text{total}} = T_{\text{capture}} + T_{\text{segmentation}} + T_{\text{depth}} + T_{\text{pose}} + T_{\text{filter}} + T_{\text{policy}} + T_{\text{communication}}.$$

Measure initialization/tracking time separately, P95/P99 latency, lost-track recovery, pose error, uncertainty calibration, and task success. A slow detector can coexist with a faster control loop only if timestamped state prediction and stale-state handling are explicit.

10. Shared Experiments and References

Compare oracle object state against RGB-D+mask+FoundationPose, SAM 2/SAM 3 mask initialization, and optional stereo/shape priors while holding the Panda task and controller fixed. Test occlusion, symmetries, camera movement, and reacquisition. Structured pose perturbations can use $\tilde{T}_t = T_t \exp(\hat{\xi}_t)$, but should include bias, correlation, missed observations and latency rather than independent Gaussian noise alone.

Division of responsibility: visual measurements and uncertainty belong here; demonstration policy fitting lives in Imitation Learning, and tool orchestration/replanning in Robot Agents.

- FoundationPose
- FoundationStereo
- SAM 2
- SAM 3
- SAM 3D Objects

Robot Dynamics for Embodied Intelligence

Course home · 简体中文 · PDF

Item	Details
Applicability	Rigid-body theory; MuJoCo and Isaac examples are conceptual, not a tested cross-engine pair
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

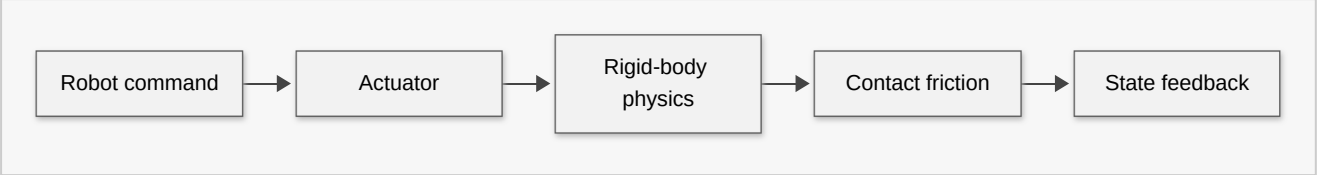
Contents

- Reading guide: follow the physical chain, not a collection of formulas
- Chapter 1. Dynamics through an embodied manipulation task
  - 1.1 Kinematics says where; dynamics explains motion
  - 1.2 State, action, and observation are different quantities
  - 1.3 One equation organizes the entire tutorial
- Chapter 2. Only the kinematics needed for dynamics
  - 2.1 Joint coordinates versus end-effector coordinates
  - 2.2 The Jacobian connects velocity and force
  - 2.3 Why inverse kinematics cannot replace dynamics
- Chapter 3. Mass, inertia, and gravity from one rigid link
  - 3.1 Translation: mass resists acceleration
  - 3.2 Rotation: inertia depends on mass distribution
  - 3.3 A one-joint pendulum already contains the main ideas
  - 3.4 Velocity-dependent coupling in three-dimensional rotation
- Chapter 4. From one link to the manipulator equation
  - 4.1 Why a mass matrix appears
  - 4.2 Deriving the standard equation from energy
  - 4.3 Forward dynamics versus inverse dynamics
  - 4.4 Energy provides a useful consistency check
- Chapter 5. A complete two-link planar manipulator example
  - 5.1 Geometry and sign conventions
  - 5.2 Kinetic energy reveals joint coupling
  - 5.3 Velocity coupling and gravity

    - 5.4 Four thought experiments worth doing
- Chapter 6. How dynamics scales to complex robots
    - 6.1 Why we do not expand every rigid-body equation
    - 6.2 Gravity compensation is a special inverse-dynamics case
- Chapter 7. Actuators: how policy actions reach the real robot
    - 7.1 A position target is not an instantaneous position jump
    - 7.2 Nonideal effects that materially change behavior
    - 7.3 Map policy action spaces to physical interfaces
- Chapter 8. Contact dynamics: grasping, pushing, sliding, and dropping
    - 8.1 Where does contact force come from?
    - 8.2 Why contact is unilateral
    - 8.3 The friction cone determines whether an object stays grasped
    - 8.4 An impact produces a rapid velocity change
    - 8.5 Distinguish four modes during a grasp
- Chapter 9. Dexterous manipulation is a coupled arm–hand–object system
    - 9.1 Why controlling the fingers alone may be insufficient
    - 9.2 When an object may be treated as extra payload
    - 9.3 Practical lessons for Tron2 with a Revo3 hand
- Chapter 10. Humanoids: from fixed-base to floating-base dynamics
    - 10.1 Why the torso cannot be driven like an ordinary joint
    - 10.2 Balance requires external support forces
    - 10.3 Only the essential centroidal dynamics and WBC concepts
- Chapter 11. Dynamics-based control that matters for embodied intelligence
    - 11.1 PD with and without gravity compensation
    - 11.2 Computed-torque control
    - 11.3 Impedance control supports compliant contact
    - 11.4 Where MPC and WBC fit
- Chapter 12. MuJoCo versus Isaac Sim / Isaac Lab: appropriate roles
    - 12.1 What each platform contributes
    - 12.2 Matching model filenames is not enough
    - 12.3 Three time scales must be identified separately
- Chapter 13. Code-free laboratory guide: free-space mechanics first
    - Lab 1 | One-link gravity and inertia
    - Lab 2 | Two-link mass matrix and coupling
    - Lab 3 | Forward–inverse dynamics consistency
    - Lab 4 | PD control versus gravity-compensated PD
    - Lab 5 | Action rate and actuator saturation
- Chapter 14. Code-free laboratory guide: contact, hands, and humanoids
    - Lab 6 | A falling block: normal contact and impact
    - Lab 7 | Sliding on a slope: friction intuition

    - Lab 8 | Dexterous grasp, rotation, and drop recovery
    - Lab 9 | Humanoid standing: contact force and center of mass
    - Lab 10 | Cross-simulator and sim-to-real sensitivity
    - Minimal one-page lab-report structure
- Chapter 15. From simulation to real robots: a concise checklist
    - 15.1 Physical parameters must remain self-consistent
    - 15.2 Prioritize sim-to-real variation by plausible error sources
    - 15.3 Four misconceptions to avoid
    - 15.4 One-page equation reference
- Appendix A. What was kept from the original course
- Appendix B. Notation, reading route, and references

Chapter concept map



Reading guide: follow the physical chain, not a collection of formulas

Robot dynamics answers an immediate question in embodied intelligence: **after a policy issues an action, why do the robot and its environment move as they do?** A single task connects the entire tutorial: a robot arm with a dexterous hand picks up a screwdriver, rotates it, and picks it up again after a drop. A humanoid that stands and reaches for the same object follows the same physical laws, except that its base is floating and its feet establish additional contacts.

The tutorial draws on the structure of Wei Zhang's MEE5114 Advanced Control for Robotics, especially the lectures on rigid-body motion, dynamics, and control. It is **not a reproduction or translation of those lecture notes**. We also use *Modern Robotics*, Featherstone's rigid-body dynamics work, and official MuJoCo / Isaac Lab documentation, focusing only on material with a clear role in robot learning.[1–8]

The central chain is: policy action a → actuator torque τ → acceleration $\ddot{q}$ and contact force λ → next physical state → camera and proprioceptive observation o → next policy action.

$$\boxed{a_t \longrightarrow \tau_t \longrightarrow (\ddot{q}_t, \lambda_t) \longrightarrow s_{t+1} \longrightarrow o_{t+1}.}$$

Included: the minimum coordinate and Jacobian concepts; mass, inertia, and gravity; manipulator equations; forward and inverse dynamics; contact and friction; actuation; floating-base dynamics; control; simulation; and sim-to-real transfer.

Deliberately omitted: Lie-group proofs, full screw theory and product-of-exponentials derivations, detailed spatial cross-product operators, low-level implementations of RNEA/ABA, general-purpose

SDP and optimal-control theory, software installation commands, and all programming code. Readers who need those topics can proceed to the references.

How to read: Chapters 1–6 build the physics foundation. Chapters 7–11 explain actuation, contact, and control. Chapters 12–14 specify reproducible **code-free** MuJoCo and Isaac Sim / Isaac Lab experiments. Chapter 15 consolidates sim-to-real considerations. The experiments describe what to perform and measure; they are not claims that simulations have already been executed.

Chapter 1. Dynamics through an embodied manipulation task

1.1 Kinematics says where; dynamics explains motion

Let $q \in \mathbb{R}^n$ denote the joint configuration and $\dot{q}$ the joint velocity. Kinematics asks: where is the palm given q, and how fast does it move given $\dot{q}$? It requires no mass, inertia, or motor-torque information.

Dynamics asks: what acceleration follows from an applied torque? How does the table's support force alter the finger motion? What happens after the object leaves contact? Dynamics thus concerns the **state, input forces, and environmental interaction**, rather than geometry alone.

Question in the task	Main concept	Why it matters
Where should a fingertip go?	Forward/inverse kinematics	Convert object pose into target joint motion
What actually drives the finger?	Actuators + dynamics	Explain torque, acceleration, and delay
Why does the screwdriver stay in the hand?	Contacts + friction	Check whether forces lie inside the friction cone
Why does a replacement hand change tracking?	Mass, inertia, gravity	Payload and center of mass have changed
Why do humanoid feet need ground forces?	Floating base + centroidal dynamics	Internal motors cannot suspend the whole body

1.2 State, action, and observation are different quantities

A physical state may include robot joint positions and velocities, object pose and velocity, and contact mode: $s = (q, \dot{q}, x_o, v_o, \text{contacts}, \ldots)$. A learned policy receives observations o, such as RGB-D frames, joint encoders, and estimated finger currents. A camera does not directly measure the mass matrix or the true friction coefficient.

The meaning of an action a depends on the control interface. It may specify desired joint position, velocity, torque, or an end-effector pose. **The same numerical action can yield different physical motions under different low-level servos.** This is the first question to ask when comparing robot-learning action spaces.

1.3 One equation organizes the entire tutorial

For a fixed-base, fully actuated rigid manipulator with independent joint coordinates,

$$\boxed{M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau + J_c(q)^T\lambda.}$$

Here M is configuration-dependent inertia, $C\dot{q}$ captures velocity-dependent coupling, g is the generalized gravity term, τ is actuator torque, λ is the force of the environment on the robot, and $J_c^T\lambda$ maps that contact force into generalized coordinates.

Define $h(q,\dot{q}) = C(q,\dot{q})\dot{q} + g(q)$. The compact form is

$$M(q)\ddot{q} + h(q,\dot{q}) = \tau + J_c^T\lambda.$$

Real systems may also contain friction, compliant transmissions, saturation, and delay. Those effects must be restored when interpreting experimental discrepancies.

Chapter 2. Only the kinematics needed for dynamics

2.1 Joint coordinates versus end-effector coordinates

For n joints, $q = (q_1, \ldots, q_n)$ describes the configuration. End-effector position is $p = f(q)$. To represent a full six-degree-of-freedom pose, use

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, \qquad R^TR = I,\ \det R = 1.$$

R is orientation; p is translation. Frame labels matter: T_{AB} maps coordinates expressed in frame B into frame A. For embodied learning, **consistent camera, robot, object, and table frames** are more important than proving every property of $SO(3)$ or $SE(3)$.

2.2 The Jacobian connects velocity and force

For end-effector position $p(q)$,

$$\boxed{\dot{p} = J_p(q)\dot{q}, \qquad \ddot{p} = J_p(q)\ddot{q} + \dot{J}_p(q,\dot{q})\dot{q}.}$$

The Jacobian first maps joint velocity to end-effector velocity. More importantly, an external Cartesian force f generates joint-space generalized force

$$\boxed{\tau_{\text{ext}} = J_p(q)^T f.}$$

The reason is **virtual work**: $f^T\delta p = f^T J_p \delta q = \tau_{\text{ext}}^T \delta q$. Thus, the table's force on a fingertip affects every joint through this mapping.

For full hand orientation and translation, one may use a six-dimensional rigid-body velocity (twist) and a six-dimensional Jacobian. The essential caution is to keep the **reference point, velocity representation, and wrench representation consistent**; a complete derivation of adjoint transformations is unnecessary here.

2.3 Why inverse kinematics cannot replace dynamics

Inverse kinematics converts a target end-effector pose into joint targets. It does not account automatically for object weight, rotational inertia, or contact forces. A geometrically feasible path may still be unrealizable within motor limits and time constraints. Picking up a heavy object changes the torques needed to hold the same pose.

Chapter 3. Mass, inertia, and gravity from one rigid link

3.1 Translation: mass resists acceleration

The center of mass of a rigid body satisfies Newton's second law:

$$\boxed{f = ma_C.}$$

For the same applied force, a larger mass generally produces a smaller center-of-mass acceleration. In grasping, identical policies can accelerate or drop objects differently when object mass changes.

3.2 Rotation: inertia depends on mass distribution

For rotation about a fixed axis, $\tau = I_O\ddot{q}$ when other torques are ignored. The rotational inertia I_O depends on **where the mass lies relative to the axis**, not just on total mass. If the center of mass is at distance r from the joint axis, the parallel-axis theorem gives

$$\boxed{I_O = I_C + mr^2.}$$

Two hands with identical masses but different mounting offsets can produce different effective joint inertia. Replacing an end-effector can therefore alter the dynamic behavior of the entire arm.

3.3 A one-joint pendulum already contains the main ideas

Consider a rigid link rotating in a vertical plane, with positive angle q measured upward from the horizontal. Its center of mass is distance r from the axis; its moment of inertia about that axis is I_O. Ignoring friction,

$$\boxed{I_O\ddot{q} + mgr\cos q = \tau.}$$

This includes the inertia term $I_O\ddot{q}$, gravity term $mgr\cos q$, and drive torque τ. A horizontal stationary link requires holding torque mgr; a vertically oriented link requires zero *static gravity* torque.

Key distinction: gravity compensation is configuration-dependent, not a fixed offset added to every joint. Being stationary does not imply zero motor torque.

3.4 Velocity-dependent coupling in three-dimensional rotation

For a rigid body expressed in a body-fixed frame, rotational motion satisfies

$$\tau_C = I_C\dot{\omega} + \omega \times (I_C\omega).$$

The second term reflects changes in angular momentum direction. It is enough to understand why the term exists: in embodied-learning practice, a multibody solver usually handles this coupling without requiring manual expansion of every link's three-dimensional equations.

Chapter 4. From one link to the manipulator equation

4.1 Why a mass matrix appears

Motion of one joint may carry many downstream links. Effective inertia therefore depends on the robot's entire configuration. Summing translational and rotational kinetic energies yields the quadratic form

$$\boxed{K(q,\dot{q}) = \tfrac{1}{2}\dot{q}^T M(q)\dot{q}.}$$

The generalized mass matrix $M(q)$ is generally **not diagonal**. Its off-diagonal entries express inertial coupling between joints. For independent generalized coordinates of a nondegenerate rigid mechanical system, $M(q)$ is symmetric positive definite.

4.2 Deriving the standard equation from energy

Let $U(q)$ be gravitational potential energy, and define the Lagrangian $L = K - U$. The Euler–Lagrange equations are

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = \tau_i.$$

Substituting quadratic kinetic energy and collecting terms gives

$$\boxed{M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau, \qquad g(q) = \nabla_q U(q).}$$

The velocity term $C\dot{q}$ contains squared velocities and products of different joint velocities. It is not an arbitrary damper or an external contact force; it comes from rigid-body motion itself. In practice, the bias vector $h = C\dot{q} + g$ is often used directly; the matrix representation of C is not unique.

4.3 Forward dynamics versus inverse dynamics

Forward dynamics predicts acceleration from state and applied torque:

$$\boxed{\ddot{q} = M(q)^{-1}[\tau - h(q,\dot{q})] \quad \text{(without contacts).}}$$

Inverse dynamics finds the torque required for a desired acceleration:

$$\boxed{\tau = M(q)\ddot{q} + h(q,\dot{q}).}$$

Physics simulation primarily uses the forward problem; gravity compensation, feedforward tracking, and certain whole-body controllers use the inverse problem. The inverse matrix above is conceptual notation; numerical implementations typically solve the associated linear system rather than explicitly computing M^{-1}.

4.4 Energy provides a useful consistency check

Without friction or external contact, total mechanical energy $E = K + U$ satisfies

$$\boxed{\dot{E} = \tau^T \dot{q}.}$$

Motor mechanical power changes stored mechanical energy. Dissipative friction removes energy; moving external contacts can add or remove energy through contact work. This identity helps

explain simulation energy drift, controller behavior, and compliance, without requiring a full stability-theory treatment.

Chapter 5. A complete two-link planar manipulator example

5.1 Geometry and sign conventions

Consider a two-revolute-joint arm moving in a vertical plane. q_1 is measured upward from the horizontal; q_2 is relative to link 1. Link 1 has length l_1, mass m_1, center-of-mass distance r_1 from joint 1, and center-of-mass inertia I_1. Link 2 has corresponding parameters m_2, r_2, I_2. Its absolute orientation is $q_1 + q_2$.

The center-of-mass positions are

$$\begin{aligned} p_1 &= (r_1 \cos q_1,\ r_1 \sin q_1)^T, \\ p_2 &= (l_1 \cos q_1 + r_2 \cos(q_1 + q_2), \\ &\qquad l_1 \sin q_1 + r_2 \sin(q_1 + q_2))^T. \end{aligned}$$

I_1 and I_2 are the link inertias about their centers of mass and normal to the plane of motion.

5.2 Kinetic energy reveals joint coupling

Total kinetic energy is

$$K = \frac{1}{2} m_1 \|\dot{p}_1\|^2 + \frac{1}{2} I_1 \dot{q}_1^2 + \frac{1}{2} m_2 \|\dot{p}_2\|^2 + \frac{1}{2} I_2 (\dot{q}_1 + \dot{q}_2)^2.$$

Collecting it as $K = \frac{1}{2} \dot{q}^T M(q) \dot{q}$ gives

$$\begin{aligned} M_{11} &= I_1 + I_2 + m_1 r_1^2 + m_2 (l_1^2 + r_2^2 + 2 l_1 r_2 \cos q_2), \\ M_{12} &= M_{21} = I_2 + m_2 (r_2^2 + l_1 r_2 \cos q_2), \\ M_{22} &= I_2 + m_2 r_2^2. \end{aligned}$$

Read the physics in the matrix: M_{11} varies with q_2, and M_{12} is usually nonzero. These are configuration-dependent inertia and cross-joint inertial coupling, respectively.

5.3 Velocity coupling and gravity

Let $b = m_2 l_1 r_2$. Then

$$C(q, \dot{q}) \dot{q} = \begin{bmatrix} -b \sin q_2 (2 \dot{q}_1 \dot{q}_2 + \dot{q}_2^2) \\ b \sin q_2 \dot{q}_1^2 \end{bmatrix}.$$

Gravitational potential energy is

$$U = m_1 g r_1 \sin q_1 + m_2 g [l_1 \sin q_1 + r_2 \sin(q_1 + q_2)],$$

so

$$g(q) = \begin{bmatrix} (m_1 r_1 + m_2 l_1) g \cos q_1 + m_2 r_2 g \cos(q_1 + q_2) \\ m_2 r_2 g \cos(q_1 + q_2) \end{bmatrix}.$$

The final result is $\tau = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q)$. We have followed the complete chain of center-of-mass positions → velocities → kinetic and potential energy → mass matrix and gravity vector. More elaborate symbolic three-dimensional calculations are unnecessary for understanding this structure.

5.4 Four thought experiments worth doing

1. **Stationary, no contact**: $\dot{q} = \ddot{q} = 0$ requires $\tau = g(q)$.
2. **Slow motion**: velocity-dependent terms are small relative to the most important inertial and gravity effects.
3. **Fast motion**: quadratic velocity terms become significant; neglecting $C\dot{q}$ introduces prediction error.
4. **A heavier second link**: both M and g change, so the same position-controller gains may need retuning.

Chapter 6. How dynamics scales to complex robots

6.1 Why we do not expand every rigid-body equation

A robot with dozens of joints is cumbersome to analyze through full symbolic expansion. Modern dynamics software exploits the kinematic tree and uses recursive algorithms. For embodied intelligence, the most important thing is to know **what each algorithm computes**, rather than memorizing its spatial-vector implementation.

Algorithm	Inputs	Output	Use in robot learning
RNEA: recursive Newton–Euler	$q, \dot{q}, \ddot{q}$	Inverse dynamics force	Gravity compensation and torque feedforward
CRBA: composite rigid-body algorithm	q	$M(q)$	Inertial coupling and model-based control
ABA: articulated-body algorithm	$q, \dot{q}, \tau$	$\ddot{q}$	Forward dynamics and physics simulation

RNEA propagates velocity and acceleration from base to tip, then accumulates forces back toward the base. ABA uses articulated-body inertia to solve accelerations efficiently. CRBA assembles the composite inertia of downstream links. With matching models and conventions in free space, forward and inverse results should agree.

6.2 Gravity compensation is a special inverse-dynamics case

Setting desired joint velocity and acceleration to zero gives, for an ideal rigid manipulator,

$$\boxed{g(q) = \mathrm{RNEA}(q, 0, 0).}$$

This is physically more appropriate than setting a constant torque at every joint. If the robot carries a payload, its mass, center of mass, and inertia must be included in the model; an algorithm can be numerically correct yet predict the wrong physics when its parameters are wrong.

Chapter 7. Actuators: how policy actions reach the real robot

7.1 A position target is not an instantaneous position jump

A robot-learning policy often outputs q_{target}. The motor does not instantly transport the robot to that configuration. A representative joint PD actuator uses

$$\boxed{\tau_{\text{servo}} = K_p(q_{\text{target}} - q) + K_d(\dot{q}_{\text{target}} - \dot{q}).}$$

The physical dynamics then determine $\ddot{q}$. The actual chain is target position → servo torque → acceleration → velocity → position. Some real robots hide this servo within a drive, exposing only a position interface to the learning system.

7.2 Nonideal effects that materially change behavior

Effect	Physical consequence	Training/deployment failure
Torque or current saturation	Acceleration cannot grow indefinitely	Requested actions are not realized
Velocity limit	Very fast motion is clipped	Tracking lag and contact timing shifts
Friction and backlash	Small commands may produce no motion	Poor precision at low speed
Delay and filtering	Commands act later than expected	Overshoot and oscillation
Motor and payload inertia	Dynamic response changes with the tool	Same gains, different results
Compliant transmission	Joints and fingers deform elastically	Different contact-force response

For an ideal gear ratio N, motor rotor inertia I_m reflects approximately as $N^2 I_m$ at the joint. Link inertia alone thus cannot characterize actuator dynamics. Real drives may also include current, velocity, and position loops plus thermal protection.

7.3 Map policy action spaces to physical interfaces

Policy action	Advantage	Physical caveat
Target joint position	Stable and common in imitation learning	Depends on servo gains and command period
Joint-position increment	Convenient for residual tracking	Drift and joint-limit handling
Target joint velocity	Directly expresses motion direction	Integration and low-level-drive differences
Joint torque	Direct interaction with dynamics	Harder training, saturation and safety
End-effector pose increment	Task-relevant representation	Needs inverse kinematics and contact handling

Conclusion: when comparing robot-learning algorithms, report the low-level actuator model as well as the nominal action space; otherwise the same action symbol may represent different physical control systems.

Chapter 8. Contact dynamics: grasping, pushing, sliding, and dropping

8.1 Where does contact force come from?

If a fingertip remains in fixed contact with a table, the contact point has zero velocity relative to the table:

$$J_c(q)\dot{q} = 0, \qquad J_c(q)\ddot{q} + \dot{J}_c(q,\dot{q})\dot{q} = 0.$$

Dynamics must also include the generalized contact force:

$$\boxed{M(q)\ddot{q} + h(q,\dot{q}) = \tau + J_c(q)^T\lambda.}$$

Acceleration and contact force are therefore **coupled**. If the robot tries to move into the table, the table supplies a normal force that prevents penetration and changes the resulting motion and joint torques. The zero-relative-velocity constraint applies only to a sustained sticking contact, not to every contact mode.

8.2 Why contact is unilateral

Let the normal gap between surfaces be $\phi(q) \geq 0$ and the compressive normal force be $\lambda_n \geq 0$. Ideal rigid unilateral contact satisfies

$$\boxed{\phi(q)\lambda_n = 0.}$$

This says: **either the surfaces are separated and the force is zero, or they touch and can sustain compression**. A normal table cannot keep pulling a screwdriver downward after it has lifted away. Adhesion, suction, and magnetism require separate models.

8.3 The friction cone determines whether an object stays grasped

For Coulomb friction, tangential contact force λ_t and normal force λ_n satisfy

$$\boxed{\|\lambda_t\| \leq \mu\lambda_n.}$$

μ is the friction coefficient. Increasing normal grip force generally increases the maximum available static friction, subject to motor limits, geometry, deformation, and safety. An object can slip because of low normal force, low friction, changing gravity direction, or increased inertial load during rapid manipulation.

8.4 An impact produces a rapid velocity change

When a finger collides with an object, large forces may act over a short interval. An impulse P approximates the change in generalized velocity across impact:

$$\boxed{M(q)(v^+ - v^-) = J_c(q)^T P.}$$

Contact solvers handle not only sustained support forces but also velocity changes during collisions. Different simulators may employ different contact compliance, time discretization, friction

approximations, and constraint solvers. Equal masses and friction coefficients do not guarantee identical collision histories.

8.5 Distinguish four modes during a grasp

Mode	Observable behavior	Implication for the policy
No contact	A gap exists between finger and object	Approach before exerting grasp force
Sticking contact	Contact points have near-zero relative velocity	A stable manipulation constraint can form
Sliding contact	Contact points move tangentially	Adjust grip, trajectory, or planning
Separation/drop	Contact forces disappear; object moves independently	Relocalize, approach, and regrasp

This explains why **drop recovery is not ordinary trajectory tracking**: the contact mode has changed, and the controller must establish new contacts rather than continue an obsolete motion reference.

Chapter 9. Dexterous manipulation is a coupled arm–hand–object system

9.1 Why controlling the fingers alone may be insufficient

Let q_r be generalized robot coordinates, including arm and hand, and let x_o be the object pose. Fingertip forces act on both the robot and the object, with action and reaction. On the robot side,

$$M_r(q_r)\ddot{q}_r + h_r = \tau_r + J_r^T \lambda.$$

On the object side, linear and angular momentum change in response to gravity and fingertip contact forces. Force signs must be chosen consistently: the two bodies do not both experience the same contact force in the same direction.

When the palm transports a rigidly held object, combined payload mass, center of mass, and inertia dominate the arm response. When the object rolls or rotates within the fingers, contact locations, friction, and object orientation become central. The arm, hand, and object should be treated as **one coupled physical system**.

9.2 When an object may be treated as extra payload

If an object remains firmly grasped and scarcely moves relative to the palm, it can be approximated as a rigid additional payload. Updating the total end-effector mass, center of mass, and inertia then supports gravity compensation and slow transport.

The approximation fails when the object rotates in the fingers, slides, changes contact points, or drops. Those phases require explicit object degrees of freedom and contact constraints.

9.3 Practical lessons for Tron2 with a Revo3 hand

After replacing a dexterous hand, verify at least: mounting frame and tool center point (TCP); payload mass, center of mass, and inertia; joint gains and force limits; fingertip collision geometry; fingertip–object friction; observation rate and command delay.

Changing the TCP is primarily a **geometric and Jacobian change**, not a change in physical inertia. Replacing the actual hand changes the physical payload. Head and wrist cameras provide object-pose observations but do not directly measure accurate contact forces. Without tactile sensors, finger current, joint behavior, and visual motion can provide uncertain proxies for contact.

Chapter 10. Humanoids: from fixed-base to floating-base dynamics

10.1 Why the torso cannot be driven like an ordinary joint

A table-mounted robot arm has a fixed base. A humanoid torso can translate and rotate freely in space, yet it generally has no motors that directly actuate these six world-space base degrees of freedom. For n joints, floating-base generalized velocity has dimension $n+6$. If orientation is represented by a quaternion, configuration variables typically have dimension $n+7$; configuration and velocity vectors are not interchangeable.

The dynamics become

$$\boxed{M(q)\dot{v} + h(q,v) = S^T\tau + J_c(q)^T\lambda,}$$

where S inserts actuated joint torques into the full generalized force vector. Base motion depends on the coupled effects of joints and external contacts.

10.2 Balance requires external support forces

For the whole-robot center of mass c,

$$\boxed{m\ddot{c} = m\mathbf{g} + \sum_i f_i.}$$

During quiet standing, $\ddot{c} \approx 0$, so ground contact forces approximately balance gravity. Without foot contact, internal joint torques cannot prevent the entire robot's center of mass from falling. Reaching, walking, and dexterous grasping all involve **the relationship between external force and available actuation**.

10.3 Only the essential centroidal dynamics and WBC concepts

The change in angular momentum about the center of mass follows external moments:

$$\dot{L}_c = \sum_i (r_i - c) \times f_i + \sum_i m_i^{\text{ext}}.$$

Whole-body control (WBC) considers end-effector tracking, torso balance, fixed foot contacts, actuator limits, friction cones, and joint limits together. Model-based WBC can explicitly optimize accelerations and contact forces; an RL locomotion policy can instead learn behavior through

simulation and reward. **Both rely on the same underlying physics**, but a learned policy need not solve inverse dynamics or a quadratic program during deployment.

Chapter 11. Dynamics-based control that matters for embodied intelligence

11.1 PD with and without gravity compensation

Given desired joint position q_d and velocity $\dot{q}_d$, a standard torque law is

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}).$$

Adding model-based gravity compensation gives

$$\boxed{\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + g(q).}$$

This is the simplest way to understand a position-action interface's low-level feedback loop. It also explains why changing payloads often requires model or gain updates. Gravity compensation alone does not guarantee stable contact behavior.

11.2 Computed-torque control

In free space, with accurate dynamics and accessible torque control, let $e = q_d - q$ and define

$$\ddot{q}^* = \ddot{q}_d + K_d\dot{e} + K_p e.$$

Choose

$$\boxed{\tau = M(q)\ddot{q}^* + h(q, \dot{q}).}$$

Under ideal assumptions the resulting error dynamics are $\ddot{e} + K_d\dot{e} + K_p e = 0$. The derivation shows how a model can compensate nonlinear mechanical coupling to yield simple tracking-error dynamics. Real robots remain subject to saturation, friction, inaccurate parameters, and contact disturbances.

11.3 Impedance control supports compliant contact

In dexterous manipulation, infinitely stiff position tracking is undesirable. A simplified desired Cartesian impedance force is

$$F_d = K_x(x_d - x) + D_x(\dot{x}_d - \dot{x}),$$

which is mapped to joint torque through the Jacobian and combined with gravity or other dynamic terms as necessary. Compared with rigid tracking, impedance control deliberately permits some displacement in response to contact force, supporting safer and more compliant interaction.

11.4 Where MPC and WBC fit

Model predictive control (MPC) predicts several future steps using dynamics and optimizes control inputs; WBC concentrates on satisfying full-body motion tasks and contact constraints at once. A

detailed optimization-solver derivation is beyond this tutorial. The important principle is that **physical constraints determine which desired accelerations, torques, and contact forces can coexist**. High-level VLA action targets are not automatically dynamically feasible.

Chapter 12. MuJoCo versus Isaac Sim / Isaac Lab: appropriate roles

12.1 What each platform contributes

MuJoCo is well suited to building mechanical intuition, measuring sensitivity to physical parameters, investigating contact behavior, and running comparatively lightweight arm and hand experiments. Its efficient simulation does not eliminate the need to understand contact models and solver settings.

Isaac Sim provides robot and scene simulation capabilities. **Isaac Lab** organizes robot-learning environments, parallel experiments, action/observation/reward pipelines, and domain randomization on top of simulation capabilities. Isaac Sim and Isaac Lab are not two independent physics engines. The active physics backend, version, and actuator semantics must be taken from the documentation matching the local installation.[4–7]

Research question	MuJoCo emphasis	Isaac Sim / Isaac Lab emphasis
Why does a link fall?	Minimal model with interpretable torque terms	Verify body properties and joint drives
Why does a grasp slip?	Geometry, friction, and numerical contact settings	Multi-contact scenes, batched environments, randomization
Why does real-robot performance degrade?	Small, diagnostic parameter sweeps	Large-scale training and sim-to-real variation
How should a controller be evaluated?	Transparent single-environment baseline	Scaling to full robots and tasks

12.2 Matching model filenames is not enough

A fair cross-platform comparison must align body mass and center of mass, inertia tensor and inertial frame, joint axes and limits, base fixation, collision geometry, gravity, actuator gains and limits, physics time step, control rate, initial conditions, and contact/friction settings.

Even after careful matching, contact compliance, discretization, and constraint solvers may differ. **Compare contact-free motion first, then gradually add contact.** Neither simulator should be treated as ground truth for the other.

12.3 Three time scales must be identified separately

The physics integration step Δt_{phys}, servo/controller update period Δt_{ctrl}, and policy action period Δt_{π} may be different. A low-frequency policy may send position targets while the low-level PD loop runs multiple times between actions and the solver performs even smaller physics steps.

A fair policy comparison must report all three frequencies and how actions are held or interpolated; otherwise nominally identical policies can experience different contact transients and delays.

Chapter 13. Code-free laboratory guide: free-space mechanics first

Use **the same one-link pendulum and the same two-link arm** in both platforms as the common starting point. Each lab gives an objective, setup, measurements, and a prediction. No commands, software snippets, or executable scripts are required to understand the exercise.

Lab 1 | One-link gravity and inertia

Objective: separate the effects of mass, center of mass, and rotational inertia. **Setup:** create a single link on a fixed rotational axis in a vertical plane. Release it from a horizontal pose, then independently vary mass, center of mass, and inertia within physically plausible ranges. **Measure:** $q(t)$, $\dot{q}(t)$, $\ddot{q}(t)$, and gravitational joint torque. **Expected:** changing center of mass changes both gravity torque and joint inertia; a stationary horizontal link requires nonzero holding torque.

Check: align axis direction, gravity, and zero-angle convention in MuJoCo and Isaac Sim, or the gravity-torque curves may differ by a sign or phase.

Lab 2 | Two-link mass matrix and coupling

Objective: understand why $M(q)$ depends on configuration. **Setup:** use the two-link parameters from Chapter 5, hold q_1 fixed, and gradually vary q_2 while examining equivalent joint inertia and static holding torque. **Measure:** diagonal and off-diagonal mass-matrix terms and $g(q)$. **Expected:** M_{11} and M_{12} vary with $\cos q_2$; joints are inertially coupled.

Check: remove contact and unintended drives when comparing mass-matrix properties, and verify SI units.

Lab 3 | Forward–inverse dynamics consistency

Objective: verify $M\ddot{q} + h = \tau$. **Setup:** choose matching nonsingular robot configurations, velocities, and desired accelerations. Determine the generalized forces required for the desired acceleration, then predict free-space acceleration under those forces. **Measure:** acceleration discrepancy and equation residual. **Expected:** values are close with consistent conventions; discretization and actuator modeling introduce differences.

Check: a position-servo target is not a joint torque. Identify whether additional low-level controllers are active.

Lab 4 | PD control versus gravity-compensated PD

Objective: connect feedback to model feedforward. **Setup:** hold a two-link arm in horizontal and downward-pointing configurations; compare pure PD against PD with gravity compensation. **Measure:** steady-state error, maximum joint effort, and settling time. **Expected:** with suitable gains and a correct model, gravity compensation reduces pose-dependent error; it cannot remove all delay or unmodeled friction.

Lab 5 | Action rate and actuator saturation

Objective: see why identical policy targets can result in different motion. **Setup:** hold the target trajectory constant but vary physics step, servo rate, policy rate, torque limit, and PD gains. **Measure:**

tracking error, peak torque, overshoot, and lag. **Expected:** slower command updates or stricter limits may increase tracking error; excessively high gains can destabilize contact interactions or coarse discretization.

Chapter 14. Code-free laboratory guide: contact, hands, and humanoids

Lab 6 | A falling block: normal contact and impact

Objective: observe the transition from free fall to supported contact. **Setup:** drop blocks with the same mass and collision geometry from the same height onto a table in MuJoCo and Isaac Sim, then vary contact settings in a controlled manner. **Measure:** height, vertical speed, penetration, normal force/impulse, and resting jitter. **Expected:** impact details can differ across platforms; after settling, average normal support force should approximately equal object weight.

Lab 7 | Sliding on a slope: friction intuition

Objective: study friction coefficient and sliding threshold. **Setup:** place a block on a slowly increasing incline and repeat with different static-friction parameters. **Measure:** onset angle, sliding velocity, and normal force. **Expected:** ideal Coulomb static friction gives $\tan\theta_c \approx \mu$; static-friction approximations and time discretization can shift this threshold in real solvers.

Lab 8 | Dexterous grasp, rotation, and drop recovery

Objective: combine mass, inertia, friction, and contact-mode changes in one task. **Setup:** keep screwdriver geometry fixed, then vary object mass/center of mass, friction, initial orientation, grip location, and manipulation speed. Define phases for grasp → lift → rotate → disturb → regrasp when necessary. **Measure:** success rate, object orientation error, slip distance, contact duration, post-drop recovery rate, and number of actions.

Expected: rapid rotation raises inertial loads and low friction increases slip risk. Successful recovery does not prove that the initial grasp was optimal; check whether the policy actually detected the new contact state. Before sim-to-real on Tron2 + Revo3, verify mounting geometry, payload data, and fingertip materials.

Lab 9 | Humanoid standing: contact force and center of mass

Objective: verify that the floating base cannot be suspended by internal actuation. **Setup:** choose a humanoid or quadruped with foot contact, and compare stable standing, a small forward body lean, and lifting one foot. **Measure:** center-of-mass projection, ground reaction force, torso pose, and joint effort. **Expected:** support forces redistribute and stability requires joint behavior consistent with contact and center-of-mass mechanics.

Lab 10 | Cross-simulator and sim-to-real sensitivity

Objective: identify the physical parameters that matter most for learning. **Setup:** first align the contact-free two-link models, then introduce friction, payload, joint delay, and contact. Vary one parameter family at a time before joint domain randomization. **Measure:** dynamics residual,

trajectory error, contact duration, task success, and robustness curves. **Expected:** free-space comparisons are easier to make consistent; contact and actuator behavior are often the most difficult factors to match.

Minimal one-page lab-report structure

Record six items for every experiment: **research question, physical state and action definition, physical parameters, intervention, measured metrics, and conclusion/limitations**. At minimum, plot position and velocity over time; for contact labs add normal force or impulse. Cross-platform plots need matching units, time axes, initial conditions, and event labels.

Chapter 15. From simulation to real robots: a concise checklist

15.1 Physical parameters must remain self-consistent

Do not randomize every inertial parameter independently without constraints. Mass, center of mass, and rotational inertia are physically related. Moving a payload's mass distribution generally changes inertia as well. Inertia tensors should be symmetric and physically realizable. Collision geometry, inertial frames, and visual meshes should have clearly distinguished roles.

15.2 Prioritize sim-to-real variation by plausible error sources

For manipulation, a practical order is: payload mass and center of mass, joint friction, actuator gains and saturation, command delay, fingertip–object friction, contact-geometry error, and initial object pose. Domain randomization should cover a **plausible real-world distribution**, not simply maximize parameter ranges.

With RGB-D observations, also assess visual noise, calibration error, and occlusion separately. Perception error changes the policy's estimate of the state; dynamics error changes how the real state evolves after an action. Both matter, but they should not be conflated.

15.3 Four misconceptions to avoid

- **"The URDF is correct, so the dynamics must be correct."** No. Collision, actuator, friction, and solver properties also matter.
- **"A position-output policy does not need dynamics."** No. Position targets still act through motor forces.
- **"Gravity compensation guarantees a stable grasp."** No. Friction, payload changes, and dynamic disturbances remain.
- **"Different simulator results mean one simulator is wrong."** Not necessarily. Contact models and time-stepping methods can produce different effective behavior.

15.4 One-page equation reference

Scenario	Core relation	Interpretation
End-effector motion	$\dot{p} = J_p \dot{q}$	Joint velocity determines Cartesian velocity

Scenario	Core relation	Interpretation
End-effector force	$\tau_{\text{ext}} = J_p^T f$	Map Cartesian force to joint generalized force
Free-space manipulator	$M\ddot{q} + h = \tau$	Actuation determines joint acceleration
Manipulator with contact	$M\ddot{q} + h = \tau + J_c^T \lambda$	Contact force changes the motion
Static gravity compensation	$\tau = g(q)$ (without contact)	Holding torque depends on configuration
Friction cone	$\lvert\lambda_t\rvert \le \mu\lambda_n$	Limits sticking-contact force
Floating base	$M\dot{v} + h = S^T\tau + J_c^T\lambda$	The base is underactuated and needs external forces
Center-of-mass translation	$m\ddot{c} = m\mathbf{g} + \sum f_i$	Support forces govern standing and lifting

Appendix A. What was kept from the original course

MEE5114 material	Location here	Editorial treatment
Lectures 2–7: poses, twists, POE, Jacobians	Chapter 2	Keep frames and Jacobians; omit detailed POE derivation
Lecture 8: rigid-body dynamics	Chapters 3–4	Retain inertia and physical intuition; omit spatial-operator details
Lecture 9: dynamics of open chains	Chapters 4–6	Retain manipulator equation and roles of RNEA / CRBA / ABA
Lecture 14: robot motion control	Chapters 7 and 11	Retain servos, gravity compensation, computed torque, impedance
Added: contact and manipulation	Chapters 8–9	Essential to embodied tasks
Added: floating-base dynamics	Chapter 10	For humanoids and legged robots
Added: two-platform experiments and sim-to-real	Chapters 12–15	Objectives and measurements, with no code

Appendix B. Notation, reading route, and references

Notation: $q, \dot{q}, \ddot{q}$: joint position, velocity, acceleration; M: joint-space mass matrix; $C\dot{q}$: Coriolis/centrifugal term; $g(q)$: generalized gravity force; $h = C\dot{q} + g$: nonlinear bias; τ: actuated generalized force; J: Jacobian; λ: environmental contact force; μ: friction coefficient; v: floating-base generalized velocity; S: actuation-selection matrix.

Shortest learning path: study Chapters 1–5 and carry out Labs 1–4; continue to Chapters 7–9 with Labs 5–8. If you work on humanoids or quadrupeds, add Chapter 10 and Lab 9. Lab 10 is designed for cross-simulator consistency and sim-to-real sensitivity.

1. MEE5114 — Advanced Control for Robotics, Wei Zhang: course structure, especially Lectures 8, 9, and 14.
2. Modern Robotics — Chapter 8: Dynamics of Open Chains: Lagrangian mechanics, forward/inverse dynamics, constraints, and actuation.

3. Modern Robotics — Chapter 11: Robot Control: joint-space and task-space control.
4. Featherstone — Rigid Body Dynamics Algorithms / Spatial Vector Resources: deeper analysis of RNEA, CRBA, and ABA.
5. MuJoCo — Computation: multibody dynamics, actuators, contacts, and constraint solving.
6. Isaac Lab — Articulations: multibody structures, fixed and floating bases, action interfaces.
7. Isaac Lab — Actuators: explicit/implicit actuator semantics; match the installed version.
8. MIT — Underactuated Robotics: Multibody Dynamics: further reading on multibody and underactuated systems.

Isaac Sim, Isaac Lab, and MuJoCo: A Complete Guide to Robot Simulation and Reinforcement Learning

Course home · 简体中文 · PDF

Item	Details
Applicability	Documented tracks: Isaac Lab 2.3.2 / Isaac Sim 5.1; Lab 3.0 Early Access / Sim 6.1. MuJoCo, MJX and MJWarp releases must be selected per task
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

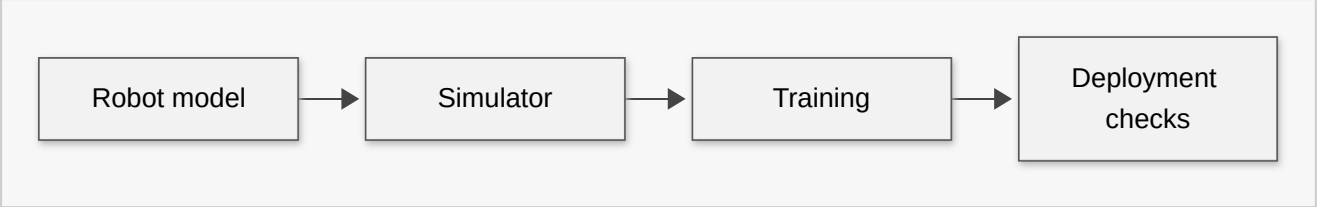
Contents

- 1. What are these platforms, and how do they differ?
  - 1.1 Their roles in the robotics workflow
  - 1.2 Which tool should you select?
- 2. Versions, installation strategy, and hardware precautions (without commands)
  - 2.1 Two sensible version tracks
  - 2.2 Where should the compute run?
  - 2.3 Acceptance checks: launching is not enough
- 3. Learning to operate both platforms without writing code here
  - 3.1 A practical MuJoCo learning sequence
  - 3.2 An Isaac Sim / Isaac Lab learning sequence
  - 3.3 A shared exercise: Panda, cube, and table
- 4. Modeling robots across URDF, MJCF, and USD
  - 4.1 There is no universally lossless conversion
  - 4.2 Ten checks after import
- 5. Dynamics, control interfaces, and timing
  - 5.1 A common mechanics perspective
  - 5.2 Five common control interfaces
  - 5.3 Keep three clocks separate
  - 5.4 Fair control comparisons
- 6. Contact, friction, vision, and sensing

    - 6.1 Why dexterous contact is difficult
    - 6.2 Standard contact validation tests
    - 6.3 Four sources of visual closed-loop error
    - 6.4 Should a policy observe vision or privileged state?
- 7. Define the RL task before training
    - 7.1 Six ingredients of a learning task
    - 7.2 A practical task hierarchy
- 8. Choosing PPO, SAC, and related methods
    - 8.1 Engineering differences between PPO and SAC
    - 8.2 A defensible selection rule
    - 8.3 Sample efficiency is not total efficiency
- 9. Designing a trainable RL environment: detailed checklist
    - 9.1 Observation design
    - 9.2 Action design
    - 9.3 Reward design
    - 9.4 Termination, time limits, and recovery
    - 9.5 Reset and reference-state initialization (RSI)
    - 9.6 Curriculum learning
- 10. A complete Isaac Lab reinforcement learning workflow
    - 10.1 From a benchmark to a custom manipulation task
    - 10.2 Selecting the Isaac Lab RL library
    - 10.3 Common Isaac Lab RL pitfalls
- 11. MuJoCo RL: classic CPU simulation, MJX, and MJWarp
    - 11.1 Classic MuJoCo: correctness first
    - 11.2 MJX and MJWarp: throughput-oriented training
    - 11.3 Ten steps for a MuJoCo RL project
- 12. PPO and SAC tuning: what to check and in which order
    - 12.1 PPO: diagnose the environment before tuning the optimizer
    - 12.2 SAC: what deserves the most attention?
    - 12.3 Three misleading signs of “successful learning”
- 13. Training efficiency: batches, vision, and bottlenecks
    - 13.1 Measure at least five quantities
    - 13.2 More environments are not always better
    - 13.3 Two routes to visual RL
- 14. Domain randomization and robust learning in a sensible order
- 15. Evaluation, ablations, and reproducibility for publishable experiments
    - 15.1 Core metrics
    - 15.2 Minimal ablation matrix
    - 15.3 Avoid unsupported conclusions
- 16. Sim-to-real transfer: from policy weights to a safe robot

    - 16.1 Six calibration layers
    - 16.2 A deployment validation ladder
- 17. End-to-end case study: screwdriver manipulation with Tron2 and Revo3
    - 17.1 Define the task precisely
    - 17.2 Recommended system development route
    - 17.3 Five critical choices
    - 17.4 Stage-specific measurements
- 18. Common failure symptoms and debugging priorities
- 19. Suggested six-week learning plan and deliverables
- 20. Final readiness checklist
- Official references and further reading

Chapter concept map



1. What are these platforms, and how do they differ?

1.1 Their roles in the robotics workflow

- **Isaac Sim** is NVIDIA's robotics simulation platform for scene authoring, USD assets, PhysX simulation, RTX rendering, camera simulation, and ROS 2 integration. Its primary concern is *how a robot and its environment operate and are perceived in simulation*.
- **Isaac Lab** is a robot learning framework built on simulation backends. It provides task definitions, vectorized environments, observations, actions, rewards, resets, termination rules, domain randomization, and interfaces to RL libraries. Its primary concern is *how to train and evaluate a policy systematically*.
- **MuJoCo** is a dynamics and contact simulation engine useful for rigid-body mechanics, constraints, actuators, control, and model debugging. Traditional MuJoCo is primarily CPU-based; MJX and MuJoCo Warp (MJWarp) enable accelerated batched simulation.

Important recent development: Isaac Lab 3.0 Early Access introduces a multiple-physics-backend architecture, including Newton/MuJoCo-Warp-based execution paths. Accordingly, “Isaac Lab always means PhysX” and “MuJoCo always means CPU simulation” are no longer generally accurate. Backend feature parity and API compatibility must still be checked for each release. [1–4]

Dimension	Isaac Sim	Isaac Lab	MuJoCo and GPU ecosystem
Primary focus	Scenes, sensing, digital twins	Robot learning and batched tasks	Dynamics, contact, control, batching

Dimension	Isaac Sim	Isaac Lab	MuJoCo and GPU ecosystem
Common assets	USD; URDF import	Backend-supported assets	MJCF; URDF import
Visualization	Strong RTX rendering	Depends on the selected renderer	Lightweight viewer; backend-dependent batched rendering
RL integration	Requires a learning/task layer	RSL-RL, RL-Games, skrl, etc.	MuJoCo Playground, Gymnasium, custom stacks
Best first use	Scene and sensor inspection	Vectorized reinforcement learning	Dynamics and control
Main pitfalls	GPU drivers, VRAM, RTX support	Version/API changes, rewards and resets	Actuation, contacts, CPU/GPU discrepancies

1.2 Which tool should you select?

Research goal	Suggested starting point	Why
Dynamics, PD/MPC, inverse dynamics	MuJoCo	Direct access to physical behavior and straightforward single-step debugging
Many parallel dexterous-hand PPO environments	Isaac Lab or MuJoCo Playground	Vectorized rollouts and structured tasks
Replay-buffer-based SAC	A compatible MuJoCo RL stack or supported Isaac Lab integration	Confirm that SAC and vectorized replay are actually implemented
Head and wrist RGB-D, lighting and occlusions	Isaac Sim / Isaac Lab	Integrated rendering and camera modeling
High-throughput state-based contact simulation	Suitable Isaac Lab backend or MJX/MJWarp	Depends on scene complexity, hardware, and data movement
Pre-deployment robustness checks	Both physics engines	Exposes dependence on one solver or actuator approximation

Selection rule: Decide on the physical task and observations before optimizing raw simulation FPS. Photorealism, contact fidelity, and real-world compatibility may be more consequential than peak steps per second.

2. Versions, installation strategy, and hardware precautions (without commands)

2.1 Two sensible version tracks

Stable reproduction track: Isaac Lab 2.3.2 with Isaac Sim 5.1. This is a practical option for replicating existing 2.x research code, RSL-RL pipelines, and established PhysX manipulation tasks. Pin the Git tag, Python, PyTorch, extensions, and GPU driver rather than installing moving targets.

New architecture track: Isaac Lab 3.0 Early Access with Isaac Sim 6.1 when full Isaac Sim integration is needed. Version 3.0 adds selected kit-less workflows and a multi-backend architecture, but includes breaking changes. As of this guide's date it is still Early Access; older tasks

and extensions should not be assumed compatible. A particularly important migration detail is the change from the common **wxyz** quaternion convention in 2.x to **xyzw** in 3.0, alongside data interface changes. [1][2]

Recommended setup sequence: (1) Identify the paper or feature to reproduce; (2) consult the relevant release compatibility table; (3) keep Isaac and MuJoCo dependencies isolated; (4) validate an official minimal scene; (5) check simulation stepping, asset import, and GPU use; and only then (6) connect the RL trainer. Avoid installing unrelated machine-learning packages before the simulator baseline works.

2.2 Where should the compute run?

- **RTX 4090, 24 GB:** Well suited to GPU physics training and carefully budgeted vision-based experiments. High-resolution multi-camera tasks still need active memory management.
- **RTX 3090, 24 GB:** Useful for compatible experiments and MuJoCo GPU training; however, Isaac Sim 5.1 lists an RTX 4080 as its minimum GPU performance tier. Treat a 3090-based Isaac Sim setup as a configuration requiring its own verification.
- **A100/A800:** Excellent for many neural-network workloads and compute paths that do not need RTX rendering. **The A100 is not supported for the full RTX-dependent Isaac Sim 5.1 experience.** High tensor throughput does not imply RTX renderer compatibility.
- **CPU machines:** Often appropriate for classic MuJoCo model validation, MPC, single-robot control, and smaller rollouts. Move to MJX/MJWarp when a measured batch-throughput need justifies it.

Official Isaac Sim 5.1 requirements specify supported Ubuntu configurations, GPU classes, memory, storage, and driver expectations. Updating every dependency to “latest” can easily break an otherwise working simulator stack. [5]

2.3 Acceptance checks: launching is not enough

Check	Acceptance criterion
Basic scene	Ground, gravity, and robot load; no unintended penetration
Joint motion	Directions, limits, home positions, and velocities verified joint by joint
Robot state	Poses, velocities, and forces agree with the model definitions
Rendering and cameras	Correct orientation, depth interpretation, timestamps, and resolution
Physics stability	No persistent resting oscillation; plausible free fall
RL connectivity	Observation-action-step-reset-reward loop operates correctly
Reproducibility	Software versions, asset hashes, GPU, driver, and random seeds recorded

3. Learning to operate both platforms without writing code here

3.1 A practical MuJoCo learning sequence

1. **Understand the model:** A robot is built from bodies, joints, geoms, actuators, sensors, and constraints. Its rendered geometry and collision geometry can be distinct.
2. **Inspect a robot:** Check joint axes, home positions, collision shapes, inertia, and tool reference frames in the viewer.
3. **Test passive motion:** Observe a pendulum or arm moving under gravity; compare physical intuition with simulated behavior.
4. **Test actuation:** Examine position, velocity, and torque control separately. An actuator's command is not necessarily an applied joint torque.
5. **Introduce contacts:** Add a tabletop and a small object. Study free fall, sliding, grasping, and release.
6. **Prepare for RL:** Specify observations, actions, resets, and randomization before integrating a trainer.

Recommended first complete robot: Franka Panda. It is sufficiently complex for useful manipulation while remaining easy to inspect; start from a maintained model in MuJoCo Menagerie. [6]

3.2 An Isaac Sim / Isaac Lab learning sequence

1. **Scene authoring in Isaac Sim:** Learn the Stage, Prims, USD assets, viewport, timeline, physical materials, and rigid-body properties.
2. **Import a robot:** Inspect the articulation root, drives, masses, joint limits, and collision geometry after URDF import.
3. **Build a simple scene:** Ground, table, fixed-base arm, manipulated object, and lighting.
4. **Validate sensors:** Place head and wrist cameras; verify intrinsics, extrinsics, depth semantics, occlusions, and synchronization.
5. **Study the Isaac Lab task abstraction:** Scene, robot assets, observations, actions, rewards, terminations, and events/domain randomization.
6. **Try maintained benchmark tasks:** Cartpole, Franka Reach, Franka Lift, then dexterous reorientation.
7. **Add RL gradually:** Start with state-based policies and limited randomization before adding vision and difficult contacts.

Manager-based vs. Direct workflows: Manager-based tasks emphasize reuse of observation, reward, reset, and event components. Direct tasks offer more explicit control for unusual contact phases or research algorithms. Both support reinforcement learning; the difference is software organization rather than inherent learning power. [7]

3.3 A shared exercise: Panda, cube, and table

Do not begin by comparing two entirely different robots in different engines. Fix the robot, cube dimensions, table height, tool frame, initial pose distribution, and action definition. Progress through **single-joint response → end-effector tracking → cube grasping → perturbed regrasping**. The same evaluation sequence can later be used with Tron2 and a Revo3 hand.

4. Modeling robots across URDF, MJCF, and USD

4.1 There is no universally lossless conversion

Modeling aspect	URDF	MJCF	USD / Isaac
Links, joints, inertia	Standard capabilities	Supported	Supported
Visual and collision meshes	Supported	Rich contact modeling	Strong scene and rendering support
Motors and actuation	Limited in core standard	Actuators, tendons, transmissions	Joint drives and physics properties
Advanced contact behavior	Usually engine-specific extensions	Detailed contact parameters	PhysX contacts and materials
Scene composition	Relatively basic	Physics-centric	Scalable scene composition

Maintain a **canonical physical parameter sheet** covering joint names, axis orientations, zero positions, limits, link masses, centers of mass, inertias, TCP, collision meshes, actuation modes, and sensor extrinsics. This provides better cross-engine consistency than an URDF file alone.

4.2 Ten checks after import

1. Base and world frames; 2. joint names and ordering; 3. positive rotation directions; 4. angle and length units; 5. tool-center-point definition; 6. mass and center of mass; 7. physically plausible inertias; 8. collision simplification; 9. appropriate self-collision filtering; and 10. actuation limits, stiffness, damping, and control period.

Additional dexterous-hand checks: Coupled joints, differential mechanisms, tendons, underactuation, fingertip material properties, and finger-joint friction. Matching the visible hand geometry is not enough to establish valid contact physics.

5. Dynamics, control interfaces, and timing

5.1 A common mechanics perspective

For rigid-body robots, the essential balance is:

Inertial effects + Coriolis/centrifugal/gravity-related bias terms = generalized actuator forces + generalized contact/constraint forces.

MuJoCo and PhysX use different numerical implementations. Before attributing discrepancies to the engine, first make robot parameters, actuation, and contact scenarios as equivalent as possible.

5.2 Five common control interfaces

Control style	Policy produces	Main caveats
Joint position targets	Desired joint angles	Hidden PD gains, torque saturation, velocity limits

Control style	Policy produces	Main caveats
Joint position increments	Changes relative to a target or state	Accumulated drift, scaling, temporal smoothness
Joint velocity	Desired joint velocity	Stability near contact, integration error
Joint torque	Actuator effort	Gravity compensation, friction, saturation, hardware safety
Cartesian pose or impedance	TCP pose or impedance quantities	IK feasibility, singularities, frame conventions

Crucial distinction: A policy that *commands* a joint position does not instantaneously *set* the simulated joint to that position. The position target must go through a comparable controller and actuator. Directly teleporting joints can bypass physical constraints and create a major sim-to-real mismatch.

5.3 Keep three clocks separate

- **Physics step:** Integrates motion, collision, and contact. Smaller steps often improve stability at higher cost.
- **Low-level control frequency:** Updates servo, impedance, or IK targets.
- **Policy and sensor frequencies:** RL decisions and camera acquisitions; these rates may be asynchronous.

Example: 500 Hz physics and 50 Hz policy inference imply **10 physics steps per policy action**. A 30 Hz camera requires explicit frame holding, timestamps, and delay handling. The deployed system additionally includes communication, estimation, filtering, and actuator response latency.

5.4 Fair control comparisons

Hold four things constant: **action meaning in joint or task space, action scaling, low-level controller semantics, and effective sampling period**. Without those controls, measured differences cannot be cleanly attributed to the physics engine.

6. Contact, friction, vision, and sensing

6.1 Why dexterous contact is difficult

Contact dynamics are highly nonlinear. Collision geometry, friction, compliance, integration time step, control stiffness, inertia, and the solver jointly determine whether an object can be held. Two engines using an identically named friction coefficient need not produce identical sliding behavior.

Debug contacts by isolation: First verify object free fall, then sliding under a known normal force, then static pinch grasps, and finally dynamic lifting and rotation. Increasing friction is not a universal fix for unstable contact simulation.

6.2 Standard contact validation tests

Test	Measurements	Typical failure
Free fall	Impact time, rebound, resting position	Penetration or excessive bounce
Planar sliding	Deceleration, stopping distance	Unrealistic slip or jitter
Fingertip grasp	Slip, contact force, holding time	Implausibly strong grasp or violent ejection
In-hand rotation	Object pose relative to hand	Unstable contacts, grasp loss
Perturbation recovery	Rediscovery and regrasp after a drop	Episode incorrectly terminates at first drop

6.3 Four sources of visual closed-loop error

This section catalogues **simulation-to-deployment observation gaps**. For SAM/Foundation vision models, temporal estimation and closed-loop reacquisition, see Visual State Estimation.

1. **Geometry:** Intrinsics, lens distortion, extrinsics, TCP, and object CAD/mesh scale.
2. **Coordinate frames:** World/base/optical frames, transform multiplication order, quaternion convention.
3. **Perception:** Missing depth, lighting, occlusions, blur, and pose-estimator drift.
4. **Timing:** Unsynchronized camera and joint readings, network delay, action holding, and reuse of stale frames.

Depth along the optical axis is not equivalent to Euclidean distance from the camera center. Align Isaac Lab depth semantics with the real sensor and any downstream pose estimator rather than relying on a generic variable name. [8]

6.4 Should a policy observe vision or privileged state?

For debugging control and reward design, start with simulator state. If the real task uses head and wrist cameras, the final deployed actor must depend only on available visual and proprioceptive observations. A useful pipeline is **privileged-state teacher → visual student**: first establish that the mechanics and control task can be learned, then introduce partial observability and sensor error. Never silently include simulator ground-truth object pose in a supposedly deployable visual policy.

7. Define the RL task before training

7.1 Six ingredients of a learning task

- **Objective:** Which object, initial configuration, desired end condition, and success dwell time?
- **Observations:** What can the robot know, and what is reserved for a training-time critic?
- **Actions:** What physical quantity is commanded, at what period, with which filters and limits?
- **Rewards:** What constitutes progress, and where can reward gaming occur?
- **Termination and truncation:** Which outcomes are terminal, and which merely reach a time limit?

- **Training/evaluation distributions:** What variation is seen during learning, and how is generalization tested?

A growing average return does not necessarily mean the robot has learned the intended task. A policy may exploit collision artifacts, unusual reset states, or poorly chosen shaping terms.

7.2 A practical task hierarchy

Level A: Reach/track; **Level B:** Approach and make contact; **Level C:** Grasp and lift; **Level D:** Reorient the object; **Level E:** Pick, move, and rotate; **Level F:** Recover after slip or object drop.

Evaluate each level independently before moving on. This isolates control, contact, stage transitions, and exploration failures.

8. Choosing PPO, SAC, and related methods

8.1 Engineering differences between PPO and SAC

Dimension	PPO	SAC
Data regime	Typically on-policy with fresh rollouts	Off-policy with reusable replay samples
Common strength	Mature GPU-parallel training, manageable stability	Data reuse when interactions are costly
Primary cost	Old rollouts are not reused indefinitely	More involved critic, temperature, and distribution tuning
Typical starting point	Isaac Lab locomotion and dexterous state control	Continuous control with a replay-friendly stack
Failure modes	Poor reward scales, unstable policy updates	Q overestimation, distribution shift, entropy/replay imbalance

Neither simulator dictates a single algorithm. A task's available algorithms are determined by its trainer and integration. RSL-RL is frequently used for GPU PPO in Isaac Lab; RL-Games and other libraries offer additional algorithms, but changing training libraries is not necessarily a plug-and-play substitution for every task. [9]

8.2 A defensible selection rule

- **Start with massively parallel state-based simulation:** PPO is a sensible baseline because maintained vectorized pipelines exist.
- **When interaction samples are expensive or replay is central:** Evaluate SAC, including replay throughput and correctness in vectorized environments.
- **With a large visual backbone:** Try freezing the encoder or distilling a smaller student before assuming PPO or SAC is the primary bottleneck.
- **When demonstrations are available:** Behavioral cloning can initialize a policy; reinforcement learning need not start from random behavior.

- **When fine-tuning foundation policies:** Separate action generators, value/reward estimation, low-level control, and inference latency. Hyperparameters from low-dimensional robot control do not transfer automatically.

8.3 Sample efficiency is not total efficiency

An off-policy learner can reuse data, but overall project cost includes physics stepping, camera rendering, image encoding, optimization, inter-device transfer, resetting, and real-world interaction. Report **environment steps, GPU hours, wall-clock time, and final task success**, not just replay ratio or number of optimizer updates.

9. Designing a trainable RL environment: detailed checklist

9.1 Observation design

State-based baseline: Joint positions and velocities, previous action, end-effector pose, object-to-goal pose error, and necessary contact/grasp signals.

Training-only privileged information: Ground-truth object pose, accurate object velocity, contact force, friction parameters, and external perturbations. These may feed a teacher or asymmetric critic, but must not leak to a deployment actor that cannot observe them.

Visual observations: RGB, depth, segmentation, or estimated object pose. Define camera rate, resolution, frame history, occlusion policy, and timestamps. With partial observability, a short history or state estimator is often better than assuming every frame fully observes the object.

Three recurring checks: Normalize quantities appropriately; prevent large-scale features from dominating; and remove leaked future targets, contact labels, or privileged state from actor observations.

9.2 Action design

Begin with **joint position increments through a realistic position servo**, or **TCP pose increments through IK together with finger joint targets**. Every action component needs a physical unit, range, filter, and safety bound.

Very large actions lead to collisions and unreachable configurations; tiny actions suppress exploration. The target rate, velocity, acceleration, and torque bounds used in training should reflect real actuator capabilities.

9.3 Reward design

Organize terms into five groups: **task success, intermediate progress, grasp/contact quality, smoothness/energy cost, and safety/failure penalties**.

Reward component	Encouraged behavior	Potential exploitation
Object approach	Reduce hand–object distance	Hover nearby without grasping
Grasp stability	Maintain realistic contact	Exploit artificially high friction

Reward component	Encouraged behavior	Potential exploitation
Pose alignment	Meet translation/rotation goals	Optimize only the easy error component
Success dwell	Hold the goal state	Repeatedly cross a success threshold
Action smoothness	Reduce impact and oscillation	Become inactive under excessive penalty

Log every reward term separately. Combine a meaningful sparse success criterion with modest dense shaping, then test whether the learned policy remains useful when shaping is reduced or the task distribution changes.

9.4 Termination, time limits, and recovery

- **Success termination:** The goal is achieved and maintained for the required interval.
- **Failure termination:** A genuinely unrecoverable or unsafe condition occurs, such as severe collisions or hard joint-limit violations.
- **Time-limit truncation:** The episode reaches its time budget without a natural terminal event. The trainer must treat bootstrap/timeout semantics correctly.
- **Recovery tasks:** If the policy must pick up a dropped object, **a drop must not always end the episode**. Allow observation, localization, approach, and regrasp unless the object leaves the recoverable workspace.

9.5 Reset and reference-state initialization (RSI)

Sample from multiple phases: normal start, just before contact, stable grasp, mid-rotation, incipient slip, and post-drop states. Use **event-biased RSI** near critical contact transitions while retaining a uniform component to avoid training on an excessively narrow distribution.

For reference-tracking tasks, randomizing the object's initial position or orientation requires a **consistent transformation of all subsequent reference poses and targets**. Otherwise, the reference and reward become contradictory. Validate each reset for penetration, discontinuous joint jumps, and unstable contact resolution.

9.6 Curriculum learning

A sensible curriculum is: fixed pose → small translation perturbations → orientation perturbations → larger workspace → uncertain dynamics → perception errors → drops and recovery. Advance based on success and failure modes, not solely on the elapsed number of training steps.

10. A complete Isaac Lab reinforcement learning workflow

10.1 From a benchmark to a custom manipulation task

Step 1 — Pick a starting task. Cartpole demonstrates the end-to-end RL loop; Franka Reach checks controls and observations; Lift adds contacts; dexterous manipulation introduces multi-stage objectives.

Step 2 — Fix the backend and release. Established 2.x PhysX tasks and new 3.0 multi-backend workflows are not automatically interchangeable. Reproduce one supported baseline first.

Step 3 — Specify the scene. Record the robot, table, object, drive settings, physics material, initial configurations, and camera geometry.

Step 4 — Define five environment interfaces. Observations, actions, rewards, termination, and reset; make sure state and cached sensor history are synchronized after every reset.

Step 5 — Overfit a single easy configuration. Use fixed objects and goals, with randomization disabled. If the policy cannot solve that configuration, inspect the environment before scaling the neural network.

Step 6 — Grow vectorization carefully. Begin with a manageable batch, then measure environment steps per second, GPU utilization, VRAM, and contact stability. More environments do not guarantee faster end-to-end learning.

Step 7 — Establish a PPO baseline. Freeze seeds, rollout horizon, optimizer settings, architecture, reward scales, and control period. Change one factor family at a time.

Step 8 — Evaluate independently. Use a fixed evaluation protocol, archive trajectories and failure videos, and measure success and safety rather than training return alone.

Step 9 — Introduce realistic variation. Add dynamics uncertainty, actuator delay, observation noise, visual occlusions, and disturbances one at a time.

Step 10 — Archive a reproducibility bundle. Preserve model assets, full task configuration, reward weights, normalization statistics, trainer version, checkpoints, evaluation implementation, and logs.

10.2 Selecting the Isaac Lab RL library

Training library	Good starting use	Caveats
RSL-RL	GPU-parallel PPO, state control, distillation	Verify actor/critic observation groups and architecture
RL-Games	Existing RL-Games projects or its supported algorithms	Capabilities and task schemas vary across releases
skrl	Wider algorithm range or PyTorch/JAX options	Needs careful integration and reproducibility checks
Stable-Baselines3	Smaller baselines and familiar analysis ecosystem	Not necessarily optimal for large GPU-resident batches

10.3 Common Isaac Lab RL pitfalls

VRAM: Memory can grow rapidly with camera count × resolution × environment count; benchmark again after adding vision. **Headless mode:** No interactive viewport does not imply that all offscreen rendering is disabled, or that camera-based RL works without configuring camera outputs. **Reset correctness:** Individual environments terminate asynchronously; reset indices and observation history must be updated correctly. **Reward timing:** Changing physics rate or policy decimation can unintentionally change accumulated reward magnitudes if time weighting is inconsistent.

11. MuJoCo RL: classic CPU simulation, MJX, and MJWarp

11.1 Classic MuJoCo: correctness first

Classic MuJoCo is well suited to isolated control tests, dynamics validation, small Gymnasium tasks, and CPU rollouts. It offers direct debugging and low single-step latency. In very large GPU-training workflows, repeated CPU-to-GPU movement of observations may become a bottleneck. [3]

Use classic MuJoCo to validate the robot model, reward, reset, contact behavior, and low-level control before deciding whether the environment needs a GPU-batched implementation.

11.2 MJX and MJWarp: throughput-oriented training

MJX-JAX uses JAX/XLA for batched computation. **MJWarp** targets NVIDIA GPU parallel simulation, especially many independent robot environments. Higher *aggregate throughput* does not imply lower *single-step latency*: MJWarp focuses on throughput, while classic MuJoCo is often preferable for low-latency single-system control. The current MJX-Warp route does not target automatic differentiation. [3][4]

MuJoCo Playground provides locomotion, dexterous manipulation, some vision-enabled tasks, and RL baselines built around MJX/MJWarp. It is a useful counterpart to Isaac Lab for evaluating state-control tasks and cross-engine robustness. [10]

11.3 Ten steps for a MuJoCo RL project

1. Choose a maintained Menagerie or validated MJCF robot model; 2. align the joints and actuators with hardware; 3. build a minimal scene; 4. validate free dynamics and contact; 5. define observations, actions, and normalization; 6. design rewards, resets, and terminal rules; 7. run a small deterministic debug experiment; 8. choose CPU, MJX-JAX, or MJWarp; 9. connect PPO or another supported learner; and 10. evaluate held-out initial states, physical parameters, and other simulation engines.

Backend feature check: GPU implementations need not have full one-to-one support for every solver feature. Revalidate tendons, unusual constraints, contact capacities, sensors, and batched resets. A CPU MuJoCo model should not automatically be assumed to produce identical results after migration to GPU simulation.

12. PPO and SAC tuning: what to check and in which order

12.1 PPO: diagnose the environment before tuning the optimizer

A useful debugging priority is **task correctness** → **reward scale** → **action scale** → **rollout batch** → **update intensity** → **exploration/entropy** → **model capacity**.

Parameter or diagnostic	Practical initial range (heuristic, not universal)	Warning sign
Learning rate	Often explore ~1e−4 to 3e−4	Oscillating performance or divergence

Parameter or diagnostic	Practical initial range (heuristic, not universal)	Warning sign
PPO clipping	Common start ~0.1–0.2	Updates too weak or excessively large
GAE λ	Starting point near 0.95	High variance or very slow learning
Discount γ	Often 0.99–0.995; adapt to control time scale	Myopic policy or poor return estimation
Rollout length per environment	Often ~16–64 policy steps; task dependent	Rollouts miss critical task transitions
Policy update epochs	Often ~3–5; monitor KL and data size	Overfitting each rollout batch

Batch size matters: Number of environments multiplied by per-environment rollout length determines samples per iteration. Raising environment count without reconsidering minibatches or optimizer passes changes the effective algorithm. Multi-stage, long-horizon tasks may need longer temporal credit assignment than short locomotion benchmarks.

12.2 SAC: what deserves the most attention?

SAC needs explicit attention to **replay coverage, replay ratio, critic stability, target-network updates, entropy temperature, and actor/critic update timing**.

When training from prior interactions, distinguish demonstrations, old-policy experience, and current-policy samples. Avoid silent replay distribution shifts. Vision-based SAC adds image augmentation, encoder-training, and replay-storage concerns. A replay buffer does not automatically guarantee higher sample efficiency.

12.3 Three misleading signs of “successful learning”

- **Reward hacking:** Average return increases while actual task success does not.
- **Reset overfitting:** The policy works only at fixed starts or a small subset of RSI states.
- **Privileged-state leakage:** The trained actor relies on exact simulator pose despite real deployment having only noisy images or estimates.

Each failure requires targeted evaluation rather than another round of blind optimization.

13. Training efficiency: batches, vision, and bottlenecks

13.1 Measure at least five quantities

Track **environment steps per second, aggregate physics steps per second, optimizer time, peak VRAM, and wall-clock time to a fixed success threshold**. For vision-based RL, also track rendering time, encoder inference, and inter-device transfers.

FPS/SPS reported by different projects may refer to one world's physics frames, aggregate vectorized steps, or policy steps. Always report environment count, physics/control frequencies, GPU model, and camera configuration alongside throughput.

13.2 More environments are not always better

Too few worlds underutilize the GPU; too many exhaust VRAM or stress contact solving, rendering, transfers, and optimization. Increase batch size incrementally and select the operating point using measured **success improvement per hour**, not an arbitrary goal of thousands of environments.

13.3 Two routes to visual RL

- **End-to-end visual RL:** Train directly from images. Conceptually simple but more expensive and sensitive to occlusion and appearance mismatch.
- **Privileged-state teacher to visual student:** Learn competent control before introducing perception; separates "the task is mechanically hard" from "the image is ambiguous."

When teacher and student update actions at different rates, distillation must align timestamps and action meanings, not merely array dimensions.

14. Domain randomization and robust learning in a sensible order

Category	Example randomized factors	Intended generalization
Geometry	Initial object pose, table height, small size errors	Workspace and layout variation
Dynamics	Mass, inertia, friction, joint damping	Modeling uncertainty
Actuation	Delay, deadband, stiffness, saturation, filtering	Real servo behavior
Observation	Encoder noise, pose bias, dropped frames	Sensing and synchronization robustness
Vision	Lighting, material, background, occlusion, camera extrinsics	Visual domain shift
Disturbances	Pushes, slip, drops, moving targets	Recovery and corrective behavior

Recommended sequence: First solve the task without randomization; then expand initial states; then dynamics and actuator uncertainty; followed by state-estimation errors, visual randomization, and finally externally induced drops/recovery. Set ranges using measured robot uncertainty or physically defensible identification intervals, not arbitrarily extreme values.

Keep training and evaluation distributions distinct. Use held-out combinations of poses, friction, mass, lighting, and calibration offsets to distinguish true robustness from interpolation within the training set.

15. Evaluation, ablations, and reproducibility for publishable experiments

15.1 Core metrics

Metric	What it measures
Success rate	Fraction of episodes meeting objective completion criteria
Time-to-success	Steps or elapsed real time until task completion
Drop/slip rate	Frequency of losing or slipping the object
Recovery success	Success after a recoverable perturbation
Pose/tracking error	Object or end-effector trajectory accuracy
Unsafe-event rate	Collisions, excessive loads, workspace violations
Environment steps / wall-clock	Sample efficiency and total training efficiency
Real-time latency	End-to-end perception, inference, and control delay

Use multiple random seeds, report central estimates and uncertainty, and avoid cherry-picking one unusually strong checkpoint. For algorithm comparisons, align assets, observations, action interfaces, training budgets, and evaluation distributions.

15.2 Minimal ablation matrix

1. No domain randomization versus domain randomization.
2. Uniform RSI versus event-biased RSI versus their mixture.
3. Proprioception only versus privileged pose versus noisy estimated pose versus images.
4. Joint-level commands versus TCP pose increments.
5. Standard grasp learning versus grasp plus drop recovery.
6. In-engine evaluation only versus MuJoCo/Isaac cross-engine evaluation.

For PPO versus SAC, additionally align interaction budgets, vectorization level, network size, rewards, termination rules, evaluation protocol, and wall-clock cost. Matching optimizer-update counts alone is generally not a fair comparison.

15.3 Avoid unsupported conclusions

- **Cross-engine comparison is not exact mechanics equivalence.** Report friction and contact-model differences.
- **Equal success rate does not imply equal robustness.** Break down failures by grasp, reorientation, release, and recovery phase.
- **Training frames are not automatically interaction samples.** Distinguish physics steps, policy steps, and vectorized totals.
- **Checkpoint weights are not the entire policy.** Retain normalization statistics, action scaling, and software dependencies.

16. Sim-to-real transfer: from policy weights to a safe robot

16.1 Six calibration layers

1. **Geometry:** Joint zero positions, TCP, robot base relative to the table.
2. **Dynamics:** End-effector payload, center of mass, inertia, friction, and gravity compensation.
3. **Actuation:** Command mode, position/velocity/torque limits, and response delay.
4. **Vision:** Head/wrist camera intrinsics and extrinsics, image timestamps, depth units.
5. **Software integration:** ROS 2 or vendor SDK timing, buffering, packet loss, and error recovery.
6. **Safety:** Emergency stops, workspace restrictions, collision thresholds, current limits, and dropped-object handling.

16.2 A deployment validation ladder

Unloaded joint motion → contact-free tracking → slow approach → low-force grasp → object lift → object rotation → small supervised disturbance → controlled drop-and-regrasp. Apply limits, protection, and human supervision before advancing to each stage.

If hardware results lag behind simulation: First inspect frames and synchronization, then action meaning and servo period, then contacts and mechanical parameters. Only afterward should you consider a larger neural network or more complex RL hyperparameter search.

17. End-to-end case study: screwdriver manipulation with Tron2 and Revo3

17.1 Define the task precisely

The robot picks up a screwdriver from **varied tabletop positions and orientations**, moves it to a target region, and executes a required rotation or reorientation. If the object slips or is dropped, the robot should use current observations to locate, approach, regrasp, and complete the objective. The assumed deployable sensors are head and wrist cameras plus robot proprioception.

17.2 Recommended system development route

Stage	Platform	Main experiment	Acceptance criterion
A. Validate model	MuJoCo	Tron2/Revo3 kinematics, mass, collision, drives	Plausible joint responses, FK and contacts
B. Learn without vision	MuJoCo or Isaac Lab	Grasp and rotate using object ground truth	Reliable success at fixed starts
C. Train at scale	Isaac Lab / MJWarp	RSI, curriculum, PPO	Stable success across randomized starts
D. Test robustness	Both engines	Pose errors, delays, friction, drop disturbances	Quantified OOD and recovery behavior
E. Distill to vision	Isaac visual backend	Head/wrist images to estimates or a visual policy	Deployable actor works without privileged state
F. Test on hardware	Tron2 + Revo3	Calibration, action limits, closed-loop regrasping	Safe real-world task completion

17.3 Five critical choices

Actions: Prefer arm TCP pose increments with IK plus finger joint-position targets initially; explore force or residual control only after establishing a stable baseline. **Observations:** Begin with simulator object pose for the teacher; subsequently train on estimated pose or head/wrist images. **Resets:** Include before-contact, post-contact, rotation, slip, and post-drop states in RSI. **Termination:** A drop cannot always end an episode if the objective includes recovery. **Randomization:** Prioritize object pose, estimation bias, friction, servo delay, and joint-tracking errors.

17.4 Stage-specific measurements

Report not only final success but also first-grasp success, in-hand rotation success, drop probability, post-drop regrasp success, time-to-completion, and results broken down by **head-camera visibility and wrist-camera occlusion**.

18. Common failure symptoms and debugging priorities

Symptom	Inspect first	Avoid the reflex to…
Robot jitters after reset	Joint zeros, stiffness, collisions, physics step	Raise gains immediately
Cube penetrates surfaces	Collision meshes, contacts, step size	Only increase friction
PPO reward rises but success stays low	Reward terms, success metric, rollout video	Just train longer
SAC critic diverges	Replay distribution, critic, target update, reward scales	Only enlarge the network
GPU remains underutilized	CPU/GPU transfers, rendering, batch size, sync points	Maximize environment count blindly
Visual training is extremely slow	Camera count, resolution, renderer, encoder	Start with full-resolution cameras
Hardware grasp is too weak	Contact geometry, materials, servo behavior, model error	Raise simulated friction only
Policy cannot recover after a drop	Termination logic, RSI, visual coverage, objective	Increase success bonuses alone
Arm tracking error is high	IK, TCP, delays, scaling, velocity limits	Retune PPO first
Behavior changes after Isaac update	Pinned dependencies, quaternion order, sensor/backend changes	Trust old checkpoints without revalidation

Global diagnostic order: Model integrity → frames and units → actuator control → physics/contact → observations and delay → reward/reset design → optimization algorithm.

19. Suggested six-week learning plan and deliverables

Week	Learning and experiment	Tangible deliverable
1	Tool landscape, Panda scene, individual joint checks	Parameter sheet, joint map, screenshots
2	MuJoCo tracking, contacts, and grasp validation	Free-fall/sliding/grasp tests
3	Isaac Lab benchmark tasks, environment design, PPO	A working fixed-start baseline and learning curves
4	RSI, curriculum, domain randomization	Random-start evaluation and first ablations
5	MJWarp/cross-engine tests and visual teacher-student	OOD report and engine discrepancy analysis
6	Tron2/Revo3 mapping, action limits, hardware testing	Safety checklist and real-world success/recovery report

The schedule is illustrative, not a promise of completion time; the real pace depends on model quality, task complexity, and sensor reliability. Maintain the same parameter records and frozen evaluation suite throughout the project.

20. Final readiness checklist

Model and physics: Verified joint order, axes, limits, masses, inertias, TCP, collisions, actuators, physics step, and friction.
RL environment: Explicit success criterion, no observation leakage, bounded actions, interpretable reward terms, correct timeout semantics, and reset coverage near key events.
Training: Fixed-start baseline solved; seeds, trainer, task, and model releases recorded; interaction count, throughput, wall time, and peak VRAM reported.
Evaluation: Multiple seeds, held-out initial states, OOD disturbances, cross-engine tests, failure categories, and drop recovery documented.
Deployment: Frames and cameras calibrated; rates and delays matched; joint/force/velocity limits, supervision, and emergency stops prepared.

> **One-sentence takeaway:** MuJoCo clarifies the mechanics and actuation; Isaac Sim provides rich scenes and realistic sensing; Isaac Lab or MuJoCo Playground structures RL training; cross-engine, out-of-distribution, and real-hardware evaluation determine whether the policy is genuinely robust.

Official references and further reading

1. Isaac Lab Releases (including 3.0 Early Access and migration notes): https://github.com/isaac-sim/IsaacLab/releases
2. Isaac Lab 3.0 Early Access installation and architecture: https://isaac-sim.github.io/IsaacLab/v3.0.0-EA/source/setup/installation/index.html
3. MuJoCo Warp documentation: https://mujoco.readthedocs.io/en/latest/mjwarp/

4. MuJoCo XLA / MJX documentation: https://mujoco.readthedocs.io/en/stable/mjx.html
5. Isaac Sim 5.1 hardware and software requirements: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/requirements.html
6. MuJoCo Menagerie: https://github.com/google-deepmind/mujoco_menagerie
7. Isaac Lab 2.3.2 Direct RL environment tutorial: https://isaac-sim.github.io/IsaacLab/v2.3.2/source/tutorials/03_envs/create_direct_rl_env.html
8. Isaac Lab 2.3.2 sensor reference: https://isaac-sim.github.io/IsaacLab/v2.3.2/source/api/lab/isaaclab.sensors.html
9. Isaac Lab RL-library comparison: https://isaac-sim.github.io/IsaacLab/main/source/overview/reinforcement-learning/rl_frameworks.html
10. MuJoCo Playground: https://github.com/google-deepmind/mujoco_playground
11. MuJoCo main documentation: https://mujoco.readthedocs.io/en/stable/
12. Isaac Sim main documentation: https://docs.isaacsim.omniverse.nvidia.com/

Reference usage: Release names, supported backends, and APIs may change. This guide deliberately omits installation commands so that version-specific commands are not accidentally used with an incompatible release.

Reinforcement Learning for Embodied AI: Algorithms, Tasks, and Use Cases

Course home · 简体中文 · PDF

Item	Details
Applicability	Algorithm and task taxonomy; no single framework environment is prescribed
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

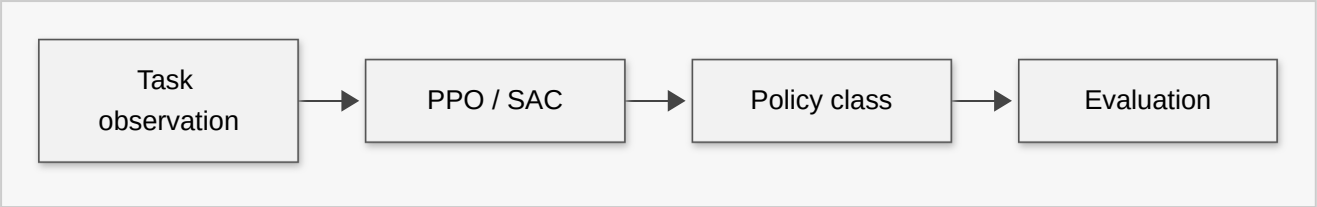
Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Overall Classification of Embodied Reinforcement Learning
  - 1.1 Classification Framework
  - 1.2 Core Classification Table
- 2. Classification by Algorithm: PPO and SAC
  - 2.1 On-policy RL: PPO
  - 2.2 Off-policy RL: SAC
  - 2.3 Comparing PPO and SAC
- 3. Classification by Task: Robot Motion-Control Reinforcement Learning
  - 3.1 Locomotion RL: Robot Motion Control
  - 3.2 Manipulation RL: Robot Manipulation Reinforcement Learning
- 4. Classification by Task: Embodied Foundation-Model RL Post-Training
  - 4.1 What Is Foundation-Model RL Post-Training?
  - 4.2 What Problems Does RL Post-Training Solve?
  - 4.3 Five Types of Foundation-Model RL Post-Training
  - 4.4 Summary of the Five Types
- 5. How PPO and SAC Are Used Across the Two Task Categories
  - 5.1 PPO + Locomotion
  - 5.2 SAC + Manipulation
  - 5.3 PPO-Style Methods + VLA
  - 5.4 SAC / Off-policy Critic-Style Methods + VLA
- 6. How to Choose an RL Method for Your Task
- 7. Typical Experimental Paths for the Two Research Directions

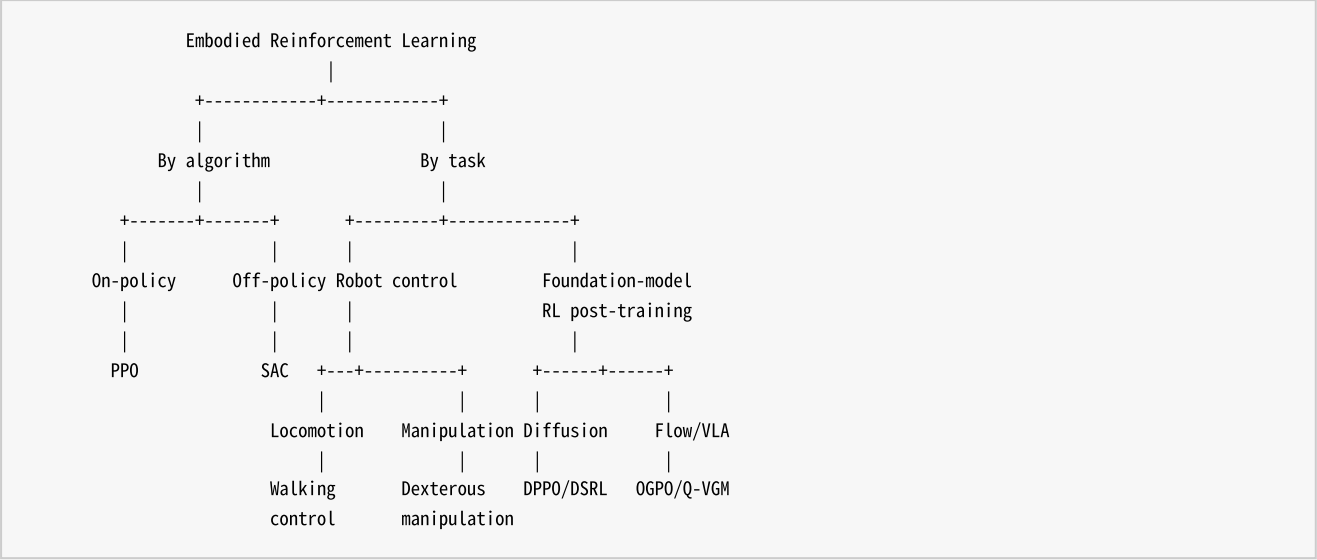
    - 7.1 Robot Motion-Control RL
    - 7.2 Embodied Foundation-Model RL Post-Training
- 8. Final Summary

Chapter concept map



1. Overall Classification of Embodied Reinforcement Learning

1.1 Classification Framework



Robot manipulation can employ either conventional low-level control policies or Diffusion Policies and VLAs.

Therefore, **the same manipulation task can fall under conventional robot-control RL or foundation-model RL post-training; what matters is the policy model and training objective.**

1.2 Core Classification Table

Classification dimension	Category	Representative algorithm or method	Main use
Algorithm	On-policy	PPO	Large-scale simulation training and motion control
Algorithm	Off-policy	SAC	Continuous control, data-efficient learning, and online improvement
Task	Locomotion RL	PPO, SAC	Bipedal, quadrupedal, and humanoid movement

Classification dimension	Category	Representative algorithm or method	Main use
Task	Manipulation RL	PPO, SAC	Grasping, in-hand manipulation, and dexterous control
Task	Foundation Model RL	DPPO, DSRL, OGPO, etc.	Improve pretrained Diffusion / Flow / VLA policies

Note that DPPO, DSRL, and OGPO were developed for generative policies and cannot all be classified directly as standard PPO or SAC.

---

2. Classification by Algorithm: PPO and SAC

2.1 On-policy RL: PPO

PPO is an on-policy optimizer using fresh rollouts and advantage estimates to control policy updates. Here the emphasis is when it is appropriate for parallel simulation and robot tasks. **For clipped surrogate objectives, GAE and full derivation, use the dedicated PPO chapter.**

2.2 Off-policy RL: SAC

SAC reuses replayed interaction data and optimizes a maximum-entropy continuous-action objective with twin critics. This overview focuses on task selection and sample efficiency. **Bellman targets, entropy tuning and Actor/Critic derivations are owned by the SAC chapter.** Applying SAC to a generative VLA is not a trivial actor replacement.

2.3 Comparing PPO and SAC

Dimension	PPO	SAC
Type	On-policy	Off-policy
Training data	Primarily the current policy	Current and historical policy data
Reuse of old data	Limited	More extensive
Replay buffer	Usually no long-term replay buffer	Required
Sample efficiency	Typically lower	Typically higher
Parallel simulation	Very mature ecosystem	Also supported
Training stability	Often easier to manage	Depends on critic quality
High-cost physical interaction	Usually less advantageous	Often more advantageous
Typical tasks	Locomotion	Manipulation, online RL
Representative framework	RSL-RL	Stable-Baselines3 and other SAC implementations

How should you choose?

Choose PPO when:

Robot simulation is fast; a large number of environments can run in parallel; and the goal is to train a stable motion-control policy from scratch.

Choose SAC when:

Physical interaction data are expensive; historical experience needs to be reused; and the goal is continued online policy improvement.

In practice, the choice also depends on the policy architecture. If the policy is a large Diffusion or Flow Matching model, standard PPO/SAC cannot simply be applied without modifications.

---

3. Classification by Task: Robot Motion-Control Reinforcement Learning

The main objective of robot motion-control RL is:

To enable a robot to learn, through interaction with its environment, to execute specific physical movements stably and efficiently.

Usually, a relatively lightweight control policy is trained directly and then deployed to the robot's control system.

3.1 Locomotion RL: Robot Motion Control

Locomotion RL mainly targets robot mobility and whole-body movement capabilities.

Typical applications

Area	Task	Main use
Quadruped Locomotion	Quadrupedal walking, running, and jumping	Robot-dog motion control
Humanoid Locomotion	Bipedal walking and stair climbing	Humanoid locomotion
Motion Tracking	Tracking human or reference motions	Humanoid motion imitation
Whole-Body Control	Coordinated whole-body movement	Mobile manipulation and complex whole-body actions
Push Recovery	Recovery from external pushes	Improve robot stability
Terrain Adaptation	Slopes, gravel, and stairs	Adaptation to challenging terrain

Typical training process

```
Robot model
    |
    v
Build a physics simulation environment
    |
    v
Define the motion task and rewards
    |
    v
Train with PPO
    |
    v
Domain Randomization
    |
    v
Sim-to-Real
    |
    v
Deploy on a physical robot
```

What are the major research topics?

1. Motion capabilities

Enable robots to perform increasingly complex motion tasks, for example:

- Progress from flat-ground walking to stairs;
- Progress from slow walking to running;
- Progress from simple gaits to whole-body motion tracking.

2. Robustness

Improve how well robots cope with different environments, for example:

- External pushes;
- Friction changes;
- Unknown payloads;
- Actuator errors.

3. Sim-to-Real

Reduce the difference between simulated and physical robots.

Main techniques include:

- Domain Randomization;
- System Identification;
- Teacher-Student Learning;
- Privileged Learning;
- Online Adaptation.

Typical technical stack

Module	Common choice
Simulation	Isaac Lab, MuJoCo
RL	PPO
Implementation	RSL-RL
Control policy	MLP, RNN
Low-level control	PD / Torque Control
Training approach	Large-scale parallel simulation
Deployment	Real-time robot control system

Defining feature of this direction:

It is often possible to train from scratch in simulation. The main bottlenecks are physics simulation, reward design, task complexity, and robustness after deployment on physical hardware.

3.2 Manipulation RL: Robot Manipulation Reinforcement Learning

Manipulation RL mainly addresses robot-arm and dexterous-hand operation.

Although it and locomotion RL are both forms of robot control RL, manipulation tasks often involve more complicated contact dynamics.

Typical applications

Area	Task	Main use
Reaching	Move a robot arm to a target	Basic end-effector control
Grasping	Grasp different objects	Robotic grasping
In-hand Manipulation	In-hand rotation and repositioning	Fine dexterous-hand manipulation
Object Reorientation	Adjust object orientation	Dexterous manipulation
Tool Manipulation	Use tools such as screwdrivers	Tool operation
Contact-Rich Manipulation	Insertion, rotation, assembly	Precision contact operations
Bimanual Manipulation	Coordinated two-arm control	Two-handed manipulation and assembly

Differences from locomotion RL

Locomotion focuses more on:

- Body balance;
- Velocity tracking;
- Ground contact;
- Periodic movement.

Manipulation focuses more on:

- Multiple contact points;
- Object pose;
- Grasp stability;
- Contact-mode transitions;
- Finger coordination;
- Precision operation.

Should you use PPO or SAC?

PPO:

Suitable for training many dexterous manipulation tasks in parallel simulation.

For example, train a dexterous hand to rotate objects from different initial object poses.

SAC:

Suitable for research on real-robot online learning and manipulation tasks requiring high environment-interaction sample efficiency.

For example, start from an existing manipulation policy and use a small number of real-world interactions to progressively improve grasp success.

Important distinction

Here, the trained policy may still be a conventional low-level controller:

```
Observation
    |
    v
MLP Actor
    |
    v
Joint Position / Torque
    |
    v
Robot
```

This is a different problem setting from VLA reinforcement-learning post-training in the next section.

---

4. Classification by Task: Embodied Foundation-Model RL Post-Training

4.1 What Is Foundation-Model RL Post-Training?

Embodied foundation models are usually pretrained on large amounts of demonstration data and then improved through reinforcement learning.

Examples include:

- Diffusion Policy;
- Flow Matching Policy;
- VLAs such as π0 and π0.5;
- Other vision-language-action generative models.

The typical training process is:

```
          Large-scale robot demonstrations
                        |
                        v
                Imitation Learning
                        |
                        v
                 Pretrained Policy
                        |
                        v
                 RL Post-training
                        |
                        v
                 Improved Robot Policy
```

The main difference from training a locomotion policy from scratch is that:

The post-training stage begins with a pretrained policy that already has some task capability.

The purpose of RL is no longer to learn basic movements from scratch, but to further improve performance based on existing capabilities.

4.2 What Problems Does RL Post-Training Solve?

Problem	Role of RL post-training
Insufficient success rate	Improve success at grasping, placing, insertion, and related tasks
Slow execution	Reduce task-completion time
Insufficient precision	Improve end-effector positioning and precision manipulation
Failure in new environments	Adapt online to environmental changes
Performance limited by imitation learning	Use autonomous experience to exceed demonstration performance
No recovery after failure	Learn recovery and retry behaviors
Few demonstrations for real tasks	Improve further through autonomous exploration

For example:

A pretrained VLA can already grasp a screwdriver, but its success rate is only 70%.

We would like to use reinforcement learning to improve success rate, eliminate unnecessary movements, and increase robustness to variations in initial pose.

4.3 Five Types of Foundation-Model RL Post-Training

These methods can be divided into five categories according to **which part of the model reinforcement learning updates.**

Type 1: Full Policy Fine-tuning

Directly update the pretrained generative policy.

Representative methods:

- DPPO;
- OGPO.

DPPO uses policy gradients to fine-tune Diffusion Policies. OGPO uses an off-policy critic and an optimization objective adapted to the generation process to perform full fine-tuning of generative control policies. (DPPO)

Main use:

Full policy fine-tuning is appropriate when a pretrained policy's capabilities are clearly insufficient and its action-generation behavior needs to change substantially.

The advantage is a broader scope for modifying the policy and potentially improving its capabilities.

The drawbacks are high training costs and the need to avoid damaging previously learned capabilities.

Type 2: Latent / Noise-Space RL

Freeze the pretrained model and optimize only its input latent variables or noise distribution.

Representative methods:

- DSRL;
- LP-DS.

DSRL improves a Diffusion Policy by learning a distribution over its initial noise, without updating the parameters of the base policy. (DSRL)

Main use:

When a pretrained model already generates fairly good actions and the main objective is to select better modes of behavior.

Advantages:

- No need to fine-tune the entire large model;
- Relatively lightweight online training;
- Reuse of the pretrained policy.

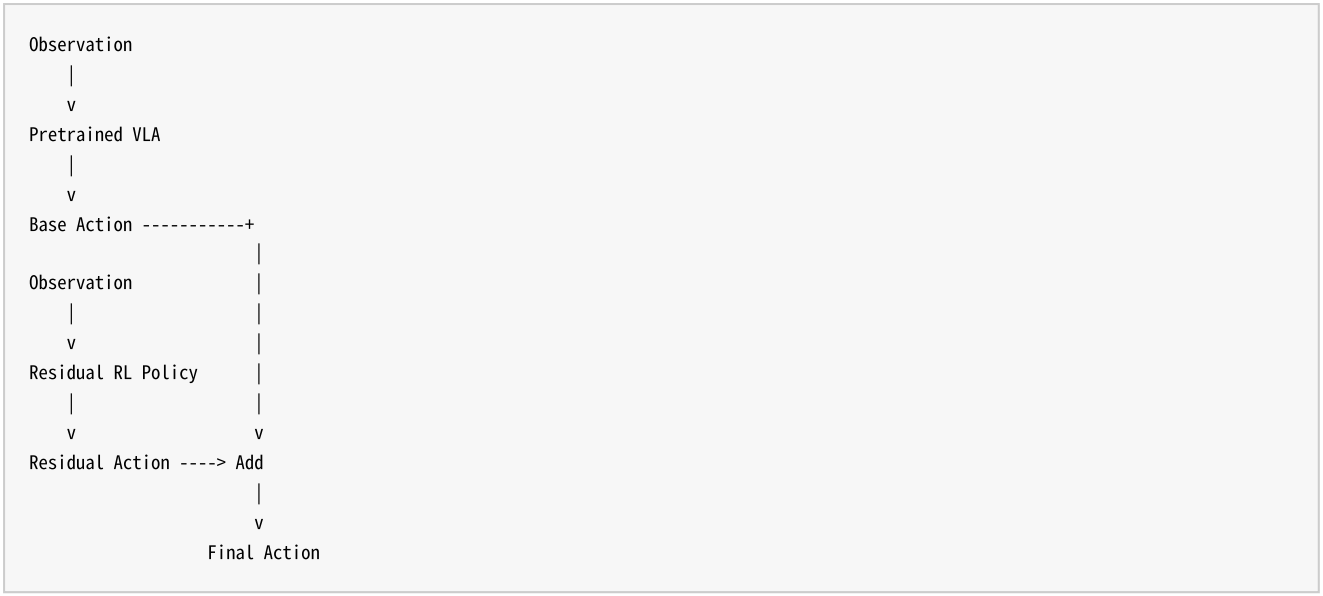
Limitation:

It is difficult to go beyond the representational capabilities of the frozen generator itself.

Type 3: Action Residual RL

Add RL-based corrections to the actions produced by a pretrained policy.

For example:



Main use:

The pretrained policy is mostly correct, but end-effector errors, pose errors, or contact control are not accurate enough.

For example:

- Increase insertion accuracy;
- Correct grasp pose;
- Add local control for tool operations;
- Improve actions during contact phases.

This category usually does not require retraining the base VLA.

Type 4: Feature / Token-Level RL

Train a lightweight RL module using internal features of a pretrained VLA.

Representative method:

- RL Token.

RL Token uses the pretrained VLA to provide compact task-relevant representations and trains a lightweight actor-critic module for online policy improvement. (RL Token)

Main use:

Online reinforcement learning for large VLAs.

Examples:

- Precision assembly;

- Insertion;
- Tool operation;
- Improving execution speed online.

This category is appropriate when the goal is to preserve the foundation model's representations while reducing online training costs.

Type 5: Velocity Field Fine-tuning

Directly optimize the velocity field of a Flow Matching Action Expert.

Representative research:

- Q-VGM;
- Approaches related to Reward / Advantage-Guided Velocity Matching.

Q-VGM uses value-gradient signals provided by a critic to improve a pretrained Flow Matching Policy through local Velocity Matching. (Q-VGM)

Main use:

When the goal is to genuinely improve a Flow Matching action-generation model without expensive backpropagation through the entire generative trajectory for every policy update.

This is a research direction highly relevant to post-training large Flow Matching VLAs.

4.4 Summary of the Five Types

Method	Representative	Updated component	Main use
Full Fine-tuning	DPPO, OGPO	Entire generative policy or main Action Expert	Improve the model's action-generation capabilities
Latent RL	DSRL, LP-DS	Input noise or latent policy	Lightweight online improvement
Action Residual RL	Residual SAC, etc.	Residual of the output action	Precision control and local action correction
Feature / Token RL	RL Token	Lightweight actor-critic module	Efficient online learning for large VLAs
Velocity Field RL	Q-VGM, etc.	Flow Matching velocity field	Directly improve the Flow Matching Action Expert

The first four categories mainly differ in where parameters are optimized; the fifth additionally exploits the generative structure of Flow Matching.

These categories are not completely mutually exclusive. For example, training only a velocity-field residual module falls under both residual adaptation and velocity-field fine-tuning.

---

5. How PPO and SAC Are Used Across the Two Task Categories

Combining the two classification dimensions gives the following core table.

	Robot motion control	Foundation-model RL post-training
On-policy PPO	Train locomotion control policies in large-scale simulation	Generative-policy fine-tuning such as DPPO
Off-policy SAC	Dexterous manipulation and real-robot online learning	Adapted methods such as DSRL based on SAC or an off-policy critic

5.1 PPO + Locomotion

Typical task:

Teach a humanoid robot to climb stairs.

Common setup:

Isaac Lab + RSL-RL + PPO.

Main advantage:

Make full use of large-scale simulation environments.

5.2 SAC + Manipulation

Typical task:

A robot learns grasping and object manipulation through online interactions.

Main advantage:

Reuse experiences in the replay buffer and reduce the need to obtain new training data.

5.3 PPO-Style Methods + VLA

Typical task:

Use DPPO to post-train a Diffusion Policy with reinforcement learning.

Main advantage:

Directly improve the generative policy using policy gradients.

5.4 SAC / Off-policy Critic-Style Methods + VLA

Typical task:

Use DSRL to optimize a pretrained generative policy's latent variables, or use OGPO, Q-VGM, and similar methods to improve the generative policy.

Main advantage:

Improve data efficiency by using historical interaction experience.

An especially important distinction is:

OGPO and Q-VGM are off-policy generative-policy optimization approaches, but they are not equivalent to standard SAC.

---

6. How to Choose an RL Method for Your Task

The following table provides practical guidelines for project selection.

Project goal	First options to consider	Reason
Train a quadruped to walk from scratch	PPO	Mature parallel simulation workflows
Train motion control for humanoids such as G1	PPO	Established motion-control RL workflows
Train a dexterous hand to rotate objects in simulation	PPO	Parallel simulation can be exploited
Improve manipulation on a physical dexterous hand online	SAC or demonstration-initialized off-policy RL	Historical data can be reused
Fine-tune an existing Diffusion Policy	DPPO, OGPO	Direct improvement of the generative policy
Keep an existing Diffusion / Flow model unchanged	DSRL	Freeze the generator and optimize the noise policy
Improve VLA precision manipulation	Residual RL, RL Token	Train lightweight modules
Directly fine-tune a Flow Matching velocity field	Q-VGM and related methods	Update the generator using value-based signals
Avoid backpropagation through the full generation chain	Latent RL, Token RL, local Velocity Matching	Reduce online optimization cost

Practical choices should also consider model size, the availability of simulation, reward sparsity, and the budget for physical interactions and computation.

---

7. Typical Experimental Paths for the Two Research Directions

7.1 Robot Motion-Control RL

Recommended path:

```
Isaac Lab / MuJoCo
        |
        v
     PPO / SAC
        |
        v
 Train motion-control policy
```

```
        |
        v
 Domain Randomization
        |
        v
   Teacher-Student
        |
        v
     Sim-to-Real
        |
        v
Physical robot control
```

This route mainly focuses on:

How to make robots move more stably and flexibly, with stronger adaptability to physical environments.

Recommended starting points:

Isaac Lab + RSL-RL

7.2 Embodied Foundation-Model RL Post-Training

Recommended path:

```
       Robot Demonstrations
                |
                v
          Pretraining
                |
                v
     Diffusion / Flow / VLA
                |
                v
           RL Post-training
                |
     +----------+----------+
     |          |          |
     v          v          v
Latent RL Residual RL Full Fine-tuning
     |          |          |
   DSRL     RL Token   DPPO / OGPO
                           |
                           v
                    Velocity Field
                       Q-VGM
                |
                v
          Improved Robot Policy
                |
                v
           Real Robot Deployment
```

This route mainly focuses on:

How reinforcement learning can improve the task capabilities of pretrained foundation models with relatively little robot interaction data.

Recommended starting path:

DSRL → DPPO → OGPO → Q-VGM

---

8. Final Summary

For reinforcement learning in embodied AI, the field is best understood using the following three-level classification.

Classification level	Main categories	Core question
Level 1: Algorithm	PPO, SAC	How can environmental feedback improve a policy?
Level 2: Task	Locomotion, Manipulation, Foundation Model Post-training	Which robot task should RL solve?
Level 3: Optimization target	Low-level controller, Residual, Latent, Action Expert, Velocity Field	Which part of the policy does RL actually update?

In particular:

Robot motion-control reinforcement learning primarily follows the technical route of PPO, large-scale physics simulation, motion control, and Sim-to-Real.

Embodied foundation-model RL post-training builds on Diffusion / Flow / VLA models and focuses on improving pretrained policies with RL, including Latent RL, Residual RL, Token-Level RL, and Full Policy Fine-tuning.

The core distinction between them is:

- **Motion-control RL:** Primarily addresses how a robot acquires stable motion and manipulation control capabilities.
- **Foundation-model RL post-training:** Primarily addresses how to improve a model that already knows how to manipulate, further increasing success rate, speed, precision, and robustness.

Thus, embodied RL is not only a choice between PPO and SAC; it also requires choices about **task type, policy architecture, the source of training data, and the scope of policy updates.**

Reinforcement Learning for Motion Control in Embodied AI: A Complete Tutorial

Course home · 简体中文 · PDF

Item	Details
Applicability	Isaac Lab / RSL-RL workflows are illustrative; task, trainer, asset and actuator versions must be bound together
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

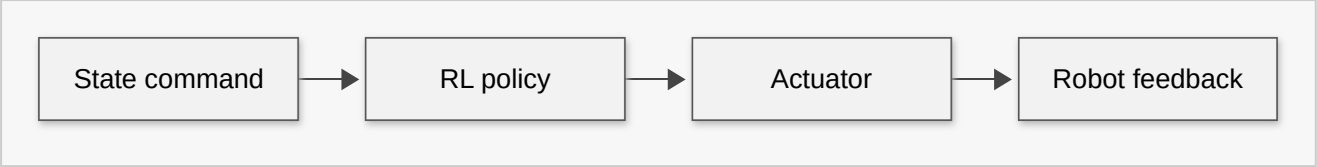
Contents

- Chapter 1: What Is Motion-Control Reinforcement Learning?
  - 1.1 Where motion-control RL fits in an embodied system
  - 1.2 Motion-control RL versus RL post-training of embodied foundation models
- Chapter 2: A Taxonomy of Motion-Control RL
  - 2.1 Classification by robotic task
  - 2.2 Classification by training strategy
- Chapter 3: Algorithms for Motion-Control RL
  - 3.1 PPO: a standard baseline for large-scale simulation
  - 3.2 SAC: improved reuse of interaction data
  - 3.3 PPO versus SAC
- Chapter 4: Designing a Motion-Control RL Environment
  - 4.1 Observation: what can the robot sense?
  - 4.2 Action: what should RL output?
  - 4.3 Reward: what behavior should the robot optimize?
  - 4.4 Reset: which initial states should be sampled?
- Chapter 5: Training a Complete Motion Controller
  - 5.1 The standard workflow
  - 5.2 Step 1: build a robot simulation model
  - 5.3 Step 2: choose control frequencies
  - 5.4 Step 3: parallel training
- Chapter 6: Motion-Imitation Reinforcement Learning

    - 6.1 The basic pipeline
    - 6.2 Motion-tracking rewards
    - 6.3 DeepMimic
    - 6.4 AMP: Adversarial Motion Priors
    - 6.5 Motion tracking versus AMP
    - 6.6 From tracking one clip to universal control
- Chapter 7: Teacher–Student Reinforcement Learning
    - 7.1 Why use teacher–student training?
    - 7.2 Standard training procedure
    - 7.3 Asymmetric actor–critic versus teacher–student
    - 7.4 RMA: Rapid Motor Adaptation
- Chapter 8: Case Study I — Training Unitree G1 Walking with PPO
    - 8.1 Task definition
    - 8.2 Observation
    - 8.3 Action
    - 8.4 Reward components
    - 8.5 Curriculum learning
    - 8.6 Open-source starting point
- Chapter 9: Case Study II — Arm and Dexterous-Hand RL
    - 9.1 Task definition
    - 9.2 Recommended control architecture
    - 9.3 Three training routes
    - 9.4 Observation design
    - 9.5 Reward design
    - 9.6 Proposed training stages
- Chapter 10: Simulation-to-Real Transfer
    - 10.1 Why simulation success does not ensure hardware success
    - 10.2 Domain randomization
    - 10.3 Sim-to-sim evaluation
    - 10.4 Real-robot deployment checklist
- Chapter 11: Evaluating Motion-Control RL
    - 11.1 Legged-robot metrics
    - 11.2 Motion-tracking metrics
    - 11.3 Dexterous-manipulation metrics
- Chapter 12: Open-Source Frameworks and Tools
    - 12.1 Isaac Lab
    - 12.2 RSL-RL
    - 12.3 MuJoCo Playground
    - 12.4 Which framework should you choose?
- Chapter 13: Representative Papers and Reading Order

- Chapter 14: Learning and Research Roadmap
  - Stage 1: basic motion-control RL
  - Stage 2: motion-tracking RL
  - Stage 3: teacher–student and sim-to-real
  - Stage 4: complex motion control
  - Stage 5: universal motion controllers
- Chapter 15: Summary and Main Takeaways
- References and Open-Source Links

Chapter concept map



Chapter 1: What Is Motion-Control Reinforcement Learning?

1.1 Where motion-control RL fits in an embodied system

A robotic system can be approximately organized into three levels:

Level	Main responsibility	Common techniques
High level: task planning	Decide what the robot should do	LLMs, VLMs, VLAs, task planning
Middle level: motion and skill control	Decide how the robot should move	Motion planning, motion tracking, RL
Low level: actuator control	Execute motor-level commands	PID, PD, impedance and current control

Motion-control RL is mostly applied at the middle level, although it may also learn parts of low-level control. For example, a high-level system may request “walk forward at 0.5 m/s.” The motion policy produces joint commands, and a low-level controller executes them.

```
User: "Walk to the table and pick up the cup"
                    |
                    v
             VLM / VLA / Planner
                    |
                    v
            Velocity / Pose Goal
                    |
                    v
             RL Motion Policy <--- Sensor feedback
                    |
                    v
            Joint / Cartesian Command
                    |
                    v
            PD / IK / Motor Driver
                    |
```

```
                    v
                Real Robot
```

A goal-conditioned motion policy can be written as

$$a_t = \pi_\theta(o_t, c_t),$$

where o_t contains observations such as joint states, IMU readings, and vision; c_t is a desired velocity, pose, or skill command; a_t is a joint or Cartesian action; and π_θ is the learned policy.

Conventional controllers typically employ explicitly designed control laws, whereas motion-control RL learns a feedback policy from interactions with a dynamical system. These approaches are complementary: practical robotic systems often combine RL with PD, inverse kinematics (IK), model-predictive control (MPC), and whole-body control (WBC).

1.2 Motion-control RL versus RL post-training of embodied foundation models

Dimension	Motion-control RL	RL post-training for foundation policies
Primary objective	Learn stable, flexible motor skills	Improve an existing policy's task performance
Typical inputs	Joints, IMU, goals, other robot states	Images, language, robot states
Typical outputs	Joint or end-effector control commands	Action chunks or action distributions
Typical models	MLPs, GRUs, Transformers	VLAs, diffusion policies, flow-matching models
Methods	PPO, SAC, imitation-based RL	PPO, SAC, offline RL, specialized fine-tuning
Main interaction source	Large-scale simulation	Pretraining data, simulation, real interaction
Typical tasks	Walking, jumping, stairs, dexterity	Visuolinguistic manipulation tasks
Main research issues	Stability, robustness, motor performance	Generalization, sample efficiency, adaptation

For instance, training a Unitree G1 to walk under velocity commands is motion-control RL. Improving the grasping success rate of an already pretrained $\pi_{0.5}$ policy through RL is foundation-policy post-training. The boundary is not absolute: both may optimize the same robot actions for manipulation, but differ in architecture, initialization, and objectives.

Chapter 2: A Taxonomy of Motion-Control RL

2.1 Classification by robotic task

Category	Typical tasks	Main challenges
Legged locomotion	Walking, running, turning	Balance and contact stability
Agile locomotion	Jumping, stair climbing, obstacles, rolls	Dynamic contact and nonlinear dynamics
Whole-body motion tracking	Dance, motion imitation, trajectory tracking	Whole-body coordination and fidelity
Manipulation control	Grasping, carrying, insertion	Contacts, accuracy, force control

Category	Typical tasks	Main challenges
Dexterous manipulation	In-hand rotation, regrasping, tool use	High-dimensional actions, multi-contact dynamics
Loco-manipulation	Walking while grasping, carrying, door opening	Coordination between locomotion and manipulation

These categories overlap. A humanoid carrying a box may simultaneously require locomotion, whole-body coordination, and manipulation.

A. Legged locomotion

The objective is to move stably under a command. A common velocity command is

$$c_t = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega_z^{\mathrm{cmd}}),$$

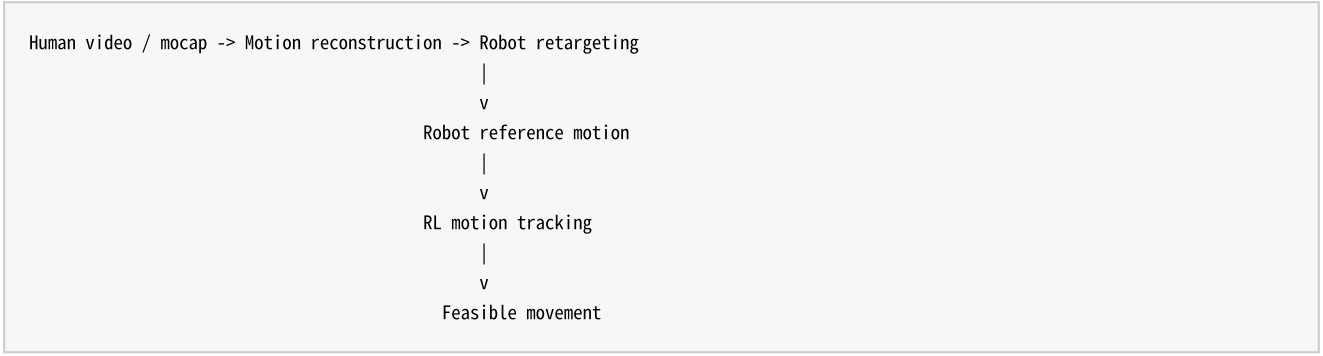
representing longitudinal, lateral, and yaw velocities. The policy outputs joint actions that track these commands. Quadruped and bipedal locomotion have developed relatively mature large-scale simulation workflows. A representative example is *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning*, which trained and deployed an ANYmal controller using massively parallel simulation [1].

B. Agile locomotion

Agile locomotion includes jumping, stair climbing, obstacle crossing, running, fall recovery, and mid-air posture control. Compared with basic walking, the robot must exploit its dynamics and contact transitions. Climbing stairs, for example, requires coordinated foot clearance, landing timing, and center-of-mass transfer.

C. Whole-body motion tracking

The robot is given a reference motion $q^{\mathrm{ref}}(t)$ and learns to track it while remaining dynamically feasible.



Representative approaches include DeepMimic, AMP, HOVER, and BeyondMimic. They explore reference imitation, motion-prior learning, multimodal whole-body control, and difficult humanoid motions [2][3][4].

D. Manipulation control

The goal is to pick up, carry, insert, or otherwise manipulate objects. RL can output joint commands or end-effector pose increments; an IK, operational-space control (OSC), or joint controller may then convert these into actuator commands. Compared with locomotion, these tasks emphasize end-effector accuracy, interaction forces, task success, collision safety, and recovery.

E. Dexterous manipulation

A multi-fingered hand controls several contacts simultaneously:

$$a_t = [a_t^{\text{finger1}}, a_t^{\text{finger2}}, \ldots, a_t^{\text{fingerN}}]^\top.$$

Tasks include stable grasps, in-hand rotation, regrasping, tool use, and recovery after object slip. The primary difficulties are high-dimensional actions and complex finger–object contact dynamics.

F. Loco-manipulation

Loco-manipulation combines moving the robot and interacting with objects. Examples include carrying while walking, using a hand for support during obstacle crossing, manipulating objects with both hands while moving, and opening a door. The objective shifts from isolated skills to coordinated whole-body behavior and contact planning.

2.2 Classification by training strategy

Strategy	Main idea	Suitable applications
RL from scratch	Explore directly with task rewards	Basic balance and walking
Imitation RL	Use rewards defined by reference motions	Dance and motion reproduction
Residual RL	Learn corrections to a baseline controller	Trajectory repair and contact compensation
Hierarchical RL	High-level skill selection and low-level control	Multistage tasks
Teacher–student learning	Train a privileged teacher and distill it	Vision-based control and sim-to-real
Adaptive RL	Adapt through historical observations	Changing friction, payloads, dynamics

These strategies can be combined. For instance, train a privileged teacher with imitation-guided RL, distill its behavior into a student that only sees deployable sensors, and transfer that student to hardware.

Chapter 3: Algorithms for Motion-Control RL

3.1 PPO: a standard baseline for large-scale simulation

Proximal Policy Optimization (PPO) is an on-policy RL algorithm widely used for quadrupeds, bipeds, humanoid motion tracking, and dexterous manipulation. A typical loop is

```
Initialize Actor / Critic
          |
          v
Create thousands of parallel environments
          |
          v
Actor collects (Observation, Action, Reward)
          |
          v
Compute Advantages
          |
          v
Update Actor and Critic -> Repeat
```

PPO is particularly effective when coupled with GPU-parallel physics simulation. Although it reuses older data only to a limited extent, thousands of simulated robots can generate large batches of fresh interactions. For instance, 4,096 environments can simulate independent copies of the same robot [1].

A widely used implementation is RSL-RL, which provides PPO, neural-network components, and training utilities aimed at robotic control [5].

3.2 SAC: improved reuse of interaction data

Soft Actor-Critic (SAC) is an off-policy algorithm. A replay buffer stores past transitions so that they can contribute to multiple updates.

```
Robot / Simulator
        |
        v
   Experience
        |
        v
   Replay Buffer
     /      \
    v        v
Critic      Actor
Update      Update
     \      /
       New Policy
```

Robot interaction may be expensive in the real world, making SAC's data reuse attractive. SAC is a useful candidate for continuous arm manipulation, online improvement with limited real-robot experience, dexterous hands, and residual control. However, PPO can be easier to engineer and scale in very high-throughput GPU simulation. Neither algorithm is universally superior; comparisons require matched environments and interaction budgets.

3.3 PPO versus SAC

Property	PPO	SAC
Category	On-policy	Off-policy
Reuse of older data	Low	High
Replay buffer	Usually unnecessary	Required
Massive parallel simulation	Highly established	Possible, with more design work
High-dimensional continuous actions	Supported	Supported
Real-world sample efficiency	Often lower	Often potentially better
Implementation complexity	Relatively straightforward	Relatively higher
Legged locomotion	Very common	Less common
Arms and dexterous hands	Common	Also used / suitable
Recommended starting point	Simulation-based control baseline	Data-expensive continuous control

Practical recommendation: reproduce PPO first for simulation-based bipedal or quadrupedal control. Investigate SAC when limited physical interaction data and online adaptation are central. Use identical tasks, interaction counts, and evaluation protocols for comparisons.

Chapter 4: Designing a Motion-Control RL Environment

The core of a training environment is

$$\text{Environment} = \{\text{Observation}, \text{Action}, \text{Reward}, \text{Reset}, \text{Dynamics}\}.$$

Goal-conditioned tasks also require a Command or Goal definition. In practice, environment design can matter more than minor changes to a PPO network.

4.1 Observation: what can the robot sense?

Observation	Meaning	Typical source
Joint position	Joint angles or positions	Encoders
Joint velocity	Joint speeds	Encoders / estimators
Base angular velocity	Body rotational velocity	IMU
Projected gravity	Gravity direction expressed in body frame	Orientation estimator
Previous action	Previous control command	Controller
Velocity command	Requested robot velocity	High-level controller
Contact state	Foot / fingertip contact estimates	Contact estimator
Height map	Terrain elevation	Depth camera or other sensors

A representative proprioceptive observation is

$$o_t = [q_t, \dot{q}_t, \omega_t, g_t, a_{t-1}, c_t].$$

Always distinguish **states available in simulation** from **observations accessible on the deployed robot**. Base linear velocity, contacts, and exact object poses often require estimators on hardware.

Observation history

Motion control is frequently partially observable. Current joint angles alone do not reveal a previous slip, an external push, or actuator delay. One solution is to use a history window:

$$o_t^{\text{hist}} = [o_{t-k}, \dots, o_{t-1}, o_t].$$

An MLP, GRU, or Transformer can process this history and help the policy infer hidden dynamical and contact properties.

4.2 Action: what should RL output?

Action space	Output	Typical characteristics
Joint position	Target joint positions	A stable and accessible baseline
Joint velocity	Target joint velocities	Suitable for velocity-control interfaces
Joint torque	Target joint torques	Direct control; harder training and safety challenges
Cartesian action	End-effector pose or velocity	Often natural for robot arms

A. Joint-position control

A common parameterization is

$$q_t^{\text{target}} = q^{\text{default}} + s_a a_t, \qquad a_t = \pi_\theta(o_t).$$

A low-level PD controller then computes

$$\tau_t = K_p(q_t^{\text{target}} - q_t) - K_d \dot{q}_t.$$

For reference tracking, one can instead use

$$q_t^{\text{target}} = q_t^{\text{ref}} + s_a a_t.$$

This is a residual action around the reference. Using PD control relieves the learned policy from modeling every aspect of actuator control.

B. Joint-torque control

The policy directly predicts $\tau_t = \pi_\theta(o_t)$. This gives greater direct control authority, but current, torque, temperature, speed, and position constraints must be enforced explicitly.

C. Cartesian control

For a robot arm, an action may be $a_t = [\Delta x, \Delta y, \Delta z, \Delta R]$. An IK or OSC module maps the Cartesian command into joint commands.

```
RL Policy -> End-effector increment / target -> IK / OSC -> Joints -> Arm
```

The orientation increment ΔR can be represented using a rotation vector or another consistent parameterization. Isaac Lab offers several joint-space and Cartesian control interfaces [6].

Suggested starting points

Robot	Initial action-space recommendation
Quadruped / biped	Joint-position targets
Humanoid motion tracking	Joint-position targets / residuals
Conventional robot arm	Cartesian pose increments
Multi-fingered hand	Finger-joint position targets
Arm plus dexterous hand	Cartesian arm action plus finger-joint action

A policy's output need not match the final actuator interface. For example, the policy may predict a Cartesian pose change, while the motor drivers execute joint-position commands.

4.3 Reward: what behavior should the robot optimize?

A useful decomposition is

$$r_t = r_t^{\text{task}} + r_t^{\text{stability}} - r_t^{\text{regularization}}.$$

For bipedal velocity tracking, possible terms are

$$r_{\text{velocity}} = \exp\left(-\frac{\|v_t - v_t^{\text{cmd}}\|^2}{\sigma_v^2}\right),$$

$$r_{\text{orientation}} = \exp\left(-\frac{\|e_t^{\text{orientation}}\|^2}{\sigma_R^2}\right),$$

$$r_{\text{smooth}} = -\|a_t - a_{t-1}\|^2.$$

One combined reward is

$$r_t = w_v r_{\text{velocity}} + w_R r_{\text{orientation}} - w_a\|a_t - a_{t-1}\|^2 - w_\tau\|\tau_t\|^2.$$

Actual tasks may add body-height, foot-slip, collision, joint-limit, energy, and contact-stability terms. Rewarding only forward velocity may encourage lunging, implausibly fast leg motion, or exploitation of simulator artifacts. Conversely, overly strong regularization may cause the robot to remain still.

Principle: rewards should capture both task accomplishment and safe, stable, deployable behavior.

4.4 Reset: which initial states should be sampled?

Simple locomotion may start from a standing pose. Motion tracking and multistage manipulation should not always begin from one fixed state, or the policy may fail to recover when entering a task midway.

Uniform Reference State Initialization (Uniform RSI): sample a time from a reference motion,

$$t_0 \sim U(0, T),$$

then initialize the robot near the corresponding reference state.

Event-biased RSI: increase the sampling probability near takeoff, landing, contact switches, grasp initiation, or release. Ensure sampled states are physically and contact consistent; unrealistic resets can make learning harder rather than easier.

Chapter 5: Training a Complete Motion Controller

5.1 The standard workflow

```
Robot model (URDF / MJCF / USD)
              |
              v
Physics simulator (Isaac Lab / MuJoCo)
              |
              v
Define Observation / Action / Reward / Reset
              |
              v
         PPO / SAC training
              |
              v
       Simulation evaluation
              |
              v
Domain Randomization / Curriculum
              |
              v
         Sim-to-sim test
              |
              v
         Sim-to-real transfer
              |
              v
              Real robot
```

5.2 Step 1: build a robot simulation model

Component	What must be modeled
Robot model	Link masses, inertias, joint axes, joint limits
Actuator	Motor torque and speed limits, friction, response
Contact	Friction, collision geometry, solver settings

Component	What must be modeled
Sensors	Encoders, IMUs, visual sensors
Control	PD gains, update frequencies, action delay
Environment	Ground, obstacles, manipulated objects

A URDF is not merely a visualization asset: it contains important dynamical information. However, the presence of a parameter in a file does not imply that it has been calibrated against the real robot.

5.3 Step 2: choose control frequencies

The following values are illustrative rather than hardware-specific:

Component	Example frequency
Physics simulation	500 Hz
Joint PD control	500 Hz
RL policy	50 Hz
Vision update	30 Hz
High-level planning	5 Hz

If $f_{\text{sim}} = 500\,\text{Hz}$ and $f_{\text{RL}} = 50\,\text{Hz}$, the simulator advances ten physics steps per policy action. This is called **action decimation**. The high-frequency control loop executes actions while RL updates its goals less often.

5.4 Step 3: parallel training

An **illustrative** PPO configuration for legged locomotion is

Parameter	Example setting
Parallel environments	4096
RL frequency	50 Hz
Rollout length	24 steps per environment
Algorithm	PPO
Actor / Critic	MLP / MLP
Action	Joint-position target
Observation	Proprioception + command

One rollout gathers $4096 \times 24 = 98{,}304$ environment transitions. This counts interaction samples, not gradient updates. Visual control or contact-heavy dexterous manipulation may require fewer parallel environments.

The actor–critic structure is

```
Observation
   |
   +-------> Actor  -----> Action -----> Simulator
   |                                       |
   +-------> Critic -----> Value <------- Reward
```

Both actor and critic are updated during training; typical deployment runs the actor only.

Chapter 6: Motion-Imitation Reinforcement Learning

Direct exploration can be difficult for jumping, dance, gymnastics, and high-dimensional dexterous motions. Reference motions provide useful guidance.

6.1 The basic pipeline

Motion Tracking / Motion Imitation RL learns physical tracking behavior with environment rewards and rollouts; it is not equivalent to supervised action-label BC. For BC/ACT/generative action-chunk training, consult Data and Imitation Learning.

```
Human motion / Teleoperation
             |
             v
     Motion reconstruction
             |
             v
      Motion retargeting
             |
             v
     Robot reference motion
             |
             v
      Motion-tracking RL
             |
             v
       Robust motion policy
```

Reference data may originate from motion capture, human-video reconstruction, teleoperation, existing controllers, or motion planners.

6.2 Motion-tracking rewards

For reference joint positions q_t^{ref}, a simple reward is

$$r_{\mathrm{track}} = \exp\left(-\frac{\|q_t - q_t^{\mathrm{ref}}\|^2}{\sigma_q^2}\right).$$

Whole-body tasks may also track root position, root orientation, joint velocity, end-effector positions, and contact schedules. Similar joint angles alone do not guarantee a physically stable movement. The policy must balance **motion fidelity, dynamic feasibility, and stability**.

6.3 DeepMimic

DeepMimic is a classic example of combining reference-motion imitation rewards with task rewards to train physical characters to perform complex actions [2].

```
Reference motion -> Tracking reward --+
                                      +--> RL policy -> Physics
Task specification -> Task reward ----+
```

Project: https://xbpeng.github.io/projects/DeepMimic/

6.4 AMP: Adversarial Motion Priors

Conventional motion tracking follows a specified reference at each time step. Adversarial Motion Priors (AMP) instead train a discriminator to provide a motion-style reward from reference datasets. A policy can then accomplish its task while exhibiting motions consistent with that prior, such as a natural-looking gait, without strictly reproducing one reference frame by frame [3].

6.5 Motion tracking versus AMP

Property	Motion-tracking RL	AMP
Requires motion data	Typically reference trajectories	Motion examples
Explicitly follows a specified trajectory	Yes	Not necessarily
Reward source	Reference tracking error	Learned motion-prior reward
Primary objective	Reproduce a target motion	Match a style / motion distribution
Major challenge	Temporal alignment and stability	Discriminator training and reward balancing

6.6 From tracking one clip to universal control

Projects such as HOVER investigate unifying multiple motion-command modes within a whole-body policy so that the controller is not restricted to reproducing a fixed sequence [4].

Trend: single-motion tracking policy → multimode / universal motion controller.

Chapter 7: Teacher–Student Reinforcement Learning

7.1 Why use teacher–student training?

A simulator can expose exact object positions and velocities, contact forces, friction coefficients, and hidden dynamical parameters. These may be unavailable or noisy in a real robot. A policy that directly depends on them may fail to transfer.

```
Full simulation state:
object poses, contact forces, friction, dynamics
                    |
```

```
                    v
                Teacher policy
                    |
                    v
                Teacher actions
                    |
                 Distillation
                    |
                    v
Real observations (Camera / Encoder / IMU) -> Student policy
```

7.2 Standard training procedure

First train a teacher with privileged state information:

$$a_t^T = \pi_T(s_t).$$

Then train a student with deployable observations only:

$$a_t^S = \pi_S(o_t).$$

A common action-distillation loss is

$$\mathcal{L}_{\text{distill}} = \mathbb{E}\left[\|a_t^S - a_t^T\|^2\right].$$

The student may use observation histories and learned state estimators to infer hidden properties. Practical distillation also needs to account for differences between the state distributions visited by teacher and student.

7.3 Asymmetric actor–critic versus teacher–student

Asymmetric actor–critic: the actor uses only deployable observations from the start of training, while the critic can use privileged simulation information.

Teacher–student: a teacher actor may itself use privileged information, and its behavior is subsequently transferred to a student actor with limited observations.

The first is a **training architecture**; the second is a **policy-transfer procedure**. They can be combined. Isaac Lab and RSL-RL support relevant observation groups and training configurations [5] [6].

7.4 RMA: Rapid Motor Adaptation

Rapid Motor Adaptation (RMA) integrates a motor policy with a mechanism for adapting to unknown dynamics. Recent observation history provides information that helps the robot respond to changing terrain, friction, payload, and dynamics [7].

Paper: https://arxiv.org/abs/2107.04034

Chapter 8: Case Study I — Training Unitree G1 Walking with PPO

8.1 Task definition

Train the G1 to move stably under $c_t = [v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega_z^{\text{cmd}}]$, including forward/backward movement, lateral movement, and turning.

8.2 Observation

Start with proprioception: joint positions, joint velocities, base angular velocity, projected gravity, previous actions, and velocity commands. For stairs or uneven terrain, gradually add height scans, contact estimates, observation histories, and visual features.

8.3 Action

Use joint-position targets:

$$q_t^{\text{target}} = q^{\text{default}} + s_a \pi_\theta(o_t),$$

executed by a low-level PD controller. This makes it easier to reuse existing hardware interfaces.

8.4 Reward components

Reward	Purpose
Linear velocity tracking	Track longitudinal and lateral velocity
Angular velocity tracking	Track yaw rate
Orientation stability	Maintain appropriate body posture
Action smoothness	Reduce oscillations
Joint-limit penalty	Avoid extreme joint configurations
Foot-slip penalty	Reduce foot sliding
Termination penalty	Penalize falls and failures

8.5 Curriculum learning

Stage	Training condition	Goal
1	Standing	Basic balance
2	Slow forward walking	Basic gait
3	Forward/backward/lateral walking	Broader speed range
4	Turning	Angular-velocity tracking
5	External pushes	Robustness and recovery
6	Uneven terrain	Contact adaptation
7	Stair climbing	Challenging locomotion

The command and terrain distributions can be expanded continuously. A separate controller need not be retrained from scratch for every stage.

8.6 Open-source starting point

Unitree RL Lab builds on Isaac Lab and includes training and deployment-related code for Unitree platforms [8]. The following commands are **illustrative for the repository interface**; verify the current tasks and dependencies in the checked-out version:

```
# List registered tasks
./unitree_rl_lab.sh -l

# Train G1 velocity tracking (illustrative task name)
./unitree_rl_lab.sh -t --task Unitree-G1-29dof-Velocity

# Evaluate a trained policy
./unitree_rl_lab.sh -p --task Unitree-G1-29dof-Velocity
```

First controlled study: reproduce flat-ground walking, then ablate action smoothness, domain randomization, observation history, and the foot-slip penalty. Compare convergence, velocity tracking, and perturbation recovery.

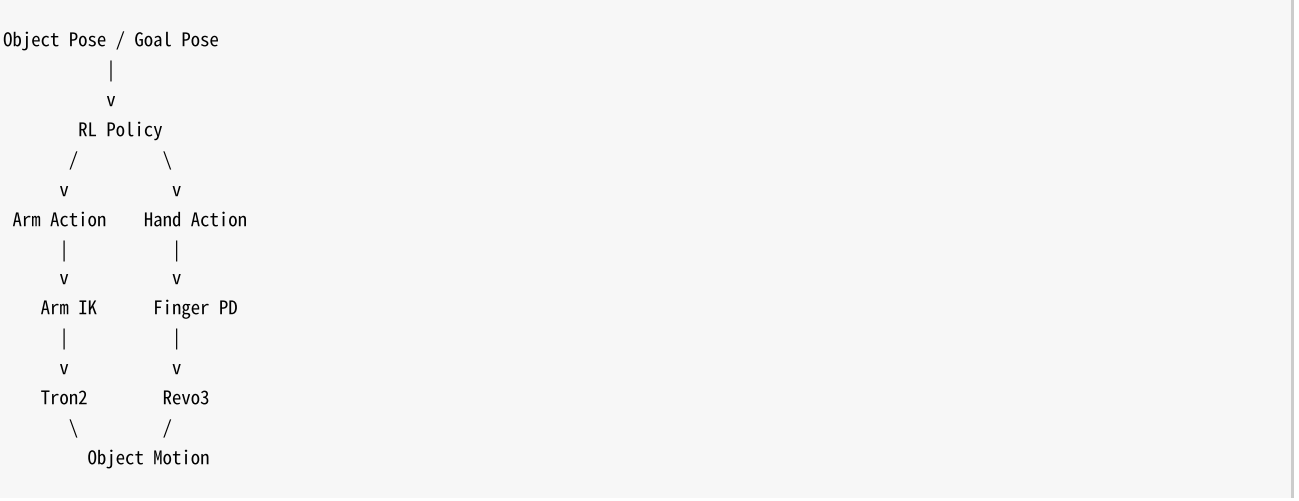
Chapter 9: Case Study II — Arm and Dexterous-Hand RL

Consider a **Tron2 arm, Revo3 dexterous hand, and screwdriver** to illustrate how locomotion-style RL environment design extends to contact-rich manipulation.

9.1 Task definition

1. Pick up a screwdriver from multiple positions on a tabletop.
2. Coordinate arm posture and finger contacts.
3. Move the screwdriver to a designated location.
4. Rotate it by a desired angle.
5. Maintain or recover the grasp after disturbances.

9.2 Recommended control architecture



```
            |
            v
      Observation Feedback
```

It is reasonable to train independent arm and hand controllers first, then move toward a coupled policy.

9.3 Three training routes

Route A: RL from scratch. Define rewards for grasping, lifting, rotation, and success:

$$r_t = r_{\text{grasp}} + r_{\text{lift}} + r_{\text{rotate}} + r_{\text{success}}.$$

This does not require reference trajectories, but exploration in a high-dimensional contact problem is difficult.

Route B: motion-tracking RL. Reconstruct reference end-effector and finger trajectories, $T^{\text{ref}}_{\text{hand}}(t)$ and $q^{\text{ref}}_{\text{finger}}(t)$, from human videos or teleoperation. This reduces exploration difficulty, but one demonstration rarely covers all initial object poses and contact variations.

Route C: residual RL. Add a constrained correction to a trajectory tracker, an IK controller, or an existing policy:

$$a_t = a_t^{\text{base}} + \Delta a_t^{\text{RL}}.$$

For example, the baseline could track an arm-end-effector trajectory while RL adjusts finger contacts; alternatively, RL can produce limited corrections for both arm and hand. Residual magnitudes, action rates, and contact forces must be constrained.

9.4 Observation design

One possible observation vector is

$$o_t = [q_{\text{arm}}, q_{\text{hand}}, \dot{q}, \hat{T}_{\text{object}}, T_{\text{goal}}, a_{t-1}].$$

Here $\hat{T}_{\text{object}}$ is estimated from head- or wrist-mounted cameras. If object velocity is not directly observable, histories or a state estimator can help infer it. **Do not silently give the actor ground-truth object state that only exists in simulation.** The critic, however, may use privileged information.

9.5 Reward design

Reward	Role
Reach	Move the hand near the target
Grasp	Establish a stable grasp
Lift	Lift the object
Position	Move the object to the target region
Rotation	Achieve the desired rotation

Reward	Role
Stability	Reduce slip and drops
Smoothness	Avoid action oscillations
Success	Reward full task completion

A policy with only a terminal rotation-success reward may rarely discover useful grasps through random exploration. A curriculum and carefully chosen intermediate rewards often help.

9.6 Proposed training stages

Stage	Task	Goal
1	Maintain a pre-established grasp	Basic contact control
2	Rotate an already-held object	In-hand manipulation
3	Reach and grasp	Grasp acquisition
4	Grasp, then rotate	Multistage operation
5	Randomize initial object pose	Generalization
6	Add object-pose estimation errors	Visual robustness
7	Add external disturbances	Recovery
8	Evaluate real-robot closed-loop control	Test transfer

When reference trajectories are available, Uniform RSI and Event-biased RSI can emphasize critical segments such as contact establishment, release, and object rotation.

Chapter 10: Simulation-to-Real Transfer

10.1 Why simulation success does not ensure hardware success

Mismatch	Typical cause
Mass / inertia	Errors in link mass and inertia
Friction	Incorrect ground or object friction
Actuator	Motor response, dead zones, internal controllers
Delay	Communication and control latency
Observation noise	Encoder, IMU, and visual error
Contact	Contact solver approximations
Geometry	Incorrect collision geometry
Calibration	Joint zero positions, frame transforms, extrinsics

For instance, a simulated motor may respond much faster than its physical counterpart; the resulting timing error can induce oscillations.

10.2 Domain randomization

Randomize environment and dynamics parameters during training:

$$\xi \sim p(\xi).$$

The vector ξ may include link mass, center of mass, friction, motor strength, action delay, and sensor noise.

Randomized parameter	Purpose
Mass	Robustness to mass mismatch
Center of mass	Robustness to COM errors
Friction	Robustness to contact variation
Motor strength	Robustness to actuator variation
Action delay	Robustness to latency
Observation noise	Robustness to sensor errors
External pushes	Robustness to disturbances

Ranges should reflect calibration, hardware specifications, and physically plausible uncertainty. Wider randomization is not automatically better.

10.3 Sim-to-sim evaluation

A policy trained in Isaac Lab can be tested in MuJoCo, or vice versa, to detect over-reliance on simulator-specific numerical behavior.

```
Isaac Lab Training -> Export Policy -> MuJoCo Evaluation
                                            |
                                            v
                                    Real Robot Test
```

If the controller behaves very differently between engines, inspect dynamics, contact, and action interfaces first. Passing sim-to-sim tests is additional evidence, not a guarantee of real-world safety.

10.4 Real-robot deployment checklist

- Match observation dimensions, ordering, and normalization to training.
- Verify joint indexing, sign conventions, units, coordinate frames, and zero positions.
- Validate action scales, PD gains, control rates, and execution timing.
- Enforce position, velocity, torque, and temperature limits.
- Provide safe fallback behavior, communication watchdogs, and emergency stops.

- Use fall protection for bipeds; guard against excessive contact forces, jamming, and dropped objects for dexterous hands.
- Begin with low-risk, small-amplitude tests before expanding the task envelope.

Advanced topic: learning the simulation–reality dynamics gap

Work such as ASAP uses data from real robots to help align simulated and physical dynamics, thereby improving sim-to-real transfer of agile whole-body behavior [9].

Chapter 11: Evaluating Motion-Control RL

An increasing training reward does not necessarily imply a better deployable controller. Use held-out conditions and independent task metrics.

11.1 Legged-robot metrics

Metric	Meaning
Success rate	Fraction of completed tasks
Fall rate	Frequency of falls
Velocity tracking error	Difference between actual and commanded velocity
Orientation error	Body pose deviation
Energy consumption	Power or energy cost
Foot slip	Unwanted foot sliding
Recovery rate	Success after perturbations
Generalization	Performance on unseen terrain or parameters

11.2 Motion-tracking metrics

Metric	Meaning
Joint tracking error	Joint trajectory deviation
Root position error	Base-position deviation
Root orientation error	Base-orientation deviation
End-effector error	End-effector tracking deviation
Contact consistency	Agreement with intended contacts
Motion completion rate	Fraction of completed motion clips
Recovery rate	Robustness after disturbances

11.3 Dexterous-manipulation metrics

Metric	Meaning
Task success rate	Fraction of successful manipulation trials
Object pose error	Final or trajectory pose deviation
Grasp stability	Ability to retain a stable grasp
Object drop rate	Frequency of dropped objects
Contact slip	Relative slip at contacts
Recovery rate	Recovery after disturbance
Completion time	Time required to finish a task
Generalization	Performance under unseen object poses

Fair comparison: match the environment and task, the interaction budget, and test conditions; report multiple random seeds, unseen initial states, and metrics independent of the training reward. Keep training success, simulated generalization, and real-world performance separate.

Chapter 12: Open-Source Frameworks and Tools

Framework	Main use	Recommendation
Isaac Lab	General robotic simulation and RL	High priority
RSL-RL	PPO, motion control, distillation	High priority
Unitree RL Lab	Unitree robot training and deployment	Useful for G1/H1
MuJoCo Playground	GPU-accelerated robot learning	Worth comparing
legged_gym	Classic legged PPO codebase	Useful for foundations
skrl	Multiple RL algorithms	Useful for comparisons

12.1 Isaac Lab

Repository: https://github.com/isaac-sim/IsaacLab

Isaac Lab covers legged locomotion, humanoid locomotion, manipulation, dexterous manipulation, and contact-rich tasks. Examples of environment IDs from an official versioned catalog include [6]:

Example environment	Task
`Isaac-Velocity-Flat-G1-v0`	G1 flat-ground velocity tracking
`Isaac-Velocity-Rough-G1-v0`	G1 rough-terrain velocity tracking
`Isaac-Velocity-Flat-Unitree-Go2-v0`	Go2 flat-ground locomotion
`Isaac-Reach-Franka-v0`	Franka reaching

Example environment	Task
`Isaac-Lift-Cube-Franka-v0`	Franka cube lifting
`Isaac-Repose-Cube-Allegro-v0`	Allegro hand object reorientation

IDs may change between releases, so consult the task registry of the installed version.

12.2 RSL-RL

Repository: https://github.com/leggedrobotics/rsl_rl

A productive code-reading sequence is **Runner** → **PPO** → **Actor/Critic** → **Rollout Storage** → **Distillation**. For PPO research in motion control, this is a practical baseline implementation [5].

12.3 MuJoCo Playground

Repository: https://github.com/google-deepmind/mujoco_playground

MuJoCo Playground provides GPU-accelerated learning tasks for quadrupeds, humanoids, dexterous hands, and robot arms. Some batched dynamics tasks can be more lightweight than running a full rendering-focused simulator, but performance depends on GPU, contact complexity, visual input, and the simulation backend [10].

12.4 Which framework should you choose?

Research goal	Suggested starting point
Learn PPO for motion control	Isaac Lab + RSL-RL
Train a G1 walking controller	Unitree RL Lab
Massive GPU-parallel RL	Isaac Lab / MuJoCo Playground
Dexterous manipulation	Isaac Lab / MuJoCo Playground
Combine vision and control	Isaac Lab
Modify PPO itself	RSL-RL
Compare SAC and PPO	An RL stack supporting both methods
Lightweight control experiments	MuJoCo Playground

Chapter 13: Representative Papers and Reading Order

Paper / project	Year	Core topic	What to learn
DeepMimic	2018	Motion-imitation RL	Reference rewards and RSI
AMP	2021	Adversarial motion priors	Style rewards from motion datasets
RMA	2021	Rapid adaptation	Observation history and dynamics

Paper / project	Year	Core topic	What to learn
Learning to Walk in Minutes	2022	Parallel PPO	Large-scale locomotion training
Learning Robust Perceptive Locomotion	2022	Perceptive locomotion	Terrain perception and transfer
HOVER	2025	Multimode whole-body control	Universal command representations
ASAP	2025	Dynamics-gap alignment	Sim-to-real
BeyondMimic	2025	Complex humanoid motions	Tracking and skill generalization
MuJoCo Playground	2025	GPU robot learning	Modern experimentation platform

Four particularly useful starting points are:

1. **Learning to Walk in Minutes:** parallel PPO, reward design, curriculum learning, and sim-to-real [1].
2. **DeepMimic:** reference motion, imitation rewards, reference-state initialization, and physics-based motion tracking [2].
3. **RMA:** privileged information, observation history, motor adaptation, and deployment [7].
4. **HOVER:** whole-body control, multiple command modalities, tracking, and distillation [4].

Chapter 14: Learning and Research Roadmap

Stage 1: basic motion-control RL

Goal: train a quadruped or biped to walk stably.

Learn Isaac Lab, RSL-RL, PPO, observation/action/reward design, and domain randomization. First reproduce velocity tracking on a platform such as G1; then ablate reward terms and observations.

Stage 2: motion-tracking RL

Goal: track a reference while maintaining dynamic feasibility and recovery ability.

Study motion retargeting, tracking rewards, RSI, curricula, and residual RL. Start from one reference motion, then vary initial states and external disturbances.

Stage 3: teacher–student and sim-to-real

Goal: eliminate dependence on simulator-only ground-truth state.

Study asymmetric actor–critic, privileged information, distillation, observation histories, and domain randomization. Train a privileged teacher, transfer its behavior to a deployable student, and test the controller in another simulator.

Stage 4: complex motion control

Direction	Example tasks
Humanoid locomotion	Stairs, obstacles, jumping, fall recovery
Dexterous manipulation	In-hand rotation, tool use, disturbance recovery

Focus on contact dynamics, task curricula, strong robustness, and closed-loop hardware control.

Stage 5: universal motion controllers

Investigate a conditional policy that can handle multiple commands:

$$a_t = \pi_\theta(o_t, c_t).$$

The condition c_t may encode desired velocity, target pose, a reference-motion segment, desired object motion, or skill identity. Related directions include goal-conditioned motion control, universal motion tracking, hierarchical motion policies, residual motor adaptation, and learning-based whole-body control.

Chapter 15: Summary and Main Takeaways

Motion-control RL is best understood along three dimensions: **task category, training strategy, and engineering workflow**.

Task	Central question
Locomotion	How can a robot move stably?
Agile locomotion	How can it execute dynamic maneuvers?
Motion tracking	How can it reproduce reference motions accurately?
Manipulation	How can it manipulate objects?
Dexterous manipulation	How can it coordinate multiple finger contacts?
Loco-manipulation	How can it coordinate movement and manipulation?

Method	Main question addressed
PPO	How can large parallel simulation train controllers?
SAC	How can interaction data be reused efficiently?
Imitation RL	How can reference demonstrations ease exploration?
Residual RL	How can an existing controller be improved?
Teacher–student	How can a privileged policy be transferred to real observations?
Adaptive RL	How can the controller handle unknown dynamics and environments?

The complete development workflow is

$$\boxed{\begin{array}{c}\text{Robot Modeling} \rightarrow \text{Task Design} \rightarrow \text{PPO/SAC Training} \\ \text{Curriculum} + \text{DR} \rightarrow \text{Sim-to-Sim} \rightarrow \text{Sim-to-Real}\end{array}}.$$

Takeaway 1: task design matters as much as the algorithm. Observations, actions, rewards, resets, simulation fidelity, and curriculum schedules strongly influence results.

Takeaway 2: PPO plus parallel simulation remains an important baseline. It is well established in legged locomotion and motion tracking. When real interaction data are scarce, algorithms with greater data reuse, such as SAC, are worth considering.

Takeaway 3: general-purpose, conditioned, robust motion controllers are a promising direction. One coherent research progression is

Motion Tracking RL → Residual RL → Robust Sim-to-Real → Goal-conditioned Universal Motion Control.

References and Open-Source Links

The following links point primarily to papers, project pages, and official repositories. Confirm final bibliographic details and currently supported task IDs from the relevant publication and software release.

1. Rudin et al., *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning* (2022)
2. Peng et al., *DeepMimic* (2018)
3. Peng et al., *AMP: Adversarial Motion Priors* (2021)
4. HOVER: Versatile Neural Whole-Body Controller for Humanoid Robots; paper
5. RSL-RL official repository
6. Isaac Lab official repository; environment catalog
7. RMA: Rapid Motor Adaptation for Legged Robots
8. Unitree RL Lab official repository
9. ASAP: Aligning Simulation and Real-World Physics
10. MuJoCo Playground official repository
11. legged_gym official repository
12. skrl official repository

Document version: October 15, 2026. Numerical settings are instructional examples, not validated safety configurations for real robots.

PPO: A Complete Derivation and Its Design Motivations

Course home · 简体中文 · PDF

Item	Details
Applicability	PPO-Clip / GAE theory; PyTorch-style examples are instructional, not a tested training package
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

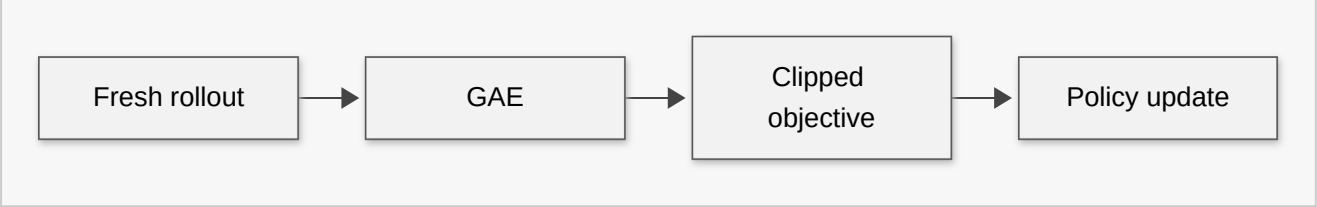
Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- Part I. From Expected Return to Policy Gradients
  - 1. Formulating the reinforcement-learning objective
  - 2. Deriving policy gradients using the log-derivative trick
  - 3. Causality and reward-to-go
- Part II. Baselines, Value Functions, and Advantage
  - 4. Why can we subtract a baseline?
  - 5. Why use the value function as the baseline?
- Part III. Performance Differences, Surrogate Objectives, and Importance Sampling
  - 6. Why can't we reuse old data indefinitely?
  - 7. Step-by-step proof of the performance difference lemma
  - 8. Approximating true improvement by a surrogate objective
  - 9. Deriving the probability ratio by importance sampling
- Part IV. TRPO: Controlling Policy Updates with a Trust Region
  - 10. Why is unconstrained maximization of CPI dangerous?
  - 11. KL divergence and the TRPO constraint
  - 12. TRPO's local second-order approximation and natural gradient
- Part V. Deriving the PPO-Clipped Objective
  - 13. From a KL penalty to a clipped surrogate
  - 14. Positive advantage: stop rewarding overly large probability increases
  - 15. Negative advantage: stop rewarding overly large probability decreases
  - 16. Why do we need `min` instead of clipping alone?

    - 17. Gradient-level interpretation of clipping
    - 18. PPO does not enforce a hard ratio or KL constraint
- Part VI. GAE: How Do We Estimate the Advantage?
    - 19. Monte Carlo advantage: motivation and cost
    - 20. One-step TD advantage from the Bellman equation
    - 21. Deriving n-step advantage estimates
    - 22. Exponential weighting and the GAE derivation
    - 23. The $O(T)$ backward recursion for GAE
    - 24. Constructing critic targets from GAE
- Part VII. The Complete Actor–Critic Loss
    - 25. Actor loss
    - 26. Critic loss
    - 27. Motivation for the entropy bonus
    - 28. Final PPO training objective
- Part VIII. Continuous-Action PPO and Robot Control
    - 29. Gaussian policy log probabilities and importance ratios
    - 30. How does a Gaussian-policy gradient move the action mean?
    - 31. Example: residual PPO for robot manipulation
- Part IX. A Complete PPO Iteration
    - 32. Ten steps from rollout collection to policy updates
    - 33. Compact, adaptable PyTorch implementations
- Part X. Hyperparameters and Diagnostics
    - 34. Common starting hyperparameters and what they mean
- Part XI. Connections to REINFORCE, Actor–Critic, and TRPO
    - 35. Algorithmic progression: one design choice per problem
- Part XII. Final Formula Sheet and Research Takeaways
    - References and further reading

Chapter concept map



Part I. From Expected Return to Policy Gradients

1. Formulating the reinforcement-learning objective

Consider a Markov decision process (MDP)

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0),$$

where $s_t \in \mathcal{S}$ denotes a state, $a_t \in \mathcal{A}$ an action, $P(s_{t+1} \mid s_t, a_t)$ the environment transition kernel, r_t the reward, ρ_0 the initial-state distribution, and $\pi_\theta(a \mid s)$ a stochastic policy parameterized by θ. The goal is

$$\boxed{\theta^* = \arg\max_\theta J(\theta)}, \qquad \boxed{J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]}. \tag{1}$$

A trajectory is $\tau = (s_0, a_0, s_1, a_1, \ldots)$, with discounted return $R(\tau) = \sum_t \gamma^t r_t$. Consequently,

$$J(\theta) = \int p_\theta(\tau) R(\tau)\, d\tau.$$

Motivation and difficulty. Policy parameters influence actions, which affect future states and subsequent rewards:

$$\theta \longrightarrow a_0 \longrightarrow s_1 \longrightarrow a_1 \longrightarrow \cdots \longrightarrow R(\tau).$$

With contact, collision, friction, or unknown dynamics, differentiating the return through the physical environment may be unavailable or impractical. We need a gradient estimator that **does not require differentiable dynamics**.

2. Deriving policy gradients using the log-derivative trick

Differentiate the expected return:

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau.$$

Apply the score-function identity

$$\boxed{\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)}. \tag{2}$$

We obtain

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[R(\tau) \nabla_\theta \log p_\theta(\tau)\right]}. \tag{3}$$

This is the **likelihood-ratio (score-function) trick**. It converts the gradient of a probability density into a derivative of its log density. The second key observation is that an MDP trajectory probability factors:

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{\infty} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Taking logarithms, and assuming neither the initial-state distribution nor the transition kernel depends on θ,

$$\begin{aligned} \log p_\theta(\tau) &= \log \rho_0(s_0) + \sum_t \log \pi_\theta(a_t \mid s_t) + \sum_t \log P(s_{t+1} \mid s_t, a_t), \\ \boxed{\nabla_\theta \log p_\theta(\tau)} &= \boxed{\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)}. \end{aligned} \tag{4}$$

Thus, formally,

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[R(\tau)\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]}. \tag{5}$$

A rigorous infinite-horizon derivation requires the usual integrability and interchange-of-limits assumptions; alternatively, derive the result in finite horizon and take a suitable limit.

What did this accomplish? We differentiate only the policy network's $\log \pi_\theta(a_t \mid s_t)$. The environment is used in the forward direction to generate trajectories and rewards, without requiring its derivatives.

3. Causality and reward-to-go

Equation (5) multiplies every action's score gradient by the entire trajectory return. But the action a_t cannot change rewards that occurred before t. The zero-mean score identity makes the expected contribution of those past-reward cross terms vanish. Define reward-to-go

$$G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_k.$$

Then

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t\right]}. \tag{6}$$

Motivation. Credit an action for outcomes that it can actually influence. Discarding past rewards often reduces variance while preserving the expected gradient. The explicit γ^t is needed for the return objective in Eq. (1); implementations commonly average sampled time steps without this explicit factor, which amounts to a particular sampling/weighting convention rather than an identical expression for every infinite-horizon formulation.

Part II. Baselines, Value Functions, and Advantage

4. Why can we subtract a baseline?

Reward-to-go still fluctuates because future outcomes are random. Introduce a state-dependent baseline $b(s_t)$:

$$g_t = \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,[G_t - b(s_t)].$$

For a fixed state, the additional term has zero expectation:

$$\begin{aligned}
&\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] \\
&= b(s) \int \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)\, da \\
&= b(s) \int \nabla_\theta \pi_\theta(a \mid s)\, da \\
&= b(s) \nabla_\theta \int \pi_\theta(a \mid s)\, da \\
&= b(s) \nabla_\theta 1 = 0.
\end{aligned} \tag{7}$$

Therefore,

$$\boxed{\mathbb{E}[\nabla \log \pi_\theta \, G_t] = \mathbb{E}[\nabla \log \pi_\theta \, (G_t - b(s_t))]}. \tag{8}$$

Motivation. Remove a common, action-independent component of future return, leaving a more informative relative-action signal. Not every baseline necessarily reduces variance; the minimum-variance baseline can depend on the squared norm of the score gradient. Using $V^\pi(s)$ is a natural and highly practical choice.

5. Why use the value function as the baseline?

Define the state-value function, action-value function, and advantage function:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s], \\ Q^\pi(s,a) &= \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a], \\ \boxed{A^\pi(s,a)} &= \boxed{Q^\pi(s,a) - V^\pi(s)}. \end{aligned} \tag{9}$$

$V^\pi(s)$ measures expected return when continuing with the current policy. $Q^\pi(s,a)$ measures expected return when first taking action a, and then following the policy. Hence:

- $A^\pi(s,a) > 0$: action a performs better than the policy's average action at s; it should become more likely.
- $A^\pi(s,a) < 0$: the action is worse than average; it should become less likely.
- $A^\pi(s,a) = 0$: this estimate supplies no relative preference signal.

Robot grasping example. At a given grasping state, suppose $V^\pi(s) = 10$. Refining a finger contact has $Q = 15$ and advantage $+5$; maintaining the current posture has $Q = 10$ and advantage 0; prematurely closing the fingers has $Q = 3$ and advantage -7. Learning should reflect **relative quality**, not merely whether a reward is positive.

Using the exact value baseline yields the advantage policy-gradient formula:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) A^{\pi_\theta}(s_t, a_t)\right]}. \tag{10}$$

This motivates the **actor–critic architecture**: the actor π_θ generates actions; the critic V_ϕ approximates state values; the advantage connects the two. An imperfect state-only baseline does not itself change the ideal score-function expectation, but bootstrapped advantage estimators constructed from an imperfect critic can be biased.

Part III. Performance Differences, Surrogate Objectives, and Importance Sampling

6. Why can't we reuse old data indefinitely?

The policy-gradient theorem describes expectations under the current policy. After data is collected with π_{θ_0} and the policy changes to π_{θ_1} or π_{θ_2}, the same rollout is still distributed according to π_{θ_0}, not the new policy. Recollecting trajectories after every optimizer step, however, is expensive.

One goal of PPO is to **perform several relatively safe first-order optimization steps on one batch of old-policy data**. Achieving this requires separating changes in the state distribution from changes in action probabilities at a given state.

7. Step-by-step proof of the performance difference lemma

Define the normalized discounted state-visitation distribution

$$\boxed{d_\pi(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^t \Pr(s_t = s \mid \pi)}. \tag{11}$$

For continuous state spaces, this notation refers to the corresponding density or probability measure. For any new and old policies, the **performance difference lemma** gives the exact identity

$$\boxed{J(\pi_\theta) - J(\pi_{\text{old}}) = \frac{1}{1-\gamma}\mathbb{E}_{\substack{s\sim d_{\pi_\theta}\\ a\sim\pi_\theta}}[A^{\pi_{\text{old}}}(s,a)]}. \tag{12}$$

Proof. The Bellman equation for the old policy implies

$$A^{\pi_{\text{old}}}(s_t,a_t) = \mathbb{E}[r_t + \gamma V^{\pi_{\text{old}}}(s_{t+1}) - V^{\pi_{\text{old}}}(s_t) \mid s_t, a_t].$$

Sum the discounted advantages along a trajectory generated by the **new** policy:

$$\begin{aligned}
&\mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t A^{\pi_{\text{old}}}(s_t,a_t)\right] \\
&= \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t r_t + \sum_t \gamma^{t+1} V^{\pi_{\text{old}}}(s_{t+1}) - \sum_t \gamma^t V^{\pi_{\text{old}}}(s_t)\right].
\end{aligned}$$

The last two sums telescope, leaving $-V^{\pi_{\text{old}}}(s_0)$ when the limiting tail vanishes. Thus

$$\mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t A^{\pi_{\text{old}}}(s_t,a_t)\right] = J(\pi_\theta) - J(\pi_{\text{old}}).$$

Re-expressing the discounted sum using the occupancy distribution gives Eq. (12).

Interpretation. A new policy improves performance when, in the states it *actually visits* and with the actions it *actually chooses*, it achieves positive advantage relative to the old policy.

8. Approximating true improvement by a surrogate objective

The challenge in Eq. (12) is the new-policy state distribution:

$$s \sim d_{\pi_\theta}.$$

We only possess trajectories from the old policy. If the policies remain close, we can make the **local approximation**

$$d_{\pi_\theta}(s) \approx d_{\pi_{\text{old}}}(s).$$

Ignoring the constant factor $1/(1-\gamma)$, define

$$\boxed{L_{\text{sur}}(\theta) = \mathbb{E}_{\substack{s\sim d_{\pi_{\text{old}}}\\ a\sim\pi_\theta}}[A^{\pi_{\text{old}}}(s,a)]}. \tag{13}$$

We have frozen the state distribution and need only reason about changes in action probabilities conditional on state. With exact advantages, this surrogate has the correct local first-order direction at the old policy (up to the discarded positive constant).

Crucial condition. The approximation is credible only while π_θ remains reasonably close to π_{old}. This is precisely the motivation behind trust regions and clipping.

9. Deriving the probability ratio by importance sampling

Equation (13) still takes actions from the new policy, whereas our stored actions were sampled by the old policy. Conditional on state s,

$$\begin{aligned}\mathbb{E}_{a\sim\pi_\theta}[A^{\pi_{\text{old}}}(s,a)] &= \int \pi_\theta(a \mid s) A^{\pi_{\text{old}}}(s,a)\, da \\ &= \int \pi_{\text{old}}(a \mid s) \frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A^{\pi_{\text{old}}}(s,a)\, da.\end{aligned}$$

Define the probability ratio

$$\boxed{r_\theta(s,a) = \frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)}}. \tag{14}$$

We arrive at a conservative-policy-iteration (CPI) surrogate:

$$\boxed{L^{\text{CPI}}(\theta) = \mathbb{E}_{\substack{s\sim d_{\pi_{\text{old}}} \\ a\sim\pi_{\text{old}}}}\left[r_\theta(s,a) A^{\pi_{\text{old}}}(s,a)\right]}. \tag{15}$$

In practice this is written $\mathbb{E}_t[r_t(\theta)\hat{A}_t]$.

For example, if the old policy assigns probability 0.2 to an action and the new policy assigns 0.3, then $r = 1.5$; a new probability of 0.1 gives $r = 0.5$. Positive advantage encourages a larger ratio; negative advantage encourages a smaller one.

Limits. This importance-sampling identity requires appropriate support / absolute continuity: an action receiving positive probability under the new policy cannot have zero probability under the old policy. Standard nondegenerate Gaussians typically satisfy this support condition. Importantly, the ratio corrects the **conditional action distribution**, not the discrepancy between d_{π_θ} and $d_{\pi_{\text{old}}}$. It therefore does not eliminate the state-distribution approximation in Eq. (13).

Part IV. TRPO: Controlling Policy Updates with a Trust Region

10. Why is unconstrained maximization of CPI dangerous?

For an action with positive advantage, $r_t\hat{A}_t$ increases as r_t grows. With $\hat{A} = 5$, ratios $1.2, 2, 10$ yield contributions $6, 10, 50$. It seems attractive to drive the probability of such an action arbitrarily high. But its advantage was estimated under the old policy, and may no longer predict performance after substantial policy changes.

A robot's changed action distribution can trigger a different contact mode, drop an object, or induce states almost absent from the old dataset. Hence increasing L^{CPI} does **not** guarantee an increase in actual return J.

11. KL divergence and the TRPO constraint

Measure changes between the action distributions at a fixed state using KL divergence:

$$D_{\mathrm{KL}}(\pi_{\mathrm{old}}\|\pi_\theta) = \mathbb{E}_{a\sim\pi_{\mathrm{old}}}\left[\log\frac{\pi_{\mathrm{old}}(a\mid s)}{\pi_\theta(a\mid s)}\right].$$

The practical TRPO optimization problem is

$$\boxed{\begin{aligned}\max_\theta \quad & \mathbb{E}_t[r_t(\theta)\hat{A}_t] \\ \text{s.t.} \quad & \mathbb{E}_{s\sim d_{\pi_{\mathrm{old}}}}[D_{\mathrm{KL}}(\pi_{\mathrm{old}}(\cdot\mid s)\|\pi_\theta(\cdot\mid s))] \le \delta.\end{aligned}} \tag{16}$$

The parameter δ controls the trust-region size. The motivation is not "always make the smallest change"; rather, **seek improvements inside a local region where the performance surrogate is comparatively reliable**.

The TRPO analysis yields performance bounds that penalize the worst-state policy discrepancy; a representative structural form is

$$J(\pi_\theta) \ge J(\pi_{\mathrm{old}}) + \frac{L_{\mathrm{sur}}(\theta)}{1-\gamma} - C\max_s D_{\mathrm{KL}}(\pi_{\mathrm{old}}\|\pi_\theta). \tag{17}$$

Here C depends on bounds on advantages / returns and on γ; precise constants and KL orientation depend on the chosen inequality. The theoretical control of a **worst-state** discrepancy is stronger than the **average KL** constraint commonly implemented. Actual neural-network TRPO therefore has no unconditional monotonic-improvement guarantee.

12. TRPO's local second-order approximation and natural gradient

Write $\Delta\theta = \theta - \theta_{\mathrm{old}}$. Taylor-expand the surrogate:

$$L^{\mathrm{CPI}}(\theta) \approx L^{\mathrm{CPI}}(\theta_{\mathrm{old}}) + g^\top\Delta\theta, \quad g = \nabla_\theta L^{\mathrm{CPI}}\big|_{\theta_{\mathrm{old}}}.$$

KL divergence is zero, and has zero first derivative, at identical distributions. Its second-order expansion is

$$\bar{D}_{\mathrm{KL}} \approx \tfrac{1}{2}\Delta\theta^\top F\Delta\theta,$$

where F is the Fisher information matrix evaluated at the old parameters. To make the evaluation point unambiguous, define $u_{\mathrm{old}}(s,a) = \nabla_\theta \log\pi_\theta(a\mid s)|_{\theta=\theta_{\mathrm{old}}}$. Then

$$F = \mathbb{E}_{s\sim d_{\pi_{\mathrm{old}}},\, a\sim\pi_{\mathrm{old}}}[u_{\mathrm{old}}(s,a)u_{\mathrm{old}}(s,a)^\top].$$

The local approximation is

$$\boxed{\max_{\Delta\theta} g^\top\Delta\theta \quad \text{s.t.} \quad \tfrac{1}{2}\Delta\theta^\top F\Delta\theta \le \delta.} \tag{18}$$

Its Lagrangian is

$$\mathcal{L}(\Delta\theta,\eta) = g^\top\Delta\theta - \eta\big(\tfrac{1}{2}\Delta\theta^\top F\Delta\theta - \delta\big).$$

Stationarity $g - \eta F\Delta\theta = 0$ yields $\Delta\theta = \eta^{-1}F^{-1}g$. Enforcing the active constraint, assuming F positive definite and $g \neq 0$, gives

$$\boxed{\Delta\theta^* = \sqrt{\frac{2\delta}{g^\top F^{-1} g}} F^{-1} g}. \tag{19}$$

The direction $F^{-1}g$ is the hallmark of **natural policy gradient**. A large policy network cannot directly invert its Fisher matrix, so TRPO relies on Fisher-vector products, conjugate gradients, line search, and often damping. **PPO's design motivation** is to retain conservative policy improvement while dispensing with this complicated second-order constrained optimization.

Part V. Deriving the PPO-Clipped Objective

13. From a KL penalty to a clipped surrogate

A straightforward first-order alternative is PPO-Penalty:

$$L^{\mathrm{KLPEN}}(\theta) = \mathbb{E}_t[r_t(\theta)\hat{A}_t] - \beta\,\mathbb{E}_t[D_{\mathrm{KL}}(\pi_{\mathrm{old}} \| \pi_\theta)]. \tag{20}$$

A small β may under-regularize updates, while a large β can make learning unnecessarily slow; adaptive tuning is commonly needed. PPO-Clip instead restricts the **additional surrogate gain** obtained from overly large changes in action probability. Define

$$\mathrm{clip}(r, 1-\epsilon, 1+\epsilon) = \min\{1+\epsilon, \max\{r, 1-\epsilon\}\}.$$

The familiar PPO objective is

$$\boxed{L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big]}. \tag{21}$$

Values around $\epsilon = 0.1$–0.2 are common starting points, not universal prescriptions. The key is to understand **why the `min` and `clip` operations must be used together**.

14. Positive advantage: stop rewarding overly large probability increases

If $\hat{A}_t > 0$, we want the probability of this action to increase, hence $r_t > 1$ is beneficial. Equation (21) reduces to

$$\boxed{L_t^{\mathrm{CLIP}} = \min(r_t, 1+\epsilon)\hat{A}_t = \begin{cases} r_t\hat{A}_t, & r_t \le 1+\epsilon, \\ (1+\epsilon)\hat{A}_t, & r_t > 1+\epsilon. \end{cases}} \tag{22}$$

Set $\hat{A}_t = 2$ and $\epsilon = 0.2$:

Ratio r_t	Unclipped $r_t\hat{A}_t$	PPO contribution
0.8	1.6	1.6
1.0	2.0	2.0
1.1	2.2	2.2
1.2	2.4	2.4
1.5	3.0	2.4

Ratio r_t	Unclipped $r_t\hat{A}_t$	PPO contribution
2.0	4.0	2.4

Once $r_t > 1.2$, that sample no longer rewards further increases in its probability; its derivative in that direction is zero. **Motivation:** Avoid excessively reinforcing a sample that appears advantageous under the old policy.

15. Negative advantage: stop rewarding overly large probability decreases

For $\hat{A}_t < 0$, reducing the action probability is beneficial. Multiplying by a negative advantage reverses the min/max relationship:

$$\boxed{L_t^{\text{CLIP}} = \max(r_t, 1-\epsilon)\hat{A}_t = \begin{cases} (1-\epsilon)\hat{A}_t, & r_t < 1-\epsilon, \\ r_t\hat{A}_t, & r_t \geq 1-\epsilon. \end{cases}} \tag{23}$$

With $\hat{A}_t = -2$ and $\epsilon = 0.2$:

Ratio r_t	Unclipped $r_t\hat{A}_t$	PPO contribution
0.5	-1.0	-1.6
0.7	-1.4	-1.6
0.8	-1.6	-1.6
1.0	-2.0	-2.0
1.2	-2.4	-2.4
1.5	-3.0	-3.0

PPO maximizes this objective, so reducing r_t is helpful for a negative-advantage sample. Once $r_t < 0.8$, however, further probability decreases produce no additional surrogate reward.

16. Why do we need `min` instead of clipping alone?

Suppose we optimized only $\text{clip}(r, 1-\epsilon, 1+\epsilon)\hat{A}$. If $\hat{A} > 0$, $r = 0.5$, and $\epsilon = 0.2$, then a good action has already been made much less likely—an **unfavorable** change. Clipping alone returns $0.8\hat{A}$ and gives zero derivative everywhere below $r = 0.8$, providing no correction signal.

With `min`, the selected term is instead $r\hat{A}$, because $r\hat{A} < 0.8\hat{A}$. The positive derivative persists and pushes the action probability back up. The symmetric argument applies to unfavorable probability increases for negative-advantage actions.

Direction beneficial but change excessive:	stop extra surrogate reward;
Direction unfavorable for the advantage:	retain corrective gradient.

Since $\min(x, y) \leq x$, the sample-wise bound $L^{\text{CLIP}} \leq L^{\text{CPI}}$ holds. This is a pessimistic **lower bound on the unclipped surrogate, not a guaranteed lower bound on the true return** J.

17. Gradient-level interpretation of clipping

Because the denominator in the probability ratio is fixed while optimizing the current rollout,

$$\boxed{\nabla_\theta r_t(\theta) = r_t(\theta)\nabla_\theta \log \pi_\theta(a_t \mid s_t)}. \tag{24}$$

Away from nonsmooth boundaries, the single-sample gradient is

$$\boxed{\nabla_\theta L_t^{\mathrm{CLIP}} = \begin{cases} 0, & \hat{A}_t > 0,\ r_t > 1+\epsilon, \\ 0, & \hat{A}_t < 0,\ r_t < 1-\epsilon, \\ \hat{A}_t r_t \nabla_\theta \log \pi_\theta(a_t \mid s_t), & \text{otherwise.} \end{cases}} \tag{25}$$

At $\theta = \theta_{\mathrm{old}}$, all ratios equal one. The initial local gradient therefore agrees with CPI / advantage policy gradient (up to the chosen normalization). **PPO does not change the initial improvement direction; it moderates the gain from further changes as repeated updates move the probability ratios away from one.**

18. PPO does not enforce a hard ratio or KL constraint

Choosing $\epsilon = 0.2$ does **not** ensure $0.8 \le r_t \le 1.2$ after an optimizer step. Clipping changes an objective, not the feasible parameter set. Parameters are shared across states and samples, and several mini-batch epochs can still drive the average KL too high.

Practical PPO systems therefore monitor

$$\bar{D}_{\mathrm{KL}}(\pi_{\mathrm{old}} \| \pi_\theta)$$

and often use target-KL early stopping, learning-rate adaptation, and/or gradient clipping. Distinguish three separate operations: **ratio clipping** limits surrogate incentives, **gradient clipping** limits gradient norms, and **action clipping** limits executed command values.

Part VI. GAE: How Do We Estimate the Advantage?

19. Monte Carlo advantage: motivation and cost

The $\hat{A}_t$ used by PPO is not directly observed. By definition $A^\pi = Q^\pi - V^\pi$. The simplest estimate substitutes the sampled reward-to-go for the action value:

$$\boxed{\hat{A}_t^{\mathrm{MC}} = G_t - V_\phi(s_t)}, \qquad G_t = \sum_{l=0}^{T-t-1} \gamma^l r_{t+l}. \tag{26}$$

For a properly completed episode, the conditional expectation of the sampled return G_t equals $Q^\pi(s_t, a_t)$. No value-function bootstrap of distant future outcomes is needed. However, imagine a grasping task where success is rewarded only at time 100. Whether an action at time 10 receives a large Monte Carlo return depends on many subsequent random events; the gradient estimate may therefore have high variance.

Question: Could we estimate advantage with shorter segments of experience instead of depending on the entire future trajectory?

20. One-step TD advantage from the Bellman equation

The Bellman identity reads

$$Q^{\pi}(s_t, a_t) = \mathbb{E}[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t, a_t].$$

Consequently, the one-step temporal-difference (TD) residual

$$\boxed{\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)} \tag{27}$$

is a candidate advantage estimator:

$$\boxed{\hat{A}_t^{(1)} = \delta_t}. \tag{28}$$

It needs just one transition s_t, a_t, r_t, s_{t+1}. Replacing random distant rewards by $V_\phi(s_{t+1})$ often reduces variance, but introduces bootstrap bias if the critic is inaccurate. At a true terminal state the future value is zero; at an artificial time cutoff it is usually appropriate to bootstrap.

Estimator	Main benefit	Main limitation
Monte Carlo	Less bias from bootstrapping future values	High variance from uncertain long-term returns
One-step TD	Lower variance and immediate feedback	Greater dependence on critic accuracy

21. Deriving n-step advantage estimates

The one-step estimate is

$$\hat{A}_t^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t) = \delta_t.$$

The two-step estimate is

$$\begin{aligned} \hat{A}_t^{(2)} &= r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t) \\ &= [r_t + \gamma V(s_{t+1}) - V(s_t)] \\ &\qquad + \gamma[r_{t+1} + \gamma V(s_{t+2}) - V(s_{t+1})] \\ &= \boxed{\delta_t + \gamma \delta_{t+1}}. \end{aligned} \tag{29}$$

Similarly, the three-step estimate is

$$\hat{A}_t^{(3)} = \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2}.$$

For a general horizon n,

$$\boxed{\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}}. \tag{30}$$

Small n emphasizes value bootstrapping; large n uses more observed future rewards. **GAE's motivation:** Rather than choosing one n, smoothly aggregate many n-step estimators.

22. Exponential weighting and the GAE derivation

Let $0 \leq \lambda < 1$. Define

$$\boxed{\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\hat{A}_t^{(n)}}. \tag{31}$$

The weights sum to one: $(1-\lambda)\sum_{n\geq 1}\lambda^{n-1} = 1$. Substitute the n-step expansion and exchange the summation order:

$$\begin{aligned}
\hat{A}_t^{\mathrm{GAE}} &= (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\sum_{l=0}^{n-1}\gamma^l\delta_{t+l} \\
&= \sum_{l=0}^{\infty}\gamma^l\delta_{t+l}(1-\lambda)\sum_{n=l+1}^{\infty}\lambda^{n-1} \\
&= \sum_{l=0}^{\infty}\gamma^l\delta_{t+l}\lambda^l.
\end{aligned}$$

Therefore,

$$\boxed{\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}}. \tag{32}$$

Equivalently,

$$\hat{A}_t = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots.$$

Two endpoints. With $\lambda = 0$, GAE becomes one-step TD: $\hat{A}_t = \delta_t$. As $\lambda \to 1$, GAE incorporates all later TD residuals; for a complete terminating episode, setting $\lambda = 1$ gives $G_t - V(s_t)$. Common initial choices are $\gamma = 0.99$ and $\lambda = 0.95$.

GAE does not mathematically guarantee lower variance in every problem. It introduces a tunable **bias–variance tradeoff**: smaller λ generally relies more on the critic, while larger λ generally relies more on sampled distant rewards.

23. The $O(T)$ backward recursion for GAE

Equation (32) implies

$$\boxed{\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}}. \tag{33}$$

Let $d_t = 1$ indicate a genuine MDP terminal transition. Then

$$\boxed{\begin{aligned}
&\delta_t = r_t + \gamma(1-d_t)V_\phi(s_{t+1}) - V_\phi(s_t), \\
&\hat{A}_t = \delta_t + \gamma\lambda(1-d_t)\hat{A}_{t+1}.
\end{aligned}} \tag{34}$$

A single backward pass from $T-1$ computes all advantages in $O(T)$ time. If a fixed-length rollout stops without the MDP actually terminating, bootstrap from $V(s_T)$. But a recursion must not accidentally propagate the next episode's advantage across a reset. Implementations must distinguish **terminated** (genuine terminal), **truncated** (external time limit), and **rollout boundary** (a sampling segment boundary).

24. Constructing critic targets from GAE

GAE estimates

$$\hat{A}_t \approx Q^{\pi_{\text{old}}}(s_t, a_t) - V_{\text{old}}(s_t).$$

Define the TD(λ) return target by

$$\boxed{\hat{R}_t^{\lambda} = \hat{A}_t^{\text{GAE}} + V_{\phi_{\text{old}}}(s_t)}. \tag{35}$$

$V_{\phi_{\text{old}}}$ denotes the critic values saved when targets for the rollout were calculated. Hold these values fixed during the PPO epochs. The construction aligns the actor's advantage estimates with the critic's regression target.

Part VII. The Complete Actor–Critic Loss

25. Actor loss

First calculate and freeze $\hat{A}_t$ for the old-policy rollout; advantage normalization is also common. The actor maximizes

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right].$$

For gradient-descent software such as PyTorch, minimize

$$\boxed{\mathcal{L}_{\text{actor}} = -L^{\text{CLIP}}}. \tag{36}$$

If the advantage estimates were produced by a differentiable critic currently inside the computational graph, detach them from the actor objective to avoid unintentionally backpropagating through the advantage targets.

26. Critic loss

Train the critic to regress the TD(λ) return:

$$\boxed{\mathcal{L}_{\text{critic}}(\phi) = \mathbb{E}_t[(V_\phi(s_t) - \hat{R}_t^{\lambda})^2]}. \tag{37}$$

Direct Monte Carlo return targets are another option. Some PPO implementations use value-function clipping or multiply the squared error by $1/2$; neither is essential to the basic PPO-Clip objective. An inaccurate critic can corrupt the sign or magnitude of advantages, so policy-update stability depends strongly on value estimation quality.

27. Motivation for the entropy bonus

A stochastic policy may become too concentrated early, ceasing to explore alternative grasps. Use the policy entropy

$$\boxed{H(\pi_\theta(\cdot \mid s)) = -\mathbb{E}_{a \sim \pi_\theta}[\log \pi_\theta(a \mid s)]}. \tag{38}$$

to encourage sufficient exploration. Let $L^{\text{entropy}} = \mathbb{E}_t[H(\pi_\theta(\cdot \mid s_t))]$. Higher entropy is not always preferable, particularly near robot deployment. The entropy coefficient may be zero or annealed during training.

28. Final PPO training objective

A common joint maximization objective combines all three terms:

$$\boxed{L^{\mathrm{PPO}}(\theta,\phi) = L^{\mathrm{CLIP}}(\theta) - c_v \mathcal{L}_{\mathrm{critic}}(\phi) + c_e L^{\mathrm{entropy}}(\theta)}. \tag{39}$$

Equivalently, minimize

$$\boxed{\mathcal{L}_{\mathrm{total}} = -L^{\mathrm{CLIP}} + c_v \mathcal{L}_{\mathrm{critic}} - c_e L^{\mathrm{entropy}}}. \tag{40}$$

- c_v determines the relative weight of value regression.
- c_e determines the strength of entropy regularization.
- With separate actor and critic parameters, the two networks can have separate updates; with shared parameters, their gradients interact.

A PPO optimization phase typically uses several mini-batch epochs over a fixed rollout, keeping the stored old log probabilities, advantages, and return targets constant during that phase.

Part VIII. Continuous-Action PPO and Robot Control

29. Gaussian policy log probabilities and importance ratios

Robot actions are often continuous vectors: joint position increments, desired velocities, or torques. A typical actor uses a diagonal Gaussian

$$\boxed{\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \mathrm{diag}(\sigma_1^2, \dots, \sigma_n^2))}. \tag{41}$$

The joint log probability of an action vector is

$$\boxed{\log \pi_\theta(a \mid s) = -\frac{1}{2} \sum_{i=1}^{n} \left[\frac{(a_i - \mu_i)^2}{\sigma_i^2} + 2 \log \sigma_i + \log(2\pi) \right]}. \tag{42}$$

The numerically convenient ratio is

$$\boxed{r_t = \exp\left(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\mathrm{old}}(a_t \mid s_t)\right)}. \tag{43}$$

Implementation detail. Sum per-dimension log probabilities to obtain the **joint** action log probability *before* exponentiation. Clipping ratios separately for each action dimension is not standard PPO for a joint policy. A tanh-squashed Gaussian requires the appropriate change-of-variables correction to its log density. If an environment directly hard-clips sampled actions, carefully distinguish the density of the sampled action from that of the executed command.

30. How does a Gaussian-policy gradient move the action mean?

Consider a one-dimensional Gaussian with fixed variance:

$$a \sim \mathcal{N}(\mu_\theta(s), \sigma^2).$$

Differentiating its log density with respect to the mean gives

$$\boxed{\frac{\partial \log \pi_\theta(a \mid s)}{\partial \mu_\theta} = \frac{a - \mu_\theta(s)}{\sigma^2}}. \tag{44}$$

The mean-related component of the unclipped policy gradient is therefore

$$\boxed{\nabla_\theta J \propto \mathbb{E}\left[\hat{A}\frac{a - \mu_\theta(s)}{\sigma^2}\nabla_\theta \mu_\theta(s)\right]}. \tag{45}$$

For $\hat{A} > 0$, this tends to move the mean toward the sampled action; for $\hat{A} < 0$, it tends to move the mean away. PPO clipping limits further *surrogate benefit* from excessive probability changes. If σ is learned, there are additional gradient terms associated with variance.

31. Example: residual PPO for robot manipulation

Suppose a baseline controller produces

$$a_t^{\text{base}} = \pi_{\text{base}}(s_t),$$

while PPO learns a stochastic residual

$$\Delta a_t \sim \pi_\theta(\cdot \mid s_t).$$

The executed command is

$$\boxed{a_t = a_t^{\text{base}} + \Delta a_t}. \tag{46}$$

The residual may adjust joint targets, velocities, end-effector pose, or finger closing actions. In a screwdriver grasp-and-rotation task, a base trajectory provides the broad motion while PPO residual actions compensate for contact-location errors, initial-pose perturbations, changes in friction, and slippage.

The ratio must correspond to the **residual action distribution**:

$$r_t = \frac{\pi_\theta(\Delta a_t \mid s_t)}{\pi_{\text{old}}(\Delta a_t \mid s_t)}.$$

When residuals are scaled, transformed nonlinearly, or filtered through a safety controller, clearly distinguish which variables have the log density used for learning and which commands are actually executed. PPO does not require differentiable contact dynamics, but does require adequate rollouts, rewards, and state/value estimation.

Part IX. A Complete PPO Iteration

32. Ten steps from rollout collection to policy updates

1. **Freeze the old policy.** Set $\pi_{\text{old}} = \pi_{\theta_k}$ and sample several episodes or fixed-length rollouts.
2. **Store rollout data.** Record (s_t, a_t, r_t, s_{t+1}), termination flags, $\log \pi_{\text{old}}(a_t \mid s_t)$, and $V_{\text{old}}(s_t)$.
3. **Compute TD errors.** Use $\delta_t = r_t + \gamma V_{\text{old}}(s_{t+1}) - V_{\text{old}}(s_t)$ with proper true-terminal masking.
4. **Calculate GAE backward.** Use $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$ and prevent recursion across unrelated episodes.

5. **Compute critic targets.** Set $\hat{R}_t^{\lambda} = \hat{A}_t + V_{\text{old}}(s_t)$ and hold the targets fixed.
6. **Optionally normalize advantages.** Use $\tilde{A}_t = (\hat{A}_t - \mu_A)/(\sigma_A + \varepsilon_{\text{num}})$.
7. **Evaluate current log probabilities on stored actions.** For the **original actions** a_t sampled by the old policy, compute current $\log \pi_\theta(a_t \mid s_t)$.
8. **Compute ratios and clipped actor loss.** Use $r_t = \exp(\log \pi_\theta - \log \pi_{\text{old}})$ and Eq. (21).
9. **Optimize actor and critic.** Minimize the joint objective or separate losses for multiple epochs, monitoring KL, clip fraction, and value prediction.
10. **Collect new on-policy data.** Use the updated policy for the next rollout rather than treating arbitrarily old replay data as ordinary PPO training data.

33. Compact, adaptable PyTorch implementations

The following function implements the central PPO actor loss. Both `new_log_prob` and `old_log_prob` must contain **joint** action log probabilities, and the old log probabilities and advantages must remain fixed within the update phase.

```
import torch


def ppo_actor_loss(new_log_prob, old_log_prob,
                   advantages, clip_eps=0.2):
    # The old log probability and advantage are fixed rollout data.
    old_log_prob = old_log_prob.detach()
    advantages = advantages.detach()

    log_ratio = new_log_prob - old_log_prob
    ratio = torch.exp(log_ratio)

    objective_unclipped = ratio * advantages
    objective_clipped = torch.clamp(
        ratio, 1.0 - clip_eps, 1.0 + clip_eps
    ) * advantages

    return -torch.minimum(
        objective_unclipped, objective_clipped
    ).mean()
```

A minimal backward GAE calculation is shown next. The `bootstrap_mask[t]` determines whether a subsequent-state value is included in the TD target. The `continue_mask[t]` determines whether the advantage recursion is allowed to cross the current transition. Correct handling of true terminations, timeouts, and rollout boundaries must come from the environment and rollout buffer.

```
import torch


def compute_gae(rewards, values, next_values,
                bootstrap_mask, continue_mask,
                gamma=0.99, lam=0.95):
    """All tensors are indexed by rollout time."""
    T = rewards.shape[0]
    advantages = torch.zeros_like(rewards)
    running = torch.zeros_like(rewards[0])

    for t in reversed(range(T)):
        delta = (rewards[t]
                 + gamma * bootstrap_mask[t] * next_values[t]
```

```
                - values[t])
        running = (delta
                   + gamma * lam * continue_mask[t] * running)
        advantages[t] = running

    returns = advantages + values
    return advantages, returns
```

For parallel simulation, tensors can have shape `[T, N_env]`; never propagate `continue_mask` across an episode reset. The `values` and `next_values` used in return targets should come from the fixed rollout-time critic.

For a diagonal Gaussian policy:

```
# dist: torch.distributions.Normal, action shape [batch, action_dim]
new_log_prob = dist.log_prob(actions).sum(dim=-1)
# entropy shape [batch]
entropy = dist.entropy().sum(dim=-1)
```

These examples implement the essential formulas; a complete training program still needs a rollout buffer, environment interaction, network definitions, optimizers, correct `terminated/truncated` handling, and metric logging.

Part X. Hyperparameters and Diagnostics

34. Common starting hyperparameters and what they mean

Hyperparameter	Common starting point	Main role
Discount γ	0.99	Weight assigned to distant rewards
GAE λ	0.95	Advantage-estimator bias–variance tradeoff
Clip ϵ	0.1–0.2	Extra surrogate gains from overly large probability ratios
Actor learning rate	3×10^{-4}	Actor update speed
PPO epochs	3–10	Number of optimization passes over one rollout
Target KL	0.01–0.03	Threshold for early stopping of actor updates
Entropy coefficient	0 or small positive	Amount of exploration encouragement
Value-loss weight	About 0.5	Relative weight of critic regression

These are experimental starting points, not universal optima. Large-scale parallel locomotion frameworks such as RSL-RL may use quite different learning rates, batch sizes, clipping ranges, and numbers of epochs.

The distinction between γ and λ: The discount factor γ defines how the learning objective values distant rewards; λ determines how much an advantage estimator incorporates distant sampled TD information. Both appear through $(\gamma\lambda)^l$ in GAE, but their conceptual roles are different.

Useful diagnostics include episode return, task success rate, value loss, explained variance, policy entropy, approximate KL, clip fraction, and gradient norm. If a surrogate improves while actual robot success decreases, investigate distribution shift, critic error, excessive KL, and reward design.

Part XI. Connections to REINFORCE, Actor–Critic, and TRPO

35. Algorithmic progression: one design choice per problem

Algorithm	Core expression / idea	Problem addressed
REINFORCE	$\nabla \log \pi_\theta \, G$	Learn a policy without environment derivatives
Advantage actor–critic	$\nabla \log \pi_\theta \, \hat{A}$	Reduce variance using a state-value baseline
TRPO	Maximize $\mathbb{E}[r\hat{A}]$ subject to KL	Keep the surrogate useful in a local trust region
PPO-Clip	$\min(r\hat{A}, \text{clip}(r)\hat{A})$	Discourage excessive probability changes with first-order optimization
PPO with GAE	$\hat{A}_t = \sum_l (\gamma\lambda)^l \delta_{t+l}$	Control the bias–variance tradeoff of advantage estimates

PPO fundamentally remains an **on-policy policy-gradient / actor–critic algorithm**. At the beginning of each rollout update, the clipped and unclipped surrogate objectives have the same first-order gradient. Their difference arises as several optimization steps on the same dataset cause probability ratios to move away from one.

PPO is especially practical for simulated robotic control because large numbers of environments can be sampled in parallel, optimization is relatively simple, differentiable dynamics are unnecessary, and Gaussian policies naturally model continuous actions. On a physical robot, however, collecting interactions is expensive, so off-policy methods such as SAC may offer important sample-efficiency advantages. Diffusion and flow-matching action policies often lack the conveniently computable explicit log probabilities required by standard PPO, motivating specialized modifications.

Part XII. Final Formula Sheet and Research Takeaways

Formula 1: Policy gradient

$$\nabla_\theta J \propto \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s) A^\pi(s,a)\right].$$

Motivation: Optimize without backpropagating through possibly nondifferentiable physical dynamics.

Formula 2: Importance sampling

$$r_t = \pi_\theta(a_t \mid s_t) / \pi_{\text{old}}(a_t \mid s_t).$$

Motivation: Construct a local new-policy surrogate from actions sampled by the old policy.

Formula 3: TRPO's trust-region idea

$$\bar{D}_{\mathrm{KL}}(\pi_{\mathrm{old}} \| \pi_\theta) \leq \delta.$$

Motivation: Control the risk that the surrogate becomes misleading under distribution shift.

Formula 4: PPO clipped surrogate

$$\boxed{L^{\mathrm{CLIP}} = \mathbb{E}_t \left[\min \left(r_t \hat{A}_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]}.$$

Motivation: Conservatively cap surrogate incentives for excessive favorable changes while retaining corrective gradients for unfavorable changes.

Formula 5: Generalized advantage estimation

$$\boxed{\hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}}, \qquad \delta_t = r_t^{\mathrm{reward}} + \gamma V(s_{t+1}) - V(s_t).$$

Motivation: Smoothly trade off reliance on long-term sampled rewards against reliance on short-horizon value bootstrapping. Here r_t^{reward} distinguishes the immediate reward from PPO's probability ratio $r_t(\theta)$.

Final loss to minimize:

$$\boxed{\mathcal{L}_{\mathrm{PPO}} = -L^{\mathrm{CLIP}} + c_v \mathcal{L}_{\mathrm{critic}} - c_e L^{\mathrm{entropy}}}.$$

> **Main research insight.** PPO is not a new method for differentiating environments. It uses policy gradients to identify local directions of improvement, importance sampling to construct a surrogate from stored actions, clipping to discourage over-optimization, and GAE to improve advantage estimation. **GAE principally addresses estimator quality; clipping principally addresses policy-update stability.** The two components solve different yet complementary problems.

References and further reading

1. Schulman, J., et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347. https://arxiv.org/abs/1707.06347
2. Schulman, J., et al. (2015). *Trust Region Policy Optimization*. ICML. https://arxiv.org/abs/1502.05477
3. Schulman, J., et al. (2015). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. ICLR 2016. https://arxiv.org/abs/1506.02438
4. OpenAI Spinning Up. *PPO Documentation*. https://spinningup.openai.com/en/latest/algorithms/ppo.html

---

Editorial note: This document expands the preceding PPO derivation with explicit support assumptions for importance sampling, precise discounted occupancy notation, the fact that PPO-Clip does not impose a hard constraint, and the difference between true episode termination and rollout truncation.

Soft Actor-Critic (SAC): Complete Derivation and Design Motivations

Course home · 简体中文 · PDF

Item	Details
Applicability	Modern continuous-action SAC; twin critics and automatic temperature tuning; pseudocode only
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

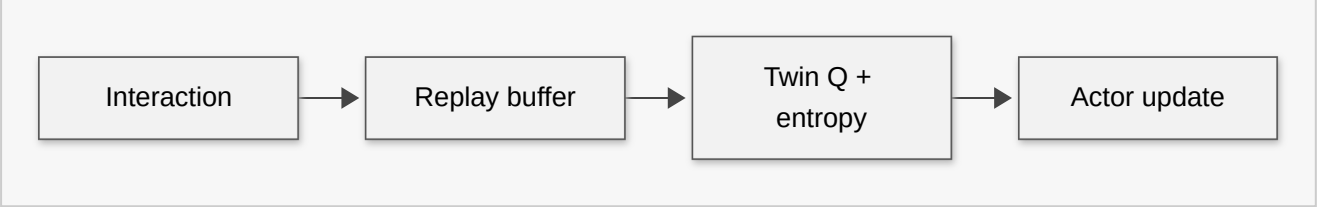
Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Starting from Standard Reinforcement Learning
- 2. Step One: Maximum-Entropy Reinforcement Learning
  - 2.1 Policy entropy and the temperature parameter
  - 2.2 Rewriting entropy as a sample-based expectation
- 3. Step Two: Deriving the Soft Bellman Equation
  - 3.1 Comparison with ordinary Bellman equations
  - 3.2 Defining soft V and soft Q consistently
  - 3.3 Soft Bellman expectation equation
- 4. Step Three: Soft Policy Improvement
  - 4.1 Why ordinary greedy improvement is insufficient
  - 4.2 Calculus of variations yields the Boltzmann policy
- 5. Step Four: Turning Policy Improvement into KL Minimization
  - 5.1 Why directly computing the Boltzmann policy is difficult
  - 5.2 Why exact soft policy iteration improves value
- 6. Step Five: From Soft Policy Iteration to Actor-Critic
- 7. Step Six: Why SAC Can Learn Off-Policy
  - 7.1 PPO and policy-dependent data
  - 7.2 Why an old transition can support a new Bellman update
- 8. Step Seven: Deriving the Complete Critic Loss
  - 8.1 From the soft Bellman target to TD regression
  - 8.2 Target networks reduce target drift

    - 8.3 Clipped double Q mitigates overestimation
    - 8.4 Final critic target and losses
- 9. Step Eight: Deriving the Actor Loss
- 10. Step Nine: The Reparameterization Trick
    - 10.1 Turning parameter-dependent sampling into a differentiable function
    - 10.2 A tanh-squashed Gaussian policy
    - 10.3 Rewriting the actor loss as a pathwise objective
- 11. Step Ten: Correcting the Tanh-Gaussian Log Probability
- 12. Step Eleven: Automatic Entropy Tuning
    - 12.1 From fixed temperature to an entropy constraint
    - 12.2 The temperature loss and its direction of change
    - 12.3 Why optimize log-alpha?
    - 12.4 Choosing target entropy
- 13. Step Twelve: Putting Together the Complete Modern SAC Algorithm
    - 13.1 Components and three optimization problems
    - 13.2 Complete sequence for one update
    - 13.3 Instructional pseudocode
- 14. Original SAC v1 Versus Modern SAC
    - 14.1 Original SAC v1: explicitly learning a V-network
    - 14.2 Modern SAC: remove the explicit V-network
- 15. Fundamental Differences Between SAC and PPO
    - 15.1 Comparing policy-update equations
    - 15.2 Comparison table
    - 15.3 Why PPO remains popular in robot simulation
- 16. Final Summary: Five Essential Formula Groups
    - The single most important PPO/SAC distinction
- References

Chapter concept map



1. Starting from Standard Reinforcement Learning

Soft Actor-Critic (SAC) is an **off-policy, stochastic-policy, actor-critic** reinforcement-learning algorithm developed primarily for continuous control. It maximizes long-term reward while encouraging an appropriate amount of policy randomness. Soft Bellman backups provide action-value estimates, and the actor is optimized using gradients through a differentiable Q-function. The

full method combines maximum-entropy objectives, soft policy iteration, a replay buffer, two Q-networks, target networks, the reparameterization trick, and automatic entropy-temperature adjustment.

Consider a discounted Markov decision process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \qquad 0 \le \gamma < 1.$$

Here $s_t \in \mathcal{S}$ is a state, $a_t \in \mathcal{A}$ is an action, $P(s_{t+1} \mid s_t, a_t)$ is the environment transition kernel, $r(s_t, a_t)$ is the immediate reward, and $\pi_\theta(a \mid s)$ is a policy parameterized by θ. Conventional RL maximizes the discounted return:

$$\boxed{J_{\mathrm{RL}}(\pi) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right].}$$

Motivation: why might maximizing reward alone be insufficient? Consider a robot choosing among three grasping actions, with the following current critic estimates:

Candidate action	Estimated Q-value
Grasp from the left	10
Grasp from the right	9.8
Grasp from above	9.5

A policy that immediately commits to the left grasp may stop exploring alternatives. Worse, the critic could be incorrect. For example, it may predict $\hat{Q}(s, a_{\text{left}}) = 10$ and $\hat{Q}(s, a_{\text{right}}) = 8$, even though their true values are $Q(s, a_{\text{left}}) = 8$ and $Q(s, a_{\text{right}}) = 10$. Rarely trying the right grasp makes it harder to correct this error.

We therefore want the policy to prefer high-value actions without eliminating randomness too early. This is the central motivation for **maximum-entropy reinforcement learning**.

2. Step One: Maximum-Entropy Reinforcement Learning

2.1 Policy entropy and the temperature parameter

For discrete actions, Shannon entropy is $\mathcal{H}(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$. For continuous actions, SAC uses differential entropy:

$$\boxed{\mathcal{H}(\pi(\cdot \mid s)) = -\int \pi(a \mid s) \log \pi(a \mid s)\, da = \mathbb{E}_{a \sim \pi}[-\log \pi(a \mid s)].}$$

For a one-dimensional Gaussian policy $\pi(a \mid s) = \mathcal{N}(\mu(s), \sigma^2(s))$:

$$\mathcal{H}(\pi) = \frac{1}{2}\log(2\pi e \sigma^2) = \log \sigma + \frac{1}{2}\log(2\pi e).$$

A larger standard deviation yields a larger entropy and a more dispersed action distribution. **Importantly**, differential entropy can be negative and changes under rescaling of the action coordinates; it is not identical to discrete entropy.

Introduce the temperature $\alpha \geq 0$ and modify the objective:

$$\boxed{J_{\mathrm{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\big)\right].}$$

The coefficient α controls the trade-off between reward and randomness. The ordinary RL objective is approached as $\alpha \to 0$. Larger values encourage more stochastic policies. The coefficient is also important because reward magnitude and entropy magnitude can differ substantially. For example, if $r \approx 100$ while $\mathcal{H}(\pi) \approx 5$, adding the two unweighted may give entropy little influence; α adjusts the relative scale.

2.2 Rewriting entropy as a sample-based expectation

Because $\mathcal{H}(\pi(\cdot \mid s)) = \mathbb{E}_{a\sim\pi}[-\log \pi(a \mid s)]$, we can write:

$$\boxed{J_{\mathrm{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) - \alpha \log \pi(a_t \mid s_t)\big)\right].} \tag{1}$$

One may interpret $\tilde{r}_t = r_t - \alpha \log \pi(a_t \mid s_t)$ as an entropy-regularized reward expression. However, this term depends on the current policy and is **not** an external environment reward fixed independently of the policy. The useful change is that continuous-control algorithms now only need to sample $a \sim \pi$ and evaluate $\log \pi(a \mid s)$, rather than explicitly integrating entropy over the entire action space.

3. Step Two: Deriving the Soft Bellman Equation

3.1 Comparison with ordinary Bellman equations

Standard value functions satisfy:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s, a)], \quad Q^\pi(s, a) = \mathbb{E}_{s'\sim P}[r(s, a) + \gamma V^\pi(s')].$$

Combining them gives:

$$Q^\pi(s, a) = \mathbb{E}_{s', a'}[r(s, a) + \gamma Q^\pi(s', a')].$$

For maximum-entropy RL, future policy entropy must also appear in the recursive value calculation.

3.2 Defining soft V and soft Q consistently

A convenient SAC convention is:

$$\boxed{V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t)\big) \,\middle|\, s_0 = s\right],}$$

$$\boxed{Q^\pi(s, a) = \mathbb{E}_\pi\left[r_0 + \sum_{t=1}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t)\big) \,\middle|\, s_0 = s, a_0 = a\right].}$$

The soft state value includes the entropy term for the current action. The soft action value conditions on a fixed initial action, so its entropy terms start at the next step. This convention makes the

recursion concise. By conditioning on the first action:

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\big].} \tag{2}$$

Equivalently:

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)] + \alpha \mathcal{H}(\pi(\cdot \mid s)).} \tag{3}$$

An ordinary state value measures expected future reward under a policy. A soft state value measures expected reward **together with** the randomness retained by that policy.

3.3 Soft Bellman expectation equation

The definitions and the Markov property imply:

$$Q^\pi(s,a) = \mathbb{E}_{s'\sim P}[r(s,a) + \gamma V^\pi(s')].$$

Substituting Equation (2):

$$\boxed{Q^\pi(s,a) = \mathbb{E}_{\substack{s'\sim P(\cdot|s,a)\\ a'\sim\pi(\cdot|s')}}\left[r(s,a) + \gamma\left(Q^\pi(s',a') - \alpha \log \pi(a' \mid s')\right)\right].} \tag{4}$$

Compared with ordinary Bellman evaluation, we have one additional term: $-\alpha \log \pi(a' \mid s')$. It is precisely how the entropy reward enters the critic's target through the next state's soft value. This equation provides **policy evaluation**, but we must still derive how to improve the policy.

4. Step Three: Soft Policy Improvement

4.1 Why ordinary greedy improvement is insufficient

Ordinary policy iteration can choose $\pi_{\text{new}}(s) \in \arg\max_a Q^{\pi_{\text{old}}}(s,a)$. Even when optimizing a stochastic distribution, maximizing $\mathbb{E}_a Q(s,a)$ alone may collapse the solution into a deterministic distribution concentrated on Q-maximizing actions. SAC wants to improve value while retaining entropy.

Fix a state s and define $Q_{\text{old}}(s,a) = Q^{\pi_{\text{old}}}(s,a)$. The soft policy-improvement problem is:

$$\boxed{\pi_{\text{new}} = \arg\max_\pi \left\{\mathbb{E}_{a\sim\pi}[Q_{\text{old}}(s,a)] + \alpha \mathcal{H}(\pi(\cdot \mid s))\right\}.} \tag{5}$$

Equivalently, maximize the functional:

$$F(\pi) = \int \pi(a \mid s) Q_{\text{old}}(s,a)\, da - \alpha \int \pi(a \mid s) \log \pi(a \mid s)\, da$$

subject to the density-normalization constraint $\int \pi(a \mid s)\, da = 1$.

4.2 Calculus of variations yields the Boltzmann policy

Assume $\alpha > 0$ and a finite normalizing constant. Introduce a Lagrange multiplier λ:

$$\begin{aligned}\mathcal{L}(\pi, \lambda) = &\int \pi(a \mid s) Q_{\text{old}}(s, a)\, da \\ &- \alpha \int \pi(a \mid s) \log \pi(a \mid s)\, da \\ &+ \lambda \left(\int \pi(a \mid s)\, da - 1 \right).\end{aligned}$$

The functional derivative with respect to the density is:

$$\frac{\delta \mathcal{L}}{\delta \pi(a \mid s)} = Q_{\text{old}}(s, a) - \alpha(\log \pi(a \mid s) + 1) + \lambda.$$

Setting it to zero:

$$\log \pi(a \mid s) = \frac{Q_{\text{old}}(s, a)}{\alpha} + \frac{\lambda}{\alpha} - 1.$$

Exponentiating and normalizing gives:

$$\boxed{\pi^*_{\text{new}}(a \mid s) = \frac{\exp(Q_{\text{old}}(s, a)/\alpha)}{Z_{\text{old}}(s)}, \quad Z_{\text{old}}(s) = \int_{\mathcal{A}} \exp(Q_{\text{old}}(s, a')/\alpha)\, da'.} \tag{6}$$

This is a **Boltzmann policy**. Higher Q-value means higher action probability. Increasing α produces a flatter distribution; as $\alpha \to 0^+$, probability concentrates on Q-maximizing actions. For example, if $Q(a_1) = 2$ and $Q(a_2) = 1$, then for $\alpha = 0.5$ the probabilities are approximately 0.881 and 0.119. With $\alpha = 2$, they become approximately 0.622 and 0.378.

Motivation recap: soft policy improvement no longer merely chooses $\arg\max_a Q(s, a)$. It learns a distribution that prioritizes high-value actions while preserving randomness.

5. Step Four: Turning Policy Improvement into KL Minimization

5.1 Why directly computing the Boltzmann policy is difficult

For a high-dimensional continuous action space, $Z(s) = \int e^{Q(s,a)/\alpha}\, da$ is generally intractable. We therefore cannot easily construct or sample the normalized ideal distribution. SAC solves this by projecting a parameterized actor π_θ onto the Boltzmann distribution in a way that eliminates the need to calculate $Z(s)$.

Define:

$$\pi_B(a \mid s) = \frac{\exp(Q_{\text{old}}(s, a)/\alpha)}{Z(s)}.$$

Consider the KL divergence with the actor as the first argument, $D_{\text{KL}}(\pi_\theta \| \pi_B)$:

$$\begin{aligned}D_{\text{KL}}(\pi_\theta \| \pi_B) &= \mathbb{E}_{a \sim \pi_\theta} \left[\log \pi_\theta(a \mid s) - \log \pi_B(a \mid s) \right] \\ &= \mathbb{E}_{a \sim \pi_\theta} \left[\log \pi_\theta(a \mid s) - \frac{Q_{\text{old}}(s, a)}{\alpha} \right] + \log Z(s).\end{aligned}$$

During the projection Q_{old} is fixed and $\log Z(s)$ does not depend on the actor parameters. Therefore:

$$\boxed{\arg\min_\theta D_{\text{KL}}(\pi_\theta \| \pi_B) = \arg\min_\theta \mathbb{E}_{a \sim \pi_\theta} \left[\alpha \log \pi_\theta(a \mid s) - Q_{\text{old}}(s, a) \right].} \tag{7}$$

This is the origin of the **SAC actor loss**: we neither compute a partition function nor enumerate all high-dimensional actions.

A useful identity makes the equivalence explicit:

$$\boxed{\mathbb{E}_{a\sim\pi_\theta}[Q(s,a)] + \alpha\mathcal{H}(\pi_\theta(\cdot \mid s)) = \alpha \log Z(s) - \alpha D_{\mathrm{KL}}(\pi_\theta \| \pi_B).} \tag{8}$$

Thus, maximizing soft value and approaching the Boltzmann policy are two views of the same optimization problem.

5.2 Why exact soft policy iteration improves value

Assume the old policy belongs to the allowed policy class and that the KL projection is solved exactly at every state. By optimality:

$$D_{\mathrm{KL}}(\pi_{\mathrm{new}} \| \pi_B) \le D_{\mathrm{KL}}(\pi_{\mathrm{old}} \| \pi_B).$$

Equation (8) then implies:

$$\boxed{\mathbb{E}_{a\sim\pi_{\mathrm{new}}}[Q^{\pi_{\mathrm{old}}}(s,a) - \alpha \log \pi_{\mathrm{new}}(a \mid s)] \ge V^{\pi_{\mathrm{old}}}(s).} \tag{9}$$

Define the soft Bellman operator for a fixed policy π:

$$(\mathcal{T}^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a'\sim\pi}\left[Q(s',a') - \alpha \log \pi(a' \mid s')\right].$$

Combining Equation (9) with Bellman evaluation gives:

$$Q^{\pi_{\mathrm{old}}} \le \mathcal{T}^{\pi_{\mathrm{new}}} Q^{\pi_{\mathrm{old}}}.$$

The operator is monotone. Under suitable boundedness assumptions, it is also a contraction:

$$\|\mathcal{T}^\pi Q_1 - \mathcal{T}^\pi Q_2\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty.$$

Repeated application of $\mathcal{T}^{\pi_{\mathrm{new}}}$ gives:

$$Q^{\pi_{\mathrm{old}}} \le \mathcal{T}^{\pi_{\mathrm{new}}} Q^{\pi_{\mathrm{old}}} \le \cdots \longrightarrow Q^{\pi_{\mathrm{new}}}.$$

Consequently:

$$\boxed{Q^{\pi_{\mathrm{new}}}(s,a) \ge Q^{\pi_{\mathrm{old}}}(s,a).} \tag{10}$$

This is the essential soft policy-improvement theorem. **Scope of the result:** the statement assumes exact evaluation and exact improvement under the stated conditions. Finite data, neural function approximation, stochastic gradients, and approximate projection in practical SAC do not inherit the same global convergence guarantee.

6. Step Five: From Soft Policy Iteration to Actor-Critic

We now have two alternating operations:

- **Policy evaluation:** evaluate $Q^\pi(s,a) = \mathbb{E}[r + \gamma(Q^\pi(s',a') - \alpha \log \pi(a' \mid s'))]$ for a fixed policy.
- **Policy improvement:** improve the policy by minimizing $\mathbb{E}_{a\sim\pi}[\alpha \log \pi(a \mid s) - Q(s,a)]$.

A tabular representation is infeasible for general continuous states and actions, so SAC uses neural-network approximations:

$$\boxed{\text{Actor: } \pi_\theta(a \mid s), \qquad \text{Critic: } Q_\phi(s,a).}$$

The actor takes a state and outputs an action distribution; the critic takes a state-action pair and outputs a scalar value. The soft Bellman equation suggests a temporal-difference error:

$$\delta = Q_\phi(s,a) - \left[r + \gamma \left(Q_\phi(s',a') - \alpha \log \pi_\theta(a' \mid s')\right)\right].$$

One can train the critic with $\frac{1}{2}\mathbb{E}[\delta^2]$ and train the actor with $\mathbb{E}[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)]$. The environment's reward affects the actor indirectly through $r \to Q_\phi \to \pi_\theta$; the environment reward is not directly backpropagated into the policy.

A practical issue remains: if a constantly changing Q_ϕ is used both as the predictor and as its own regression target, learning can be unstable. Section 8 introduces target networks and double-Q learning.

7. Step Six: Why SAC Can Learn Off-Policy

7.1 PPO and policy-dependent data

PPO's clipped objective depends on data collected by the old policy, and on the importance ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

As a result, standard PPO predominantly uses recent on-policy rollouts. Very old data can cause substantial distribution mismatch.

7.2 Why an old transition can support a new Bellman update

In a fixed MDP, the environment transition law $P(s' \mid s,a)$ does **not** depend on which behavior policy generated the state-action pair. A previously observed transition (s,a,r,s') therefore remains a valid sample of the environment dynamics. To construct the **current policy's** soft Bellman target, we simply resample a *new next action* $a' \sim \pi_\theta(\cdot \mid s')$:

$$y = r + \gamma\big(Q(s',a') - \alpha \log \pi_\theta(a' \mid s')\big).$$

Variable	Source
s, a, r, s'	Historical transition from the replay buffer
a'	Newly sampled from the current actor $\pi_\theta(\cdot \mid s')$
$\log \pi_\theta(a' \mid s')$	Evaluated using the current actor

Maintain a replay buffer:

$$\boxed{\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}, d_t)\}.}$$

Gradient steps sample mini-batches from $\mathcal{D}$ rather than always requiring fresh rollouts. The same transitions can be reused repeatedly; multiple optimization steps may follow a single environment step.

Important limitation: off-policy does not mean that arbitrary distribution mismatch is harmless. If the buffer provides little coverage of important state-action regions, the neural critic can extrapolate very poorly. Finite data, function approximation, and bootstrapping may jointly produce instability. SAC improves sample reuse but does not remove these challenges.

8. Step Seven: Deriving the Complete Critic Loss

8.1 From the soft Bellman target to TD regression

Sample a next action from the current policy, $a' \sim \pi_\theta(\cdot \mid s')$. A direct single-sample target would be:

$$y = r + \gamma \left[Q_\phi(s', a') - \alpha \log \pi_\theta(a' \mid s') \right].$$

The associated regression loss is:

$$J_Q(\phi) = \frac{1}{2} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\phi(s, a) - y \right)^2 \right].$$

This is not yet practical SAC: the target depends on the same network being updated, and the actor may exploit positive errors in the critic's estimates.

8.2 Target networks reduce target drift

Maintain a slowly changing target parameter vector $\bar{\phi}$ and a target critic $Q_{\bar{\phi}}$. Update them using Polyak averaging:

$$\boxed{\bar{\phi} \leftarrow (1 - \tau)\bar{\phi} + \tau \phi, \qquad 0 < \tau \ll 1.} \tag{11}$$

A typical value is $\tau = 0.005$. The target network slows changes in the bootstrap target; it does not, by itself, eliminate value-estimation bias.

8.3 Clipped double Q mitigates overestimation

Suppose a learned value has an error $\hat{Q}(s, a) = Q(s, a) + \epsilon(s, a)$. An actor that prefers actions with high *estimated* Q may select actions mostly because of positive estimation error. For example, if two actions both truly have value 10 but are estimated at 12 and 8, the actor may favor the first one and amplify its accidental overestimate.

SAC therefore trains two independently parameterized critics:

$$Q_{\phi_1}(s, a), \qquad Q_{\phi_2}(s, a),$$

with corresponding target networks:

$$Q_{\bar{\phi}_1}(s, a), \qquad Q_{\bar{\phi}_2}(s, a).$$

A conservative minimum is used in the target:

$$\boxed{Q_{\text{target}}(s', a') = \min_{j=1,2} Q_{\bar{\phi}_j}(s', a').}$$

Taking the minimum reduces certain overestimation effects but can introduce underestimation; it does **not** make the estimator unbiased by construction.

8.4 Final critic target and losses

Let $d = 1$ denote a true terminal transition, not merely a time-limit truncation. The full target is:

$$\boxed{y = r + \gamma(1 - d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right], \quad a' \sim \pi_\theta(\cdot \mid s').} \tag{12}$$

Each critic fits the same target:

$$\boxed{J_{Q_i}(\phi_i) = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[\left(Q_{\phi_i}(s, a) - y\right)^2\right], \qquad i = 1, 2.} \tag{13}$$

$$\boxed{J_Q(\phi_1, \phi_2) = J_{Q_1}(\phi_1) + J_{Q_2}(\phi_2).} \tag{14}$$

Update $\phi_i \leftarrow \phi_i - \eta_Q \nabla_{\phi_i} J_{Q_i}$. The target branch must have its gradients stopped: $y \leftarrow \text{stopgrad}(y)$. For a **true terminal** transition, $d = 1$ and $y = r$. If an episode is truncated only because of an external time limit and the state is not terminal, bootstrapping should generally be retained.

9. Step Eight: Deriving the Actor Loss

The KL projection in Section 5 gave:

$$\pi_{\text{new}} = \arg\min_{\pi} \mathbb{E}_{a \sim \pi}\left[\alpha \log \pi(a \mid s) - Q(s, a)\right].$$

Replace the policy and value by a parameterized actor and critic, and draw states from the replay buffer:

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s \sim \mathcal{D} \\ a \sim \pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s, a)\right].} \tag{15}$$

A widely used twin-critic version is:

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s \sim \mathcal{D} \\ a \sim \pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_{\phi_i}(s, a)\right].} \tag{16}$$

Some implementations instead use only the first critic for the actor update. Equation (16) contains two complementary objectives. Minimizing the expectation of $\alpha \log \pi_\theta$ encourages higher entropy, while minimizing $-Q_\phi$ favors high-value actions.

The key difference from PPO: PPO mainly uses sampled-action advantages and probability ratios. Continuous SAC aims to pass gradients through the action itself, using $\nabla_a Q_\phi(s, a)$ to communicate a local direction for improving action value. Because the action is randomly sampled, this requires a differentiable representation of the sampling procedure: the reparameterization trick.

10. Step Nine: The Reparameterization Trick

10.1 Turning parameter-dependent sampling into a differentiable function

Suppose the actor predicts a mean and standard deviation:

$$\mu_\theta(s), \qquad \sigma_\theta(s).$$

The unsquashed action is Gaussian:

$$u \sim \mathcal{N}\big(\mu_\theta(s), \mathrm{diag}(\sigma_\theta^2(s))\big).$$

Directly viewing sampling as $u \sim \pi_\theta$ makes the sampling distribution depend on θ. Instead, use the equivalent representation:

$$\boxed{u_\theta(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).} \tag{17}$$

Now the noise ϵ is independent of the parameters, and u_θ is differentiable with respect to θ. For example, $\partial u/\partial \mu = I$ and $\partial u/\partial \sigma = \mathrm{diag}(\epsilon)$.

10.2 A tanh-squashed Gaussian policy

Robot actions are generally bounded (for example, to $[-1,1]^d$), whereas a Gaussian has unbounded support. SAC applies an elementwise hyperbolic tangent:

$$\boxed{a_\theta(s,\epsilon) = \tanh\big(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon\big).} \tag{18}$$

For a physical action interval $[a_{\min,i}, a_{\max,i}]$, one may further use $a_i^{\mathrm{env}} = c_i + b_i \tanh(u_i)$, where $c_i = (a_{\max,i} + a_{\min,i})/2$ and $b_i = (a_{\max,i} - a_{\min,i})/2$.

10.3 Rewriting the actor loss as a pathwise objective

Write $a = f_\theta(s,\epsilon)$ and replace policy-dependent action randomness with a fixed noise distribution:

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ \epsilon\sim\mathcal{N}(0,I)}} \left[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - Q_\phi(s, f_\theta(s,\epsilon))\right].} \tag{19}$$

Let $a_\theta = f_\theta(s,\epsilon)$. Differentiation gives:

$$\boxed{\nabla_\theta J_\pi = \mathbb{E}\left[\alpha \frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s, a_\theta) \frac{\partial a_\theta}{\partial \theta}\right].} \tag{20}$$

The derivative of $\log \pi_\theta(a_\theta \mid s)$ is a **total derivative**: the log-density depends on θ explicitly and through a_θ:

$$\frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) = \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} + \nabla_a \log \pi_\theta(a_\theta \mid s) \frac{\partial a_\theta}{\partial \theta}.$$

Thus the full gradient is:

$$\boxed{\begin{aligned}\nabla_\theta J_\pi = \mathbb{E}\Bigg[& \alpha \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} \\ & + \big(\alpha \nabla_a \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s, a_\theta)\big) \frac{\partial a_\theta}{\partial \theta}\Bigg].\end{aligned}} \tag{21}$$

The control-relevant term has a clear interpretation:

$$\underbrace{\nabla_a Q_\phi(s, a_\theta)}_{\text{direction of better actions}} \qquad \underbrace{\frac{\partial a_\theta}{\partial \theta}}_{\text{how to change actor parameters}}.$$

This gives SAC a direct connection to deterministic policy gradients such as DDPG: SAC exploits action derivatives of the critic but does so for a **reparameterizable stochastic policy**, together with entropy regularization. A pathwise gradient often has lower variance than a direct score-function estimator, although its usefulness still depends on critic quality.

11. Step Ten: Correcting the Tanh-Gaussian Log Probability

The Gaussian variable u and the transformed action $a = \tanh(u)$ do not have the same distribution. Therefore, $\log \mathcal{N}(a; \mu, \Sigma)$ is not the correct log-density for the squashed action. For an invertible change of variables $a = g(u)$:

$$\pi(a \mid s) = p_U(u \mid s) \left| \det \frac{\partial a}{\partial u} \right|^{-1},$$

$$\log \pi(a \mid s) = \log p_U(u \mid s) - \log \left| \det \frac{\partial a}{\partial u} \right|.$$

For elementwise $a_i = \tanh(u_i)$, $\partial a_i / \partial u_i = 1 - \tanh^2(u_i)$. The Jacobian is diagonal, and its determinant is $\prod_i (1 - \tanh^2(u_i))$. Hence:

$$\boxed{\log \pi_\theta(a \mid s) = \log \mathcal{N}(u; \mu_\theta(s), \Sigma_\theta(s)) - \sum_{i=1}^{d} \log\big(1 - \tanh^2(u_i)\big).} \tag{22}$$

If the environment action is additionally scaled as $a_i^{\text{env}} = c_i + b_i \tanh(u_i)$ with $b_i > 0$, then:

$$\boxed{\log \pi_\theta(a^{\text{env}} \mid s) = \log \mathcal{N}(u; \mu_\theta, \Sigma_\theta) - \sum_{i=1}^{d} \log\big(b_i(1 - \tanh^2(u_i))\big).} \tag{23}$$

Near saturation at ± 1, the direct $\log(1 - \tanh^2 u)$ expression can be numerically unstable. A stable identity is $\log(1 - \tanh^2 u) = 2(\log 2 - u - \mathrm{softplus}(-2u))$; implementations may alternatively add a small numerical stabilizer. Many implementations evaluate log-probabilities in normalized action coordinates and then scale actions for the environment. The crucial point is to use a consistent action-coordinate convention for density and entropy.

Motivation recap: both the SAC actor objective and automatic temperature tuning use $\log \pi$. Omitting the tanh Jacobian changes the objective and produces incorrect entropy gradients.

12. Step Eleven: Automatic Entropy Tuning

12.1 From fixed temperature to an entropy constraint

Manually choosing α is sensitive to reward scaling and action dimensionality. Rather than prescribing an entropy weight, we may require the policy to maintain a desired entropy level. A compact averaged formulation is:

$$\boxed{\begin{aligned} \max_{\pi} \quad & J_R(\pi) \\ \text{s.t.} \quad & \bar{\mathcal{H}}(\pi) \geq \mathcal{H}_{\text{target}}. \end{aligned}} \tag{24}$$

Here $\bar{\mathcal{H}}$ is the average policy entropy under the relevant state visitation distribution. The original automatic-temperature derivation uses per-time-step expected entropy constraints; we use the average form to convey the same design principle without unnecessary notation.

Introduce a nonnegative Lagrange multiplier α:

$$\boxed{\mathcal{L}(\pi, \alpha) = J_R(\pi) + \alpha\big(\bar{\mathcal{H}}(\pi) - \mathcal{H}_{\text{target}}\big).} \tag{25}$$

This yields the optimization form:

$$\boxed{\max_{\pi} \min_{\alpha \geq 0} \mathcal{L}(\pi, \alpha).} \tag{26}$$

For fixed α, optimizing the policy recovers the maximum-entropy objective. For fixed policy, optimizing α yields the dual update that aims to enforce the entropy target.

12.2 The temperature loss and its direction of change

Since $\mathcal{H}(\pi) = -\mathbb{E} \log \pi$, the dual-temperature loss is:

$$\boxed{J_\alpha(\alpha) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta} \left[\alpha \left(-\log \pi_\theta(a \mid s) - \mathcal{H}_{\text{target}} \right) \right].} \tag{27}$$

Equivalently, $J_\alpha = -\mathbb{E}[\alpha(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})]$. Differentiating:

$$\boxed{\frac{\partial J_\alpha}{\partial \alpha} = \bar{\mathcal{H}}(\pi_\theta) - \mathcal{H}_{\text{target}}.} \tag{28}$$

Thus:

$$\boxed{\begin{cases} \bar{\mathcal{H}}(\pi) < \mathcal{H}_{\text{target}} & \Rightarrow \alpha \uparrow, \\ \bar{\mathcal{H}}(\pi) > \mathcal{H}_{\text{target}} & \Rightarrow \alpha \downarrow. \end{cases}} \tag{29}$$

If policy entropy falls below the target, gradient descent increases α and encourages exploration. If entropy exceeds the target, α decreases and lets the actor concentrate more strongly on high-Q actions.

12.3 Why optimize log-alpha?

Directly updating α may produce negative values, which are inadmissible. Set $\eta = \log \alpha$ and $\alpha = e^\eta > 0$. Substituting exactly into the dual loss gives:

$$J_\eta(\eta) = -\mathbb{E}\left[e^\eta \left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}}\right)\right].$$

Its gradient includes the factor $e^\eta = \alpha$. Many popular implementations instead use a surrogate with a different gradient scale but the same interior stationary condition:

$$\boxed{J_\eta^{\text{surrogate}}(\eta) = -\mathbb{E}\left[\eta \left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}}\right)\right].} \tag{30}$$

During temperature optimization, detach the log-probability tensor so that the temperature loss does not unintentionally update actor parameters.

12.4 Choosing target entropy

For a normalized d-dimensional continuous action, a popular heuristic is $\mathcal{H}_{\text{target}} = -d$. For $d = 12$, one might initially try -12. This is not a theorem or universally optimal choice. Differential entropy can be negative and depends on the coordinate system used for action density. High-dimensional hands, robot arms, and action-chunk policies may require a task-specific target-entropy search.

13. Step Twelve: Putting Together the Complete Modern SAC Algorithm

13.1 Components and three optimization problems

A modern continuous-action SAC implementation typically maintains:

Component	Parameters or object	Purpose
Actor	θ	Generates reparameterizable stochastic actions
Critics 1 and 2	ϕ_1, ϕ_2	Estimate soft Q and mitigate overestimation
Target critics	$\bar{\phi}_1, \bar{\phi}_2$	Produce slowly changing TD targets
Temperature	α or $\eta = \log \alpha$	Regulates entropy weight
Replay buffer	$\mathcal{D}$	Stores historical (s, a, r, s', d) transitions

The three optimization problems are:

$$\boxed{\min_{\phi_1,\phi_2} J_Q, \qquad \min_{\theta} J_\pi, \qquad \min_{\alpha \geq 0} J_\alpha.}$$

13.2 Complete sequence for one update

Step 1: interact with the environment. At state s_t, sample $a_t \sim \pi_\theta(\cdot \mid s_t)$; execute the action and store $(s_t, a_t, r_t, s_{t+1}, d_t)$. During early training, random actions can be used to obtain initial coverage.

Step 2: sample past transitions. Draw a mini-batch $B = \{(s_i, a_i, r_i, s_i', d_i)\}_{i=1}^N$ from $\mathcal{D}$.

Step 3: compute targets. At the next states, sample $a_i' \sim \pi_\theta(\cdot \mid s_i')$ using the current actor and compute:

$$\boxed{y_i = r_i + \gamma(1 - d_i)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s'_i, a'_i) - \alpha \log \pi_\theta(a'_i \mid s'_i)\right].}$$

Step 4: update both critics. Each critic fits the same target:

$$J_{Q_j} = \frac{1}{2N}\sum_{i=1}^{N}(Q_{\phi_j}(s_i, a_i) - y_i)^2.$$

Step 5: update the actor. Resample $\epsilon_i \sim \mathcal{N}(0, I)$ and set $\tilde{a}_i = f_\theta(s_i, \epsilon_i)$. Minimize:

$$\boxed{J_\pi = \frac{1}{N}\sum_{i=1}^{N}\left[\alpha \log \pi_\theta(\tilde{a}_i \mid s_i) - \min_j Q_{\phi_j}(s_i, \tilde{a}_i)\right].}$$

The critic **weights** should be frozen during the actor step, but gradients through the critic **input action** must be retained: $\nabla_a Q_\phi$ is necessary.

Step 6: update temperature. Adjust α using the difference between $-\log \pi_\theta$ and the target entropy. Detach the log-probability during the temperature update.

Step 7: update target critics. For each critic, apply $\bar{\phi}_j \leftarrow (1 - \tau)\bar{\phi}_j + \tau\phi_j$. Repeat the cycle.

13.3 Instructional pseudocode

```
initialize actor pi_theta
initialize critics Q1, Q2
initialize target critics Q1_targ, Q2_targ from Q1, Q2
initialize replay buffer D and log_alpha

for each environment interaction:
    action = pi_theta.sample(obs)  # warm-up may use random actions
    next_obs, reward, terminated, truncated = env.step(action)
    D.add(obs, action, reward, next_obs, terminated)
    obs = env.reset() if (terminated or truncated) else next_obs

    for each gradient update:
        bs, ba, br, bs_next, bd = D.sample(batch_size)
        alpha = exp(log_alpha)

        # 1) Critic target: no gradient through target branch
        with no_grad():
            a_next, logp_next = pi_theta.rsample(bs_next)
            q_next = min(Q1_targ(bs_next, a_next),
                         Q2_targ(bs_next, a_next))
            y = br + gamma * (1 - bd) * (q_next - alpha * logp_next)

        loss_q1 = mean((Q1(bs, ba) - y) ** 2)
        loss_q2 = mean((Q2(bs, ba) - y) ** 2)
        update(Q1, loss_q1)
        update(Q2, loss_q2)

        # 2) Actor: freeze Q weights, keep dQ/da
        a_pi, logp_pi = pi_theta.rsample(bs)
        q_pi = min(Q1(bs, a_pi), Q2(bs, a_pi))
        loss_pi = mean(alpha.detach() * logp_pi - q_pi)
        update(pi_theta, loss_pi)

        # 3) Automatic entropy tuning (log-alpha surrogate)
        loss_alpha = -mean(log_alpha *
                           (logp_pi.detach() + target_entropy))
```

```
        update(log_alpha, loss_alpha)

        # 4) Polyak target update
        for Q, Q_targ in [(Q1, Q1_targ), (Q2, Q2_targ)]:
            Q_targ.params = (1 - tau) * Q_targ.params + tau * Q.params
```

> **Implementation warning:** this is explanatory pseudocode, not a complete runnable library. Production code must correctly handle optimizer gradient resets, parameter freezing, action scaling, density corrections, exploration warm-up, environment resets, and batch dimensions. It must also distinguish `terminated` from `truncated`.

14. Original SAC v1 Versus Modern SAC

14.1 Original SAC v1: explicitly learning a V-network

The original SAC formulation trained an additional state-value network $V_\psi(s)$, besides the actor and two Q-functions:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\big].$$

One can therefore fit:

$$\boxed{J_V(\psi) = \frac{1}{2}\mathbb{E}_s\left[\left(V_\psi(s) - \left(\min_i Q_{\phi_i}(s,\tilde{a}) - \alpha \log \pi_\theta(\tilde{a} \mid s)\right)\right)^2\right].}$$

When training V_ψ, the target expression is treated as a constant. Its Q-function backup is:

$$\boxed{y_Q = r + \gamma(1-d)V_{\bar{\psi}}(s').}$$

14.2 Modern SAC: remove the explicit V-network

The identity:

$$V^\pi(s') = \mathbb{E}_{a'\sim\pi}\big[Q^\pi(s',a') - \alpha \log \pi(a' \mid s')\big]$$

can be approximated directly with a newly sampled action:

$$V^\pi(s') \approx \min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s').$$

Substitution gives the target in Equation (12) without requiring an additional V-network. This removes one set of function-approximation errors and simplifies the model. Today, the phrase "standard SAC" usually refers to this modern twin-Q version, frequently combined with automatic temperature adjustment.

15. Fundamental Differences Between SAC and PPO

15.1 Comparing policy-update equations

PPO uses sampled-action advantage estimates and old-to-new policy likelihood ratios:

$$\boxed{J_{\mathrm{PPO}}(\theta) = \mathbb{E}\left[\min\left(\rho_t(\theta)\hat{A}_t, \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right],}$$

where $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\mathrm{old}}(a_t \mid s_t)$. The optimizer **maximizes** this surrogate objective, and clipping restrains large policy changes. Its typical policy-gradient signal is related to $\nabla_\theta \log \pi_\theta(a \mid s)\, A^\pi(s,a)$.

SAC instead **minimizes**:

$$\boxed{J_{\mathrm{SAC}}(\theta) = \mathbb{E}\left[\alpha \log \pi_\theta(a_\theta \mid s) - Q_\phi(s, a_\theta)\right], \quad a_\theta = f_\theta(s,\epsilon).}$$

Its gradient includes $\nabla_a Q_\phi(s,a_\theta)\, \partial a_\theta/\partial\theta$. PPO generally does not require $\nabla_a Q$; standard continuous-action SAC does.

15.2 Comparison table

Dimension	PPO	SAC
Data regime	On-policy	Off-policy
Actor	Usually stochastic	Reparameterizable stochastic policy
Critic	Commonly $V(s)$	Commonly $Q(s,a)$
Policy signal	Advantage and probability ratio	Q-value, action gradient, entropy
Main gradient estimator	Score-function estimator	Pathwise / reparameterization
Reuse of old data	Limited; fresh rollouts dominate	Replay buffer with repeated sampling
Entropy term	Common but optional	Central to the objective
Stabilization	Clipped policy update	Double Q and target critics
Sample efficiency	Often lower	Often higher
Typical strength	High-throughput parallel simulation	Expensive environment interactions
Main challenge	Needs many new transitions	Critic extrapolation and bootstrapping stability

PPO clipping and SAC double Q are **different stabilization mechanisms**: the former primarily restricts policy probability changes, while the latter addresses Q overestimation. Standard SAC does not impose the same explicit probability-ratio clipping as PPO.

15.3 Why PPO remains popular in robot simulation

On platforms such as Isaac Lab, thousands of parallel simulated environments may generate substantial experience at low marginal cost. PPO can exploit high-throughput rollouts and achieve excellent training speed. SAC reuses samples, but frequent critic updates, bootstrapped learning, and replay-buffer access also incur computational costs. Hence:

$$\boxed{\text{Sample Efficiency} \neq \text{Wall-Clock Efficiency.}}$$

Contact nonlinearity, sparse rewards, and major state-distribution shifts can also make SAC critics difficult to train. Conversely, when every physical robot interaction is expensive, SAC's off-policy reuse is often attractive, subject to safe exploration and coverage constraints.

16. Final Summary: Five Essential Formula Groups

(1) Maximum-entropy objective:

$$\boxed{J(\pi) = \mathbb{E}_{\tau}\left[\sum_{t=0}^{\infty} \gamma^t \big(r_t - \alpha \log \pi(a_t \mid s_t)\big)\right].}$$

Motivation: maximize reward while retaining exploration through entropy regularization.

(2) Soft Bellman equation:

$$\boxed{Q^{\pi}(s,a) = \mathbb{E}\left[r + \gamma\big(Q^{\pi}(s',a') - \alpha \log \pi(a' \mid s')\big)\right].}$$

Motivation: recursively decompose the maximum-entropy objective for TD learning.

(3) KL-based soft policy improvement:

$$\boxed{\pi_{\text{new}} = \arg\min_{\pi} D_{\mathrm{KL}}\left(\pi(\cdot \mid s) \,\middle\|\, \frac{\exp(Q^{\pi_{\text{old}}}(s,\cdot)/\alpha)}{Z(s)}\right),}$$

which is equivalent to minimizing $\mathbb{E}_{a\sim\pi}[\alpha \log \pi(a \mid s) - Q^{\pi_{\text{old}}}(s,a)]$. *Motivation:* approach a high-value Boltzmann distribution without evaluating its normalizing constant.

(4) Modern SAC critic and actor:

$$\boxed{J_{Q_i} = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[\big(Q_{\phi_i}(s,a) - y\big)^2\right], \quad y = r + \gamma(1-d)\left[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_{\theta}(a' \mid s')\right],}$$

$$\boxed{J_{\pi} = \mathbb{E}_{s,\epsilon}\left[\alpha \log \pi_{\theta}(f_{\theta}(s,\epsilon) \mid s) - \min_j Q_{\phi_j}(s, f_{\theta}(s,\epsilon))\right].}$$

Motivation: make soft policy iteration practical with trainable networks, historical transitions, twin critics, and pathwise gradients.

(5) Automatic entropy tuning:

$$\boxed{J_{\alpha} = -\mathbb{E}\left[\alpha(\log \pi_{\theta}(a \mid s) + \mathcal{H}_{\text{target}})\right].}$$

Motivation: automatically adapt exploration pressure to a desired entropy level.

The single most important PPO/SAC distinction

PPO mainly uses $\nabla_{\theta} \log \pi_{\theta}(a \mid s) A^{\pi}(s,a)$ to change the **probabilities of sampled actions**. Continuous SAC uses $\nabla_a Q_{\phi}(s,a)\, \partial a_{\theta}/\partial \theta$, together with the entropy gradient, to change the **differentiable action generator** toward higher-value actions.

The main logical progression is:

$$\boxed{\begin{array}{c}\text{Maximum-Entropy RL} \longrightarrow \text{Soft Bellman} \longrightarrow \text{KL Policy Improvement} \\ \longrightarrow \text{Reparameterization} \longrightarrow \text{Off-Policy Actor-Critic}\end{array}}$$

Double Q, target critics, and automatic temperature tuning make this theory useful for challenging practical continuous-control problems.

References

1. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). **Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.** ICML. https://arxiv.org/abs/1801.01290 — Theory of soft policy iteration, KL improvement, and stochastic actors.
2. Haarnoja, T., Zhou, A., Hartikainen, K., et al. (2018). **Soft Actor-Critic Algorithms and Applications.** https://arxiv.org/abs/1812.05905 — Automatic temperature adjustment and the modern SAC design.
3. OpenAI Spinning Up. **Soft Actor-Critic.** https://spinningup.openai.com/en/latest/algorithms/sac.html — Modern SAC losses and implementation details.

Document note: This Markdown is the single source used to generate the corresponding English PDF. Section structure, equations, tables, and references are identical across the two formats.

Data Collection and Imitation Learning

Course home · 简体中文 · PDF

Item	Details
Applicability	Demonstrations, BC, DAgger, ACT and generative chunk policies
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Methods survey and proposed experiments; no validated training results. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. From Demonstrations to a Trainable Dataset
  - 1.1 What One Demonstration Must Contain
  - 1.2 Time and Frames Come Before Architecture
- 2. Collection and Quality Control
  - 2.1 Beyond Successful Demonstrations
  - 2.2 Splits Matter More Than a Headline Score
- 3. Imitation Learning: BC to Interactive Correction
  - 3.1 Why Low Training Loss May Still Fail
  - 3.2 What DAgger Adds
- 4. From Single Actions to Generative Action Chunks
  - 4.1 ACT in One Picture
  - 4.2 Diffusion Policy and Flow Matching
- 5. Augmentation, Recalibration and Simulation
  - 5.1 Contact Events and Recovery Samples
  - 5.2 Safe Augmentation Boundaries
- 6. A Shared Panda Tabletop Experiment
  - 6.1 Minimum Experiment Matrix
  - 6.2 Debugging Checklist
- 7. Unified Data Model: From Demonstrations to Action Chunks
  - 7.1 Distinguish Latent State, Observations, and Commands
  - 7.2 The Action-Chunk Prediction Problem
  - 7.3 Time Alignment, Frames, and Action Normalization
- 8. Behavior Cloning: Objectives, Intuition, and Failure Modes
  - 8.1 Deterministic and Probabilistic BC

    - 8.2 Covariate Shift and Compounding Error
    - 8.3 Chunk Loss and Smoothness Trade-offs
- 9. Interactive Imitation and Recovering from Failure
    - 9.1 DAgger: Expert Queries on Learner-Visited States
    - 9.2 Failures, Recoveries, and Event-Biased Sampling
- 10. ACT, Diffusion Policy, Flow Matching, and RTC: Training vs. Execution
    - 10.1 ACT: Conditional VAE and Transformer Chunks
    - 10.2 Diffusion Policy: Predict Noise in Action Space
    - 10.3 Flow Matching: Generative Velocity Is Not Joint Velocity
    - 10.4 Fair Comparisons Against BC
    - 10.5 Why RTC Matters: From Synchronous Chunks to Asynchronous Control
    - 10.6 Inference-Time RTC: Inpainting With a Frozen Prefix and Soft Overlap
    - 10.7 Training-Time RTC: Learn the Prefix Constraint From Demonstrations
    - 10.8 Comparing RTC Against Temporal Ensembling and Naive Async
- 11. Human Video Retargeting, Contact, and Augmentation
    - 11.1 Human-to-Robot Is Not Pixel Imitation
    - 11.2 Augmentation Must Preserve Geometry and Physics
- 12. Training and Evaluation Protocol
    - 12.1 Offline Metrics, Closed-Loop Success, and Recovery
    - 12.2 A Shared Panda Tabletop Benchmark
- 13. Where This Fits and References

Learning outcomes: Identify demonstration quality risks; understand BC, DAgger, ACT, and diffusion/flow chunk training; distinguish inference-time and training-time RTC; evaluate timing, continuity, and task success.

1. From Demonstrations to a Trainable Dataset

Imitation learning is not just storing a successful video. It turns observations, robot states, actions and task conditions into aligned and reproducible supervision. A closed-loop robot ultimately learns a conditional policy $\pi_\theta(a_t \mid o_{\le t}, g)$, where g is the goal.

1.1 What One Demonstration Must Contain

Each episode should at least log task/scene IDs, image timestamps, RGB or RGB-D, joint feedback, executed actions, action reference frame, control mode, success, and exceptional events. **The action label must identify the command actually issued**; do not mix commanded joint targets, measured positions, and joint velocities. For vision, specify whether camera extrinsics are fixed or time-varying.

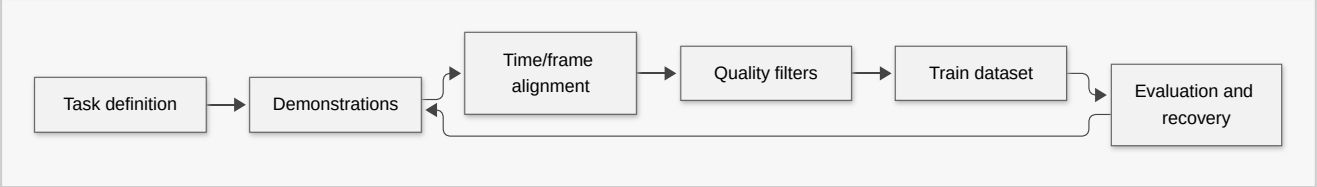
1.2 Time and Frames Come Before Architecture

Use one clock basis or verified timestamp alignment. Define T_B^C, T_B^E and T_B^O under one transform convention. Cameras often run slower than proprioception; match past observations to action

timestamps or use documented delay-aware interpolation, never future frames.

2. Collection and Quality Control

Common sources include teleoperation, kinesthetic teaching, scripted experts, simulated RL experts, and retargeted human video. They differ in throughput, action quality, contact fidelity, and hardware cost.



Practical principle: start with a smaller coherent dataset rather than accumulating large volumes of unaligned video.

2.1 Beyond Successful Demonstrations

Retain informative failures, corrective behavior, and recovery segments, with event boundaries such as first contact, slip, and regrasp. Do not blindly imitate every failed action: distinguish useful recovery demonstrations from unsafe commands. Record consent, data permissions, licensing, and robot safety review.

2.2 Splits Matter More Than a Headline Score

Split by object instance, scene layout, operator, initial configuration, and collection session. **Never put chunks from the same continuous trajectory in both train and test sets.** Report episode success, not only per-action MSE. Group synchronized views and adjacent frames together to prevent leakage.

3. Imitation Learning: BC to Interactive Correction

Behavior cloning (BC) fits actions from demonstrations. Deterministic regression can minimize $\mathcal{L}_{BC} = \mathbb{E}_{(o,a)\sim D}\|\pi_\theta(o) - a\|_2^2$; stochastic policies maximize conditional action likelihood. Neither objective automatically fixes deployment-time distribution shift.

3.1 Why Low Training Loss May Still Fail

In closed-loop execution, small action errors change future observations and push the robot outside the demonstration distribution. Record state drift, failed contacts, interventions, and recovery from perturbed states. Safe, supervised recovery demonstrations can expand coverage.

3.2 What DAgger Adds

DAgger collects states visited by the learner, queries an expert for suitable actions, and aggregates labeled data. A real robot needs safe intervention procedures. Offline retraining without expert relabeling of learner-visited states should not be called DAgger.

4. From Single Actions to Generative Action Chunks

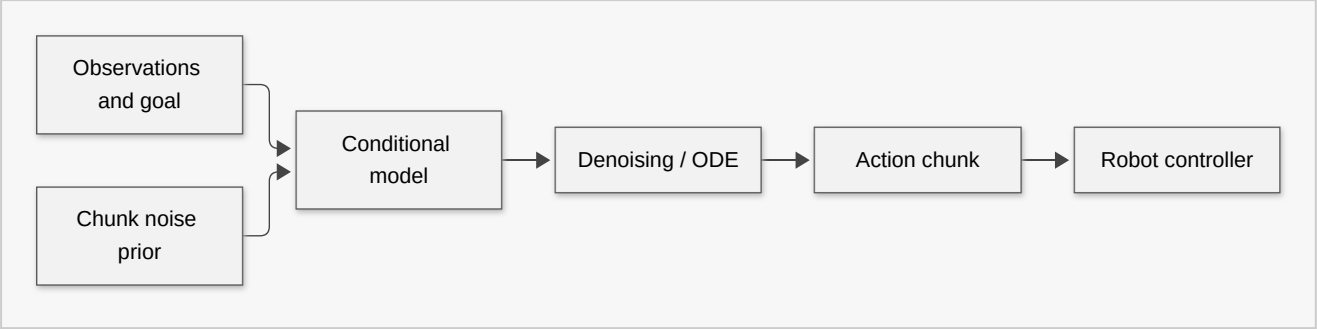
Action chunks reduce the need to replan every servo tick but may use stale observations during execution. Distinguish prediction horizon, execution horizon, and policy query rate. The chunk's command frame must agree with the actual controller interface.

4.1 ACT in One Picture

ACT uses a Transformer to predict a sequence of actions; the original formulation includes a latent-variable mechanism for demonstration variation. Chunks suit continuous structured demonstrations, but alignment and overlap policies still matter.

4.2 Diffusion Policy and Flow Matching

Diffusion Policy and Flow Matching can both represent multimodal action distributions, but **denoising dynamics or generative velocity fields are not robot joint velocities**.



For fair comparison, control observation modalities, demonstration budget, inference budget, controller rate, and evaluation scenarios. Offline action loss alone is insufficient.

5. Augmentation, Recalibration and Simulation

Geometric augmentation must transform images, object goals, reference trajectories, and action labels consistently. Rotating images without transforming the appropriate action target can create incorrect supervision. Simulated expert trajectories expand coverage, but contact, rendering, and latency gaps require separate validation.

5.1 Contact Events and Recovery Samples

Start from a uniform-sampling baseline before oversampling near first contact, grasp failure, or object drop. Event sampling changes the training distribution; document weights and always evaluate on an unbiased test distribution.

5.2 Safe Augmentation Boundaries

Do not propagate arbitrary initial conditions into physically inconsistent contact trajectories. Check reachability, collision, contact consistency, and actuator limits after rotation, translation, scaling, or timing perturbations; short physics replay is useful before admitting data.

6. A Shared Panda Tabletop Experiment

Task: in Isaac Lab or MuJoCo, use a Panda arm to grasp a randomized cube; collect scripted or teleoperated demonstrations and train BC, ACT, and a generative policy.

Control variables: camera view, task count, command interface, control rate, object split, and episode count. Simple BC is a required baseline for generative methods.

Metrics: success, contact failure, trajectory drift, recovery, policy inference latency, servo latency, and demonstrations used. Report seeds and failure modes rather than claiming one family always wins.

6.1 Minimum Experiment Matrix

Comparison	Purpose
One-step BC	Verify labels and controller
Chunked BC	Isolate chunking benefit
ACT	Test Transformer chunk prediction
Flow/Diffusion	Test multimodal action modelling
Recovery demos	Test closed-loop robustness

6.2 Debugging Checklist

Check timestamps, command/feedback confusion, camera extrinsics, action scale, latency, execution horizon, data splits, and initial-state coverage before scaling the model. Training-set action MSE is not a substitute for robot task success.

7. Unified Data Model: From Demonstrations to Action Chunks

7.1 Distinguish Latent State, Observations, and Commands

In real manipulation the full world state s_t (object motion, friction, contact modes) is usually not observed. We record images o_t^{cam}, joint feedback q_t, actual actuator commands a_t, and a goal g. A demonstration may be represented as

$$\tau_i = \{g_i, (o_t^{\text{cam}}, q_t, a_t, r_t^{\text{event}})_{t=0}^{T_i-1}\}.$$

The event label r_t^{event} may encode contact, slip, recovery, or completion; it need not be an RL reward. Ground-truth simulated object poses, future expert commands, or post-hoc outcome labels must not leak into the deployed policy observations. Keep issued commands distinct from delayed joint feedback.

Expert vs. behavior policy: Let π_E denote the expert and μ the actual data-collection policy. When demonstrations include exploratory actions, corrective trials or multiple operators, μ need not equal π_E. Preserve action provenance and evaluate on unbiased held-out episodes.

7.2 The Action-Chunk Prediction Problem

Construct conditioning information $c_t = (o_{t-L+1:t}, q_t, g)$ from an observation window, current proprioception and goal. The policy predicts an H-step sequence:

$$A_t = (a_t, a_{t+1}, \ldots, a_{t+H-1}), \qquad p_\theta(A_t \mid c_t).$$

Prediction horizon H is **not** the number of steps that must run open-loop. Execute only $h \le H$ steps, observe, then replan. Document control interval, inference call interval, queueing and interpolation. For example, 30-Hz vision and 100-Hz servo feedback do not imply that the large VLA must run at 100 Hz; it does require a safe interpolation, hold and timeout policy.

7.3 Time Alignment, Frames, and Action Normalization

At action time t_k^a, training inputs must already have been available. Accounting for acquisition/transport lag δ, choose a historical frame satisfying $t_j^{\text{image}} \le t_k^a - \delta$ rather than a future frame. For Cartesian pose commands, base-frame left-multiplied increments $T_{BE,t+1} = \Delta T_{B,t} T_{BE,t}$ differ from end-effector-frame right-multiplied increments $T_{BE,t+1} = T_{BE,t} \Delta T_{E,t}$.

Estimate action normalization statistics only on the training set:

$$\tilde{a} = (a - \mu_D)/(\sigma_D + \varepsilon), \qquad a_{\text{deployed}} = \tilde{a}(\sigma_D + \varepsilon) + \mu_D.$$

Different joints, gripper apertures, rotations and force commands require consistent units and independent safety limits. Never use test data to fit normalization parameters.

8. Behavior Cloning: Objectives, Intuition, and Failure Modes

8.1 Deterministic and Probabilistic BC

Given expert samples $(c, a) \sim d_E$, a deterministic baseline minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(c,a)\sim d_E} \|\pi_\theta(c) - a\|_2^2.$$

Under squared loss it converges toward the conditional mean $\mathbb{E}[a \mid c]$. This works for sufficiently unimodal behavior. If going left and going right around an obstacle are equally valid, the mean could collide with the obstacle. A probabilistic policy instead minimizes the negative log-likelihood

$$\mathcal{L}_{\text{NLL}}(\theta) = -\mathbb{E}_{(c,a)\sim d_E} \log p_\theta(a \mid c).$$

For a Gaussian distribution with fixed isotropic variance, NLL is MSE up to scaling and constants. Mixtures or generative action distributions can represent multiple modes. **High dimensionality alone is not proof that ordinary BC is inadequate.**

For general Gaussian $p_\theta(a \mid c) = \mathcal{N}(\mu_\theta(c), \Sigma_\theta(c))$, the NLL includes $\frac{1}{2}(a-\mu_\theta)^\top \Sigma_\theta^{-1}(a-\mu_\theta)$ and $\frac{1}{2}\log|\Sigma_\theta|$; it reduces to a weighted MSE only when the covariance is fixed.

8.2 Covariate Shift and Compounding Error

Offline BC learns on the expert occupancy d_{π_E}, while the deployed policy visits d_{π_θ}:

$$\min_\theta \mathbb{E}_{s\sim d_{\pi_E}} \ell(\pi_\theta(s),\pi_E(s)) \quad \not\Rightarrow \quad \min_\theta \mathbb{E}_{s\sim d_{\pi_\theta}} \ell(\pi_\theta(s),\pi_E(s)).$$

A small contact error may move the object to a state absent from expert training data. Collect intervention and recovery data; evaluate under deliberate safe perturbations, not solely on shuffled held-out action labels.

8.3 Chunk Loss and Smoothness Trade-offs

For variable-length episodes use a validity mask $m_{t,k}$ and optional horizon weight w_k:

$$\mathcal{L}_{\text{chunk}} = \frac{\sum_{t,k} m_{t,k} w_k \|\hat{a}_{t+k|t} - a_{t+k}\|_1}{\sum_{t,k} m_{t,k} w_k}.$$

A smoothness penalty $\lambda_s \sum_k \|\hat{a}_{k+1} - \hat{a}_k\|^2$ may suppress unwanted oscillation, yet can also erase necessary changes at contact onset or release. Measure task success and contact-force peaks before choosing it based on visual smoothness.

9. Interactive Imitation and Recovering from Failure

9.1 DAgger: Expert Queries on Learner-Visited States

DAgger gathers the states the current learner actually visits and requests safe expert labels. At iteration i, a mixture $\pi_i = \beta_i \pi_E + (1-\beta_i)\pi_{\theta_i}$ visits $s \sim d_{\pi_i}$; the expert supplies $\pi_E(s)$:

$$D_{i+1} = D_i \cup \{(s,\pi_E(s)) : s \sim d_{\pi_i}\}, \qquad \theta_{i+1} = \arg\min_\theta \sum_{(s,a)\in D_{i+1}} \ell(\pi_\theta(s),a).$$

The expert mixing probability β_i is often decreased gradually. Hardware requires human intervention authority, velocity/force limits and emergency stops. Merely retraining from static offline trajectories is not DAgger.

One optional event-biased sampler uses $w_t = 1 + \lambda \sum_j \exp[-(t-t_j)^2/(2\sigma^2)]$ around contact/slip events t_j.

9.2 Failures, Recoveries, and Event-Biased Sampling

Classify demonstrations as successful, recoverable failure, irrecoverable failure, or intervention. Annotate first contact, slip, drop, reacquisition and regrasp. Imitate only **safe and meaningful recovery actions**; unsafe failures may still be kept as negative outcome labels.

Compare uniform time sampling against event-biased sampling. A weighted loss is $\mathcal{L} = \sum_t w_t \ell_t / \sum_t w_t$, but modified sampling changes the training distribution. Always report unbiased test performance as well.

10. ACT, Diffusion Policy, Flow Matching, and RTC: Training vs. Execution

10.1 ACT: Conditional VAE and Transformer Chunks

Action Chunking with Transformers (ACT) predicts a sequence of commands with a Transformer. Its original formulation uses a conditional variational autoencoder (CVAE) to represent variation in demonstrations. An encoder approximates $q_\phi(z \mid A, c)$, and a decoder models $p_\theta(A \mid z, c)$:

$$\mathcal{L}_{\mathrm{ACT}} = \mathbb{E}_{z\sim q_\phi}[-\log p_\theta(A \mid z, c)] + \beta D_{\mathrm{KL}}(q_\phi(z \mid A, c)\|p(z)).$$

The KL term regularizes the posterior against the latent prior. At inference there is no future ground-truth action chunk A; use the declared prior or a deterministic representative. Large β can cause posterior collapse; tiny β can create a mismatch between training posterior and inference prior.

10.2 Diffusion Policy: Predict Noise in Action Space

Let A_0 be a demonstrated action chunk, $\epsilon \sim \mathcal{N}(0, I)$, and k the diffusion step. Construct

$$A_k = \sqrt{\bar{\alpha}_k}A_0 + \sqrt{1-\bar{\alpha}_k}\epsilon$$

and fit the conditional noise prediction objective

$$\mathcal{L}_{\mathrm{diff}} = \mathbb{E}_{A_0,c,k,\epsilon}\|\epsilon - \epsilon_\theta(A_k, k, c)\|_2^2.$$

At deployment, iterative denoising produces an action chunk. Execute a subset and replan from the next observation. The denoising step count, visual encoder and network architecture all affect latency; "generative" does not imply real-time.

10.3 Flow Matching: Generative Velocity Is Not Joint Velocity

Use data endpoint $A_1 \sim p_{\mathrm{data}}(\cdot \mid c)$ and noise $A_0 \sim \mathcal{N}(0, I)$. For a straight interpolation $A_\tau = (1-\tau)A_0 + \tau A_1$, one possible training loss is

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{c,A_0,A_1,\tau}\|v_\theta(A_\tau, \tau, c) - (A_1 - A_0)\|_2^2.$$

Sample by solving $dA_\tau/d\tau = v_\theta(A_\tau, \tau, c)$. This vector field is a derivative in **generative time**, not the robot's physical $\dot{q}$. Path choices and time orientations vary by implementation. The continuity equation, conditional-expectation argument and OpenPI time convention are derived exclusively in the Flow Matching tutorial; this chapter focuses on the imitation-learning interface.

10.4 Fair Comparisons Against BC

Use the same demonstrations, observation encoder, action representation, prediction/execution horizons and evaluation protocol for one-step BC, chunked BC, ACT, Diffusion Policy and Flow Matching. Record data volume, model calls, generation steps, inference latency and closed-loop success. Do not mistake the benefit of action chunking for the benefit of a Transformer, or of extra test-time computation for the learning algorithm itself.

10.5 Why RTC Matters: From Synchronous Chunks to Asynchronous Control

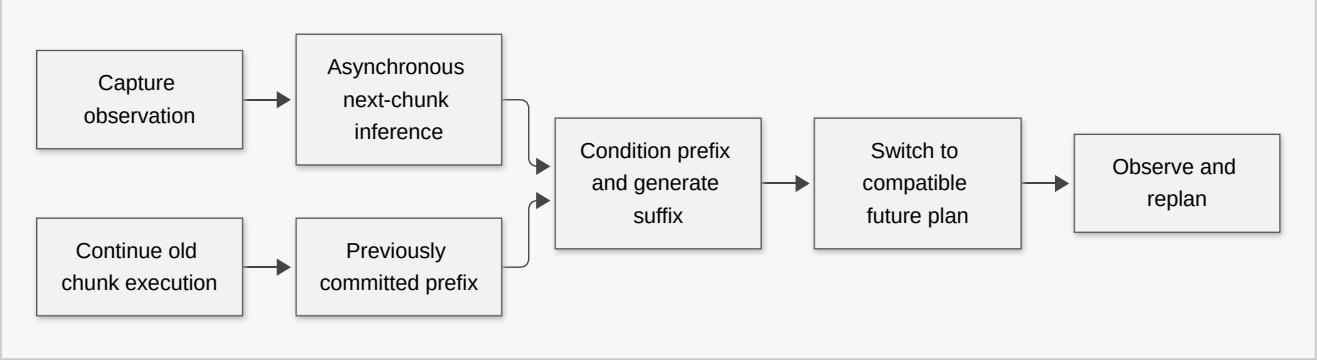
Real-Time Chunking (RTC) is not a new BC objective or a standalone motor policy. It is an execution/conditional generation technique for keeping diffusion- or flow-based chunk policies responsive despite inference latency. Synchronous chunk execution pauses at chunk boundaries.

Naive asynchronous switching removes the pause but may jump between different valid yet incompatible future trajectories. Temporal ensembling averages predictions, which can produce an invalid average of two legitimate modes in contact-rich tasks.

Let H be the prediction horizon, $s \le H$ the execution horizon, Δt the servo timestep, and δ the end-to-end latency from observation to available chunk. Measure inference delay in servo steps:

$$d \approx \left\lceil \frac{\delta}{\Delta t} \right\rceil, \qquad A_t = (a_t, \dots, a_{t+H-1}).$$

The ceiling is a **conservative engineering approximation**; the original RTC derivation uses $\lfloor \delta/\Delta t \rfloor$ and estimates future delay from a rolling history. For the asynchronous schedule in the original paper, require $d \le s \le H - d$: $d \le s$ permits inference to complete before the next handoff, and $d \le H - s$ leaves sufficient actions in the previous chunk to cover the inference delay. Leave additional margin for jitter, initialization, and timeouts. At 50 Hz ($\Delta t = 20$ ms), a measured 140 ms latency is roughly seven steps. Choosing $H = 50$ and $s = 20$ can leave enough old-chunk actions to cover inference, but does not replace timeout, packet-loss handling or emergency-stop logic.



10.6 Inference-Time RTC: Inpainting With a Frozen Prefix and Soft Overlap

Let Y be the old planned actions **aligned with the new chunk by absolute controller time**, and A^τ the new generated chunk at generative time $\tau \in [0, 1]$. The first d new-chunk actions correspond to time already committed to the old plan, so they must not change. Beyond that, some actions overlap the old plan but may be updated using new observations. The original RTC soft mask uses

$$W_i = \begin{cases} 1, & 0 \le i < d, \\ c_i(e^{c_i} - 1)/(e - 1), & d \le i < H - s, \\ 0, & H - s \le i < H, \end{cases}$$

$$c_i = \frac{H - s - i}{H - s - d + 1}.$$

The committed prefix receives weight one; the middle overlap decays; the last s actions are newly planned. **These are guidance weights inside generation, not post-hoc averaging of two action sequences.**

For a noise-to-data flow policy, a local fully-denoised endpoint prediction is

$$\widehat{A}^1 = A^\tau + (1 - \tau) v_\theta(A^\tau, \tau, c).$$

The original paper's pseudoinverse guidance (Eq. 2–4) can be written equivalently as a negative gradient of a weighted endpoint error:

$$\tilde{v}_\theta = v_\theta - \lambda_\tau \nabla_{A^\tau} \left[\frac{1}{2} \| W^{1/2} \odot (\widehat{A}^1 - Y) \|_2^2 \right].$$

The exact clipped guidance coefficient in the original paper is

$$r_\tau^2 = \frac{(1-\tau)^2}{\tau^2 + (1-\tau)^2}, \qquad \lambda_\tau = \min\left(\beta, \frac{1-\tau}{\tau\, r_\tau^2}\right).$$

The clipping parameter β prevents an overly large guidance correction. Implementations must protect numerical limits near $\tau = 0$ or $\tau = 1$. Integrate $A^{\tau+\Delta\tau} = A^\tau + \Delta\tau\, \tilde{v}_\theta$. This keeps the generated trajectory compatible with committed actions while still allowing future corrections. The correction involves a vector–Jacobian product during generative integration: **inference-time RTC does not require retraining, but adds inference computation.** The original method assumes a guidable continuous diffusion/flow sampler; an arbitrary deterministic ACT output cannot simply be passed through the same inpainting solver.

10.7 Training-Time RTC: Learn the Prefix Constraint From Demonstrations

Training-Time Action Conditioning for Efficient Real-Time Chunking incorporates the committed prefix into the imitation-learning objective. It learns

$$p_\theta(A_{d:H} \mid c, A_{0:d}),$$

where $A_{0:d}$ comprises indices $0, \ldots, d-1$ and the postfix $A_{d:H}$ comprises indices $d, \ldots, H-1$ (half-open interval notation). Given an expert chunk A, sample deployment-like delay $0 \le d < H$, flow time τ, and Gaussian noise ϵ. Use per-action flow times and inputs

$$\bar{\tau}_i = \begin{cases} 1, & i < d, \\ \tau, & i \ge d, \end{cases} \qquad \tilde{A}_i = \bar{\tau}_i A_i + (1 - \bar{\tau}_i)\epsilon_i.$$

The first d tokens are **clean ground-truth prefix actions**, while the remaining tokens learn to denoise. A compatible action transformer must support token-wise flow-time conditioning. Mask the loss so that only the postfix is supervised:

$$\mathcal{L}_{\mathrm{RTC-train}} = \mathbb{E}_{A,c,d,\tau,\epsilon} \left[\frac{1}{H-d} \sum_{i=d}^{H-1} \| v_\theta(\tilde{A}, \bar{\boldsymbol{\tau}}, c)_i - (A_i - \epsilon_i) \|_2^2 \right].$$

At deployment, supply the previously committed actions as a hard prefix and generate **only the editable suffix**. This avoids inference-time gradient guidance, but requires training/fine-tuning for a distribution of delays and does not support the original method's flexible soft overlap in exactly the same way. Training prefixes come from expert demonstrations; deployed prefixes come from previous model predictions, so rollout evaluation must cover this distribution shift.

This method belongs naturally in imitation learning because it changes **sample construction, token conditioning, and loss masking**. The full flow-matching mathematical derivation remains in Flow Matching. Sample d from measured latency statistics rather than assuming an arbitrary universal delay range; the cited $\pi_{0.6}$ experiments evaluate randomly sampled prefix lengths at both training and inference time.

10.8 Comparing RTC Against Temporal Ensembling and Naive Async

Execution mode	Waits at chunk boundaries?	How inter-chunk consistency is handled	Changes training?
Synchronous chunking	Usually yes	Replan after a pause	No
Naive asynchronous	Not necessarily	Immediate switch, possible jerk/mode jump	No
Temporal ensembling	Can be asynchronous	Average overlapping predictions; can fail with multimodality	No
Inference-time RTC	No	Frozen committed prefix + soft generative guidance	**No**, but extra inference cost
Training-time RTC	No	Prefix-conditioned denoising; hard prefix at inference	**Yes**, cheaper inference

Minimum evaluation: Fix the same flow policy, task seeds, controller rate, and safety stack. Compare all five execution modes with 0/100/200 ms injected latency. Report task success, completion time, task throughput, inference p50/p95 latency, missed servo deadlines, boundary jump $\|a_t - a_{t-1}\|$, acceleration/jerk, contact failure, and drop recovery. Distinguish model compute time, handoff delay, and physical execution time. Offline FPS is not enough.

References: RTC original paper (NeurIPS 2025), training-time RTC (2025), Physical Intelligence project/videos, and LeRobot RTC documentation. Extensions such as Soft RTC (2026) and DiscreteRTC (2026) explore soft overlap priors and native inpainting for discrete action samplers; their reported benefits are not universal guarantees for the original RTC method.

11. Human Video Retargeting, Contact, and Augmentation

11.1 Human-to-Robot Is Not Pixel Imitation

A realistic pipeline is hand/tool pose recovery → scale and frame calibration → embodiment-specific retargeting → contact event detection → short physics replay and trajectory repair → policy training. Humans and robot hands differ in kinematics, reachability and contact friction. One task-dependent objective is

$$\mathcal{L}_{\text{retarget}} = w_p \|p_{\text{robot}} - p_{\text{target}}\|^2 + w_R d_{SO(3)}(R_{\text{robot}}, R_{\text{target}})^2 + w_c \mathcal{L}_{\text{contact}} + w_j \mathcal{L}_{\text{limits}}.$$

Define $\mathcal{L}_{\text{contact}}$ for the task, e.g. contact point error, surface normals, grasp maintenance or object slip. Mere proximity does not certify physical contact.

11.2 Augmentation Must Preserve Geometry and Physics

When perturbing object position, scale or orientation, transform task goals, observable geometry, trajectories and reference-frame-dependent labels consistently. Rotating an image without adjusting Cartesian action labels is typically invalid. Contact phases should be physically replayed to check penetration, limits, slip and unreasonable forces before accepting augmented episodes.

12. Training and Evaluation Protocol

12.1 Offline Metrics, Closed-Loop Success, and Recovery

Test	Report	Does not replace
Held-out demonstrations	NLL / action error and chunk alignment	Closed-loop task success
Randomized starts	Success, completion time, collision/contact failures	Perturbation tests
Drop or disturbance	Recovery success, recovery latency, regrasp count	Nominal performance
Held-out objects/scenes	Generalization success with grouped splits	Data leakage checks
Runtime	Policy p50/p95 latency, actuation latency	A bare GPU FPS number

For k successes in N independent episodes, report $\hat{p} = k/N$ and a confidence interval. The rough standard error is $\sqrt{\hat{p}(1-\hat{p})/N}$; use a Wilson interval for small N or near-boundary success rates. Frames from the same episode are not independent trials.

12.2 A Shared Panda Tabletop Benchmark

Use the same Panda arm, cube, and tabletop in simulation. First validate control labels, then compare one-step versus chunked BC, then ACT versus diffusion/flow, and finally add recovery demonstrations and contact perturbations. When transferring across Isaac and MuJoCo, disclose differences in contact solvers, actuator dynamics and timing.

Chapter ownership: this tutorial owns demonstration quality, supervised imitation, distribution shift and action-policy comparisons. Complete PPO/SAC derivations belong to their respective chapters; formal FM derivations belong to the Flow Matching tutorial; the visual model toolbox belongs to Visual Perception and State Estimation.

13. Where This Fits and References

This chapter bridges camera calibration, visual state estimation, Flow Matching, and GR00T / SONIC.

- ACT / ALOHA
- Diffusion Policy
- DAgger
- Flow Matching for Generative Modeling
- LeRobot

Verification boundary: this chapter is a methods and experiment-design primer, not a claim that those policies were reproduced in the shared Panda setup.

Flow Matching: A Step-by-Step Mathematical Derivation

Course home · 简体中文 · PDF

Item	Details
Applicability	CFM theory; OpenPI pi0.py code reference pinned below; examples are not a tested fine-tuning run
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

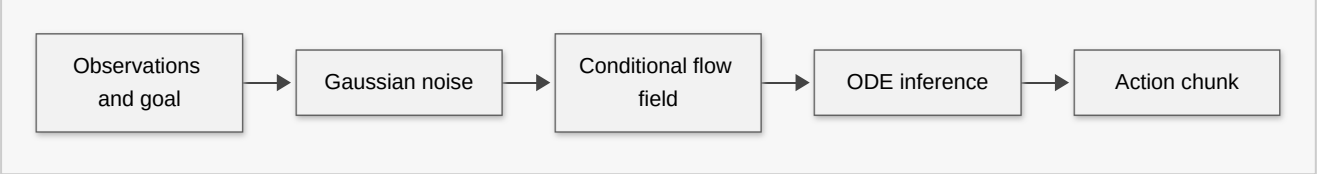
Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- Step 1 — Why do we need Flow Matching?
  - 1.1 What problem does a generative model solve?
  - 1.2 Start from an easily sampled Gaussian distribution
- Step 2 — Why introduce an ODE?
  - 2.1 Decompose a complicated transformation into small continuous steps
  - 2.2 A control-theoretic interpretation
- Step 3 — Why do we need the continuity equation?
  - 3.1 Begin with moving particles
  - 3.2 Derivation of the continuity equation
  - 3.3 Physical meaning of the continuity equation
- Step 4 — What is the ideal Flow Matching objective?
  - 4.1 Suppose the correct velocity field were known
  - 4.2 Why cannot we train with this ideal objective directly?
- Step 5 — How is Conditional Flow Matching constructed?
  - 5.1 Construct two endpoints
  - 5.2 Obtain a directly trainable loss
- Step 6 — Why does conditional velocity regression recover the global field?
  - 6.1 Individual-path velocity versus marginal velocity
  - 6.2 A rigorous derivation of the marginal velocity field
  - 6.3 Equivalent expression using probability densities
- Step 7 — Why does MSE regression learn the conditional expectation?

    - 7.1 Orthogonal decomposition of least squares
    - 7.2 Why is this such an important result?
    - 7.3 Why can generated trajectories curve if training paths are straight?
- Step 8 — Why introduce more general Gaussian conditional paths?
    - 8.1 A general Gaussian conditional path
    - 8.2 The simplest linear Gaussian path
    - 8.3 Derive the velocity field
- Step 9 — How are Flow Matching models trained and sampled?
    - 9.1 Training procedure
    - 9.2 Sampling (inference) procedure
- Step 10 — Why can Flow Matching generate multimodal distributions?
- Step 11 — How does π0.5 use Flow Matching?
    - 11.1 The generated object is an action chunk
    - 11.2 Conditional Flow Matching for action chunks
    - 11.3 Important implementation detail: OpenPI reverses the time direction
- Step 12 — How can Flow Matching be fine-tuned with reinforcement learning?
    - 12.1 What does pretrained Flow Matching learn?
    - 12.2 Why learn a residual velocity?
    - 12.3 Why cannot reward be used directly as a velocity label?
    - 12.4 A natural design: Advantage-Weighted Flow Matching
- Step 13 — Synthesis: the complete mathematical logic of Flow Matching
    - 13.1 Eight essential steps
    - 13.2 Three ideas to remember

Chapter concept map



Step 1 — Why do we need Flow Matching?

1.1 What problem does a generative model solve?

Assume the training data follow an unknown distribution:

$$x \sim p_{\text{data}}(x).$$

In image generation, x represents an image; in video generation, it represents a video; in robot control, it may represent an **action chunk**; and in dexterous manipulation, it may represent a sequence of joint actions over several future timesteps.

Our goal is to learn a generative model such that

$$x_{\text{generated}} \sim p_{\text{data}}(x).$$

Real-world data distributions are often complicated. For example, a robot may grasp a cup from either the left or the right. The resulting action distribution can be multimodal:

$$p_{\text{data}}(x) = \frac{1}{2}p_{\text{left}}(x) + \frac{1}{2}p_{\text{right}}(x).$$

Suppose we instead train a deterministic network $f_\theta(o) \to x$ using squared error:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\|f_\theta(o) - x\|^2\right].$$

Its population-optimal prediction is the conditional mean:

$$f_\theta^*(o) = \mathbb{E}[x \mid o].$$

That mean may correspond to neither a valid left grasp nor a valid right grasp.

Motivation. We want to learn the **full data distribution**, rather than output only its mean.

1.2 Start from an easily sampled Gaussian distribution

Sampling from a standard Gaussian is easy:

$$\boxed{x_0 \sim p_0(x) = \mathcal{N}(0, I).}$$

Instead of predicting a real sample directly, imagine learning a transformation

$$F_\theta : \mathbb{R}^d \to \mathbb{R}^d, \qquad x_1 = F_\theta(x_0),$$

that satisfies

$$\boxed{(F_\theta)_{\#}p_0 = p_{\text{data}}.}$$

The symbol # denotes the **pushforward distribution**. Intuitively, if we transform many Gaussian samples through F_θ, the distribution of their outputs should match the data distribution:

$$\underbrace{\mathcal{N}(0, I)}_{\text{simple Gaussian}} \xrightarrow{F_\theta} \underbrace{p_{\text{data}}}_{\text{complex multimodal distribution}}.$$

The question now becomes: **How can we learn this distribution transformation?**

Step 2 — Why introduce an ODE?

2.1 Decompose a complicated transformation into small continuous steps

Learning $x_1 = F_\theta(x_0)$ in a single step may be difficult. Instead, introduce a continuous **generative time** variable:

$$t \in [0, 1].$$

By convention, $t = 0$ corresponds to the Gaussian distribution, and $t = 1$ corresponds to the data distribution:

$$t = 0 : \ x_0 \sim p_0, \qquad t = 1 : \ x_1 \sim p_{\text{data}}.$$

Between these endpoints, a sample moves along a continuous trajectory:

$$x_0 \longrightarrow x_t \longrightarrow x_1.$$

Define a time-dependent **velocity field**

$$v_\theta(x, t) : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d$$

and the ordinary differential equation (ODE)

$$\boxed{\frac{dx_t}{dt} = v_\theta(x_t, t).} \tag{1}$$

With $x_0 \sim \mathcal{N}(0, I)$ as the initial condition, integration gives

$$x_1 = x_0 + \int_0^1 v_\theta(x_t, t)\, dt.$$

Motivation. Rather than predicting a final sample in one shot, the network answers a local question: *Given the current location x_t and time t, in which direction should the sample move?* A global generation task becomes local velocity prediction.

This construction is an instance of a **Continuous Normalizing Flow (CNF)**.

2.2 A control-theoretic interpretation

From a dynamical-systems perspective, Flow Matching learns the nonautonomous system

$$\dot{x} = v_\theta(x, t).$$

Its goal, however, is not to drive a single state toward an equilibrium. It aims to evolve an **entire probability distribution** along a desired path:

$$p_0 \longrightarrow p_t \longrightarrow p_1.$$

A conventional control problem often focuses on $x(0) \to x_{\text{target}}$; Flow Matching instead focuses on

$$\boxed{p_0(x) \to p_{\text{target}}(x).}$$

We do not have to specify in advance which particular noise sample should correspond to which particular training example. Matching the generated **distribution** is the central requirement.

Step 3 — Why do we need the continuity equation?

3.1 Begin with moving particles

Consider many particles $x_t^{(1)}, \ldots, x_t^{(N)}$, each satisfying

$$\frac{dx_t^{(i)}}{dt} = v_t(x_t^{(i)}).$$

Together, they induce a probability density $p_t(x)$. When particles flow into a region, its density increases; when they leave, its density decreases.

Thus, we need a precise connection between the **velocity field** and the **time evolution of probability density**.

3.2 Derivation of the continuity equation

Take an arbitrary smooth test function $\phi : \mathbb{R}^d \to \mathbb{R}$. Consider

$$\mathbb{E}[\phi(x_t)] = \int \phi(x) p_t(x)\, dx.$$

Differentiate with respect to time, assuming sufficient regularity to interchange differentiation and integration:

$$\frac{d}{dt}\mathbb{E}[\phi(x_t)] = \int \phi(x) \frac{\partial p_t(x)}{\partial t}\, dx.$$

On the other hand, the chain rule gives

$$\frac{d}{dt}\phi(x_t) = \nabla \phi(x_t)^\top \frac{dx_t}{dt} = \nabla \phi(x_t)^\top v_t(x_t).$$

Consequently,

$$\frac{d}{dt}\mathbb{E}[\phi(x_t)] = \int \nabla \phi(x)^\top v_t(x) p_t(x)\, dx.$$

Assuming the boundary term vanishes, integrate by parts:

$$= -\int \phi(x) \nabla \cdot [p_t(x) v_t(x)]\, dx.$$

Equating the two expressions yields

$$\int \phi(x) \left[\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x) v_t(x)) \right] dx = 0.$$

Because this holds for every suitable ϕ, we obtain

$$\boxed{\frac{\partial p_t(x)}{\partial t} + \nabla \cdot [p_t(x) v_t(x)] = 0.} \tag{2}$$

This is the **continuity equation**.

3.3 Physical meaning of the continuity equation

Expand the divergence term:

$$\frac{\partial p_t}{\partial t} + v_t^\top \nabla p_t + p_t \nabla \cdot v_t = 0.$$

The term $v_t^\top \nabla p_t$ describes advection of the density, while $\nabla \cdot v_t$ measures the local divergence of the velocity field.

If $\nabla \cdot v_t > 0$, nearby particles tend to spread out, and density tends to decrease. If $\nabla \cdot v_t < 0$, particles tend to converge, and density tends to increase.

Along an ODE trajectory, we also have

$$\boxed{\frac{d}{dt} \log p_t(x_t) = -\nabla \cdot v_t(x_t).} \tag{3}$$

This identity is the basis for evaluating sample log-densities in CNFs.

Motivation. An ODE describes the motion of an individual particle, whereas the continuity equation describes how the **distribution of particles** changes. Together they link generative dynamics to the target probability distribution.

Step 4 — What is the ideal Flow Matching objective?

4.1 Suppose the correct velocity field were known

Assume we choose a probability path $p_t(x)$ in advance, satisfying

$$p_{t=0} = p_0, \qquad p_{t=1} = p_{\text{data}},$$

and a velocity field $u_t(x)$ exists such that

$$\frac{\partial p_t}{\partial t} + \nabla \cdot (p_t u_t) = 0.$$

We could then regress directly onto this field:

$$\boxed{\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}[0,1] \\ x \sim p_t}} \left[\| v_\theta(x,t) - u_t(x) \|^2 \right].} \tag{4}$$

If $v_\theta = u_t$ is learned exactly, the induced ODE realizes the selected distribution evolution.

4.2 Why cannot we train with this ideal objective directly?

In practice, we generally observe only

$$x_1 \sim p_{\text{data}}.$$

We do not observe the intermediate process by which Gaussian samples evolve into data samples. Therefore, neither the intermediate $p_t(x)$ nor the required $u_t(x)$ is usually known.

Central motivation. Can we construct a path that is easy to sample **and** whose velocity is directly computable, thereby obtaining a supervised learning signal for the network?

That question leads to **Conditional Flow Matching (CFM)**.

Step 5 — How is Conditional Flow Matching constructed?

5.1 Construct two endpoints

Sample independently from the Gaussian source and the data distribution:

$$x_0 \sim p_0 = \mathcal{N}(0, I), \qquad x_1 \sim p_{\text{data}}.$$

The simplest pairing is independent:

$$x_0 \perp x_1.$$

Given an endpoint pair (x_0, x_1), prescribe constant-speed linear motion:

$$\boxed{x_t = (1-t)x_0 + t x_1.} \tag{5}$$

The boundary conditions are immediate: $x_t = x_0$ at $t = 0$, and $x_t = x_1$ at $t = 1$.

For example, let

$$x_0 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, \quad x_1 = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Differentiating (5),

$$\boxed{\frac{dx_t}{dt} = x_1 - x_0.} \tag{6}$$

The target velocity in this example is $[4, 2]^\top$. It is independent of t because we deliberately chose constant-speed linear interpolation.

Why a straight line? For fixed endpoints, it is extremely easy to compute, has an explicit velocity label, and requires neither an auxiliary model nor simulation of an intermediate trajectory.

5.2 Obtain a directly trainable loss

During training, sample

$$x_0 \sim \mathcal{N}(0, I), \quad x_1 \sim p_{\text{data}}, \quad t \sim \mathcal{U}[0, 1].$$

Construct

$$x_t = (1-t)x_0 + t x_1, \qquad u_t = x_1 - x_0.$$

Train the neural network to predict the velocity:

$$v_\theta(x_t, t) \approx x_1 - x_0.$$

The resulting loss is

$$\boxed{\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1}\left[\left\| v_\theta((1-t)x_0 + t x_1, t) - (x_1 - x_0) \right\|^2\right].} \tag{7}$$

This is the familiar **straight-line Conditional Flow Matching objective**.

One training update requires just a data sample, Gaussian noise, a randomly chosen time, an interpolation, a target velocity, one network forward pass, and a mean-squared-error (MSE) loss.

Major advantage. There is no need to solve the generative ODE during training. This is commonly called **simulation-free training**.

A deeper question remains: the network receives only (x_t, t), not the original endpoints (x_0, x_1). Why does regressing onto $x_1 - x_0$ recover an appropriate global velocity field?

Step 6 — Why does conditional velocity regression recover the global field?

6.1 Individual-path velocity versus marginal velocity

For fixed endpoints (x_0, x_1), the velocity of that individual path is

$$u_t(x \mid x_0, x_1) = x_1 - x_0.$$

It depends on the chosen endpoints, and we call it a **conditional velocity**.

At inference time, however, the final data endpoint is unknown. The network must predict a field that depends only on the current location and time:

$$v_t(x).$$

Many different straight paths can pass through the same (x, t), potentially with different velocities. What should the network output? The answer is the **conditional expectation** of those path velocities.

6.2 A rigorous derivation of the marginal velocity field

Use capital letters for random variables. Define

$$Z = (X_0, X_1), \qquad X_t = (1-t)X_0 + tX_1,$$

and

$$U_t = X_1 - X_0.$$

Since $dX_t/dt = U_t$, for any smooth test function ϕ,

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \mathbb{E}[\nabla\phi(X_t)^\top U_t].$$

Apply the tower property of conditional expectation:

$$= \mathbb{E}\left[\nabla\phi(X_t)^\top \mathbb{E}[U_t \mid X_t]\right].$$

Define

$$\boxed{v_t^*(x) = \mathbb{E}[U_t \mid X_t = x].} \tag{8}$$

Then

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \int \nabla\phi(x)^\top v_t^*(x) p_t(x)\, dx.$$

Integration by parts gives

$$= -\int \phi(x)\nabla \cdot [p_t(x)v_t^*(x)]\,dx.$$

But we also have

$$\frac{d}{dt}\mathbb{E}[\phi(X_t)] = \int \phi(x)\partial_t p_t(x)\,dx.$$

Hence,

$$\boxed{\partial_t p_t(x) + \nabla \cdot [p_t(x)v_t^*(x)] = 0.} \tag{9}$$

In other words, the conditional expectation defines a velocity field that realizes the target **marginal probability path**:

$$\boxed{v_t^*(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x].} \tag{10}$$

This is one of the central mathematical statements behind CFM.

6.3 Equivalent expression using probability densities

More generally, let z denote a conditioning variable, $p_t(x \mid z)$ a conditional probability path, and $u_t(x \mid z)$ its conditional velocity. The marginal distribution is

$$p_t(x) = \int p_t(x \mid z)q(z)\,dz.$$

We can define

$$\boxed{v_t^*(x) = \frac{\int u_t(x \mid z)p_t(x \mid z)q(z)\,dz}{p_t(x)}.} \tag{11}$$

Equivalently,

$$v_t^*(x) = \mathbb{E}[u_t(x \mid Z) \mid X_t = x].$$

For deterministic interpolation between fixed endpoints, the conditional path is a Dirac distribution concentrated at the interpolated point. The random-variable argument in the previous subsection is therefore more convenient.

Important subtlety. The continuity equation can admit more than one valid velocity field. Flow Matching does not discover a unique, path-independent "true velocity field." We choose a probability path and learn a field that realizes it.

Step 7 — Why does MSE regression learn the conditional expectation?

7.1 Orthogonal decomposition of least squares

For a fixed t, define

$$m_t(X_t) = \mathbb{E}[U_t \mid X_t].$$

Consider the training loss

$$\mathcal{L}(\theta) = \mathbb{E}[\|v_\theta(X_t, t) - U_t\|^2].$$

Add and subtract m_t inside the difference:

$$v_\theta - U_t = (v_\theta - m_t) + (m_t - U_t).$$

Expand the squared norm:

$$\begin{aligned}\|v_\theta - U_t\|^2 = \|v_\theta - m_t\|^2 + \|m_t - U_t\|^2 \\ + 2(v_\theta - m_t)^\top (m_t - U_t).\end{aligned}$$

By the defining property of conditional expectation,

$$\mathbb{E}[m_t - U_t \mid X_t] = 0.$$

Furthermore, $v_\theta(X_t, t) - m_t(X_t)$ is a function of X_t. Thus the expected cross term vanishes:

$$\mathbb{E}[(v_\theta - m_t)^\top (m_t - U_t)] = 0.$$

Therefore,

$$\boxed{\begin{aligned}\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}[\|v_\theta(X_t, t) - m_t(X_t)\|^2] \\ + \mathbb{E}[\|U_t - m_t(X_t)\|^2].\end{aligned}} \tag{12}$$

The second term does not depend on θ. Therefore, under the same sampling distribution and squared-error definition,

$$\boxed{\arg\min_\theta \mathcal{L}_{\mathrm{CFM}}(\theta) = \arg\min_\theta \mathcal{L}_{\mathrm{FM}}(\theta).} \tag{13}$$

Provided differentiation and expectation can be interchanged, we also have

$$\boxed{\nabla_\theta \mathcal{L}_{\mathrm{CFM}} = \nabla_\theta \mathcal{L}_{\mathrm{FM}}.} \tag{14}$$

7.2 Why is this such an important result?

What we want to learn is

$$v_t^*(x) = \mathbb{E}[U_t \mid X_t = x],$$

but computing this conditional expectation explicitly is difficult.

CFM avoids calculating it. Instead, it **regresses directly onto sampled values of** U_t. By the least-squares projection principle, the optimal predictor is automatically the conditional expectation:

$$\underbrace{\text{hard-to-compute marginal velocity}}_{v_t^*(x)} \quad \Longleftarrow \quad \underbrace{\text{sampled conditional velocity}}_{X_1 - X_0}.$$

This is the fundamental reason Conditional Flow Matching can be trained efficiently.

7.3 Why can generated trajectories curve if training paths are straight?

For each endpoint pair, the training path is indeed a straight line:

$$X_t = (1-t)X_0 + tX_1.$$

But the network ultimately learns the conditional average

$$v_t^*(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x].$$

For example, suppose two paths cross at the same location with velocities

$$u_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad u_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

If both paths have equal conditional weight at that location,

$$v_t^*(x) = \frac{1}{2}u_1 + \frac{1}{2}u_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

This velocity equals neither individual path velocity. Therefore, the trajectory obtained by solving $\dot{x}_t = v_t^*(x_t)$ can be curved.

Key insight. Straight-line **training interpolations** do not imply straight-line **generative ODE trajectories**.

Step 8 — Why introduce more general Gaussian conditional paths?

Straight-line interpolation is only one special case of Flow Matching. The broader framework allows more general conditional probability paths.

8.1 A general Gaussian conditional path

For a given data sample x_1, define

$$\boxed{p_t(x \mid x_1) = \mathcal{N}\left(x; \mu_t(x_1), \sigma_t^2 I\right).} \tag{15}$$

Here $\mu_t(x_1)$ specifies how the center of the distribution moves, while σ_t determines how the distribution expands or contracts.

We can sample from this distribution using

$$\boxed{X_t = \mu_t(x_1) + \sigma_t \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).} \tag{16}$$

Differentiate with respect to t:

$$\frac{dX_t}{dt} = \dot{\mu}_t(x_1) + \dot{\sigma}_t \epsilon.$$

Equation (16) implies

$$\epsilon = \frac{X_t - \mu_t(x_1)}{\sigma_t}.$$

Substituting back gives the **conditional velocity field**

$$\boxed{u_t(x \mid x_1) = \dot{\mu}_t(x_1) + \frac{\dot{\sigma}_t}{\sigma_t}[x - \mu_t(x_1)].} \tag{17}$$

This derivation assumes $\sigma_t > 0$ at the relevant time. If the endpoint variance approaches zero, the endpoint is understood by taking a limit.

Motivation. Once we choose μ_t and σ_t, we can compute the conditional velocity explicitly and train with the same MSE framework. Straight-line interpolation is not the only available path.

8.2 The simplest linear Gaussian path

Choose

$$\mu_t(x_1) = tx_1, \qquad \sigma_t = 1 - (1 - \sigma_{\min})t,$$

where $0 \leq \sigma_{\min} \ll 1$. Then

$$\boxed{X_t = tx_1 + [1 - (1 - \sigma_{\min})t]\epsilon.} \tag{18}$$

At $t = 0$, $X_0 = \epsilon \sim \mathcal{N}(0, I)$.

At $t = 1$, $X_1 = x_1 + \sigma_{\min}\epsilon$, so

$$X_1 \mid x_1 \sim \mathcal{N}(x_1, \sigma_{\min}^2 I).$$

When $\sigma_{\min} = 0$, the endpoint is exactly the data sample. When $\sigma_{\min} > 0$, the final marginal distribution is the data distribution convolved with a small Gaussian, not exactly the original data distribution.

8.3 Derive the velocity field

Differentiate (18):

$$\frac{dX_t}{dt} = x_1 - (1 - \sigma_{\min})\epsilon.$$

The target velocity is therefore

$$\boxed{u_t = x_1 - (1 - \sigma_{\min})\epsilon.} \tag{19}$$

Setting $\sigma_{\min} = 0$ gives

$$X_t = (1 - t)\epsilon + tx_1,$$

and

$$\boxed{u_t = x_1 - \epsilon.} \tag{20}$$

This recovers the earlier straight-line Flow Matching objective.

Takeaway. The familiar target $x_1 - \epsilon$ is a special case of Gaussian Conditional Flow Matching. Different probability paths change the supervised target, the learning problem, and the behavior of

numerical integration.

Step 9 — How are Flow Matching models trained and sampled?

9.1 Training procedure

Given a data sample

$$x_1 \sim p_{\text{data}},$$

perform the following operations:

1. Sample Gaussian noise: $x_0 \sim \mathcal{N}(0, I)$.
2. Sample a random time: $t \sim \mathcal{U}(0, 1)$.
3. Construct the interpolated sample: $x_t = (1 - t)x_0 + tx_1$.
4. Construct the target velocity: $u_t = x_1 - x_0$.
5. Predict the velocity: $\hat{u}_t = v_\theta(x_t, t)$.
6. Optimize the parameters using $\mathcal{L} = \|\hat{u}_t - u_t\|^2$.

No numerical integration of the generative ODE is required during training.

A minimal PyTorch implementation:

```
import torch


def flow_matching_loss(model, x1, cond=None):
    # x1 shape: [B, D] or [B, H, D]
    B = x1.shape[0]

    # 1. Gaussian noise
    x0 = torch.randn_like(x1)

    # 2. Random flow timestep
    t = torch.rand(B, device=x1.device)
    t_view = t.view(B, *([1] * (x1.ndim - 1)))

    # 3. Linear interpolation
    xt = (1 - t_view) * x0 + t_view * x1

    # 4. Target velocity
    target_v = x1 - x0

    # 5. Predict velocity
    pred_v = model(xt, t, cond)

    # 6. Mean squared error
    loss = ((pred_v - target_v) ** 2).mean()
    return loss
```

9.2 Sampling (inference) procedure

At inference time, a real sample x_1 is no longer needed. Start by sampling

$$x_0 \sim \mathcal{N}(0, I),$$

then solve

$$\frac{dx_t}{dt} = v_\theta(x_t, t).$$

The simplest numerical integrator is **forward Euler**. Divide $[0, 1]$ into K steps with $\Delta t = 1/K$:

$$\boxed{x_{k+1} = x_k + \Delta t\, v_\theta(x_k, k\Delta t).} \tag{21}$$

The final state x_K approximates the desired generated sample x_1.

PyTorch pseudocode:

```
@torch.no_grad()
def sample_flow(model, shape, cond=None, K=10, device="cuda"):
    x = torch.randn(shape, device=device)
    dt = 1.0 / K

    for k in range(K):
        t = torch.full((shape[0],), k * dt, device=device)
        v = model(x, t, cond)
        x = x + dt * v

    return x
```

Other options include Heun, midpoint, Runge–Kutta, and adaptive ODE solvers. Higher-order methods can reduce numerical error, but they often require more neural-network evaluations per step. The design trade-off is between accuracy and the **number of function evaluations (NFE)**.

Step 10 — Why can Flow Matching generate multimodal distributions?

Suppose an observation o permits two different grasps:

$$p(A \mid o) = \frac{1}{2} p_{\text{left}}(A \mid o) + \frac{1}{2} p_{\text{right}}(A \mid o).$$

A conventional MSE policy $f_\theta(o)$ sees only the observation. Its optimal prediction is the conditional mean $\mathbb{E}[A \mid o]$, potentially falling between the two valid actions.

In contrast, a Flow Matching policy has the form

$$v_\theta(x_t, t, o),$$

with a random initial sample

$$x_0 \sim \mathcal{N}(0, I).$$

Different initial noise samples can be transported to different action modes.

Even if the ODE output is deterministic once x_0 and o are fixed, X_0 remains random. Therefore

$$X_1 = \Phi_1(X_0; o)$$

is also a random variable, with conditional distribution

$$\boxed{p_\theta(A \mid o) = (\Phi_1(\cdot; o))_{\#}\mathcal{N}(0, I).} \tag{22}$$

Key distinction. Flow Matching also uses MSE, but it regresses onto a **transport velocity field**, not onto a single final-action mean. This permits modeling multimodal action distributions.

Step 11 — How does π0.5 use Flow Matching?

11.1 The generated object is an action chunk

This chapter treats action chunks as high-dimensional random variables for generative modelling. Demonstration datasets, chunk execution horizons, temporal alignment and replanning belong in Data and Imitation Learning.

A robot action model aims to learn

$$\boxed{p(A \mid o, \ell),}$$

where

$$A = [a_0, a_1, \ldots, a_{H-1}]$$

is a sequence of future actions, o denotes visual observations and robot state, and ℓ denotes language or higher-level task context.

If each action has dimension d_a, then

$$A \in \mathbb{R}^{H \times d_a}.$$

For example, $H = 50$ and $d_a = 18$ give an action chunk with 900 continuous variables. Flow Matching generates this entire high-dimensional action chunk jointly rather than autoregressively sampling every action component.

11.2 Conditional Flow Matching for action chunks

First use the time convention adopted throughout these notes: $t = 0$ means noise and $t = 1$ means data. Let

$$\epsilon \sim \mathcal{N}(0, I),$$

and define

$$\boxed{A_t = (1 - t)\epsilon + tA.} \tag{23}$$

The target velocity is

$$\boxed{u_t = A - \epsilon.} \tag{24}$$

Train the **Action Expert** using

$$\boxed{\mathcal{L}_{\text{FM}} = \mathbb{E}\left[\|v_\theta(A_t, t, o, \ell) - (A - \epsilon)\|^2\right].} \tag{25}$$

Compared with unconditional Flow Matching, we additionally supply the observation and language condition to the velocity network. The ideal conditional velocity is

$$\boxed{v_t(x \mid o, \ell) = \mathbb{E}[U_t \mid X_t = x, o, \ell].} \tag{26}$$

The network therefore learns how to convert Gaussian noise into plausible robot action chunks **given the current observation and task**.

Deployment connection: Learning the generative velocity field is separate from handling chunk-handoff latency. See Real-Time Chunking in Data and Imitation Learning for asynchronous execution, prefix inpainting, training-time conditioning, and timing metrics.

11.3 Important implementation detail: OpenPI reverses the time direction

When reproducing π0.5, be careful to distinguish the explanatory time convention above from the convention used in actual code.

In the relevant Flow Matching implementation of OpenPI's official `pi0.py`, the convention is

$$t = 1 : \text{noise}, \qquad t = 0 : \text{real action}.$$

The interpolation is

$$\boxed{A_t = t\epsilon + (1 - t)A.} \tag{27}$$

Consequently,

$$\boxed{\frac{dA_t}{dt} = \epsilon - A.} \tag{28}$$

The velocity regression target is

$$\boxed{\mathcal{L} = \|v_\theta(A_t, t, o) - (\epsilon - A)\|^2.} \tag{29}$$

The model predicts **noise minus action**, not **action minus noise**. Both conventions are mathematically valid, but the velocity target and direction of integration must be consistent.

Sampling starts with $A_1 \sim \mathcal{N}(0, I)$ and integrates in the direction $t : 1 \to 0$. With K Euler steps, $\Delta t = -1/K$:

$$A_{t+\Delta t} = A_t + \Delta t\, v_\theta(A_t, t, o),$$

or equivalently,

$$\boxed{A_{t-1/K} = A_t - \frac{1}{K} v_\theta(A_t, t, o).} \tag{30}$$

The terminal state A_0 is the generated action chunk. Relevant OpenPI configurations commonly use 10 integration steps and can cache the visual-language backbone's prefix key/value tensors, avoiding repeated full-prefix computation at every flow step. Exact settings depend on the software version and configuration.

Flow Matching velocity is not robot joint velocity

It is essential to distinguish two different notions of time:

- **Generative time t:** the artificial ODE time used to turn noise into an action chunk.
- **Physical time:** the time during which the robot actually executes $a_0, a_1, \ldots, a_{H-1}$.

If $A_t \in \mathbb{R}^{50\times 18}$, then

$$v_\theta(A_t, t, o) \in \mathbb{R}^{50\times 18}.$$

This velocity describes how the entire **noisy action chunk** changes with generative time; it is not the physical joint velocity of the robot arm or dexterous hand.

Step 12 — How can Flow Matching be fine-tuned with reinforcement learning?

This section connects the derivation to research on **π0.5, DSRL, and residual velocity adaptation**.

12.1 What does pretrained Flow Matching learn?

Imitation learning produces a velocity field

$$v_{\text{pre}}(x, t, o),$$

whose generated distribution approximately matches the demonstration action distribution $p_{\text{demo}}(A \mid o)$.

Solving

$$\frac{dA_t}{dt} = v_{\text{pre}}(A_t, t, o)$$

produces actions resembling those in the demonstrations, but these actions need not maximize reward on the current task.

For example, several grasps may all be valid, but differ in success rate, stability, or execution time. The goal of RL post-training is to shift the generated action distribution toward **higher-return behavior**.

12.2 Why learn a residual velocity?

One natural approach freezes the pretrained velocity field and trains a smaller residual correction:

$$\boxed{v_{\text{new}}(x, t, o) = v_{\text{pre}}(x, t, o) + \Delta v_\phi(x, t, o).} \tag{31}$$

The generative ODE becomes

$$\boxed{\frac{dA_t}{dt} = v_{\text{pre}}(A_t, t, o) + \Delta v_\phi(A_t, t, o).} \tag{32}$$

Motivation. This resembles **residual control**: preserve useful behavior learned by a pretrained system while making incremental adjustments for a new reward or task objective.

This is a proposed post-training algorithmic choice—not a structural requirement of Flow Matching itself.

12.3 Why cannot reward be used directly as a velocity label?

During pretraining, the conditional target

$$u_t = A - \epsilon$$

comes from differentiating an explicitly defined path equation.

Reinforcement learning typically provides

$$R(A, o),$$

which evaluates a final action or executed trajectory. It does **not** directly specify the appropriate residual velocity at each intermediate generative state:

$$\Delta v_\phi(A_t, t, o).$$

We therefore cannot simply interpret an advantage estimate as a velocity label in generative space. A principled connection is required:

Reward → Target action distribution → Velocity-field update.

12.4 A natural design: Advantage-Weighted Flow Matching

Suppose online exploration yields a dataset

$$\mathcal{D} = \{(o_i, A_i, R_i)\}_{i=1}^N.$$

Assign larger **positive weights** to higher-return actions:

$$w_i = \exp\left(\frac{\widehat{\mathrm{Adv}_i}}{\beta}\right), \qquad \beta > 0.$$

Consider the following weighted residual-velocity regression objective:

$$\boxed{\begin{aligned}\mathcal{L}(\phi) = \mathbb{E}_{\mathcal{D},\epsilon,t}\Big[w(A,o)\big\| v_{\mathrm{pre}}(A_t,t,o) \\ + \Delta v_\phi(A_t,t,o) - (A-\epsilon)\big\|^2\Big],\end{aligned}} \tag{33}$$

where

$$A_t = (1-t)\epsilon + tA.$$

A residual regularizer may be added:

$$\lambda\, \mathbb{E}\left[\|\Delta v_\phi(A_t, t, o)\|^2\right]$$

to penalize deviations from the pretrained velocity field.

What is the mathematical motivation?

Recall that the optimal MSE regressor is the conditional expectation of velocities under the chosen joint distribution.

For fixed observation o, define a reward-weighted action distribution

$$\boxed{p_w(A \mid o) = \frac{w(A,o)p_{\mathcal{D}}(A \mid o)}{Z(o)},} \tag{34}$$

where

$$Z(o) = \mathbb{E}_{A\sim p_{\mathcal{D}}(\cdot|o)}[w(A,o)].$$

If the dataset has sufficient coverage, the weights are nonnegative and integrable, and the model and optimization are sufficiently expressive, weighted CFM can be interpreted as learning the velocity field associated with the **reweighted target action distribution**.

The research rationale is therefore

$$\boxed{\text{Advantage} \longrightarrow \text{weighted action distribution} \longrightarrow \text{Flow Matching} \longrightarrow \text{new velocity field.}}$$

Important limitation. Weighted Flow Matching is not automatically equivalent to an online policy-gradient method, and it does not guarantee monotonic policy improvement. Practical performance still depends on data coverage, advantage estimation, distribution shift, numerical integration, and the design of update constraints.

Step 13 — Synthesis: the complete mathematical logic of Flow Matching

13.1 Eight essential steps

1. Define the generative goal. Transport a Gaussian distribution into the data distribution:

$$p_0 = \mathcal{N}(0, I) \longrightarrow p_1 = p_{\text{data}}.$$

2. Represent continuous transport with an ODE.

$$\dot{x}_t = v_\theta(x_t, t).$$

Its induced probability path should obey

$$\partial_t p_t + \nabla \cdot (p_t v_\theta) = 0.$$

3. The ideal objective regresses onto a global velocity field.

$$\mathcal{L}_{\text{FM}} = \mathbb{E}[\|v_\theta(x,t) - v_t^*(x)\|^2],$$

but v_t^* is generally unknown.

4. Construct a computable conditional path.

$$x_t = (1-t)x_0 + t x_1, \qquad u_t = x_1 - x_0.$$

5. Prove that the marginal velocity is a conditional expectation.

$$\boxed{v_t^*(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x].}$$

6. Use the L^2 projection property of MSE.

$$\boxed{\mathbb{E}\|v_\theta - U_t\|^2 = \mathbb{E}\|v_\theta - v_t^*\|^2 + C.}$$

7. Arrive at the simple training objective.

$$\boxed{\min_\theta \mathbb{E}_{\substack{t\sim\mathcal{U}[0,1] \\ x_0\sim\mathcal{N}(0,I) \\ x_1\sim p_{\text{data}}}} \Big[\big\|v_\theta((1-t)x_0 + t x_1, t) - (x_1 - x_0)\big\|^2\Big].}$$

8. Generate samples by integrating the ODE.

$$\boxed{\begin{cases} x_0 \sim \mathcal{N}(0, I), \\ \dot{x}_t = v_\theta(x_t, t), \\ x_1 \sim p_\theta \approx p_{\text{data}}. \end{cases}}$$

The last line means that the **distribution of generated samples** approximates the data distribution; it does not imply that every generated sample must equal an existing training example.

13.2 Three ideas to remember

First, Flow Matching learns a dynamical system for probability distributions.

$$\boxed{\dot{x}_t = v_t(x_t) \quad \Longleftrightarrow \quad \partial_t p_t = -\nabla \cdot (p_t v_t).}$$

This links generative modeling to continuous-time dynamical systems.

Second, the essence of CFM is conditional expectation and L^2 projection.

$$\boxed{v_t^*(x) = \mathbb{E}[U_t \mid X_t = x].}$$

This explains why simple supervision from random conditional paths can train a velocity field for complicated distributions.

Third, straight-line training interpolation does not imply a straight-line sampling trajectory. The neural network learns a global marginal velocity field determined jointly by conditional paths and their probability distribution.

For online post-training of Flow Matching policies, a particularly useful research direction is

$$\boxed{\text{changing the action distribution} \quad \Longleftrightarrow \quad \text{changing the transport velocity field.}}$$

How to use rewards or advantages to modify the velocity field **stably**, while retaining the generative capabilities of a pretrained model, remains an optimization problem worth studying in its own right.

Document scope. This English edition preserves the streamlined 13-section structure: foundational derivations, conditional flow matching, training and inference, $\pi 0.5$, and residual-velocity RL fine-tuning. The previously requested chapters on *Optimal Transport* and on the *Flow Matching–Diffusion mathematical connection* remain excluded. Later sections and numbered equations have been renumbered continuously.

Humanoid Robotics: System 1 (VLA) and System 0 (Whole-Body Control)

Course home · 简体中文 · PDF

Item	Details
Applicability	SONIC paper integration: GR00T N1.5; documented G1 workflow: N1.7, 78-dim actions, 2.5 Hz inference / 50 Hz stream. Checkpoint and observation config must match
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

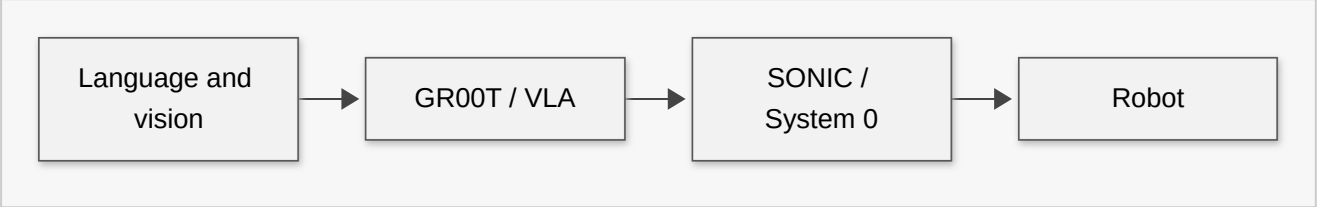
Verification scope: Documentation organization and upstream reference snapshot; no tutorial runtime, training or hardware validation. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Overall architecture: the two distinct problems
  - 1.1 Why not use a single model for everything?
  - 1.2 Key differences
  - 1.3 Two feedback loops, not an open-loop chain
- 2. System 1: how GR00T understands tasks and generates actions
  - 2.1 What is a VLA?
  - 2.2 Two components of GR00T
  - 2.3 Why predict more than one action at once?
  - 2.4 Flow matching, intuitively
  - 2.5 What action representation can a VLA predict?
  - 2.6 How is GR00T trained?
- 3. System 0: how SONIC learns whole-body motion control
  - 3.1 What is whole-body control (WBC)?
  - 3.2 A useful baseline: lower-body RL + upper-body IK
  - 3.3 SONIC's training task: motion tracking
  - 3.4 SONIC architecture: encoders, motion tokens, decoder
  - 3.5 How does SONIC turn a token into joint commands?
  - 3.6 PPO, tracking rewards, and robustness
  - 3.7 What SONIC knows—and what it does not
- 4. Connecting the two systems through motion tokens
  - 4.1 The central interface

    - 4.2 Verified action-space dimensions
    - 4.3 How are the rates coordinated?
    - 4.4 Why can this decomposition improve generalization?
    - 4.5 Practical integration checks
- 5. Worked example: walking, grasping, pedal pressing, disposal
    - 5.1 Task description
    - 5.2 Division of responsibility
    - 5.3 An important capability boundary
- 6. Training, demonstration collection, and real-robot deployment
    - 6.1 Step 1: obtain a working System 0 controller
    - 6.2 Step 2: collect teleoperated demonstrations
    - 6.3 Step 3: fine-tune GR00T to predict SONIC actions
    - 6.4 Step 4: deploy with PolicyServer and SONIC
    - 6.5 Step 5: evaluate the complete system
- 7. Research directions, ablations, and common misconceptions
    - 7.1 Where should improvements be targeted?
    - 7.2 Three minimal ablation studies
    - 7.3 Five common misconceptions
- 8. Summary and references
    - 8.1 Three takeaways
    - 8.2 Papers and official resources

Chapter concept map



1. Overall architecture: the two distinct problems

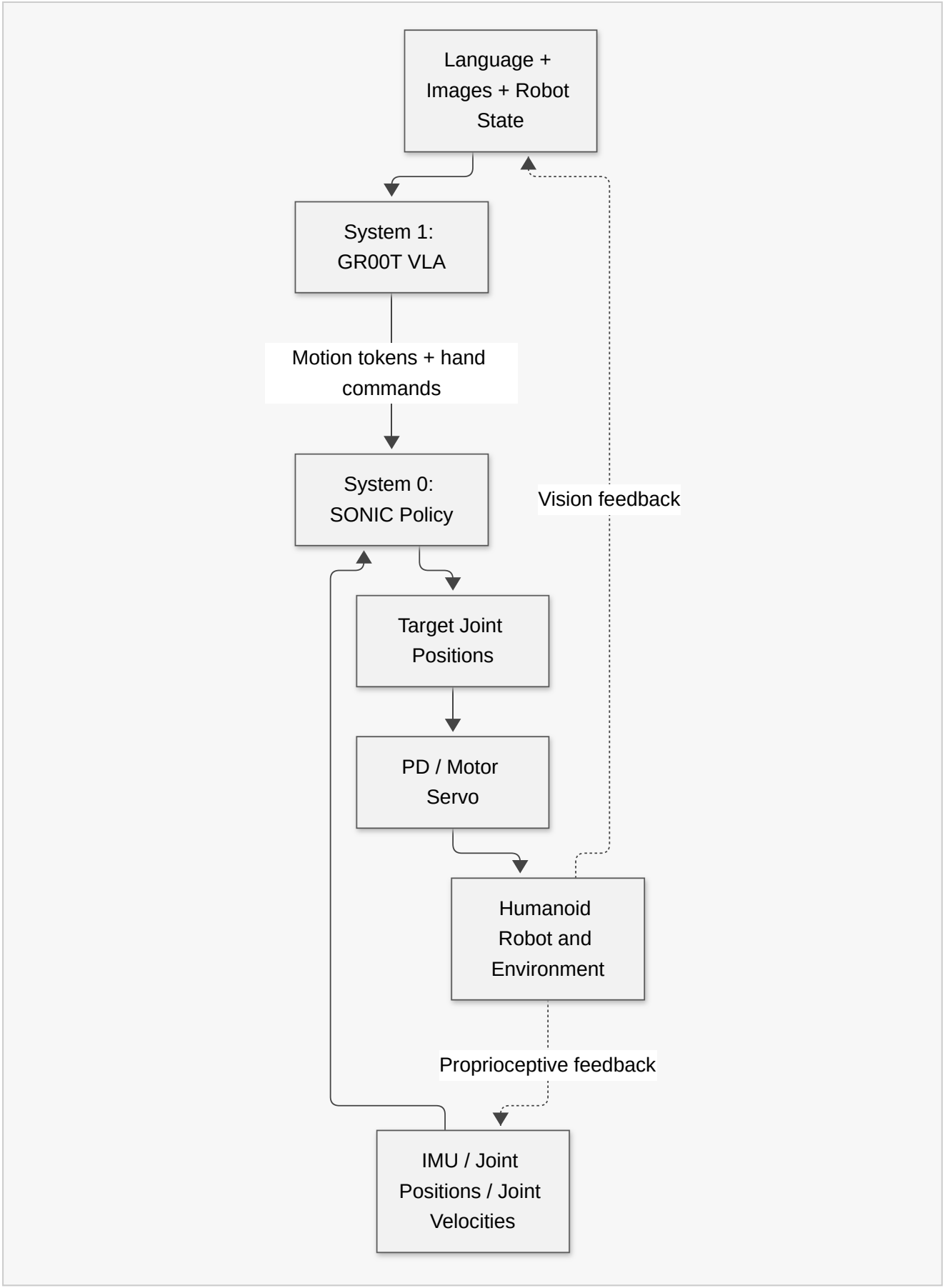
1.1 Why not use a single model for everything?

A humanoid faces two fundamentally different challenges. The first is **semantic task reasoning**: locating a cup, understanding “put the cup in the bin,” and choosing suitable motions. The second is **dynamics and contact**: controlling foot placement, shifting the center of mass, maintaining support, and avoiding falls while reaching and bending.

A practical engineering design separates these responsibilities:

- **System 1 / VLA (high level):** Processes camera images, language instructions, and robot state to predict future actions or motion latents.

- **System 0 / WBC (low level):** Processes motion references, joint positions and velocities, and IMU observations to generate full-body joint targets in real time.



Important: GR00T is not merely a classical symbolic task planner: it can generate short-horizon actions. SONIC is not a passive trajectory player, either: its outputs depend on the robot's measured state at each control step.

1.2 Key differences

Dimension	System 1: GR00T	System 0: SONIC
Main objective	Choose task-appropriate actions	Track motion while maintaining stability
Main inputs	Images, language, robot state	Motion reference, joints, IMU
Main outputs	Future action chunks	Full-body joint position targets
Typical architecture	VLM + flow-matching DiT	Motion encoder + RL controller
Typical training	Pretraining, imitation fine-tuning	Motion tracking + PPO
Central challenges	Generalization, perception, long tasks	Contact, balance, coupling, disturbances
Feedback	Vision and task state, lower rate	Proprioception, high rate

1.3 Two feedback loops, not an open-loop chain

The VLA replans from updated camera observations. Meanwhile SONIC updates joint targets from fresh proprioceptive observations. The point of the hierarchy is not that the VLA can ignore physics; rather, it should not need to solve fast full-body dynamics from scratch at every expensive vision-model inference call.

---

2. System 1: how GR00T understands tasks and generates actions

2.1 What is a VLA?

A **VLA (Vision-Language-Action Model)** turns visual and linguistic context into robot actions, rather than only producing text as a conventional vision-language model (VLM) would.

A VLM commonly maps observations to a text response:

$$(\text{Image}, \text{Language}) \longrightarrow \text{Text}.$$

A VLA predicts actions instead:

$$(\text{Image}, \text{Language}, \text{Robot State}) \longrightarrow \text{Action}.$$

For “pick up the cup on the table,” it must reason about the cup's location, the current robot configuration, and the sequence of motions needed to approach and grasp the object.

2.2 Two components of GR00T

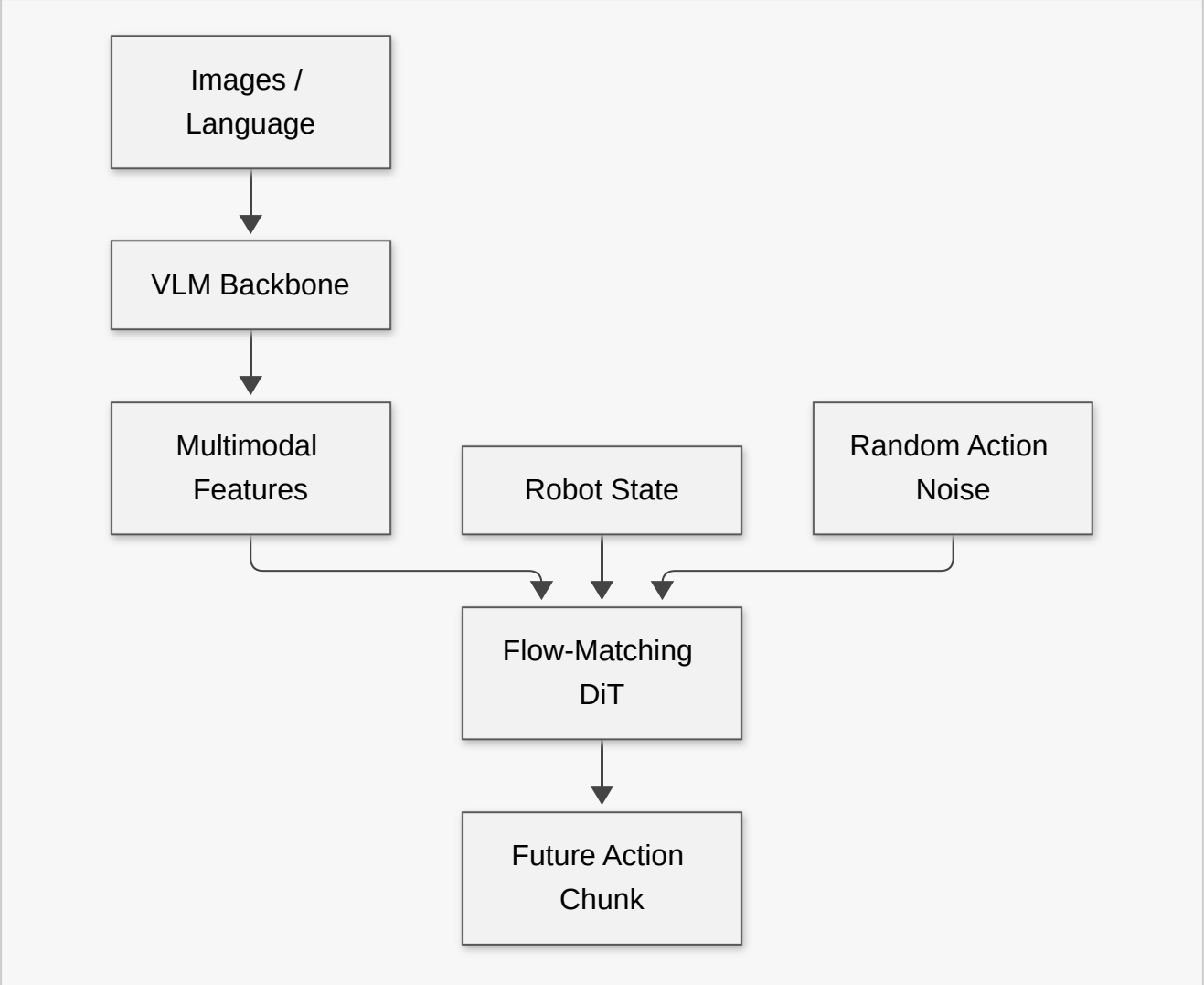
Using **GR00T N1.7** as the example, the main components are: [R3]

1. Vision-language backbone (VLM)

It extracts multimodal conditioning features from visual and linguistic observations. The public N1.7 code uses **Cosmos-Reason2-2B**, built on the Qwen3-VL architecture, as its backbone. This differs from the earlier N1.5/N1.6 configurations.

2. Flow-matching action head

A **Diffusion Transformer (DiT)** action head conditions on multimodal features and robot state to produce a sequence of actions via flow matching. Flow matching here is an action-generative learning method, not a replacement for the robot's physical control loop.



2.3 Why predict more than one action at once?

Instead of outputting a single instantaneous command, GR00T can predict a short sequence of length H:

$$A_t = [a_t, a_{t+1}, \ldots, a_{t+H-1}].$$

Here a_t is one action step, and H is the **action horizon**. N1.7 defaults to **40 steps**. [R3]

Action chunking reduces how often a large VLA must be invoked and promotes temporally coherent actions. **It does not mean that all 40 steps must be executed before a new observation is processed.** A receding-horizon controller can refresh the plan before the current chunk ends.

2.4 Flow matching, intuitively

GR00T's action expert produces an observation-conditioned action chunk from a noise prior. The **generative velocity field is not joint velocity**; chunk commands reach the System 0 controller through a specified interface and timing scheme. The complete CFM, continuity equation and ODE derivation belongs in Flow Matching.

2.5 What action representation can a VLA predict?

Action representation	Meaning	Advantage / limitation
Joint actions	Direct joint targets or increments	Simple interface; embodiment-specific
End-effector poses	Desired hand poses	Useful for manipulation; needs IK/WBC
Whole-body poses	Desired full-body postures	Expressive; physical feasibility is difficult
Latent actions	Learned motion representations	Leverages pretrained control; decoder compatibility is essential

The distinctive **GR00T + SONIC** choice is to use a **latent motion interface**. The VLA learns to predict the motion-token representation defined by SONIC; SONIC then translates that representation into full-body joint targets.

2.6 How is GR00T trained?

1. **Pretraining:** Learn multimodal and action priors from diverse robot demonstrations and human videos. N1.7 emphasizes human-video pretraining and cross-embodiment relative end-effector representations; not every embodiment uses SONIC latents. [R3]
2. **Task fine-tuning:** Collect task-specific demonstrations containing synchronized images, instructions, robot state, and target actions. Optimize a supervised action-generation objective.
3. **Closed-loop evaluation:** Repeatedly process new images, generate chunks, execute a portion of each chunk, and examine failures.

A dataset can be represented as:

$$\mathcal{D} = \{(I_t, l, s_t, a_t)\}_{t=1}^{N}.$$

In the SONIC integration, a_t may encode motion tokens and hand commands rather than raw full-body joint positions.

Note: Imitation fine-tuning is central to the official workflow. Online reinforcement-learning post-training is an interesting research option, but it is not required to use SONIC.

---

3. System 0: how SONIC learns whole-body motion control

3.1 What is whole-body control (WBC)?

A stationary robot arm mostly needs to move its end effector to a desired pose. A humanoid must simultaneously manage both legs, the torso, foot-ground contact, and its overall center of mass. Reaching, crouching, crawling, and pressing a pedal with one foot are strongly coupled activities.

Whole-body control is not simply many independent joint PID loops. It must account for whole-body dynamics and contact interactions.

3.2 A useful baseline: lower-body RL + upper-body IK

NVIDIA's **Decoupled WBC**, used with GR00T N1.5/N1.6, employs reinforcement learning for lower-body locomotion and balance, with inverse kinematics (IK) for upper-body reaching. [R4]

This design is modular and relatively easy to debug. However, the upper body and legs may require additional coordination for deep bending, one-leg-supported manipulation, or actions involving the feet.

SONIC instead uses a **unified RL motion-tracking policy** over diverse whole-body movements. More precisely, SONIC is a learned general-purpose whole-body motion tracker, not a conventional quadratic-programming WBC solver.

3.3 SONIC's training task: motion tracking

SONIC's central hypothesis is that one should train a single controller to track a broad distribution of human motion rather than build a separate reward-engineered policy for running, squatting, crawling, and so on. [R1]

The training procedure is conceptually simple:

1. Collect diverse **human motion-capture (mocap)** trajectories.
2. **Retarget** those trajectories into feasible robot reference motions.
3. Run physics simulations with randomized robot states and ask a policy to track the references.
4. Train with **PPO**, rewarding accurate tracking, stability, and smooth execution.
5. Apply **domain randomization** over relevant physical parameters and disturbances to improve sim-to-real robustness.

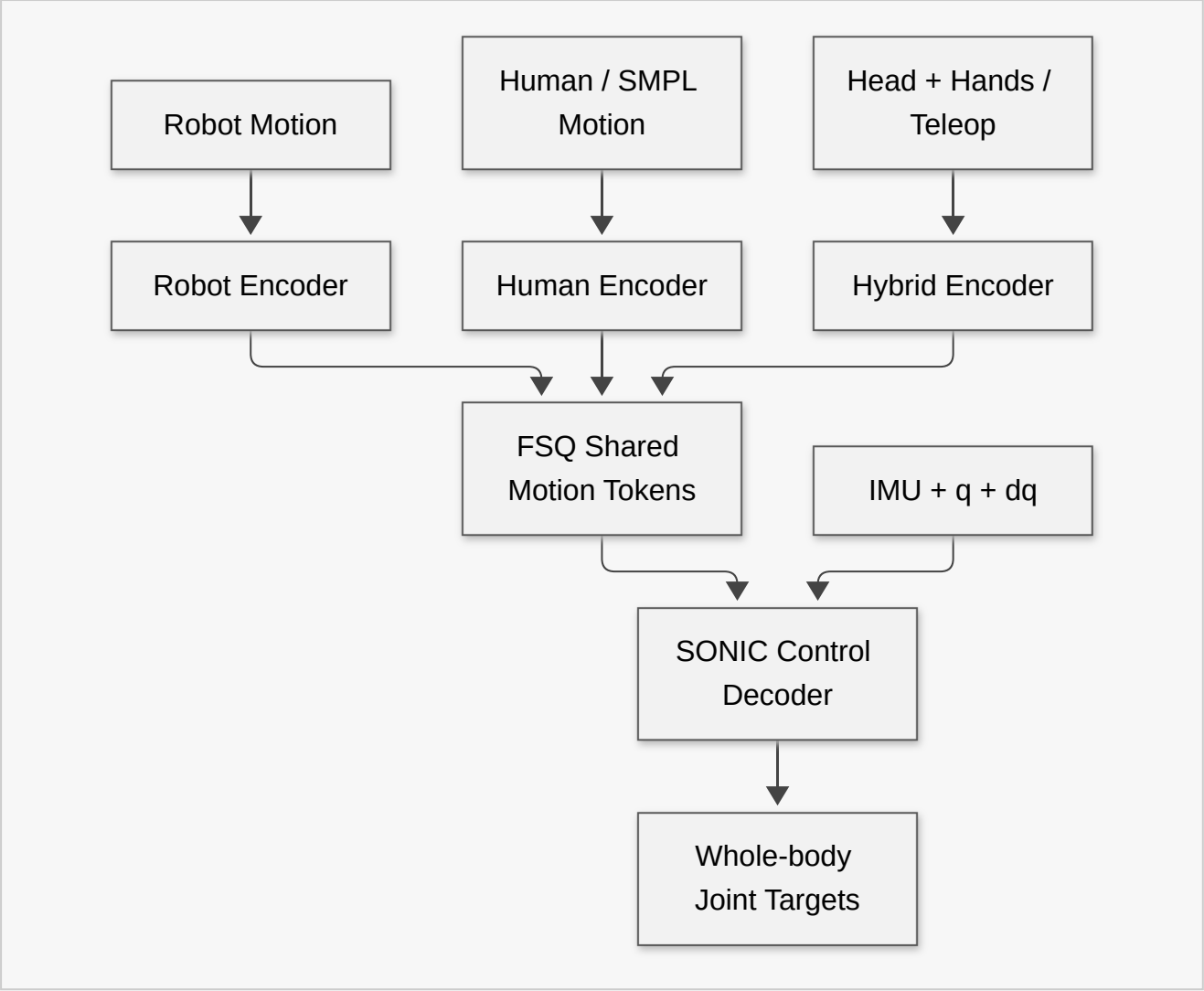
The paper reports roughly **700 hours of mocap, more than 100 million motion frames, a maximum policy size of about 42 million parameters, and around 21,000 GPU-hours**. These describe the published research scale, not the minimum resources needed to reuse a released controller. [R1]

3.4 SONIC architecture: encoders, motion tokens, decoder

SONIC maps different kinds of motion commands into a shared latent space:

- **Robot encoder:** Receives robot joint-motion references.
- **Human encoder:** Receives human / SMPL motion references.
- **Hybrid or teleoperation encoder:** Receives head-and-hand tracking targets and other kinematic constraints.

Their outputs are mapped into shared **universal motion tokens** using **finite scalar quantization (FSQ)**. A control decoder then combines a token with robot proprioception to output joint targets. [R1–R2]



Key intuition: A command can originate from a human operator, VR, a motion planner, a video-to-motion system, or a VLA. If it is encoded as a compatible motion token, the same controller can execute it.

3.5 How does SONIC turn a token into joint commands?

The interface can be summarized as:

$$\boxed{q_t^{\text{target}} = \pi_{\text{SONIC}}(s_t, z_t)}.$$

- s_t: Current proprioceptive observations, including joint positions/velocities and IMU measurements.
- z_t: Motion token describing the requested movement.
- q_t^{target}: Target full-body joint positions.

SONIC controls **29 body joints** in the official Unitree G1 configuration. [R2]

A lower-level servo may track those targets with a PD controller. With zero target joint velocity, a simplified expression is:

$$\tau_t = K_p(q_t^{\text{target}} - q_t) - K_d\dot{q}_t.$$

This equation illustrates a motor-level control law; it is **not** SONIC's learned objective. Real motor interfaces also include gain tuning, saturation, safety limits, and transport latency.

3.6 PPO, tracking rewards, and robustness

SONIC uses **PPO** with dense motion-tracking feedback. A simplified reward description is:

$$r_t \approx r_{\text{tracking}} - r_{\text{penalty}}.$$

Tracking terms encourage reference-consistent root motion, poses, keypoint positions, and velocities. Penalties discourage joint-limit violations, discontinuous commands, and undesirable contacts. The real objective comprises multiple weighted terms; the equation only communicates the idea.

An **asymmetric actor–critic** setup can give the critic accurate simulator-only information while restricting the actor to deployable observations. Only the actor / control decoder is required for real-time control.

Domain randomization may vary mass, friction, actuation, pushes, and initial conditions. It helps policies tolerate modeling errors, but does not guarantee recovery from arbitrarily severe disturbances.

3.7 What SONIC knows—and what it does not

SONIC learns coordinated full-body motion tracking under the dynamics of its robot embodiment. It does **not** independently recognize objects, interpret arbitrary language instructions, or solve high-level task planning. A VLA, human operator, or motion planner must still provide appropriate movement intent.

---

4. Connecting the two systems through motion tokens

4.1 The central interface

GR00T predicts the motion token plus hand commands; SONIC uses the token to compute body joint targets:

$$\begin{aligned}(z_t, h_t) &= \pi_{\text{GR00T}}(I_t, l, s_t),\\ q_t^{\text{target}} &= \pi_{\text{SONIC}}(s_t, z_t).\end{aligned}$$

The separate hand command h_t should **not be confused with SONIC's 29-dimensional body joint targets**.

4.2 Verified action-space dimensions

For NVIDIA's published **Unitree G1** + **SONIC** + **GR00T N1.7** example: [R4–R5]

$$\underbrace{a_t^{\text{VLA}}}_{78} = \big[\underbrace{z_t}_{64}, \underbrace{h_t^L}_{7}, \underbrace{h_t^R}_{7}\big].$$

Item	Specification
Motion token	64-dimensional learned whole-body latent
Left-hand actions	7 dimensions
Right-hand actions	7 dimensions
Total VLA action	78 dimensions per step
Action horizon	40 time steps
SONIC body joints	29 target joint positions

The **64 dimensions** describe a learned quantized motion representation, not 64 predefined verbal commands such as "walk," "turn," or "squat."

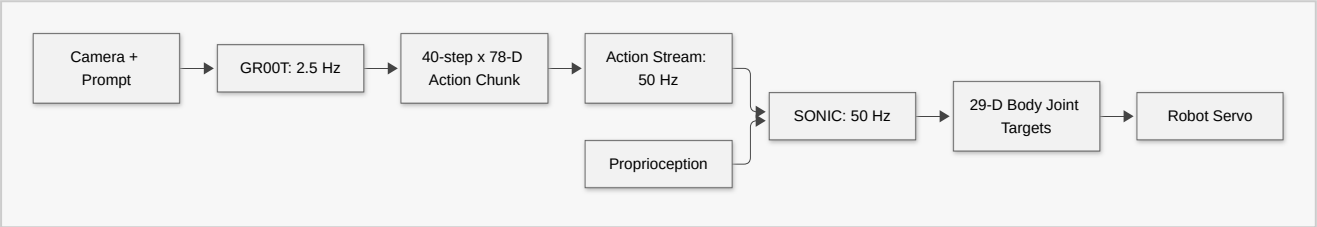
4.3 How are the rates coordinated?

The official N1.7 workflow specifies the following example rates: [R4–R5]

Component	Example rate	Interpretation
GR00T VLA inference	2.5 Hz	Refresh action chunk around every 400 ms
Action stream / SONIC	50 Hz	Update motion/joint targets every 20 ms
Action horizon	40 steps	0.8 s of future actions at 50 Hz
Low-level motor servo	Hardware-dependent	May run faster than the RL policy

Crucially, the **0.8 s prediction horizon is not necessarily an 0.8 s open-loop execution period**. The system can replace an in-progress chunk with new predictions after the next camera update.

Cross-chapter note: Real-Time Chunking (RTC) is a candidate technique for latency-tolerant asynchronous chunk handoff, but it needs integration and validation; this is not a claim that the referenced GR00T/SONIC implementation already runs RTC by default.



4.4 Why can this decomposition improve generalization?

If the VLA predicts every body joint directly, it must learn task-conditioned actions, body coordination, and balance simultaneously. Motion tokens expose the movement capabilities already

learned by SONIC, reducing the burden on the high-level model.

If a disturbance changes the state s_t, SONIC can respond by producing different joint targets even when z_t remains unchanged. **However, a physically impossible token or a severe unexpected contact can still cause failure.**

4.5 Practical integration checks

- **Timestamp synchronization:** Align image acquisition, action chunks, and joint/IMU observations.
- **Joint order and units:** Verify radians/degrees and the mapping of 29 body and 14 hand dimensions.
- **Checkpoint compatibility:** Match the VLA fine-tune, SONIC encoder/decoder checkpoint, and observation configuration.
- **Latency and chunk switching:** Avoid discontinuities; define a safe fallback when VLA inference stalls.
- **Safety:** Validate joint limits, emergency stop, communication loss behavior, and protected sim-to-real testing.

---

5. Worked example: walking, grasping, pedal pressing, disposal

5.1 Task description

Instruction: "Walk to the table, pick up the soda can, approach the bin, press the lid pedal with your foot, and drop the can inside."

This is a typical **loco-manipulation** task: the robot must locomote, grasp with its hands, use its foot to interact with the environment, and balance throughout.

5.2 Division of responsibility

Stage	GR00T / System 1	SONIC / System 0
Find the can	Perceive and select the target	Does not recognize the object
Approach table	Generate short-horizon approach actions	Execute gait and maintain support
Reach and grasp	Produce arm motion and finger commands	Coordinate torso, arms, and legs
Approach bin	Update motion based on visual feedback	Balance during locomotion
Press foot pedal	Specify intended foot/body movement	Coordinate single-leg support and contact
Dispose of can	Generate the placement / throwing action	Execute coordinated body motion

The original SONIC paper reports **60% success over 10 trials** on the soda-can-to-bin sequence, with approximately **75% mean success** across its grouped VLA loco-manipulation tasks. This supports feasibility while underscoring that reliability in unconstrained settings remains an open challenge. [R1]

5.3 An important capability boundary

SONIC can track a motion that places the foot near a pedal, but it does not infer the pedal's location from a language-only instruction. The high-level policy must provide sufficiently appropriate movement intent.

---

6. Training, demonstration collection, and real-robot deployment

6.1 Step 1: obtain a working System 0 controller

Start by validating an official SONIC checkpoint in simulation and a protected setting whenever possible. Training from scratch is a larger research undertaking.

The training pipeline is:

```
Human mocap / robot motion
       ↓ Retargeting + filtering
Robot motion-reference library
       ↓ Massively parallel Isaac Lab simulation
PPO motion tracking + auxiliary losses
       ↓ Validation / sim-to-sim / sim-to-real
SONIC actor + motion encoders
```

NVIDIA publishes SONIC training scripts, configuration guidance, and BONES-SEED processing instructions. [R2] **The public BONES-SEED subset should not automatically be equated with the entire 700-hour mocap corpus reported in the paper.**

Check standing stability, stepping and turning, squatting and single-leg support, disturbance response, and motion-transition continuity before collecting high-level VLA demonstrations.

6.2 Step 2: collect teleoperated demonstrations

With a VR headset or mocap system, a human provides motion references. A SONIC encoder converts those references into motion tokens, and the SONIC controller executes the commanded motions. Record synchronized:

- Robot camera images, covering the target and relevant contact regions;
- Language labels, timestamps, and robot state;
- **64-dimensional motion tokens** and **7+7 hand action dimensions**;
- Episode success, resets, interruptions, and discarded segments.

The official workflow exports demonstrations in **LeRobot** format and recommends collecting sufficient successful trajectories before cleaning and fine-tuning. [R4]

6.3 Step 3: fine-tune GR00T to predict SONIC actions

The supervision target becomes:

$$(I_t, l, s_t) \longmapsto (z_t, h_t).$$

The official N1.7 workflow uses the specific `UNITREE_G1_SONIC` embodiment tag:

```
# Example key flags from the official repository.
# Install Isaac-GR00T and prepare a LeRobot dataset first.
uv run python gr00t/experiment/launch_finetune.py \
  --base-model-path nvidia/GR00T-N1.7-3B \
  --dataset-path /path/to/lerobot_dataset \
  --embodiment-tag UNITREE_G1_SONIC \
  --modality-config-path gr00t/configs/data/embodiment_configs.py \
  --num-gpus 4 \
  --output-dir /path/to/output
```

This is an illustrative command showing essential integration flags, not a complete environment setup. Consult the version-matched official guide for all required options and dependencies. [R4]

6.4 Step 4: deploy with PolicyServer and SONIC

A GPU machine can run the GR00T PolicyServer, while the robot-side client receives camera and proprioceptive data, streams action chunks, and executes them with SONIC's C++ inference stack. [R5]

```
GPU workstation: GR00T PolicyServer
     ↕ images / states / action chunks
Robot compute: VLA client + SONIC C++ inference
     ↕ high-rate proprioception and joint targets
Motor servos + humanoid robot
```

Perform **sim-to-sim tests** before moving to protected, low-speed robot trials. NVIDIA provides MuJoCo testing and deployment tooling; direct unprotected first deployment is not advisable. [R2, R5]

6.5 Step 5: evaluate the complete system

Layer	Useful metrics	What they reveal
VLA	Task success, generalization, inference latency	Whether task-conditioned actions are appropriate
Motion tokens	Reconstruction error, compatibility, smoothness	Whether movement intent is faithfully represented
SONIC	Tracking success, keypoint error, fall rate	Whether motions are executed robustly
End-to-end	Overall success, delay, recovery, safety events	Interaction and integration reliability

Debug failures by layer: First inspect whether the VLA selected an appropriate motion, then whether the token carries that intent, then whether SONIC could track it. Not every task failure is a high-level VLA failure.

---

7. Research directions, ablations, and common misconceptions

7.1 Where should improvements be targeted?

Objective	Primary focus	Potential approaches
Few-shot and object generalization	System 1	Data augmentation, pretraining, imitation fine-tuning
Long-horizon tasks and corrections	System 1	Feedback replanning, task-level post-training
Natural full-body motion	System 0	Large-scale motion tracking
Disturbance and sim-to-real robustness	System 0	PPO, domain randomization, system identification
Complex hand-foot contact	Both layers and interface	Contact-rich demonstrations and evaluations
Reliable hierarchy	Interface / deployment	Token design, timing, safety filtering

7.2 Three minimal ablation studies

Ablation A: action representation

With as much else as possible held constant, compare (i) raw joint actions, (ii) explicit whole-body poses, and (iii) SONIC motion tokens. Measure success, smoothness, and sensitivity to infeasible commands. Such a comparison changes more than dimensionality; do not attribute all performance differences to compression alone.

Ablation B: feedback rate

Freeze SONIC while varying VLA refresh rate, the number of actions executed from each chunk, and imposed inference delay. Test moving targets, contact changes, and disturbance recovery.

Ablation C: low-level robustness

Freeze the high-level tokens and perturb simulator mass/friction, IMU measurements, and external pushes. Compare tracking success, posture error, and fall rate.

7.3 Five common misconceptions

1. **"System 1 only writes a plan; System 0 generates all motions."** Not exactly. A VLA can generate short-horizon actions directly.
2. **"SONIC understands language and recognizes objects."** No. It is primarily a full-body motion controller.
3. **"Motion tokens are fixed skill IDs."** No. Here they belong to a learned quantized motion latent space.
4. **"A 40-step action chunk requires 0.8 s of open-loop execution."** No. Prediction and execution horizons can differ.
5. **"A new SONIC checkpoint is always drop-in compatible with the VLA."** No. The latent space, encoders, and observation schemas must match.

---

8. Summary and references

8.1 Three takeaways

First, division of labor: GR00T turns visual-linguistic context into future actions; SONIC executes full-body motions using real-time proprioceptive feedback.

Second, the interface: Motion tokens bridge high-level task intelligence and low-level physical skills. In the published G1 + SONIC configuration, the motion-token vector is 64-dimensional and the total VLA action is 78-dimensional after adding both hands.

Third, training: SONIC primarily acquires general motion skills via large-scale motion tracking and PPO. GR00T learns multimodal action generation using pretraining and task-specific demonstrations.

The combined system can be summarized as:

$$\boxed{(I_t, l, s_t) \xrightarrow{\text{GR00T}} (z_t, h_t) \xrightarrow[s_t]{\text{SONIC}} q_t^{\text{target}} \xrightarrow{\text{Servo}} \text{Robot}}.$$

8.2 Papers and official resources

- **[R1] SONIC paper and project:** Luo et al., *SONIC: Supersizing Motion Tracking for Natural Humanoid Whole-Body Control*, *Science Robotics* 11(117), 2026. Project · arXiv · DOI
- **[R2] Official SONIC docs and training:** GR00T-WholeBodyControl Documentation · Training Guide
- **[R3] GR00T N1.7 official code:** NVIDIA/Isaac-GR00T
- **[R4] Official GR00T + SONIC workflow:** VLA Workflow: Collect, Fine-tune, Deploy
- **[R5] SONIC inference/deployment:** VLA Inference Guide
- **[R6] Whole-body-control code:** NVlabs/GR00T-WholeBodyControl

Prepared October 16, 2026. Model versions, dimensions, and frequencies refer to the cited public implementations, not to a universal humanoid robotics standard.

Advanced Robot Agents: Planning, Skills, Verification and Recovery

Course home · 简体中文 · PDF

Item	Details
Applicability	RAX, Gemini Robotics ER, SayCan, Code as Policies: conceptual comparison
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Research workflow and methodology discussion; no robot deployment validated. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Responsibilities of Agents and Policies
- 2. Closed-Loop Workflow
  - 2.1 Executable Skill API
- 3. Representative Robot Agent Approaches
- 4. Planning, Verification and Belief State
- 5. Architectures and Safety
- 6. Shared Panda Experiment and Ablation
- 7. System Architecture: Constraining Robot-Directed Agent Actions
  - 7.1 Hierarchical Tools, Skills, and Task State
  - 7.2 A Closed Loop Requires Acceptance Tests, Not Just Feedback
  - 7.3 Different Time Scales Must Stay Separate
- 8. Detailed Contemporary Approaches
  - 8.1 RAX: Online Exploration, Compilation, and Held-Out Evaluation
  - 8.2 Gemini Robotics ER 2: Streaming Video and Tool Orchestration
  - 8.3 Comparing the Workflows
- 9. Independent Verification, Memory, and Risk
  - 9.1 Task Verification Beyond the Pose Estimate
  - 9.2 Choosing a Skill Under Benefits, Costs and Risks
  - 9.3 A Skill Library Must Preserve Evidence
- 10. Failure Recovery and Real-Robot Interfaces
  - 10.1 Diagnose Failures by Layer
  - 10.2 Shared Panda and Dexterous Tool Case

- 11. Evaluating Robot Agents Scientifically
  - 11.1 Minimum Ablation Matrix
  - 11.2 Metrics and Boundaries
- 12. Knowledge Boundaries

1. Responsibilities of Agents and Policies

A Robot Agent handles goal decomposition, skill selection, state updates, verification and replanning. VLA/imitative policies generate conditional robot actions; a motion controller handles timely execution. Write the hierarchy as $g \to z_k \to \pi_{\text{skill}}(a_t \mid o_t, z_k)$, where g is a task and z_k a parameterized skill. An agent must **not replace a high-rate servo loop**.

2. Closed-Loop Workflow

Starting from a language goal, an agent reads visual/proprioceptive state, proposes a reachable subgoal, calls a skill, then observes again and verifies success, recovery or stop conditions. Finishing a skill is not the same as satisfying the task. An occluded or unknown outcome cannot be marked successful.

2.1 Executable Skill API

Represent a skill as (name, preconditions, arguments, controller, timeout, success_predicate, fallback). A pick(object_id, grasp_pose) skill checks object identity, pose confidence and collision feasibility; on timeout it relocalizes or exits safely instead of retrying forever.

3. Representative Robot Agent Approaches

Gemini Robotics ER illustrates vision-language and spatial reasoning for task orchestration; RAX (Robot Agent Exploration) explores feedback-driven experience and skill reuse. Related ideas include SayCan for language-grounded skill affordances and Code as Policies for programmatic compositions. Capabilities depend on model version and robot integration: a paper demonstration does not imply calibration-free autonomous operation on arbitrary robots.

4. Planning, Verification and Belief State

For partial observability maintain a belief $b_t(s) = p(s_t \mid o_{\le t}, a_{<t})$. A grasp verifier such as $V_{\text{grasp}} = \mathbf{1}[\text{object lifted and retained}]$ should use visual and proprioceptive evidence over time. Low-confidence outcomes enter UNKNOWN and trigger more observation, rather than being labeled SUCCESS.

5. Architectures and Safety

Compare LLM/VLM plus fixed skills, VLM agents plus VLA, world-model candidate planning plus skills, and exploration agents plus experience memory. Predicted world-model imagery is not a ground-truth state measurement. Gate all high-level commands through workspace, collision,

force/velocity, latency, confidence and emergency-stop constraints. Low-level safety cannot depend on remote LLM response.

6. Shared Panda Experiment and Ablation

Shared Panda cube-on-table task: pick from randomized locations, place at goal, then recover from an externally displaced object. Compare A fixed script, B agent plus skills, C visual verification, D replanning, and E reusable memory while fixing the controller and evaluation distribution. Report success $\hat{p} = \frac{1}{N}\sum_i \mathbf{1}[\text{goal achieved}]$, latency, collisions, interventions, retries, recovery and tool/model costs.

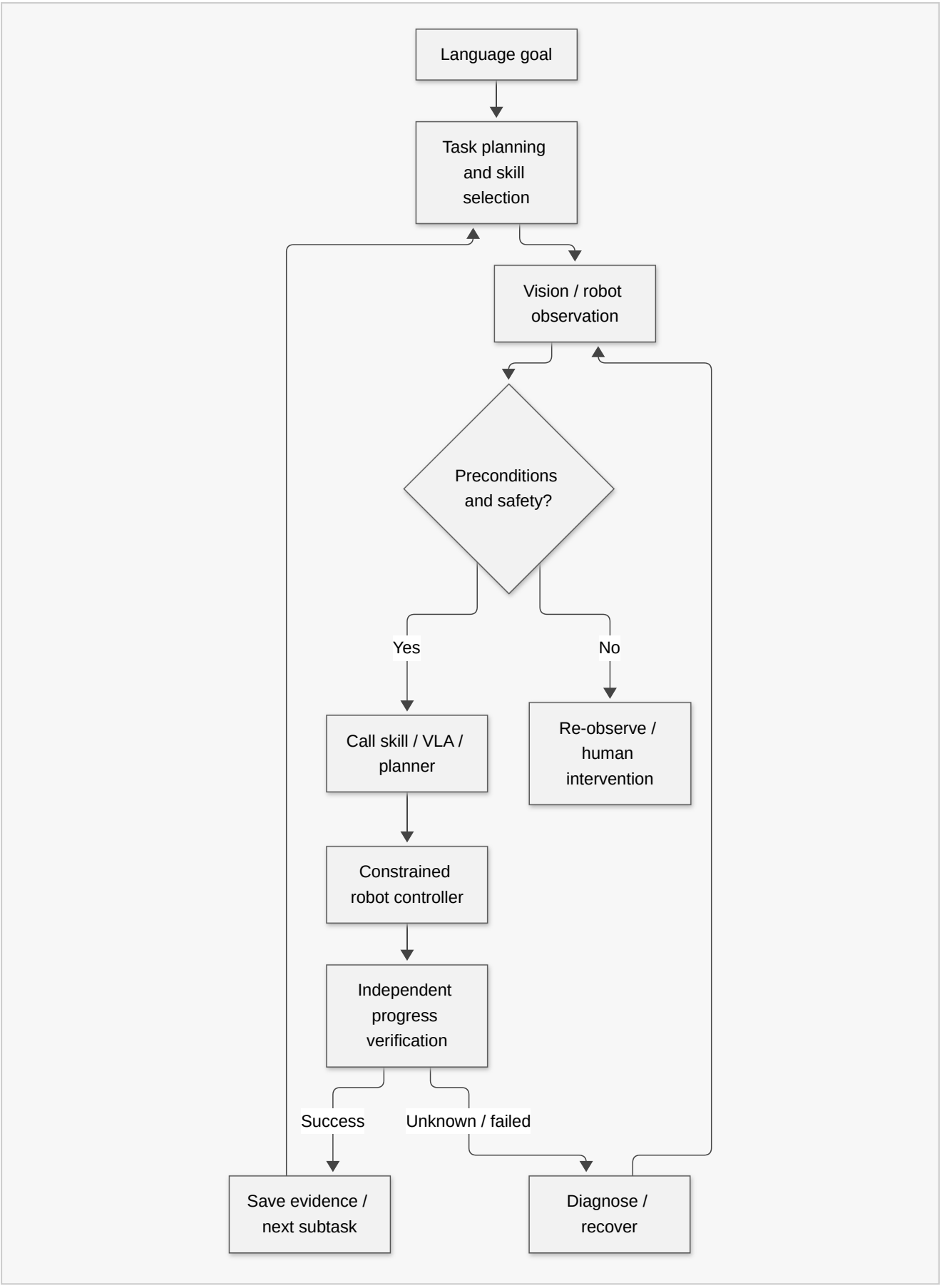
7. System Architecture: Constraining Robot-Directed Agent Actions

7.1 Hierarchical Tools, Skills, and Task State

Use structured task memory rather than an ever-growing free-form text log: object IDs, observation/pose timestamps, confidence, completed steps, gripper state, tool results, retry count and safety events. A skill $\omega = (I_\omega, \pi_\omega, \beta_\omega)$ has an initiation set I_ω, an internal policy π_ω and a termination rule β_ω; reliable deployment also needs a separate success verifier and recovery rule.

Useful tools include target search, pose query, navigation, gripper control, low-level VLA execution, progress verification and safe stop. The API contract must state coordinate frame, unit, parameter bounds, allowed control mode, timeout, observable completion, errors and audit logging. **The tool returning 'command accepted' is not proof the grasp succeeded.**

7.2 A Closed Loop Requires Acceptance Tests, Not Just Feedback



Separately check whether the command has terminated, the physical task outcome is achieved, and the next operation can begin safely. A language model saying 'done' is not a physical postcondition. Perception mathematics belongs in Visual State Estimation.

7.3 Different Time Scales Must Stay Separate

The top-level agent may be event-triggered; the estimator updates with camera observations; VLA policies may produce chunks; and the robot servo loop operates independently at higher frequency. Agent tokens must not directly drive servo commands. Bound each stage with explicit timeouts, stale-state detection and safe fallback. Select numerical rates from measured hardware timings, not universal assumptions.

8. Detailed Contemporary Approaches

8.1 RAX: Online Exploration, Compilation, and Held-Out Evaluation

RAX uses a general coding agent, including publicly documented Codex CLI or Claude Code setups, to manipulate through a robot tool API, collect observations and interaction feedback, remember useful preconditions, and compile successful explorations into a standalone robot controller program. The conceptual loop is **Explore → Compile → Evaluate**. Execution of the compiled program can avoid frequent calls to the original coding agent.

Published demonstrations cover different robots in simulation and hardware; the reported scaling results are tied to those evaluation tasks, not a universal real-robot success guarantee. The distinctive idea is episodic online exploration, persisted skill memory, compilation and evidence-backed testing; **RAX need not train a new VLA with gradients**.

8.2 Gemini Robotics ER 2: Streaming Video and Tool Orchestration

Gemini Robotics ER 2 is designed as a high-level embodied reasoner. It consumes video and language, plans steps, calls developer-defined robot APIs or lower-level VLA models, monitors progress, handles changes, and supports multi-robot coordination. It **does not normally replace the real-time motor control loop by emitting every joint torque itself**.

Continuous video enables decision-making during execution rather than once before a long open-loop episode. Progress classification and critical-event recognition help determine when to finish or correct steps; they still need physical postconditions and the independent hardware safety stack. The published Model Card should be used to check capabilities, limits, and supported interface versions.

8.3 Comparing the Workflows

Dimension	RAX	Gemini Robotics ER 2
Key mechanism	Explore robot API and compile learned skills	Streaming multimodal reasoning and tool selection
Adaptation	Trial-and-error, experience memory, verified code	Perceptual/linguistic generalization and feedback planning

Dimension	RAX	Gemini Robotics ER 2
Runtime	Compiled program may run without coding agent	Ongoing reasoning/stream processing may be used
Motor execution	Robot backend planners/controllers or learned policies	Attached VLA, navigation or robot tools
Primary challenges	Exploration safety, skill transfer, failed branches	Video mistakes, cost, latency, permission and safety

A possible hybrid uses RAX-style experience gathering to produce auditable skills and an ER-style reasoner to schedule them. This is a **proposed system design**, not a claim that the papers already established gains for such a hybrid.

9. Independent Verification, Memory, and Risk

9.1 Task Verification Beyond the Pose Estimate

Define an independent postcondition $V_\omega(o_{0:t}, q_{0:t}, g)$ for each skill. A grasp should verify stable object motion with the end effector, not just a closed gripper; insertion should check alignment, contact and final seating, not simply that the end effector reached a target. Verification states should distinguish SUCCESS, FAILURE and UNKNOWN. **UNKNOWN requires more sensing or safe human confirmation; it is not automatically SUCCESS.**

9.2 Choosing a Skill Under Benefits, Costs and Risks

A conceptual skill-choice objective (not an asserted internal optimization of RAX/ER 2) is

$$\omega_t^\star = \arg\max_{\omega \in \Omega(b_t)} \left[\mathbb{E}[R_{\text{goal}} \mid b_t, \omega] - \lambda_c C(\omega) - \lambda_r \text{Risk}(b_t, \omega)\right].$$

Only skills satisfying their preconditions and safety permissions enter $\Omega(b_t)$. The actuator service, not the language model, must enforce hard workspace and motor limits; this formula is not a hardware safety certificate.

9.3 A Skill Library Must Preserve Evidence

Store robot hardware, camera/URDF/controller versions, preconditions, execution traces, failure modes, postcondition checkers and unattended test count. The public RAX Skills Library illustrates retaining executable code with logs and success evidence. A skill that worked on another gripper or calibration revision must be revalidated before deployment.

10. Failure Recovery and Real-Robot Interfaces

10.1 Diagnose Failures by Layer

Failure	Evidence	Local recovery
Lost target	LOST state, invalid mask, stale image	Hold and reacquire

Failure	Evidence	Local recovery
Object slip	Relative pose change, gripper signal	Retreat safely and regrasp
Unreachable target	Failed IK / workspace limit	Change approach or skill
Tool timeout	No completion, excessive latency	Stop commands, inspect robot
Unknown progress	Conflicting verifier signals	Observe again or ask operator
Physical emergency stop	Hardware stop input	Stop immediately; no autonomous restart

Give the agent access only through a logged robot service. Preserve vendor safety controls, hard emergency stops and independent workspace/force/speed constraints. Uncertain observations must never justify arbitrary high-force exploratory movements.

10.2 Shared Panda and Dexterous Tool Case

In Panda + cube + table, preserve the same robot and controller while varying object starts, occlusions, displacement and grasp failures. The agent should localize → query trusted depth/pose → choose grasp skill → execute and verify → place → verify final state, recovering only affected subtasks. A screwdriver and dexterous-hand variant adds tool orientation, contact stability and reacquisition. Perception and policy training are taught in their own chapters; the agent chapter owns multi-step orchestration.

11. Evaluating Robot Agents Scientifically

11.1 Minimum Ablation Matrix

Compare A) fixed FSM/behavior tree, B) one-shot VLM planning, C) agent with execution feedback, D) independent success verifier added, E) failure replanning added, and F) persistent skill memory added. Additionally compare exploratory agent runs against compiled skill execution. Keep camera, robot API, controllers and held-out layouts fixed so improvements can be attributed to the orchestration layer.

11.2 Metrics and Boundaries

Report unseen-scene task success with confidence intervals, subtask completion, failure-detection recall, recovery success, tool-call count, interventions, inference/token cost, wall-clock completion time, controller latency and blocked unsafe actions. Exploration budgets and deployed inference costs must be reported separately. On real hardware, specify operator takeover conditions, maximum retries and permitted skill set.

12. Knowledge Boundaries

This chapter owns task-level orchestration, verification and recovery only. Consult Visual State Estimation for masks/depth/pose, Imitation Learning for collecting demonstrations and fitting BC/ACT/Flow policies, Flow Matching for full derivations, and GR00T/SONIC for system interfaces.

References: RAX · RAX code · skill library · Gemini Robotics ER 2 · SayCan · Code as Policies. No hardware benchmark or reproduction is claimed.